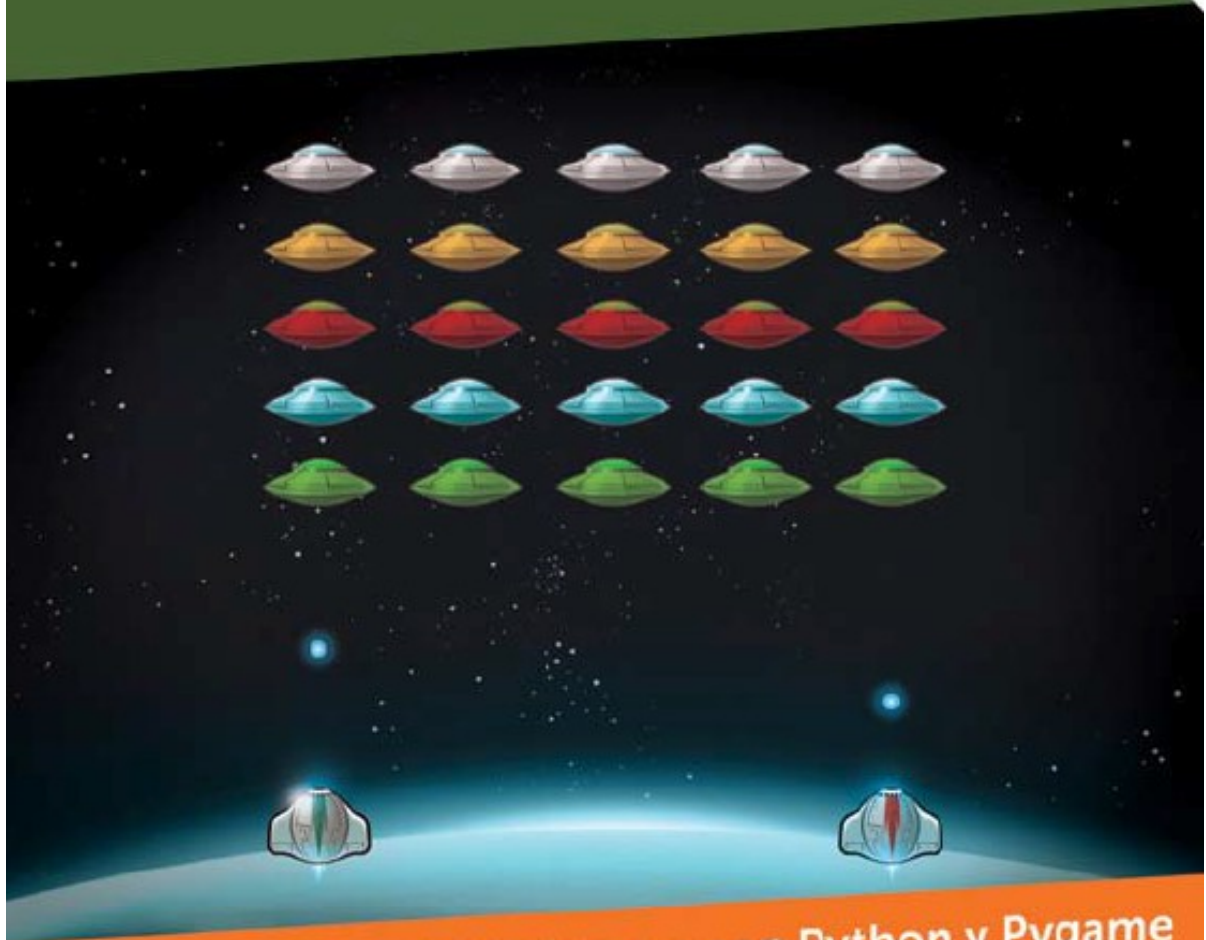


¡HACETE TU VIDEOJUEGO!



Programación de Videojuegos con Python y Pygame

Fernando Sansberro

Indice

Indice.....	2
¡Hacete tu Videojuego!.....	9
Nota para los Padres y Educadores.....	9
¿Porqué se Escribió este Libro?.....	10
Sobre el Libro.....	10
Descarga de los Materiales.....	10
¿Para Quién es este Libro?.....	11
¿Qué Nivel alcanza el Libro?.....	11
¿Por qué es tan Importante la Programación?.....	12
Requisitos para Comenzar.....	12
1. Introducción a la Programación de Videojuegos.....	13
La Pantalla.....	13
Resolución de la Pantalla.....	14
FPS (Frame Rate, Cuadros por Segundo).....	14
El Game Loop.....	16
La Estructura de un Videojuego.....	17
Inicialización del Sistema y Carga de Assets.....	18
Obtener la Entrada del Jugador.....	19
Actualizar los Objetos del Juego (la función update()).....	20
Dibujar los Objetos del Juego (la función render()) y Actualizar la Pantalla.....	21
Liberación de los Recursos Utilizados.....	22
2. Nuestro Primer Programa en Pygame.....	23
¿En qué lenguajes se programa un juego?.....	23
Librerías Gráficas y APIs.....	23
Pygame y SDL.....	24
Cómo Instalar Python y Pygame.....	26
Principales Características de Pygame.....	26
Los Programas de Ejemplo y el Editor.....	27
Nuestro Primer Programa con Pygame.....	27
Inicializar Pygame.....	30
Establecer el Modo de Video.....	31
Crear la Superficie del Fondo.....	32
La Inicialización del Juego.....	32
El Game Loop.....	33
El Reloj.....	35
Procesar los Eventos.....	35
Refrescar la Pantalla.....	36
Liberar la Memoria al Final.....	36
Encoding.....	37
¿Qué hace el Game Loop en un Videojuego?.....	37
Modo Fullscreen.....	39
Cambiar entre modo Fullscreen y modo Ventana.....	40
Blit.....	41

Double Buffering.....	42
3. Moviendo Nuestro Primer Objeto.....	44
El Sistema de Coordenadas.....	44
Ubicando un Objeto en la Pantalla.....	46
Moviendo nuestro primer Objeto.....	47
Crear el Objeto.....	50
Colocar el Objeto en su Posición Inicial.....	51
Mover el Objeto.....	52
Dibujar el Objeto.....	53
Agregando Otro Objeto al Juego.....	54
Chequear los Bordes de la Pantalla.....	57
Evitar Código Duplicado.....	65
Escribiendo la Clase CCuadrado.....	66
4. Manejo de Imágenes.....	74
Usando Imágenes.....	74
Crear una Superficie.....	75
Los Colores en Pygame.....	76
Cargar una Imagen.....	78
Dibujar una Imagen.....	80
Cargar Imágenes con Transparencia.....	81
Establecer Valores por Defecto.....	86
5. Estructura de un Videojuego.....	88
El Ciclo init() / update() - render() / destroy().....	88
Destrucción de Objetos.....	92
Estructura de los Objetos del Juego.....	93
El Game Object.....	94
Packages api / game.....	99
Encapsulamiento de los Objetos.....	102
Tipos de Objetos.....	105
6. Movimiento de Objetos.....	106
Introducción a la Física.....	106
Física Real vs. Física Simulada.....	107
Vectores.....	107
Velocidad y Rapidez.....	108
Rapidez.....	109
Tiempo y Frames.....	110
Movimiento a Velocidad Constante.....	111
¿Qué Velocidad debe tener un Objeto en el Juego?.....	115
Movimiento en Diagonal.....	115
Aceleración.....	117
¿Cómo saber dónde va a estar un objeto en el futuro?.....	117
Simplificando las Ecuaciones en los Juegos.....	118
¿Cómo saber qué velocidad tendrá un objeto en el futuro?.....	119
Implementando la Aceleración.....	119
7. Controlando el Jugador con el Teclado.....	122
Manejo de Eventos.....	122
Chequear la Pulsación de Teclas.....	124
Eventos del Teclado.....	126

La función <code>pygame.key.get_pressed()</code>	127
Agregando el Jugador.....	128
La Clase <code>CKeyboard</code>	139
Una Clase Singleton.....	143
Mejorando la Clase <code>CKeyboard</code>	144
8. Movimiento de Sprites.....	145
¿Qué es un Sprite?.....	145
¿Cuál es la diferencia entre un Sprite y un Game Object?.....	145
¿Qué información tiene un Sprite?.....	146
La clase <code>CSprite</code>	146
Chequeo de los Bordes de la Pantalla.....	152
Manejo de Colisiones con los Bordes.....	165
9. Disparos y el Manager de Sprites.....	170
Disparando Balas.....	170
El Manager de Balas.....	174
La función <code>fire()</code> de <code>CKeyboard</code>	180
Eliminando Balas del Manager.....	183
Destruyendo los Objetos.....	185
La Clase <code>CGameConstants</code>	186
10. El Manager de Enemigos.....	189
El Manager de Enemigos.....	189
Disparo de los Enemigos.....	193
Inicialización de los Enemigos - La Formación.....	198
Agregando otro Jugador - Modo Dos Players.....	200
Unificando los Managers.....	208
11. Detectando las Colisiones.....	214
Colisiones entre Dos Sprites.....	214
Sprites Rectangulares.....	214
Colisiones entre Rectángulos.....	216
Colisiones de un Sprite contra una Lista de Sprites (Manager).....	217
Colisiones de las Balas del Jugador contra los Enemigos.....	219
Reaccionando a las Colisiones.....	220
12. Animación de Sprites.....	223
Los Cuadros de una Animación.....	223
La Clase <code>CAnimatedSprite</code>	223
La Función <code>setImage()</code> de la Clase <code>CSprite</code>	224
Animando un Sprite.....	228
Cargando las Imágenes.....	228
Inicializando la Animación.....	232
Actualizando la Animación en <code>update()</code>	234
Controlando la Velocidad de la Animación del Sprite.....	235
Animando la Nave del Jugador.....	237
Optimizando la Memoria.....	240
13. Máquinas de Estados de los Objetos.....	242
¿Qué es una Máquina de Estados?.....	242
¿Cómo Luce una Máquina de Estados?.....	242
Implementando los Estados <code>NORMAL</code> y <code>DYING</code>	244
Agregando más Estados al Jugador.....	251

Mejorando la clase AnimatedSprite: gotoAndPlay() y gotoAndStop()	256
Agregando la Animación de la Explosión al Morir el Jugador	264
Agregando Explosiones a los Enemigos	268
Ignorando las Colisiones de las Balas contra las Explosiones	274
Reorganizando el Código de CSprite y CGameObject para Reutilizar	275
Haciendo Parpadear un Sprite (Visible / Invisible)	281
14. Audio: Música y Efectos	285
El Audio en los Juegos	285
Formatos de Archivos de Audio	285
Inicializar el Sistema de Audio	285
Cargar y Ejecutar un Sonido	286
Disparo del Jugador	286
Agregando Todos los Sonidos al Juego	289
Agregando la Música	292
El Manager de Audio	294
15. Manejo de Textos y Fuentes	300
Dibujando un Texto	300
Usando una Fuente del Sistema (System Font)	304
Usando una Fuente Propia del Juego (Custom Font)	304
Creando el HUD: Agregando los Puntajes y las Vidas	305
Controlando el Puntaje de los Jugadores	310
Controlando las Vidas de los Jugadores	315
Condición de Ganar y Condición de Perder	318
16. Leyendo el Mouse	321
Eventos del Mouse	321
Detectando los Botones del Mouse	322
La Clase CMouse	323
El Puntero del Mouse	326
Ocultar o Hacer Visible el Mouse del Sistema	330
17. Las Pantallas del Juego	332
La clase CGame	332
La clase CGameState	333
La clase CLevelState	333
La Máquina de Estados del Juego	333
Agregando la Pantalla Principal (Menú)	345
Sentencia return Luego de Cambiar de Estado	349
Estado Inicial del Juego	350
Sprites de Texto	351
Garbage Collector	356
La Pausa del Juego	357
Imports y las Referencias Circulares en Python	360
18. Terminando el Nivel y Puliendo	363
Matar la Bala de los Enemigos al Pegarle al Jugador	363
Limitando la Cantidad de Disparos	364
La Formación de Enemigos Baja	366
Velocidad Incremental de los Enemigos	370
Ocultar o Mostrar el Mouse	374
Máquina de Estados del Nivel	376

Animación de la Llama (Usar un Sprite dentro de Otro).....	386
Modo de Uno o Dos Jugadores.....	391

Un aspirante siempre se pregunta, ¿por dónde empiezo?, y la respuesta no solo no es sencilla, sino que una mala guía nos puede costar mucho tiempo, frustraciones y desánimo. La idea para escribir este libro es el poder tener una referencia y un punto de partida confiable para quienes quieren empezar en el desarrollo de videojuegos, sin perderse en cuestiones como cuál lenguaje es el mejor, que motor conviene usar, etc.

La idea de este libro es que pueda usarse para aprender fácilmente a desarrollar videojuegos, en forma profesional.

Yo amo desarrollar videojuegos, y espero que con este libro tú también lo hagas.

Hay muchos temas que me gustaría haber incluido en el libro, pero el mundo del desarrollo de videojuegos es gigante y espero poder hacerlo con más tiempo. Tengo muchos materiales sobre varios temas que iré incluyendo en futuros libros.

¡Hacete tu Videojuego! te introducirá en el apasionante mundo del desarrollo de videojuegos de una forma práctica y, por sobre todo, aplicada. Al desarrollar videojuegos debemos “ver” y “usar” la matemática, cosa que suena intimidatoria, pero como hay una aplicación real detrás de los conceptos teóricos, es relativamente sencillo de aprender.

Dentro de los beneficios que tiene el aprender a desarrollar videojuegos, sea tanto en forma amateur, como hobby, o en forma profesional, se encuentran el uso de la matemática, la solución de problemas de programación, el desarrollo de la creatividad, el trabajo en equipo, la formación del carácter y el uso del idioma inglés, entre otras. Estas son habilidades que son sumamente necesarias en los tiempos actuales.

Espero ver las creaciones que hagas junto con tus amigos. Espero que este libro te sirva para desarrollar tus propios juegos. Este es el libro que yo desearía haber tenido cuando comencé a estudiar cómo hacer juegos.

Se agradece la difusión, y cualquier comentario o pregunta es bienvenida.

¡Disfruta el libro!
Fernando Sansberro
Batovi Games

Copyright © 2021 - Fernando Sansberro - Batovi Games.
Twitter: @sansberro
Batovi Games: www.batovi.com

Hacete tu Videojuego

Versión 1.0.

1a. Edición.



Copyright © 2021 - Fernando Sansberro - Batovi Games.

Algunos Derechos Reservados. “Hacete tu Videojuego” está publicado bajo licencia Creative Commons, Attribution-Noncommercial-Share Alike 3.0.

Usted es libre para:

Compartir — copiar y redistribuir el material en cualquier medio o formato.

Adaptar — remezclar, transformar y crear a partir del material.

Bajo los siguientes términos:

Atribución — Usted debe reconocer el crédito de una obra de manera adecuada, proporcionar un enlace a la licencia, e indicar si se han realizado cambios. Puede hacerlo en cualquier forma razonable, pero no de forma tal que sugiera que tiene el apoyo del licenciante o lo recibe por el uso que hace.

NoComercial — Usted no puede hacer uso del material con fines comerciales.

CompartirIgual — Si usted mezcla, transforma o crea nuevo material a partir de esta obra, usted podrá distribuir su contribución siempre que utilice la misma licencia que la obra original.

El resumen de la licencia se encuentra en este enlace:

<http://creativecommons.org/licenses/by-nc-sa/4.0/deed.es>

El texto completo del código legal de la licencia se puede encontrar en este enlace:

<http://creativecommons.org/licenses/by-nc-sa/4.0/legalcode>

¡Hacete tu Videojuego!

¡Júntate con tus amigos y arma un equipo, diseña tu propio videojuego, libera tu creatividad, desarróllalo, y busca maneras de distribuir tu juego y comercializarlo!

En esta serie de libros se encuentra todo lo que necesitas para desarrollar un videojuego, desde la idea inicial a su realización.

Este libro en particular te enseñará a programar videojuegos desde cero y una vez que aprendas los conceptos fundamentales, siguiendo el libro y aplicando sus ejemplos, serás capaz de crear tus propios juegos.

Este libro enseña a programar videojuegos utilizando el lenguaje *Python* y utilizando la biblioteca gráfica *Pygame*, aunque todos los conceptos que aprendas serán aplicables en cualquier lenguaje. Todo lo que necesitas es una computadora y los materiales incluidos en el proyecto *¡Hacete tu Videojuego!*

Nota para los Padres y Educadores

El proyecto *¡Hacete tu Videojuego!* fue pensado para que el estudiante pueda seguir el curso de forma autodidacta. Esta decisión ha sido tomada basada en el hecho de que no existe una formación establecida (educación formal) para el desarrollo de videojuegos, y entre las principales dificultades para su formalización se encuentra el hecho de que esta actividad es relativamente nueva, además de estar compuesta por diversas áreas como la programación, el arte, el diseño de juegos, el audio, etc.

Los videojuegos y los ejemplos de este libro, pueden ser una excelente opción para ser utilizados en clases curriculares. Por ejemplo, al hablar de temas como la velocidad, el movimiento, la gravedad de los planetas, etc. Es posible modificar los parámetros de los programas de ejemplo para ver aplicados los conceptos y fórmulas matemáticas detrás del funcionamiento de las cosas.

El libro toma un enfoque totalmente práctico, dado que un videojuego es algo totalmente visual y lo que programamos se ve reflejado en la pantalla. En lugar de enseñar los conceptos en forma abstracta y sin aplicación, el enfoque del libro es mostrar el código de los ejemplos y explicar sus conceptos, por lo cual cada concepto teórico se muestra aplicado en un ejemplo que el estudiante puede modificar a su gusto y realizar las pruebas que desee. Una vez entendido el concepto y apropiado el código, lo puedes aplicar en tu propio juego.

En el aula, los docentes pueden usar los ejemplos de videojuegos para enseñar conceptos en diferentes materias, como por ejemplo, matemática, física, astronomía, arte, informática, etc. Los ejemplos se pueden utilizar para crear videojuegos o simplemente demostraciones de conceptos de programación, conceptos matemáticos aplicados, ejemplos de física, etc. Se agradece el apoyo de los docentes en facilitar este material a los estudiantes, realizar actividades en clase con ellos y fomentar su estudio y realización de trabajos prácticos, concursos, etc.

¿Porqué se Escribió este Libro?

Este libro viene a llenar un vacío existente, en el caso cuando una persona quiere aprender a programar videojuegos y no es un programador experimentado, que va desde el momento en que se aprende a programar hasta llegar a crear un videojuego de calidad comercial.

Enseñando desarrollo de videojuegos, cuando quería brindar material de lectura, he visto que no existe una bibliografía completa, fiable, y centralizada en el tema, que comience desde cero y que cubra todos los puntos claves de hacer un juego. Menos aún en el idioma español.

La inmensa mayoría de los libros de programación de juegos están vinculados a una plataforma o a un lenguaje de programación determinado, haciendo muy difícil su lectura. Cuando a los alumnos les he dado material de lectura, les he tenido que aclarar cosas como “en este libro lean esto, pero no miren el código porque no está sólido”, o “este libro explica bien el concepto, pero el código depende mucho de la plataforma”, o “en este libro el código está bien realizado, pero no se explican los conceptos en profundidad”, etc.

Esto hace que no sea sencillo el estudio cuando alguien falta a una clase o se ha quedado atrás por problemas de abstracción o de asimilación de los conceptos. Desde ese lugar es que me ha surgido la necesidad de contar con materiales de la calidad acorde al curso impartido, y por otra parte, no siendo un tema nada menor, en idioma español. Este es el libro que me hubiera gustado tener cuando empecé a estudiar cómo programar videojuegos.

Sobre el Libro

Este libro es el producto de la experiencia de muchos años en el dictado de clases sobre desarrollo de videojuegos (materias: Programación de Videojuegos, Producción y Game Design) y los ejemplos se encuentran basados en los materiales desarrollados para dichos cursos.

El autor, *Fernando Sansberro*, ha escrito este libro durante 2013 a 2021, y junto con sus materiales han sido publicado con licencia *Creative Commons* como parte del proyecto de formación denominado *¡Hacete tu Videojuego!*, desarrollado por *Batovi Games*.

Se ha hecho público el libro bajo licencia *Creative Commons* (ver la página de licenciamiento al inicio del libro), lo que permite copiar y distribuir los materiales, con tal que se mantenga la atribución hacia el autor y que no sea utilizado con fines comerciales.

Descarga de los Materiales

El libro y los materiales son de libre distribución. El sitio para descargar el libro en formato

PDF, el código de los ejemplos y los materiales es: www.fsansberro.com.

Si encuentras algún error, tienes alguna sugerencia para realizar o quieres hacernos llegar tus comentarios, por favor dirígete a la sección de contacto en el sitio web y envíanos tu mensaje.

Los materiales (los ejemplos de juegos y assets) que acompañan este libro han sido liberados bajo licencia *Creative Commons (CC-BY-SA)*. Esto quiere decir que los materiales pueden adaptarse y/o ser usados para crear juegos comerciales.

¿Para Quién es este Libro?

Este libro está dirigido a cualquier persona que tenga ganas de desarrollar un videojuego y que cuente con algún tiempo libre para eso.

El libro y los materiales están basados en las experiencias que he tenido dando clases de desarrollo de videojuegos a gente con poco o ningún conocimiento previo de programación. Naturalmente que si se tienen conocimientos más profundos de programación, más rápido se llegará al nivel esperado, y por lo tanto mejores juegos se podrán hacer, aunque el ser mejor programador no siempre significa hacer un mejor juego. Cualquiera que aprenda programación puede llegar a realizar un excelente juego.

Este libro se encuentra orientado a adolescentes en etapa liceal y pre-universitaria, pero estos materiales han sido utilizados durante años en la carrera de desarrollo de videojuegos que imparto, tanto a jóvenes como a adultos. Cualquiera puede ser capaz de comenzar a desarrollar un videojuego desde un conocimiento de nivel cero.

¿Qué Nivel alcanza el Libro?

El libro cubre todos los aspectos fundamentales del desarrollo de videojuegos, y explica todos los conceptos sin asumir ningún conocimiento previo. Sin embargo, cada sección del libro explica estos conceptos en profundidad, siendo útil incluso para quien ya conozca del tema y desee formalizar los mismos.

Este quizás sea el desafío más grande que tiene el libro como objetivo, el servir tanto de guía a los principiantes como ser el libro de cabecera para los cursos de desarrollo de videojuegos que se impartan en el futuro.

Dicho esto, cualquier desarrollador, incluso con experiencia, puede beneficiarse de la lectura en profundidad de este libro. Habiendo trabajado en más de un centenar de juegos de diferentes géneros y plataformas como desarrollador, game designer y productor en *Batovi Games*, he visto innumerables aciertos y errores a nivel de equipo, proyectos y personas, y este libro plasma muchas de las experiencias adquiridas, motivo por el cual el libro le servirá tanto a los desarrolladores principiantes como a los desarrolladores más avanzados.

¿Por qué es tan Importante la Programación?

Este libro es básicamente sobre programación de videojuegos, aunque incluye algunos conceptos sobre diseño de videojuegos (*game design*) y producción, que son tres de los pilares fundamentales para realizar un videojuego (más el arte y el audio para nombrar a los otros pilares).

El libro se enfoca en la programación de videojuegos simplemente por el hecho de que es indispensable saber programar para hacer un juego. Un programador, por ejemplo, puede realizar un buen juego con un arte muy simple, pero generalmente es muy difícil que un artista pueda hacer un buen juego con una programación simple.

Alguien que tenga aspiraciones de ser artista para videojuegos, debe comprender el proceso de creación de un videojuego, y tener elementos de programación como los que se imparten en este libro, de forma tal que su arte pueda integrarse fácilmente en el juego, evitando problemas que son comunes, y entregando los materiales de arte en la forma en la que se esperan para poder ser integrados correctamente en el juego.

Requisitos para Comenzar

Este libro se ha escrito asumiendo que se tienen nociones básicas de programación (puede ser en cualquier lenguaje). Como es un libro que enseña a programar videojuegos de forma profesional, se ha evitado la enseñanza de conceptos básicos de programación, enfocándose en los conceptos que son necesarios para el desarrollo de videojuegos.

En este libro se explica desde cero el uso de la biblioteca gráfica *Pygame*, que es el módulo que debemos utilizar para hacer juegos 2D con gráficos y sonido en lenguaje *Python*.

El libro está organizado en capítulos de forma tal de que en cada capítulo se cubre un tema específico, mientras se van aplicando los conceptos adquiridos en un videojuego (un juego tipo *Space Invaders*) que se va realizando desde los primeros pasos, cuando comenzamos a dibujar y mover objetos por la pantalla, hasta que tenemos un control total del motor del juego.

En el *Apéndice A*, se hace una introducción al lenguaje de programación *Python*. Si eres nuevo a la programación, o eres nuevo en este lenguaje, es necesario comenzar por este apéndice. Luego de leer sobre el funcionamiento de *Python*, para practicar la programación, debes poder escribir y ejecutar código en un editor. Debes consultar a un profesor de informática, a un desarrollador o a algún amigo para que te explique cómo escribir y ejecutar un programa si es que no puedes hacerlo luego de leer el *Apéndice A*.

1. Introducción a la Programación de Videojuegos

Antes de ponernos a programar con *Pygame*, veremos una introducción a cómo se programa un videojuego, explicando las partes que lo componen y la arquitectura básica del mismo.

En este capítulo se muestran las principales partes que componen un videojuego y se realiza una introducción al tema, mostrando los conocimientos básicos que son necesarios para programar un juego.

La Pantalla

La pantalla (*screen* o *display*) es en donde el jugador ve el videojuego. Hay que pensar en la pantalla como una grilla o matriz de pequeños cuadraditos, los cuales se denominan *píxeles*. Cada uno de estos cuadraditos tiene un color y todos juntos forman la imagen que ve el jugador.

Nota: En español la palabra *pixel* lleva tilde, pero a lo largo del libro usaremos varios términos en inglés. El primer motivo de esto es que muchas veces es más fácil buscar documentación en *Internet* si los términos se escriben en inglés. El segundo motivo es que hoy en día es muy común hablar con estos términos en inglés, y los mismos ya son parte del vocabulario (jerga) de desarrollo de videojuegos.



Figura 1-1: La pantalla y las imágenes se componen de píxeles.

En la figura 1-1, vemos una parte ampliada de la imagen o de la pantalla, mostrando cómo la imagen se compone de píxeles de colores. Cada píxel tiene un color determinado.

Resolución de la Pantalla

La pantalla tiene una cantidad determinada de píxeles que la forman, y a la cantidad de píxeles que entran en horizontal y en vertical se le denomina *resolución* de la pantalla. La resolución de la pantalla viene a ser el tamaño de la pantalla en píxeles. Por ejemplo, si decimos que una pantalla tiene una resolución de 800 x 600 píxeles, lo que estamos diciendo es que la pantalla tiene 800 píxeles horizontales (columnas) por 600 píxeles verticales (filas).

No existe una única resolución de pantalla. Cada computadora o cada tarjeta de video puede soportar desde una sola a varias resoluciones de pantalla. Los juegos de PC, por ejemplo, pueden correr en diferentes resoluciones en diferentes máquinas. Las resoluciones de pantalla más comunes a lo largo de la historia son 320 x 240, 640 x 480, 800 x 600, 1024 x 768, 1366 x 768 y 1920 x 1080 píxeles, entre otras. Hoy en día con diferentes computadoras, laptops, tablets y celulares, hay prácticamente resoluciones de todos los tamaños y aspectos.

Nota: Siempre hay que tener en cuenta que cuanto más grande es la resolución de la pantalla, más píxeles hay que dibujar en cada cuadro del juego, y que cuanto más chica es la resolución, el juego funcionará más rápido porque hay menos área de pantalla para dibujar en cada cuadro.

La razón de querer usar una resolución menor en un juego es para dibujar menos. El programa tiene que dibujar en la pantalla todos los elementos del juego varias veces por segundo. Si la superficie a dibujar es más chica, eso toma menos tiempo y el juego puede alcanzar muchos cuadros (*frames*) por segundo. Si la resolución es mayor, la operación de dibujar tomará más tiempo, pero los gráficos se verán mejor.

FPS (Frame Rate, Cuadros por Segundo)

El manejo del tiempo es muy importante al programar videojuegos. Los juegos deben correr lo más rápido posible en la computadora. Los juegos hacen un uso intensivo del procesador y del display de la máquina (la tarjeta de video). Esto es porque un videojuego debe hacer muchos cálculos y dibujar muchas cosas, varias veces por segundo.

Un videojuego es como una película en cuanto a que varias veces por segundo (casi siempre 30 o 60 veces por segundo) tiene que mostrar un cuadro (denominado *frame*). Para eso, debe calcular dónde van a estar los objetos del juego y qué es lo que hacen, y luego dibujarlos en la pantalla al armar la escena y mostrarla al jugador. Es por esta razón que para jugar videojuegos necesitamos una máquina rápida, porque la computadora debe trabajar mucho para mostrar la escena que se genera 30 o 60 veces por segundo. A cada imagen completa que se muestra en pantalla varias veces por segundo se le llama *cuadro de animación*, o *frame*.

A la velocidad a la cual se realiza la actualización del dibujo se le denomina *frame rate*. O sea, el *frame rate* es cuantas veces por segundo se muestra un cuadro de animación (se abrevia *FPS* por sus siglas en inglés: *frames per second*).

Un *frame rate* mayor implica una animación más fluida, pero necesita más poder de procesamiento. Un *frame rate* bajo es una animación que se ve lenta. Si el *frame rate* es demasiado bajo, simplemente se ve mal (se pueden ver “saltos” en la animación). Por ejemplo, si movemos algo por la pantalla a un bajo *frame rate*, se verán los saltos de posiciones donde se dibuja y no se verá como un movimiento continuo. Las animaciones para que parezcan reales necesitan actualizarse muchas veces por segundo.



Figura 1-2: Un juego debe correr preferentemente a 60 FPS para sentirse y verse bien.

Un frame rate normal son 30 *FPS*. La televisión, por ejemplo, funciona a 24 *FPS*. Cuanto más *FPS* corra un videojuego, más capacidad de procesamiento de la computadora necesitará.

El objetivo siempre será lograr el *frame rate* más alto posible para que el juego se vea bien y corra bien en velocidad. También hay que lograr que el *frame rate* sea consistente. Si se logra que el *frame rate* sea constante, el juego se verá bien, y debería serlo para no ver saltos en las animaciones.

Hay un máximo *frame rate*, el cual está dado por la velocidad de refresco de la pantalla, o también por la capacidad del ser humano para detectarlo. Lo óptimo para un videojuego son 60 *FPS*, pero esto a veces es difícil de alcanzar (por el lenguaje, la máquina o la plataforma). Si este es el caso, lo mejor es apuntar a correr el juego a 30 *FPS*, lo cual es alcanzable y se verá razonablemente bien. El frame rate mínimo aceptable para un juego (en caso extremo) debería ser 24 *FPS*. En algunas plataformas como en *web* o en algunos dispositivos *handheld*, se suele apuntar a este *frame rate*.

El Game Loop

Uno de los grandes secretos de la programación de videojuegos es el concepto de *game loop*, el cual es un *bucle* o *ciclo* (*loop* en inglés) que se ejecuta permanentemente mientras el juego está en funcionamiento. Es lo primero que deberemos programar (o ya tenerlo hecho de antemano) al comenzar a desarrollar un juego.

En otras formas de programación que generalmente se enseñan en las carreras tradicionales de informática, como por ejemplo en la *programación orientada a eventos*, un programa no hace nada hasta que el usuario toca un botón (por ejemplo, en un formulario, en el cual se llenan los datos y luego se pulsa un botón “Enviar”). En este caso, el programa reacciona ante el evento de apretar el botón “Enviar”. Incluso, hasta es común que el botón mismo tenga código o ejecute un programa cuando se lo presiona. Pero mientras no se pulse el botón, el programa no hace prácticamente nada.

En los videojuegos, es todo lo contrario. Siempre hay muchos objetos moviéndose por la pantalla, como por ejemplo, animaciones, partículas, etc. y por lo tanto el programa siempre tiene que estar haciendo algo. Para lograr esto hay un loop repitiéndose indefinidamente hasta que se sale del programa. Esto es así tanto en los juegos 2D como en los juegos 3D. Generalmente el *game loop* es una estructura `while`, de la cual se sale cuando se cierra la ventana (la *aplicación*), o se decide salir del juego, o el juego termina de alguna manera.



Figura 1-3: El *game loop* se encarga de mover los objetos del juego y dibujarlos.

El *game loop* estará encargado de leer la entrada del usuario (mediante teclado, mouse, joystick, pantalla táctil, etc.) y determinar qué quiere hacer el jugador, actualizar y mover a

todos los objetos del juego, chequear las colisiones entre los objetos y dibujar en pantalla los objetos visibles. Todo eso se hace para cada cuadro del juego.

Como se ve, toda la lógica del juego se encuentra dentro del *game loop*. El *frame rate* del juego depende de la velocidad de proceso del *game loop*, por lo cual, en realidad, el *frame rate* es la frecuencia del *game loop*. Si hay demasiado que procesar en el *game loop*, el juego correrá más lento (por lo tanto, tendrá un bajo *frame rate*). Lo ideal es correr el *game loop* 60 veces por segundo (60 *FPS*).

Un juego simple puede correr normalmente a 60 o más *FPS*, pero si hay muchos cálculos en el juego, o hay que dibujar mucho, es probable que el juego corra más lento.

La Estructura de un Videojuego

Casi todos los juegos tienen la misma estructura. Comienzan con una inicialización del sistema (por ejemplo, establecer la resolución o crear la ventana del juego) y luego viene la carga de los archivos necesarios (a éstos se le llaman los *assets* del juego: los gráficos, sonidos, niveles, o lo que sea necesario). Luego de inicializar el sistema y cargar los *assets*, se ingresa en el loop principal.

Entonces, todos los juegos que hagamos van a tener básicamente la siguiente estructura:

- Configuración del sistema y de la pantalla.
- Inicialización de los objetos del juego (función `init()`).
- Carga de los *assets* necesarios (gráficos, sonidos, etc.).
- El game loop:
 - Obtener la entrada del jugador.
 - Limpiar la imagen actual (limpiar la pantalla).
 - Actualizar los objetos del juego (función `update()`).
 - Dibujar los objetos del juego (función `render()`).
 - Actualizar la pantalla (volcar el frame dibujado a la pantalla, para que se vea).
- Al final, al salir del game loop, liberar la memoria utilizada por todos los objetos y salir del programa (función `destroy()`).



Figura 1-4: En cada *frame* se mueven y dibujan todos los objetos de un juego.

Al principio del programa se configura lo necesario del sistema y de la pantalla, se cargan todos los *assets* necesarios (imágenes, sonidos, niveles, etc.) y se crean los datos que se necesitan para que el juego pueda comenzar. Luego de que todo esté inicializado, se entra al *game loop*, del cual no se saldrá hasta que el usuario decida irse del juego.

Dentro del *game loop*, lo que se hace es procesar los eventos para manejar la entrada del jugador (leer el teclado, el mouse, etc.), se actualizan los objetos en el juego (se ejecuta una función `update()` que corre la lógica de los objetos), se dibujan los objetos en la pantalla (se ejecuta una función `render()` que los dibuja), y al final se dibuja la imagen creada en la pantalla para que ésta sea visible al jugador.

A continuación se explica brevemente cada una de las partes. En capítulos posteriores iremos profundizando cada parte, a medida que vayamos construyendo nuestro juego.

Inicialización del Sistema y Carga de Assets

Antes de entrar al *game loop*, el programa debe configurar correctamente la ventana (llamada también *display* según el contexto), inicializar el sistema de video y de audio, por ejemplo, entre otras muchas cosas.

También debemos cargar los *assets* necesarios (generalmente las imágenes y los sonidos) y crear los objetos del juego e inicializarlos correctamente.

Es importante crear e inicializar todos los objetos y componentes que se van a usar en el juego antes de entrar en el *game loop* y no durante el mismo. Esto es para que el *game loop* no se tranque en ningún momento durante su ejecución.



Figura 1-5: Los *assets* del juego se deben cargar antes de entrar al *game loop*.

Si cargamos todos los *assets* durante la inicialización del juego, luego el *game loop* ya los tendrá listo en memoria y no demorará en cargarlos (cosa que haría que el juego se trancara un instante mientras se cargan los gráficos). Como no queremos que pase eso, siempre los gráficos se cargan en la parte de inicialización, antes de que el juego en sí comience (o sea, antes de entrar al *game loop*).

Una vez inicializados todos los componentes necesarios para el juego, se ingresa al *game loop*. A continuación se explican las partes que componen el mismo.

Obtener la Entrada del Jugador

El principal aspecto de un videojuego es la interactividad (si no tuviera interactividad, sería simplemente una animación). Para conseguir la interactividad, debemos leer los dispositivos de entrada que utilice el juego, como por ejemplo, el teclado, el mouse, la pantalla táctil (*touch*), el acelerómetro, un joystick, etc. y responder a las acciones del usuario. Por ejemplo, el personaje del jugador puede moverse y saltar si apretamos las teclas correspondientes, con el mouse podemos controlar los botones del juego, etc.



Figura 1-6: Un juego puede tener varios controles.

Pygame tiene funciones para leer los dispositivos de entrada, el teclado por ejemplo, y saber si el jugador está apretando una tecla determinada en ese momento. Entonces leemos el estado de esa tecla y el juego responde a ella (por ejemplo, el jugador dispara si se apreta la tecla de disparar).

Actualizar los Objetos del Juego (la función `update()`)

En cada *frame*, el programa debe calcular dónde va a estar cada objeto del juego (a los objetos en el juego también se le llaman *sprites*, *game objects*, *entidades* o *game entities*). Para cada objeto del juego, se calcula dónde va a estar en el siguiente frame de acuerdo a:

- La entrada del usuario (qué es lo que el jugador hace).
- La velocidad que tiene el objeto y la dirección que lleva (el movimiento o la física).
- Si choca con otro objeto o no (si choca hay que resolver las colisiones, explotar, etc.).
- Las acciones que realiza el objeto (su comportamiento o *inteligencia artificial*).

De esta manera, en cada *frame* se determina la nueva posición para cada uno de los objetos del juego. Si el objeto se está moviendo, se calcula su próxima posición, o se determina si debe morir o no (por ejemplo, si ha chocado con una bala), o se determina si una bala debe destruirse porque se fue de la pantalla, o lo que sea necesario hacer, por supuesto, esto dependiendo del juego del que se trate (la lógica o reglas del juego).



Figura 1-7: Todo lo que pasa en un cuadro se calcula en la función `update()`.

Cada objeto tiene su lógica propia según lo que se deba hacer en el mismo. Por ejemplo, un fantasma del juego *Pacman* nos perseguirá, una nave del juego *Space Invaders* se moverá y cada tanto disparará una bala, etc. Entonces, decimos que la lógica del juego se encuentra en la función `update()`. Más adelante escribiremos esta función.

Todos los cambios que se realizan en los objetos, se hacen en sus propias variables (denominadas *atributos* del objeto), como por ejemplo, el cambio de sus coordenadas cuando se mueve, su velocidad, su aceleración, etc. El jugador ve los cambios porque hay una actualización visual en la pantalla (por ejemplo, un enemigo se mueve diez píxeles hacia la derecha y luego se dibuja en esa posición, con lo cual parece dar la idea de que se ha movido).

Dibujar los Objetos del Juego (la función `render()`) y Actualizar la Pantalla

Luego que en la función `update()` calculamos qué es lo que hace cada objeto del juego en el *frame* actual, sabemos dónde va a estar ubicado cada objeto y qué imagen debemos mostrar. Con esta información, ya se puede dibujar la pantalla.

A la operación (que se realiza en cada *frame*) de dibujar los objetos del juego, se le llama *actualizar* la pantalla y consiste en crear una imagen con el *frame* que se va a mostrar en la pantalla. Esta función `render()` lo que hace es dibujar cada uno de los objetos del juego que están visibles en la pantalla. Más adelante escribiremos esta función.

Antes de dibujar los objetos donde están en el *frame* actual, es necesario borrar la imagen del *frame* anterior. Es posible borrar toda la imagen del *frame* anterior y recrear el nuevo *frame* completamente, aunque es un proceso lento, o podemos optimizar y dibujar solamente lo que ha cambiado desde el *frame* anterior, lo cual es más rápido. Más adelante veremos cómo

hacer todo esto.

Liberación de los Recursos Utilizados

Al salir del *game loop*, la última tarea que realiza el programa, antes de terminar, es liberar todos los recursos utilizados por el juego. Es necesario hacer esto para evitar problemas, como por ejemplo, que el sistema operativo se quede sin memoria o que aparezcan otros problemas relacionados.

Como regla general, cada vez que no usemos más un objeto, lo deberemos destruir, liberando la memoria utilizada. Más adelante veremos cómo hacer esto. Por ahora solamente diremos que todos los objetos del juego tendrán una función `destroy()`, que escribiremos más adelante, que se encargará de eliminar todo lo que ha creado. De la misma manera, al salir del programa, todos los recursos pedidos al sistema deben liberarse.

2. Nuestro Primer Programa en *Pygame*

En este capítulo comenzaremos hablando de los lenguajes de programación disponibles para hacer videojuegos, y nos centraremos en el lenguaje *Python* y en la librería para juegos *Pygame*, que son los que utilizaremos en este libro.

Durante este capítulo construiremos el esqueleto de un juego, con un bucle principal (el *game loop*), y al final del capítulo ya tendremos todo pronto como para comenzar a agregar objetos que se muevan por la pantalla.

A medida que se avanza en el libro, lo que vayamos programando en cada capítulo nos servirá de base para todos los juegos que hagamos en el futuro.

¿En qué lenguajes se programa un juego?

Existen varios lenguajes de programación, como por ejemplo, *C++*, *C#*, *Java*, *Actionscript*, *Python*, *LUA*, *JavaScript*, etc., que son los lenguajes más comunes para desarrollar videojuegos.

En lenguajes de bajo nivel, como por ejemplo el lenguaje *C++*, lleva muchas líneas de código para lograr hacer que algo aparezca en la pantalla. En general, cuando una tarea lleva muchas líneas de código, los programadores hacen *módulos* (*bibliotecas* o *librerías*) que podemos utilizar a través de llamadas a funciones, para hacer las cosas más fáciles sin saber los detalles que hay internamente.

En lenguajes de más alto nivel, como *LUA*, es más sencillo programar, dado que con menos sentencias realizamos más cosas. Cada lenguaje, por supuesto, tiene sus pros y sus contras.

Librerías Gráficas y APIs

Hace muchos años, cuando se quería hacer un videojuego, había que trabajar directamente con el hardware. Por ejemplo, si queríamos mostrar una imagen, debíamos acceder directamente a la tarjeta de video, y esto era muy complicado, porque cada tarjeta era diferente, había varias tarjetas en el mercado, y había que programar a cada una por separado.

Al principio se programaba en lenguaje *Assembler*, un lenguaje de códigos muy cercanos al código que ejecuta el procesador. Había que programar en este lenguaje porque era lo único suficientemente rápido para hacer juegos. La dificultad que traía esto es que había que escribir muchas líneas de código para hacer algo puntual, y que cada procesador y computadora eran diferentes, por lo cual había que reescribir el código para cada máquina.

A medida que fue pasando el tiempo, y que las computadoras fueron siendo cada vez más rápidas, fue posible utilizar lenguajes de más alto nivel. Al principio, lenguajes como *C++* y *Pascal*, y hoy día tenemos lenguajes como *C#*, *Java*, *Actionscript*, *Python*, *JavaScript* o *LUA*, por nombrar a los más utilizados para juegos.

Con cada lenguaje, utilizaremos lo que se conoce como *módulos* o *bibliotecas* (*library* en inglés) o a veces se les llama *librerías*, que son un conjunto de módulos y/o funciones para realizar determinadas tareas. También utilizaremos lo que se conoce como *APIs* (*Application Programming Interface*), que son librerías que le permiten al programador trabajar con el hardware de una forma más abstracta, a más alto nivel. De esta forma, por ejemplo, podemos mostrar una imagen en la pantalla, usando una sola línea de programación, invocando a una función de la librería gráfica que en realidad hace un montón de cosas complejas a nivel del procesador de la máquina, y que no necesitamos saber en detalle qué es lo que hace internamente.

Entonces, para programar un juego, necesitamos saber programar una de estas *APIs*. Sin esto, no podríamos acceder a la parte gráfica ni al sistema de audio de la computadora, por ejemplo.

Entre las *APIs* más comunes para programar videojuegos se encuentran las que nos proporcionan los fabricantes de los sistemas operativos, siendo las más comunes *DirectX* (creada por *Microsoft* para el sistema operativo *Windows*), *OpenGL* (es una biblioteca de código abierto, que funciona en varios sistemas operativos) y *SDL* (también de código abierto y para varias plataformas).

Nota: Cuando un lenguaje, API o videojuego funciona en varios sistemas operativos, se dice que es *multiplataforma*. Se denomina *plataforma* a un entorno, máquina o lenguaje, o incluso a la combinación de ellas. Por ejemplo, las plataformas comunes para correr videojuegos son *PC* (*Windows*), *Mac*, *Linux*, *Web*, *iOS*, *Android*, *Consolas*, etc.

Pygame y SDL

Cada biblioteca gráfica tendrá su versión para diferentes lenguajes de programación. La biblioteca que vamos a utilizar es *SDL* (*Simple Directmedia Library*), que está escrita en lenguaje *C*, es muy poderosa, es multiplataforma, es gratuita y es de código abierto.

Como estaremos programando en lenguaje *Python*, no podremos acceder a *SDL* directamente, dado que *SDL* está programada en lenguaje *C*. Ahí es donde entra *Pygame*, que es lo que se conoce como un *wrapper* de *SDL*. Esto quiere decir que es un conjunto de funciones en *Python* que se comunica con el lenguaje *C* de *SDL*.

Para poder trabajar con *Python* y *Pygame*, deberemos instalarlos en la computadora. Más adelante veremos cómo hacer esto.

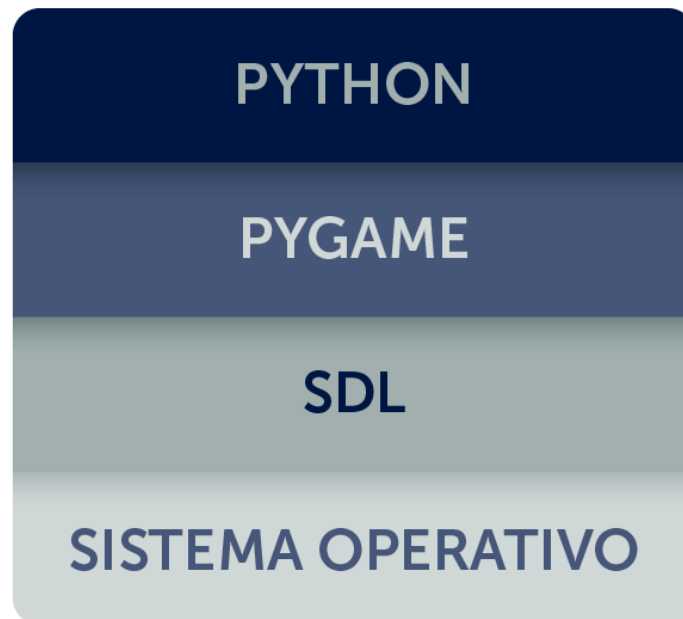


Figura 2-1: Esquema de las capas de software.

Por ejemplo, supongamos que queremos crear una ventana para el juego. Si usáramos una biblioteca de bajo nivel (o sea, más cercana al hardware de la computadora), la forma de hacerlo sería distinta para cada plataforma. Deberíamos usar diferente programación para cada una.

Al utilizar un lenguaje de alto nivel (o sea, más abstraída del hardware de la computadora), junto con una biblioteca de alto nivel, las tareas se realizan de la misma manera para todas las diferentes plataformas. Entonces, para crear una ventana, simplemente llamaríamos a una función que hará la tarea, e internamente se encargará de cosas que no nos interesa saber cómo funcionan en detalle y que tienen relación con el hardware.

Por ejemplo, para crear la ventana llamaremos a una función del estilo:

```
screen = pygame.display.set_mode(...)
```

Internamente se realizan tareas relacionadas con el hardware de la computadora para que la ventana sea creada y se ponga visible. De esta forma simple, escribimos en lenguaje *Python* y llamamos a una función de *Pygame* para crear la ventana. Esta función se comunica con *SDL* y utiliza la función de *SDL* correspondiente para crear la ventana, y a su vez *SDL* se comunica con el sistema operativo para indicarle que cree la ventana. Esta cadena o estructura de capas, se puede ver en la figura 2-1.

Esta es una de las grandes ventajas de usar una *API* como *Pygame*. O sea, escribiremos el programa una sola vez y luego lo llevaremos fácilmente a otras plataformas. Siempre tendremos que tener en cuenta factores inevitables como diferentes resoluciones de pantalla, o diferentes controles, pero es mejor adaptar el programa, a que tener que escribirlo todo nuevamente.

Nota: A la acción de programar un juego y luego modificarlo para que corra en otra plataforma se le llama *portar* el juego.

Cómo Instalar *Python* y *Pygame*

Antes de comenzar a trabajar con *Python* y *Pygame*, debemos instalarlos en la computadora.

Para instalar *Python*, hay que ir al sitio <https://www.python.org/>, descargar la última versión de *Python* para el sistema operativo que usemos, e instalarlo. A la hora de escribir este libro se usó *Python 3.10.0*.

Por ejemplo, para instalar en el sistema operativo *Windows*, debemos ejecutar el instalador que descargamos, y seleccionar la opción "Add Python to environment variables". Con esto, podremos ejecutar *Python* desde la terminal de comandos.

Ante cualquier duda, podemos consultar cómo instalar *Python* desde la web del mismo, o pedirle ayuda a un profesor de informática, o a algún programador.

Para instalar *Pygame* hay que ir al sitio <http://www.pygame.org> y seguir las instrucciones para su instalación en el sistema operativo que usemos.

Para instalar *Pygame* en *Windows*, la forma más sencilla es escribir el siguiente comando en la terminal de comandos:

```
py -m pip install -U pygame --user
```

Este libro y los programas de ejemplo fueron creados utilizando *Python 3.10.0* y *Pygame 2.0.2*, que eran las últimas versiones disponibles al momento de escribir este libro. De todas formas, se ha tratado, dentro de lo posible, no usar programación que dependa de versiones específicas. Si usáramos una versión diferente de *Python* o *Pygame*, algunas cosas podrían no funcionar y habría que corregirlas, pero el cambio a realizar debería ser mínimo.

Nota: Para ver la versión de *Python* instalada en el sistema, abrir la terminal de comandos y ejecutar el siguiente comando en la carpeta donde se instaló: `python -V`. Cuando se entra a la consola de *Python* (escribiendo `python` en la terminal de comandos, o abriendo el programa *IDLE* en *Windows*) se muestra un mensaje con la versión de *Python*. Para ver la versión de *Pygame* instalada, una vez dentro de la consola de *Python*, escribimos las sentencias: `import pygame, y luego print (pygame.ver)`.

Principales Características de *Pygame*

Pygame es una biblioteca que contiene varios módulos, cada uno con muchas funciones para trabajar con los distintos aspectos que tiene un juego, entre los cuales se encuentran:

- Manejo del *Display* (la pantalla).
- Manejo de *Superficies* (Imágenes y objetos *Rects*).
- Manejo de la Entrada (Teclado, Mouse, Joystick, etc.).
- *Sprites* (manejo de grupos de sprites, colisiones, etc.).
- Audio.
- Módulos con utilidades (manejo del tiempo, transformaciones, etc.).

Iremos viendo por partes, cada una, a medida que vayamos introduciendo nuevas funcionalidades en nuestro videojuego. También se irán explicando los términos utilizados en el desarrollo de videojuegos, a medida que aparezcan.

En este punto ya sabemos en dónde estamos parados en lo que respecta a *Python* y *Pygame*. ¡Ya podemos comenzar a aprender a programar nuestro primer videojuego!

Los Programas de Ejemplo y el Editor

La mejor forma de aprender a programar videojuegos es, justamente, programando videojuegos y probando cosas nuevas en él. De la misma manera, la mejor forma de aprender a utilizar un lenguaje como *Python* y una biblioteca como *Pygame*, es escribir nuestros propios programas con ellos y practicar mucho.

El libro contiene los ejemplos ya armados, para poder ejecutarlos, modificarlos y de esta forma aprender con la práctica. Debes tener la habilidad de escribir el texto del programa utilizando algún editor de texto y poder ejecutarlo y ver el resultado. Un programa se puede escribir utilizando un editor de texto sin formato y se puede usar la terminal para ejecutar el programa. Si no puedes escribir y ejecutar un programa *Python*, debes pedirle asistencia a un docente o a un experto. Una vez que puedas escribir un programa y ejecutarlo, el resto es sencillo. En *Windows*, se puede usar el editor *IDLE* que viene incluido con la instalación de *Python*.

Nota: Los ejemplos y materiales que acompañan el libro pueden ser descargados del sitio web: www.fsansberro.com.

Los ejemplos se encuentran clasificados por capítulos, de forma tal que sea sencillo ubicarlos. Cuando el libro utilice uno de estos ejemplos, se indicará dónde está ubicado para su fácil localización.

Para ejecutar los ejemplos y en general para programar en *Python*, no es necesario usar ningún editor en especial, aunque ayuda mucho el uso de un entorno de desarrollo integrado (lo que se conoce como *IDE*, por las iniciales de *Integrated Development Environment*). Un excelente editor para *Windows*, es *PyCharm* (<https://www.jetbrains.com/pycharm>).

Nuestro Primer Programa con *Pygame*

A continuación escribiremos un programa que será el esqueleto básico de todo videojuego

que hagamos con *Pygame*. Escribiremos el *game loop* o bucle principal del juego.

Comencemos por ejecutar el primer ejemplo, el cual se encuentra en la carpeta: `capitulo_02\001_game_loop`. Para ver el ejemplo en funcionamiento, debemos ejecutar el archivo `main.py`.

Al ejecutar este programa solamente veremos una ventana azul. Si pulsamos la tecla `[Esc]` el programa se termina y la ventana se cierra. También podemos tocar en la cruz de la ventana para salir.

Para hacer cualquier videojuego, partiremos desde este esqueleto. A continuación se explicará cada una de las partes que componen nuestro primer programa, y que será la base para el resto. Pero primero, veamos el listado del programa completo:

```
# -*- coding: utf-8 -*-

#-----
# Ejemplo de game loop.
#
# Autor: Fernando Sansberro - Batovi Games.
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar e inicializar Pygame.
import pygame
pygame.init()

# Crear la ventana y poner el tamaño.
screen = pygame.display.set_mode((640, 360))
# Poner el título de la ventana.
pygame.display.set_caption("Mi Juego")

# Crear la superficie del fondo o background.
imgBackground = pygame.Surface(screen.get_size())
imgBackground = imgBackground.convert()
imgBackground.fill((0, 0, 255))

# Inicializar las variables de control del game loop.
clock = pygame.time.Clock()
salir = False

# Loop principal (game loop) del juego.
while not salir:

    # Timer que controla el frame rate.
```

```

clock.tick(60)

# Procesar los eventos que llegan a la aplicación.
for event in pygame.event.get():

    # Si se cierra la ventana se sale del programa.
    if event.type == pygame.QUIT:
        salir = True

    # Si se pulsa la tecla [Esc] se sale del programa.
    if event.type == pygame.KEYUP:
        if event.key == pygame.K_ESCAPE:
            salir = True

# Actualizar la pantalla.
screen.blit(imgBackground, (0, 0))
pygame.display.flip()

# Cerrar Pygame y liberar los recursos que pidió el programa.
pygame.quit()

```

Si aún no has usado algún *IDE*, para escribir y ejecutar el programa, puedes crear el texto con un editor sin formato y luego, desde la consola de comandos, ejecutar `python main.py`, estando en el mismo directorio donde se encuentra el archivo `main.py`.

Si te encuentras en *Windows*, abre el programa con *IDLE* y ejecuta el programa (pulsando [F5] o mediante el menú *Run | Run Module*).

Si estás usando algún *IDE* como *PyCharm*, puedes pulsar con el botón derecho del mouse sobre el archivo `main.py`, abrirlo con *PyCharm* y ejecutarlo con el menú *Run*. En caso de no tener configurado el intérprete de Python, aparecerá una ventana donde podemos seleccionar la versión instalada.

Nota: Es muy importante escribir el programa nosotros mismos. Los ejemplos del libro están para poder consultar en caso que algo no nos funcione, pero nunca va a haber una mejor experiencia y una mejor forma de aprender que escribiendo nosotros mismos el código.

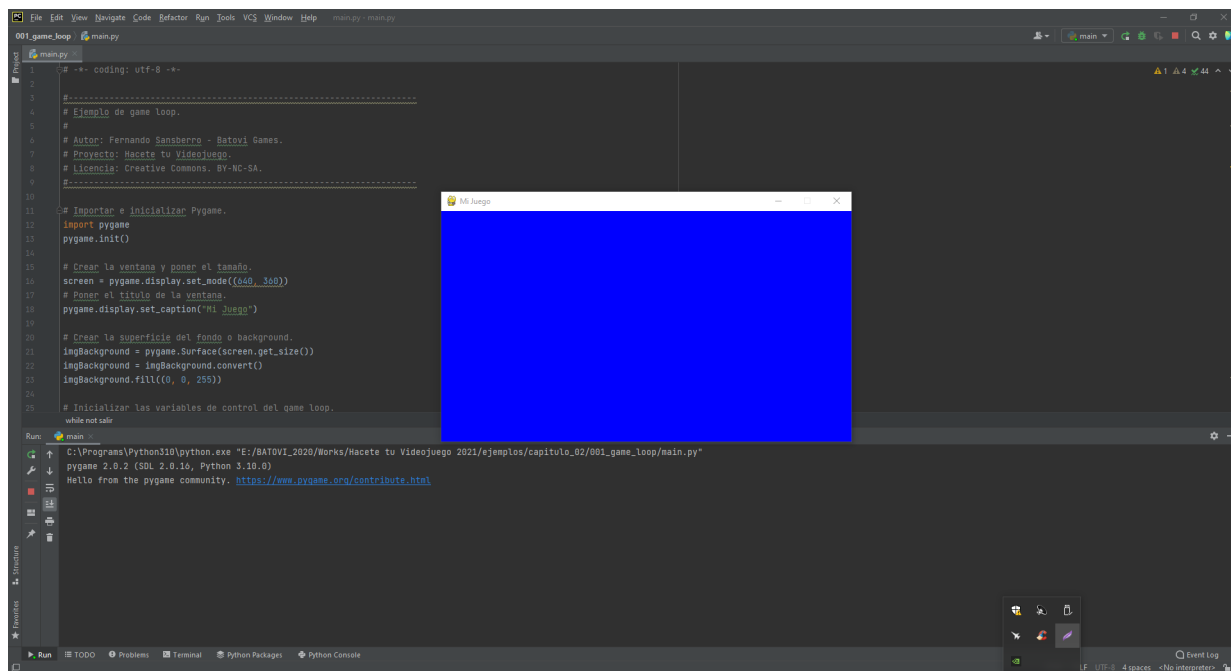


Figura 2-2: Ejecutando nuestro primer programa en *Pygame*.

Nuestro primer programa parece un programa grande, pero es chico comparado con uno que haga lo mismo en otros lenguajes, como por ejemplo en *C++*. *Python* es un lenguaje que permite hacer las cosas con bastante menos código.

Es muy importante escribir este programa y compilarlo para ver que funcione sin errores, y luego entender bien qué es lo que hace cada sentencia, lo cual se explicará a continuación.

El programa tiene básicamente dos partes. La primera parte carga e inicializa los elementos necesarios de *Pygame* para que el programa funcione, y la segunda parte consiste en el *game loop* o el bucle del juego propiamente dicho (la estructura *while*), que es en donde luego ocurrirá toda la acción del juego. A continuación se explica el programa paso a paso.

Inicializar *Pygame*

Antes que nada, lo primero que hay que hacer es importar el módulo `pygame`, para tener acceso a las funciones de *Pygame*. Luego de eso, debemos llamar a la función `pygame.init()` para inicializar la biblioteca *Pygame*.

```
# Importar e inicializar Pygame.  
import pygame  
pygame.init()
```

No podremos utilizar ninguna función de *Pygame* hasta que no hayamos invocado a `pygame.init()`, por lo cual esto es lo primero que debemos hacer en el programa.

Nota: Cuando necesites saber más acerca del funcionamiento de *Pygame*, o lo que hace alguna función en particular, consulta la documentación online en el siguiente sitio: <http://www.pygame.org/docs/>. Consultar la documentación es una excelente forma de profundizar los conocimientos sobre un tema en particular y aprender.

Establecer el Modo de Video

Lo que sigue luego de inicializar *Pygame* es inicializar el modo de video. Los videojuegos generalmente utilizan un modo de video dedicado. Esto consiste en decir si el juego correrá en una ventana o en la pantalla completa. Si es en una ventana, necesitamos indicar su tamaño, y si se trata de pantalla completa, necesitamos indicar su resolución. En ambos casos debemos pasarle dos números a la función: el ancho y el alto en píxeles.

En las computadoras *PC*, por ejemplo, podremos correr el juego en pantalla completa y a determinada resolución de pantalla (esto se conoce como el *modo de video*, por ejemplo: 640 x 480, 800 x 600, 1024 x 768 píxeles, etc.). Cuando corremos el programa en una ventana, decimos que lo corremos en “modo ventana”. Cuando lo corremos en la pantalla completa, decimos que lo corremos en “modo pantalla completa” (*fullscreen*).

Lo que haremos con nuestro primer programa, será correrlo en modo ventana, con un tamaño de 640 x 360 píxeles. Hemos elegido esta resolución baja porque los juegos que haremos en esta serie de libros son con estilo *pixel art*.

```
# Crear la ventana y poner el tamaño.
screen = pygame.display.set_mode((640, 360))
# Poner el título de la ventana.
pygame.display.set_caption("Mi Juego")
```

Con la primera sentencia decimos que el modo de video va a ser en ventana, a una resolución de 640 x 360 píxeles. La función `pygame.display.set_mode()` recibe como parámetro una tupla con el tamaño de la ventana. En *Python*, una tupla son valores separados con coma, delimitados entre paréntesis. Más adelante veremos cómo hacer que el juego corra en pantalla completa.

La función `pygame.display.set_caption()` sirve para ponerle un título a la ventana (el título de la aplicación o juego). Cuando ejecutamos el programa, vemos que la ventana tiene como título el texto que le pasamos como argumento a esta función.

Nota: Recuerda que cuando pasamos valores a una función, a estos valores se le llaman *argumentos* y cuando la función los recibe se les llama *parámetros*. Hoy día se estila mucho usar solo la palabra parámetro, que es la que usaremos en el libro, tanto para referirnos al valor que se pasa como al parámetro que lo recibe.

Crear la Superficie del Fondo

Una superficie es un rectángulo en memoria en el cual se puede dibujar (se puede decir que es sinónimo de una “imagen en la memoria”). Necesitamos tener una imagen que va a ser la que utilizaremos para el fondo de la ventana. Entonces, creamos una superficie del mismo tamaño de la ventana y la llenamos con el color azul (color *RGB*: 0, 0, 255).

```
# Crear la superficie del fondo o background.
imgBackground = pygame.Surface(screen.get_size())
imgBackground = imgBackground.convert()
imgBackground.fill((0, 0, 255))
```

La sentencia `pygame.Surface()` crea una nueva superficie y tiene como parámetro el tamaño de la superficie que vamos a crear (una tupla con el ancho y el alto en píxeles). En este caso le pasamos `screen.get_size()` para que sea del mismo tamaño que tiene la ventana (`screen.get_size()` nos retorna una tupla con el ancho y alto de la ventana). La referencia a la superficie creada se almacena en la variable `imgBackground`.

La sentencia `imgBackground.convert()` convierte el formato de la imagen al formato de *Pygame* (al formato de la pantalla). Debemos hacer esto para todas las imágenes que creamos o que cargamos, porque por ejemplo si una imagen tiene un formato con compresión (como es el caso de imágenes *PNG* o *JPG*), al mostrar la imagen, *Pygame* tiene que descomprimirla y eso demora. Por esta razón, siempre utilizaremos la función `convert()` luego de crear o cargar cualquier imagen.

Por último, la sentencia `imgBackground.fill((0, 0, 255))` lo que hace es llenar la imagen con el color (0, 0, 255), lo que corresponde a rojo = 0, verde = 0 y azul = 255, o sea, el color azul puro. Esta forma de indicar un color se denomina formato de *color RGB*, y está dado por tres valores correspondientes a los componentes rojo, verde y azul (en inglés: *red, green, blue*). Más adelante veremos cómo se codifican los colores con el formato *RGB*.

Nota: La explicación en este libro siempre tenderá a ser lo más simple posible, de forma que sea sencillo y práctico de seguir. Recuerda que para profundizar en cualquier función de *Pygame*, o conocer en profundidad lo que hace *Pygame* en su interior, es necesario recurrir a la documentación online en el sitio: <http://www.pygame.org/docs/>.

La Inicialización del Juego

Hemos visto que el código que se ejecuta antes del *game loop* se encarga de inicializar todos los elementos que son necesarios en el programa. En un juego grande, esto incluirá inicializar los *assets* (los gráficos, los sonidos, los niveles, etc.). No es conveniente cargar o inicializar objetos o elementos cuando el juego mismo está corriendo, porque sino el juego se podría trancar un instante (porque por ejemplo necesita acceder al disco y leer). Por este motivo,

siempre habrá una etapa de inicialización antes de comenzar el juego en sí.

Una vez inicializado todo lo necesario, entramos al *game loop* para que corra el juego en sí (en este ejemplo, el juego no hará nada, pero ya dejaremos pronta la base para hacer un juego).

Estas dos líneas inicializan las variables que controlarán el loop principal del juego (el *game loop*):

```
# Inicializar las variables de control del game loop.
clock = pygame.time.Clock()
salir = False
```

En la primera línea, inicializamos el reloj (creamos un objeto `clock`), que en el *game loop* va a hacer latir al juego 60 veces por segundo (el juego va a correr a 60 *FPS*). Luego, la variable `salir` hará que se salga del *game loop* cuando esta variable valga `True`, lo cual ocurrirá cuando se pulse la tecla `[Esc]` o cuando se cierre la ventana.

El Game Loop

Llegamos a la parte principal del programa: el *game loop*. El *game loop* es una estructura `while`, de la cual no se sale hasta que la variable `salir` sea `True`. El *game loop* correrá en forma continua 60 veces por segundo hasta que el usuario cierre la ventana o decida salir pulsando la tecla `[Esc]`. Veamos el código del *game loop* y luego explicaremos una a una sus sentencias.

```
# Loop principal (game loop) del juego.
while not salir:

    # Timer que controla el frame rate.
    clock.tick(60)

    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():

        # Si se cierra la ventana se sale del programa.
        if event.type == pygame.QUIT:
            salir = True

        # Si se pulsa la tecla [Esc] se sale del programa.
        if event.type == pygame.KEYUP:
            if event.key == pygame.K_ESCAPE:
                salir = True
```

```
# Actualizar la pantalla.  
screen.blit(imgBackground, (0, 0))  
pygame.display.flip()
```

El flujo del programa se puede ver en la figura 2-2. Básicamente se ejecuta una estructura `while` mientras la variable `salir` sea `False`. Se sale del *game loop* cuando la variable `salir` es `True`. Dentro del *game loop*, lo que se hace es manejar los eventos, mover los objetos del juego y dibujarlos (esto se hará más adelante cuando agreguemos los objetos del juego). Al final, se limpia la memoria usada por los objetos creados por el programa.

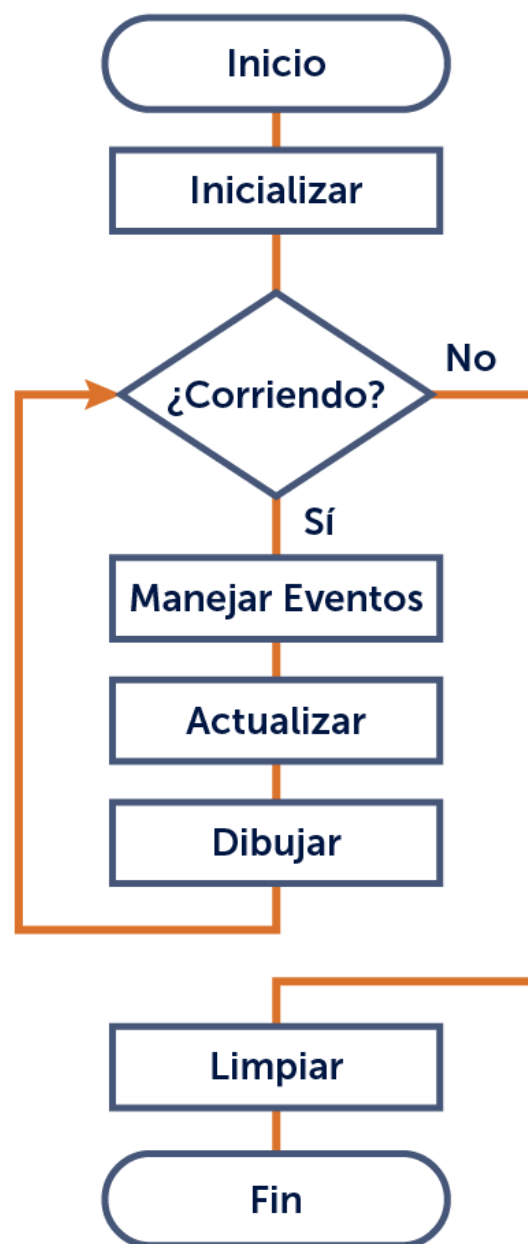


Figura 2-3: Diagrama de flujo de nuestro primer programa *Pygame*.

A continuación se explicará paso por paso las sentencias que componen nuestro *game loop* y veremos algunos conceptos necesarios.

El Reloj

La primera línea del *game loop*, es `clock.tick(60)`. Lo que hace esta sentencia es esperar el tiempo necesario para que el *game loop* (y por lo tanto, el juego) corra a 60 *frames* por segundo. Se hace esto para que el *frame rate* del juego sea constante.

Procesar los Eventos

Dentro del *game loop*, en cada pasada del ciclo, procesamos los eventos de la aplicación (esto es, los eventos que el sistema operativo le envía a nuestra ventana).

```
# Procesar los eventos que llegan a la aplicación.
for event in pygame.event.get():

    # Si se cierra la ventana se sale del programa.
    if event.type == pygame.QUIT:
        salir = True

    # Si se pulsa la tecla [Esc] se sale del programa.
    if event.type == pygame.KEYUP:
        if event.key == pygame.K_ESCAPE:
            salir = True
```

Con estas sentencias, estamos capturando el evento `pygame.QUIT` que es el evento que el sistema operativo le envía a la aplicación (a nuestro juego) cuando se cierra la ventana. También estamos chequeando si se pulsa la tecla *[Esc]*. Si ocurre alguna de estas condiciones, la variable `salir` se pondrá en `True`, lo que hará que se salga del *game loop*.

En un capítulo más adelante, veremos en profundidad el manejo de eventos. Por ahora diremos que utilizando `for event in pygame.event.get()` se recorren todos los eventos que han ocurrido desde el *frame* anterior. Para cada evento en la lista de eventos, se pregunta si es el evento `pygame.QUIT` o el evento `pygame.KEYUP` el que ha llegado a la ventana. Si el evento es `pygame.KEYUP`, significa que se ha pulsado una tecla, por lo cual preguntamos si esa tecla es la tecla *[Esc]*.

Si ocurre uno de estos eventos, se cambia la variable de control del *game loop* (`salir`) para que se salga del loop en el siguiente *frame*.

Más adelante veremos cómo en esta parte del *game loop* (denominada manejo de eventos), capturamos otros eventos, como por ejemplo cuando el jugador pulsa una tecla para moverse o disparar, pulsa o mueve el mouse, etc.

Refrescar la Pantalla

Luego de manejar los eventos, el programa movería los objetos del juego y luego los dibujaría en la pantalla. Como por ahora no hay objetos en el juego, pasamos a la parte que refresca la pantalla:

```
# Actualizar la pantalla.  
screen.blit(imgBackground, (0, 0))  
pygame.display.flip()
```

Recordemos que `imgBackground` contiene la imagen del fondo (*background*) que usamos en el juego. La primera sentencia copia la imagen de *background* a la pantalla. Las coordenadas `(0,0)` es para decirle que la muestre en la posición `(0,0)`, correspondiente a la esquina superior izquierda de la pantalla. Como la imagen tiene el mismo tamaño que la ventana, la sobrescribe completamente. Más adelante hablaremos sobre el sistema de coordenadas.

La sentencia `pygame.display.flip()` lo que hace es volcar a la pantalla del monitor (volcar al *display*) el contenido de la pantalla (todo lo que hemos dibujado en `screen`). Es necesario invocar a esta función, o de lo contrario no veríamos nada en la pantalla.

Liberar la Memoria al Final

Una vez que salimos del *game loop*, el programa deberá liberar los recursos que ha pedido al sistema (a esto se le llama *liberar* la memoria).

```
# Cerrar Pygame y liberar los recursos que pidió el programa.  
pygame.quit()
```

La función `pygame.quit()` la llamaremos al final del programa y lo que hace es liberar de memoria los módulos de *Pygame* que se hayan inicializado. En este punto del programa deberemos, además, liberar los recursos que hemos pedido para nuestros objetos.

Con esto hemos terminado nuestro primer programa *Pygame* y hemos implementado el *game loop*, que será la base para nuestro primer juego. Ahora veremos algunos conceptos más y

luego ya pasaremos a la parte de la acción, en la cual moveremos objetos por la pantalla.

Encoding

Si queremos poner tildes en los comentarios, debemos poner esta línea al principio de los archivos:

```
# -*- coding: utf-8 -*-
```

Como veremos en todos los ejemplos del libro, todos los archivos comienzan con esta línea. El editor que usemos para editar debe grabar el texto con encoding *UTF-8*.

Nota: En este libro escribiremos el código en inglés y los comentarios en español. El motivo de escribir el código en inglés es que es una práctica profesional, dado que es muy factible que se comparta el código con gente de otras nacionalidades y en la industria de videojuegos se espera que el programador escriba el código en inglés. Hemos puesto los comentarios en español para hacerlo más fácil al lector, pero cuando se trabaje profesionalmente, ha de escribirse en inglés.

¿Qué hace el *Game Loop* en un Videojuego?

En un videojuego hay muchos objetos (también llamados entidades) que se mueven en forma autónoma. ¿Cómo se logra esto?

Pues bien, el *game loop* es quien se encarga de eso. Como hemos visto, el *game loop* consiste en un bucle infinito (una estructura `while`), del cual se sale solamente cuando se termina el juego. En cada ciclo del loop, se actualiza la posición de cada objeto del juego y luego se dibuja en su nueva ubicación. La función que actualiza los objetos del juego se llamará `update()` y la función que los dibuja se llamará `render()`. Más adelante implementaremos estas funciones. Repasemos la estructura del *game loop* mirando la siguiente figura.

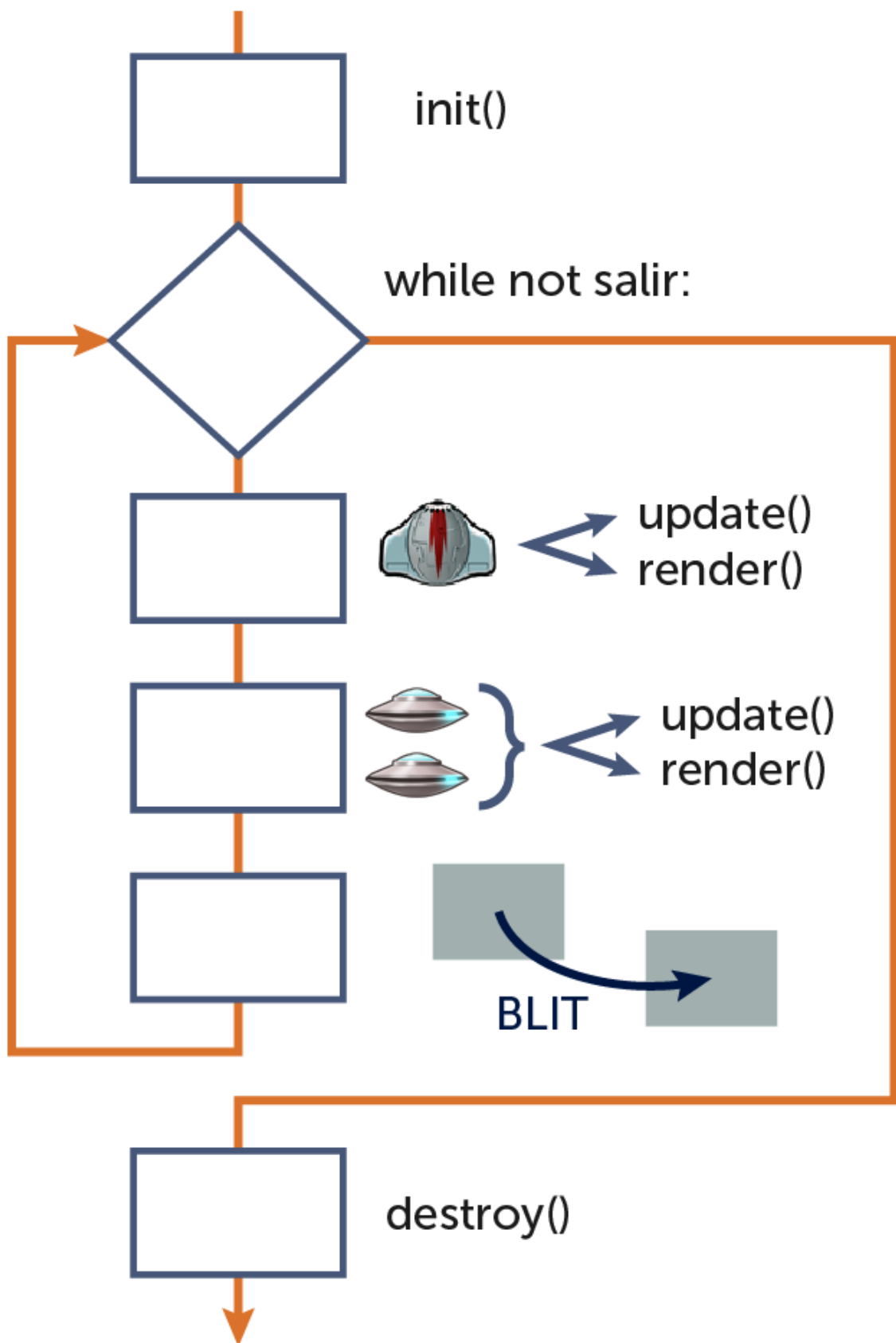


Figura 2-4: El game loop en pseudocódigo.

La primera parte del *game loop* corresponde a procesar los eventos y, por ejemplo, ver si el jugador ha pulsado alguna tecla o ha hecho algo con el *mouse* o *joystick* para controlar a su personaje. Luego la función `update()` recorrerá los objetos del juego uno a uno y los moverá según la velocidad que tengan, controlando las colisiones entre ellos y las reacciones. En el caso del personaje que controla el jugador, lo moverá según los controles.

Luego de movidos los objetos, se chequean las colisiones entre ellos para saber si algún objeto debe morir porque fue alcanzado por una bala, por ejemplo, o si se debe crear una bala porque el personaje ha disparado, o si un objeto debe eliminarse porque ha salido de la pantalla, etc. Al final de todo, se invoca a la función `render()` para dibujar a todos los objetos del juego en su nueva posición y luego se mostrará el *frame* actual en la pantalla.

A cada ciclo del *game loop* se le denomina *update*, que significa actualización. Cada paso del *game loop* corresponde a una invocación a las funciones `update()` y `render()`, y es lo que mantiene funcionando un videojuego.

Modo *Fullscreen*

Generalmente los juegos corren en modo de pantalla completa (denominado modo *fullscreen*) porque un juego que corre en pantalla completa, será más inmersivo (nos crearemos más la realidad del juego).

Veamos el segundo ejemplo, ubicado en la carpeta `capitulo_02\002_fullscreen`. Este ejemplo es el mismo que el anterior, pero agrega un parámetro al crear la ventana para crearla en modo *fullscreen*.

Si corremos el programa en PC, veremos que el programa se muestra en la pantalla completa. Si no funciona, para que corra, debería cambiarse la resolución a alguna soportada por la tarjeta de video (1920 x 1080, 1280 x 720 si es un notebook, etc.). Ya hablaremos sobre esto más adelante.

La única línea que cambia con respecto al ejemplo anterior, es cuando se inicializa el modo de video. Ahora se pasa un parámetro extra, `pygame.FULLSCREEN`, para indicarle a *Pygame* que queremos usar la pantalla completa.

```
# Poner el modo de video fullscreen e indicar la resolución.  
screen = pygame.display.set_mode((640, 360), pygame.FULLSCREEN)
```

Nota: Recordemos que cuando una palabra aparece en mayúsculas es porque se trata de una constante. En este caso, la constante `pygame.FULLSCREEN` es necesaria para que *Pygame* sepa que queremos correr el juego en pantalla completa.

Hay que tener en cuenta que no todas las tarjetas de video pueden poner en *fullscreen*

cualquier resolución que le indiquemos. Hoy día es posible que sí, pero existe la posibilidad de que falle (dando un error), sobre todo con equipos viejos o en hardware limitado. Las resoluciones históricas más comunes en modo *fullscreen* para PC son 640 x 480, 800 x 600, 1024 x 768, 1920 x 1080 píxeles, etc.

Nota: Cuando hagamos un videojuego para publicar, deberemos chequear que la resolución sea soportada por la computadora donde corre, y si no se soporta la resolución indicada, permitirle al jugador seleccionar una. Es seguro asumir que un PC soporta fullscreen en 640 x 480, u 800 x 600, que son resoluciones históricas. En modo ventana la mayoría de las tarjetas de video pueden manejar tamaños variables y no habrá errores.

Nota: En los ejemplos, utilizaremos el modo de ventana en 640 x 360, de modo que los mismos funcionen en todas las computadoras. Si se desea correr los ejemplos en fullscreen, hay que tener en cuenta que la resolución sea soportada por la tarjeta de video. Como el objetivo de este libro es enseñar las bases de la programación de videojuegos, nos olvidaremos de este tema hasta que debamos publicar el juego, y trabajaremos en una resolución de 640 x 360.

Cambiar entre modo *Fullscreen* y modo *Ventana*

En esta sección modificaremos el programa para que pulsando la tecla *[F]* podamos cambiar entre el modo *fullscreen* y el modo *ventana*. Veamos el ejemplo que se encuentra en la carpeta `capitulo_02\003_cambiar_ventana_fullscreen`.

Al principio del programa agregamos una constante y una variable para controlar el pasaje entre el modo *fullscreen* y el modo *ventana*:

```
# Control del modo ventana o fullscreen.
RESOLUTION = (640, 360)
isFullscreen = False

# Poner el modo de video en ventana e indicar la resolución.
screen = pygame.display.set_mode(RESOLUTION)
# Poner el título de la ventana.
pygame.display.set_caption("Mi Juego")
```

La constante `RESOLUTION` es una tupla que contiene los valores del tamaño de la ventana o resolución de la pantalla. La variable `isFullscreen` valdrá `True` cuando se ejecute en modo *fullscreen*, y valdrá `False` cuando se ejecute en modo *ventana*. Al comienzo se empieza en modo *ventana*, por lo cual esta variable vale `False`.

Luego, en el *game loop*, en la parte del proceso de eventos, chequeamos si el jugador pulsa la tecla *[F]*. Si lo hace, se cambia la variable `isFullscreen` entre los valores `True` y `False`, y a continuación se vuelve a establecer el modo de video correspondiente.


```
# Si se pulsa la tecla [F], se cambia entre ventana y fullscreen.
if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_f:
        isFullscreen = not isFullscreen
        if isFullscreen:
            screen = pygame.display.set_mode(RESOLUTION,
                                              pygame.FULLSCREEN)
        else:
            screen = pygame.display.set_mode(RESOLUTION)
```

Nota: Cuando una línea no entra en el código listado en el libro, se baja para que quede legible. Solo recuerda que en el programa esto no se puede hacer. Las líneas deben de ser continuas. De lo contrario el programa dará error.

Con esto hemos terminado con la programación de este capítulo. Veamos a continuación un par de conceptos que necesitamos saber y que hemos pasado un poco por alto.

Blit

En el programa hemos utilizado esta sentencia para mostrar la imagen de background:

```
# Actualizar la pantalla.
screen.blit(imgBackground, (0, 0))
pygame.display.flip()
```

La función o método `blit()` lo que hace es, aplicada a una imagen o superficie, dibuja la imagen que se le pasa como parámetro en las coordenadas indicadas. A la función `blit()` se le pasa la imagen a dibujar y las coordenadas de la esquina superior izquierda. En este caso estamos dibujando en la coordenada `(0, 0)` de la pantalla, porque la imagen a dibujar es del mismo tamaño que la ventana.

La operación de dibujar una imagen se denomina *blitting*. *Blit* es una abreviación de *bit blit*, que significa “*binary block transfer*” (o copia *bit* a *bit*). Es una técnica que se usa con la memoria para copiar un área rectangular de un lado a otro. Cuando decimos que copiamos una imagen, lo que estamos diciendo es que estamos haciendo *blit* a una nueva posición. Esta operación de blit se encuentra disponible en la mayoría de las tarjetas gráficas.

Nota: Cuando una función se aplica a un objeto, o dicho de otra forma, un objeto tiene adentro una función, a esa la función se le denomina *método*. Como la diferencia entre *función* y *método* depende de si se encuentra dentro de un objeto o no, y ambas son funciones, generalmente se utiliza la palabra función.

Double Buffering

Para la computadora, desplegar gráficos en la pantalla es un proceso lento. Por este motivo, se dibuja en una superficie que se encuentra en la memoria (porque es mucho más rápido manipular la memoria) y luego, al final, se vuelca a la pantalla.

Con *Pygame* debemos utilizar la función `pygame.display.flip()` para copiar la imagen de *background* al *display* (a la pantalla). Al proceso de dibujar en una imagen de fondo y luego volcarla a la pantalla se le denomina *double buffering*. Esta técnica permite una mejor calidad de animación, evitando el parpadeo (no usar *double buffer* produce un parpadeo de la pantalla). Los videojuegos siempre utilizan esta técnica para dibujar y *Pygame* ya lo hace por nosotros.

Nota: La palabra *buffer* significa que es una parte de la memoria utilizada para almacenar temporalmente, en este caso, la imagen de la pantalla.

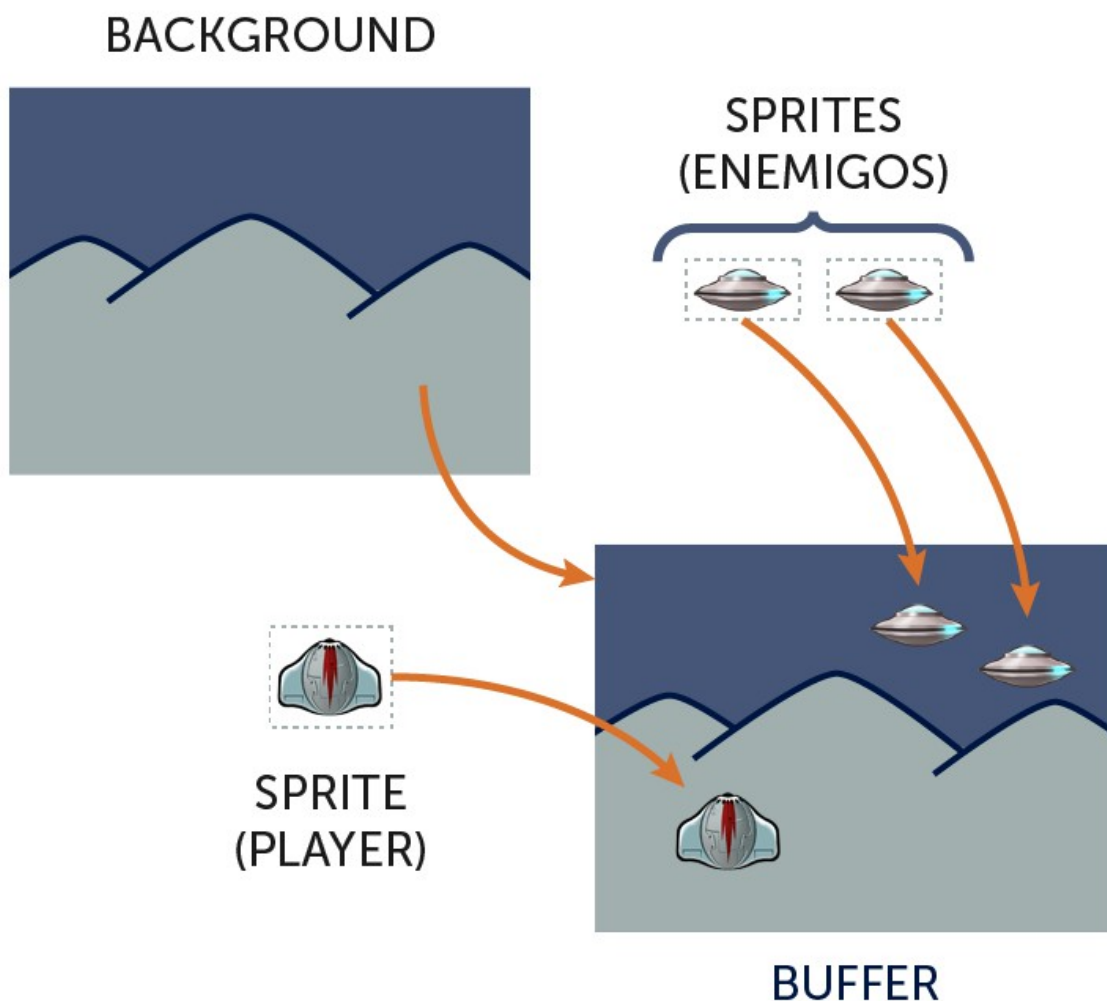


Figura 2-5: Cómo funciona el *double buffer*. Paso 1.

En la figura 2-5 vemos la primera etapa del *double buffer*, en la cual se dibuja en el *back buffer* cada uno de los elementos del juego. En este caso se dibuja el fondo, luego los enemigos y por último el jugador. En este momento tenemos armada la imagen en el *buffer*, pero no está visible en la pantalla.

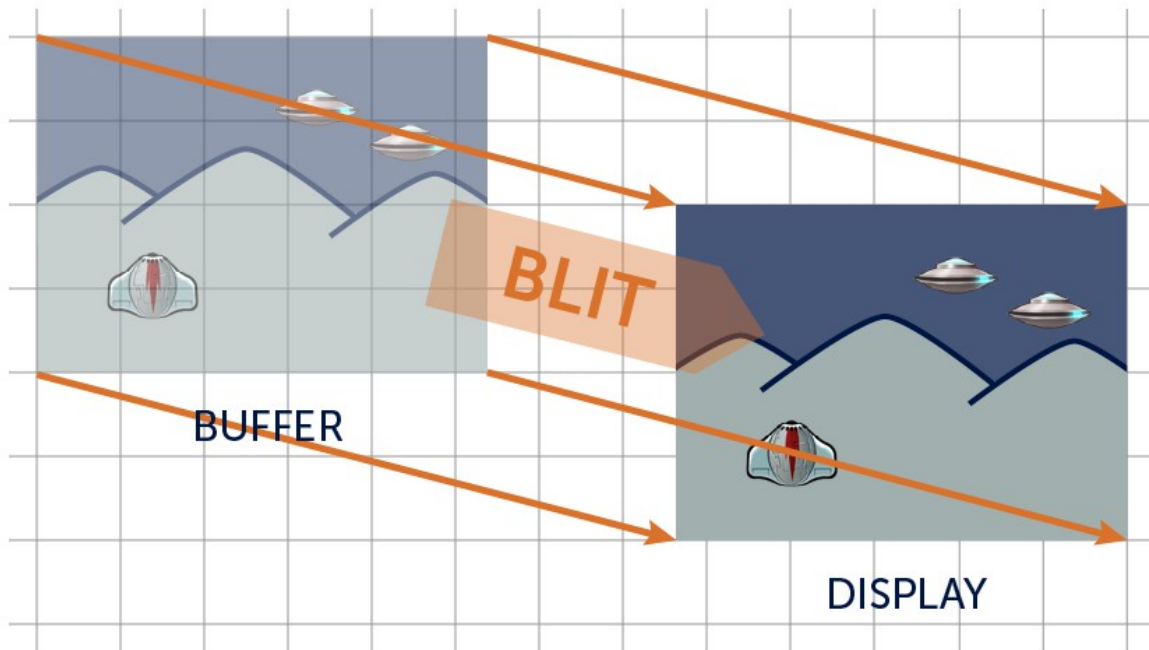


Figura 2-6: Cómo funciona el *double buffer*. Paso 2.

En la figura 2-6 vemos cómo al llamar a la función `pygame.display.flip()`, el contenido del *back buffer* es volcado a la pantalla y en ese momento es que se ve realmente en la pantalla. El *double buffer* entonces implica dibujar al *back buffer* y al final volcar el *back buffer* al *display*. Por suerte, *Pygame* hace todo fácil para nosotros y lo único que haremos es utilizar la función `blit` para dibujar en `screen`, que representa el *back buffer*, y al final usamos `pygame.display.flip()` para hacer visible el frame actual.

Con esto, hemos terminado la base para comenzar a construir nuestro primer videojuego, cosa que haremos en los próximos capítulos.

3. Moviendo Nuestro Primer Objeto

En este capítulo veremos cómo crear y mover objetos por la pantalla. Empezaremos con el sistema de coordenadas de *Pygame*, y luego veremos cómo se crean los objetos en el juego y cómo los movemos teniendo en cuenta los bordes de la pantalla. Por último, crearemos nuestra primera clase, para un objeto movable. Más adelante en el libro, utilizaremos *Sprites* para hacer movimientos más avanzados y con animaciones.

El Sistema de Coordenadas

Pygame usa un sistema de coordenadas denominado *Sistema de Coordenadas Cartesianas*, el cual está compuesto por una grilla de cuadrados (píxeles) con un par de ejes de coordenadas que forman un ángulo recto. El eje horizontal es el eje x y el eje vertical es el eje y .

Es esencial para un desarrollador de videojuegos conocer los elementos matemáticos necesarios para la programación, porque se utilizan en todo momento. Por ejemplo, si queremos colocar un objeto del juego en una posición en la pantalla, lo que tenemos que hacer es darle su ubicación utilizando sus coordenadas (x, y) , mediante sentencias de programación que ya veremos más adelante.

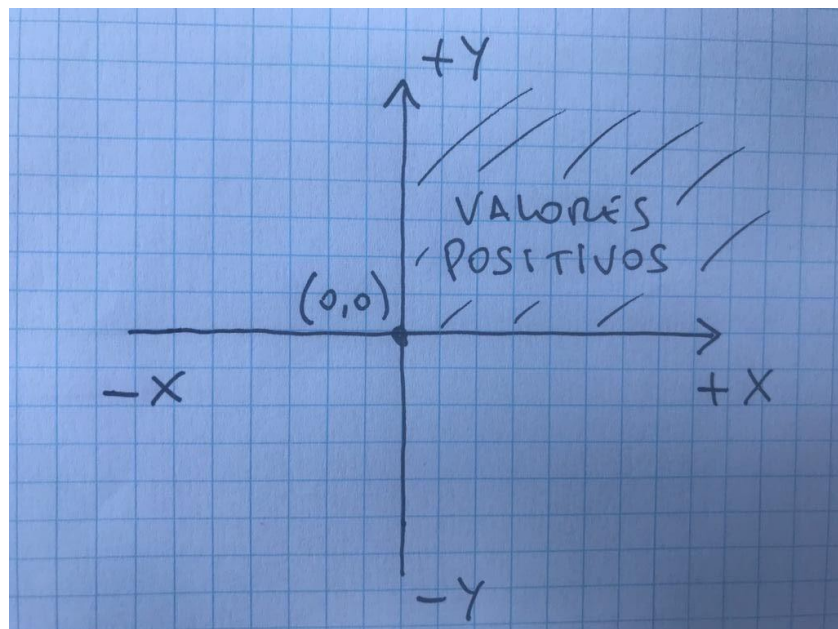


Figura 3-1: El Sistema de Coordenadas en Matemática.

Como hemos visto en cualquier clase de matemática, la coordenada horizontal crece hacia la derecha y la coordenada vertical crece hacia arriba. Esto se muestra en la figura 3-1.

En *Pygame*, y en general en la mayoría de los sistemas o programas de computación, el sistema de coordenadas cambia y la coordenada vertical crece hacia abajo. La figura 3-2 muestra el sistema de coordenadas que vamos a usar en los juegos.

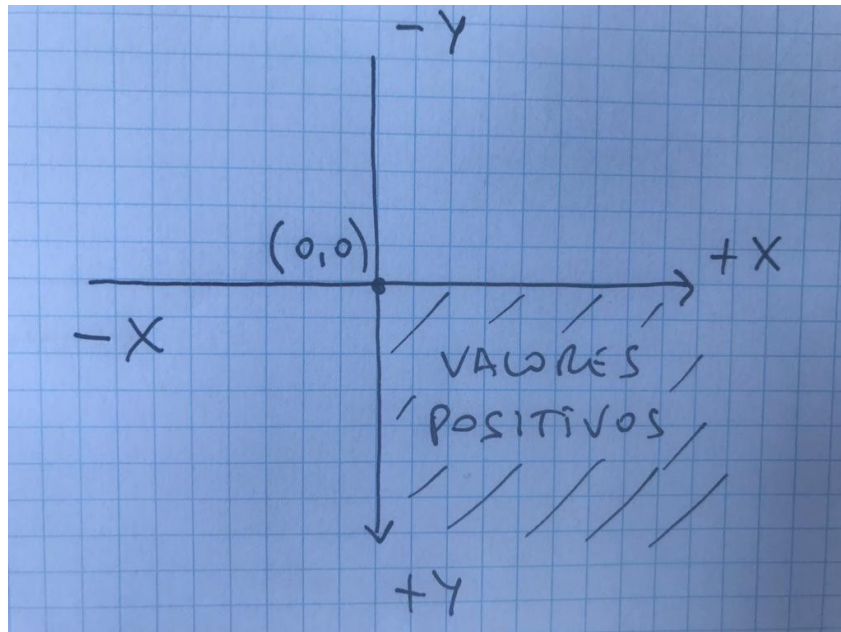


Figura 3-2: El Sistema de Coordenadas en *Pygame*.

De esta forma, en *Pygame*, el eje de coordenadas tiene como origen a la esquina superior izquierda de la pantalla, esto es, el punto $(0, 0)$. Recordemos que el origen del sistema de coordenadas es el punto donde los ejes se cruzan. Luego, la coordenada horizontal (x) crece hacia la derecha y la coordenada vertical (y) crece hacia abajo.

En realidad, salvo la dirección del eje y , no hay diferencia entre los dos sistemas de coordenadas. El sistema de coordenadas de la pantalla lo usaremos de la misma forma que lo hacemos en matemática. Lo único que cambia es la forma en la que lo observamos, pero las fórmulas que utilizemos se aplicarán igual que a lo que estamos acostumbrados de la matemática normal.

Como vemos en la figura 3-2, los valores positivos para x y para y se encuentran dentro de la pantalla (si no son muy grandes). Debemos imaginar la pantalla del juego como una grilla de píxeles, en un sistema de coordenadas. Miremos la figura 3-3.

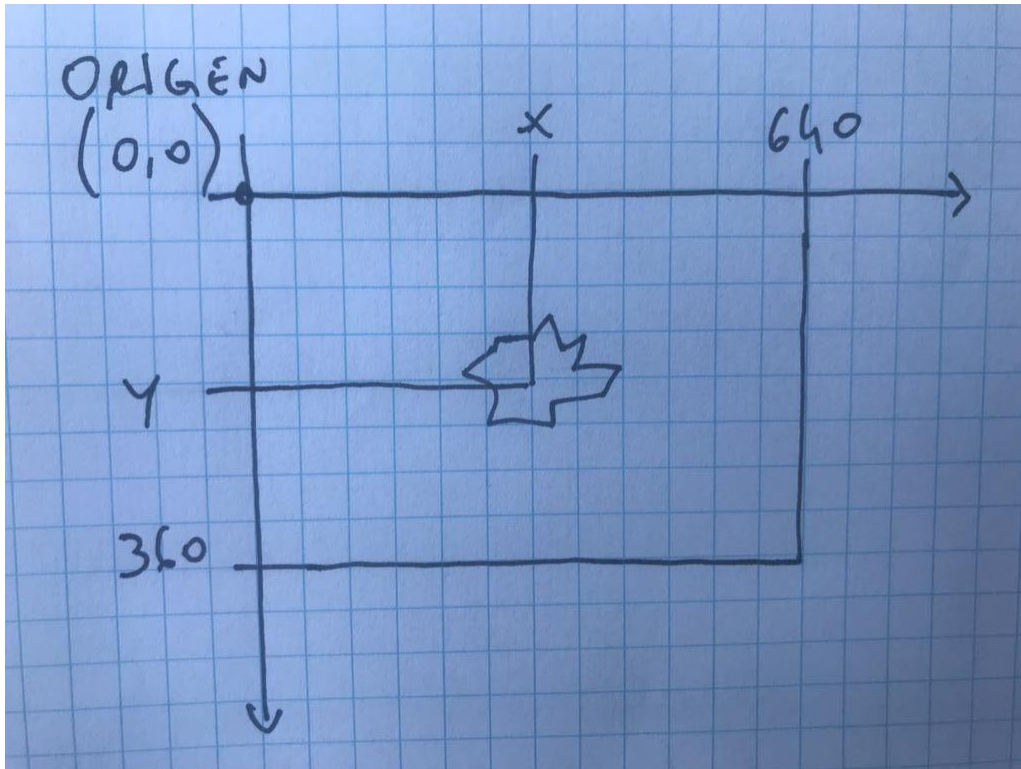


Figura 3-3: El juego visto en el sistema de coordenadas de la pantalla.

Si la pantalla tiene un tamaño de 640 x 360 píxeles, todo lo que se encuentre entre los puntos con coordenadas $(0, 0)$ y $(639, 359)$ se verá en la pantalla.

Ubicando un Objeto en la Pantalla

Para ubicar un objeto en la pantalla (o sea, en el sistema de coordenadas), lo que hacemos es dar un par de coordenadas que indican dónde está el objeto. En matemática, las dos coordenadas se agrupan con paréntesis, como por ejemplo: $(5, 6)$. Mira la figura 3-4.

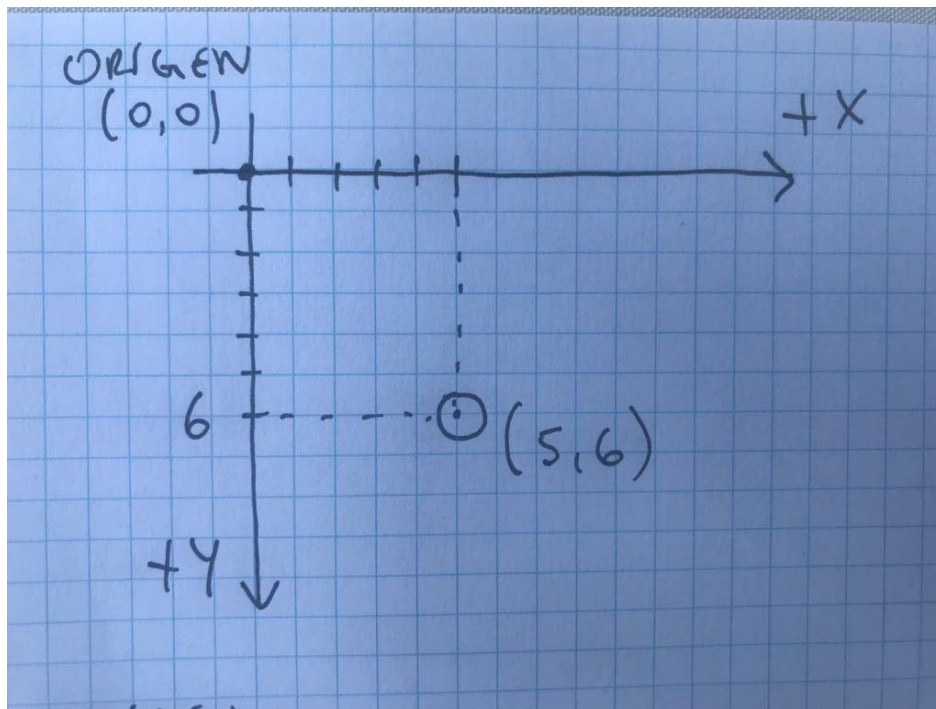


Figura 3-4: Ubicando un objeto en el sistema de coordenadas.

La primera coordenada representa la coordenada horizontal (la coordenada x) y la segunda coordenada representa la coordenada vertical (la coordenada y). Si decimos que un objeto está ubicado en $(5, 6)$, esto quiere decir que el 5 es la distancia sobre el eje x , y el 6 es la distancia sobre el eje y .

Moviendo nuestro primer Objeto

Ahora que sabemos cómo ubicar un objeto en la pantalla, vamos a poner un cuadrado en la pantalla y a moverlo (más adelante cambiaremos el cuadrado por un gráfico). Para esto partiremos desde el *game loop* que construimos en el capítulo anterior y le iremos agregando código para hacer esto.

Lo primero que haremos será crear una superficie cuadrada (una imagen) y agregaremos dos variables llamadas `cuadradoX` y `cuadradoY` que tendrán las coordenadas del cuadrado. Luego, dentro del *game loop*, cambiaremos esas variables para que el cuadrado cambie de posición y también dentro del *game loop* dibujaremos el cuadrado en su nueva posición. Antes de dibujar el cuadrado en la nueva posición, dibujaremos el fondo, para borrar de esta forma lo que había antes en la pantalla.

Mover un objeto en el juego, se resume en los siguientes pasos:

- Crear el objeto.
- Inicializar sus coordenadas (establecer la posición inicial).
- Dentro del *game loop*:
 - Mover el objeto (cambiar sus coordenadas).

- Dibujar el fondo (borrar el objeto de la pantalla de su posición anterior).
- Dibujar el objeto en su nueva posición.

Abre y ejecuta el ejemplo de la carpeta `capitulo_03\001_mover_cuadrado`. Examina un poco el código y observa las diferencias que hay con el último ejemplo del capítulo anterior.

Nota: Los ejemplos en el libro se van haciendo incrementales, lo que quiere decir que se le van agregando partes, de a una a la vez. Cuando algo se hace por primera vez, las sentencias correspondientes se mostrarán en negrita en el listado de código, para que sea más fácil de entender.

Nota: El código está completamente documentado. Como programador tendrás tus propias preferencias con respecto a los comentarios en el código. A algunos programadores les gusta documentar bastante y a otros poco o nada. Es conveniente encontrar un equilibrio. Yo recomiendo documentar absolutamente todo.

Al correr el ejemplo, vemos que un cuadrado se mueve por la pantalla de izquierda a derecha y se va de la pantalla. A continuación se explican los pasos necesarios para mover un objeto. Primero veamos el listado completo del programa, que es similar al *game loop* del ejemplo anterior, y en negrita se muestran las líneas que se han agregado.

```
# -*- coding: utf-8 -*-

#-----
# Mover un cuadrado por la pantalla.
#
# Autor: Fernando Sansberro - Batovi Games.
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar e inicializar Pygame.
import pygame
pygame.init()

# Control del modo ventana o fullscreen.
RESOLUTION = (640, 360)
isFullscreen = False

# Poner el modo de video en ventana e indicar la resolución.
screen = pygame.display.set_mode(RESOLUTION)
# Poner el título de la ventana.
pygame.display.set_caption("Mi Juego")

# Crear la superficie del fondo o background.
imgBackground = pygame.Surface(screen.get_size())
imgBackground = imgBackground.convert()
```



```

imgBackground.fill((0, 0, 255))

# Crear un cuadrado de 32x32.
imgCuadrado = pygame.Surface((32, 32))
imgCuadrado = imgCuadrado.convert()
imgCuadrado.fill((255, 0, 0))

# Coordinadas del cuadrado.
cuadradoX = 0
cuadradoY = 180

# Inicializar las variables de control del game loop.
clock = pygame.time.Clock()
salir = False

# Loop principal (game loop) del juego.
while not salir:

    # Timer que controla el frame rate.
    clock.tick(60)

    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():

        # Si se cierra la ventana se sale del programa.
        if event.type == pygame.QUIT:
            salir = True

        # Si se pulsa la tecla [Esc] se sale del programa.
        if event.type == pygame.KEYUP:
            if event.key == pygame.K_ESCAPE:
                salir = True

        # Si se pulsa la tecla [F], se cambia entre ventana y
        fullscreen.
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_f:
                isFullscreen = not isFullscreen
                if isFullscreen:
                    screen = pygame.display.set_mode(RESOLUTION,
                                                       pygame.FULLSCREEN)
                else:
                    screen = pygame.display.set_mode(RESOLUTION)

    # Modificar la posición del cuadrado (mover el cuadrado).
    cuadradoX += 2

    # Dibujar el fondo.

```

```

screen.blit(imgBackground, (0, 0))

# Dibujar el cuadrado en la nueva posición.
screen.blit(imgCuadrado, (cuadradoX, cuadradoY))

# Actualizar la pantalla.
pygame.display.flip()

# Cerrar Pygame y liberar los recursos que pidió el programa.
pygame.quit()

```

Hemos resaltado en negrita las líneas agregadas, de forma tal de entender dónde es que va cada parte. Por ejemplo, la parte de inicialización va al comienzo del programa. Luego, la parte de movimiento y de dibujo, va dentro del *game loop*. A continuación explicaremos las sentencias necesarias para crear el cuadrado y, en el *game loop*, moverlo y dibujarlo en la pantalla.

Al ejecutar el programa, veremos el cuadrado rojo atravesando la pantalla como muestra la figura 3-5..

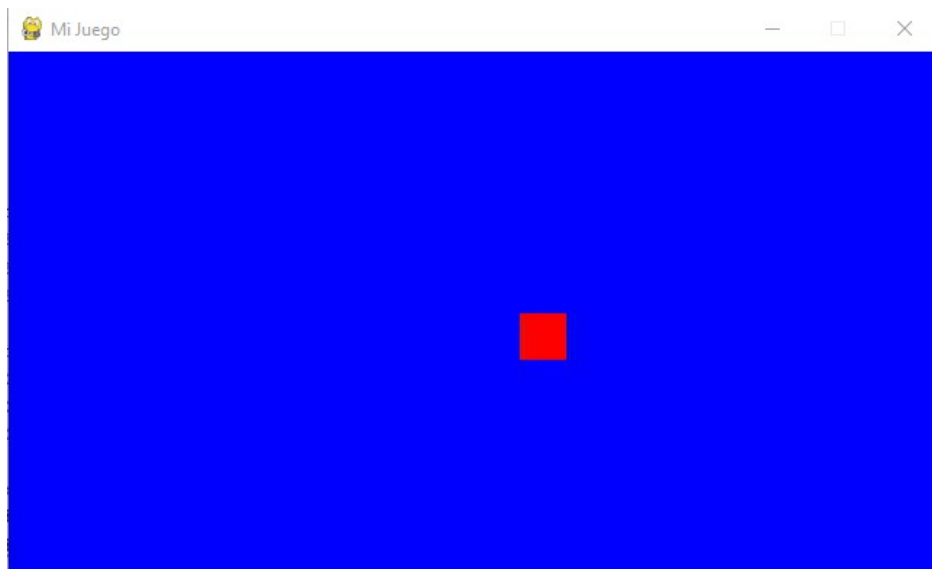


Figura 3-5: El cuadrado moviéndose por la pantalla.

Crear el Objeto

El cuadrado va a ser una superficie de 32 x 32 píxeles. Para eso creamos una superficie y la usaremos con una variable que llamaremos `imgCuadrado`. Esto lo hacemos antes de entrar al *game loop*.

```
# Crear un cuadrado de 32x32.  
imgCuadrado = pygame.Surface((32, 32))  
imgCuadrado = imgCuadrado.convert()  
imgCuadrado.fill((255, 0, 0))
```

Con este código, creamos la superficie utilizando la función `pygame.Surface()`, al igual que lo hicimos antes para el fondo, y le pasamos el ancho y el alto de la superficie, que en este caso será de 32 píxeles de ancho y 32 píxeles de alto. Luego, convertimos la superficie creada al formato de *Pygame*, al igual que como lo hicimos con el *background*, y con la función `fill()` llenamos la superficie con un color, en este caso, con el color rojo (255, 0, 0).

Recordemos que un color en formato *RGB* tiene tres números: el primer número corresponde al componente rojo, el segundo al componente azul y el tercero al componente verde.

Colocar el Objeto en su Posición Inicial

Cada vez que creamos un objeto, deberemos colocarlo en su posición inicial. En general, al crear un objeto en el juego, siempre le daremos unos valores iniciales. A esta tarea se le denomina *inicializar* el objeto. En este caso, usaremos dos variables para mantener la posición del cuadrado. La coordenada *x* irá en la variable `cuadradoX` y la coordenada *y* irá en la variable `cuadradoY`.

```
# Coordenadas del cuadrado.  
cuadradoX = 0  
cuadradoY = 180
```

El cuadrado, inicializado con las coordenadas (0, 180), comenzará situado en el borde izquierdo y en la mitad de la altura de la ventana (esto es así, porque la ventana mide 640 x 360 píxeles). El punto usado para situar el cuadrado, es la coordenada de la esquina superior izquierda del cuadrado. La siguiente figura muestra el cuadrado en su posición inicial.

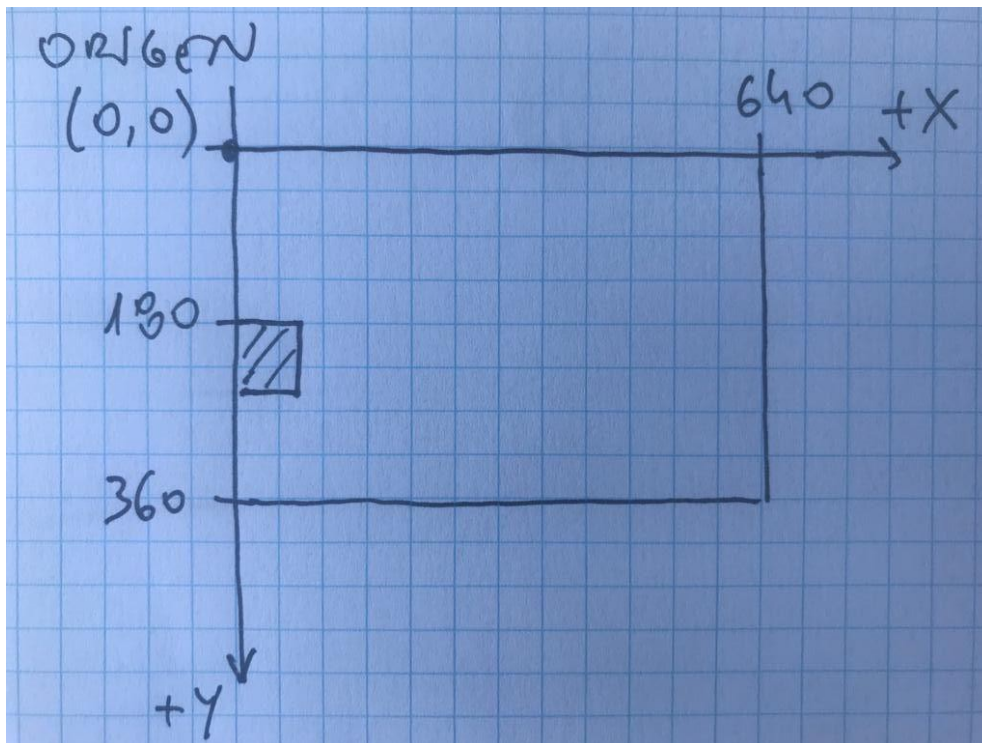


Figura 3-6: El cuadrado en su posición inicial: (0, 300).

Una vez que tenemos inicializado el objeto, ya podremos pasar al *game loop*, en donde en cada *frame* moveremos y dibujaremos el objeto.

Mover el Objeto

Dentro del *game loop* movemos el cuadrado cambiando su posición. Por ejemplo, lo que vamos a hacer en cada *frame* es mover el cuadrado 2 píxeles hacia la derecha.

```
# Modificar la posición del cuadrado (mover el cuadrado).  
cuadradoX += 2
```

Esta sentencia lo que hace es incrementar el valor de la coordenada x del cuadrado en 2 píxeles. Por sí solo, esta variable no mueve el cuadrado, solamente almacena su nueva posición. Pero cómo utilizamos esta variable para dibujar el cuadrado en su nueva posición, esto será lo que realmente causa que el cuadrado se mueva en la pantalla.

En la figura 3-7 podemos ver cómo cambiando la posición del objeto y luego dibujándolo en la nueva posición, haremos que el objeto se mueva.

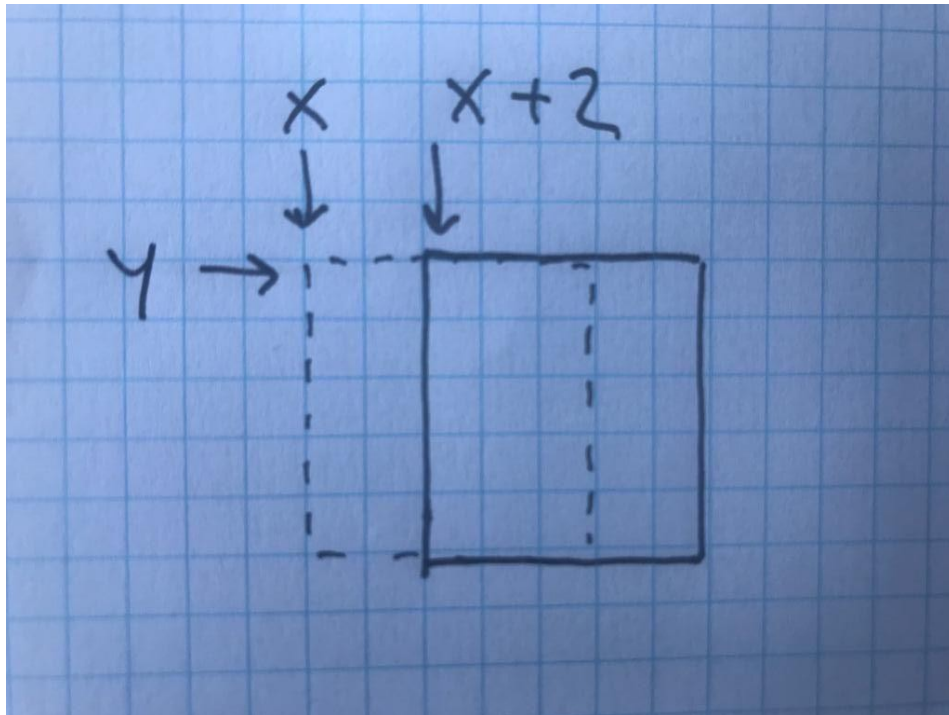


Figura 3-7: Al cambiar las coordenadas del objeto, cambia su posición.

Dibujar el Objeto

Si recordamos cómo funciona el *game loop*, habíamos dicho que una parte actualizaba los objetos y otra los dibujaba. En la sección anterior movimos el objeto de lugar (cambiamos sus coordenadas), pero esto por sí solo no hace nada si no se dibuja. Ahora dibujaremos el objeto en su nueva posición.

Al final del *game loop* tenemos la parte que dibuja la escena:

```
# Dibujar el fondo.
screen.blit(imgBackground, (0, 0))

# Dibujar el cuadrado en la nueva posición.
screen.blit(imgCuadrado, (cuadradoX, cuadradoY))

# Actualizar la pantalla.
pygame.display.flip()
```

La primera línea dibuja el fondo, al igual que como lo hacíamos en el ejemplo anterior. Esto lo que hace es dibujar completamente el fondo, haciendo que el cuadrado que había en el frame anterior, sea borrado.

Luego viene la sentencia `screen.blit(imgCuadrado, (cuadradoX, cuadradoY))` que lo que hace es dibujar el cuadrado en la pantalla, en las coordenadas indicadas por las

variables `cuadradoX` y `cuadradoY` (que son las coordenadas que ya han sido actualizadas al moverlo). Por esta razón es que cada vez que se cambien estas coordenadas, el cuadrado se moverá de lugar. La tercera línea actualiza la pantalla como ya hemos visto antes (realiza el *flip* o volcado de pantalla).

Nota: Estas líneas tienen que estar en orden. Si primero dibujamos el cuadrado y luego el fondo, entonces el fondo taparía el cuadrado. Por esto es que tenemos que dibujar primero el fondo y encima del fondo dibujamos el cuadrado. Al dibujar el fondo, esto tiene como efecto que el cuadrado se borra de su posición anterior (porque es tapado por el fondo).

Al ejecutar el ejemplo vemos que el cuadrado va de izquierda a derecha y se va de la pantalla. Esto sucede porque aún no estamos controlando los bordes y la coordenada `x` continúa incrementándose indefinidamente, y llega un momento que el cuadrado tiene un valor de `x` que supera el ancho de la pantalla (y por lo tanto, queda fuera de la pantalla). Más adelante, en este capítulo, controlaremos los bordes de la pantalla para que el cuadrado no se vaya de la pantalla.

Podemos mover el cuadrado más rápido si aumentamos el incremento de la variable `cuadradoX`, o también podemos hacer que vaya hacia la izquierda si restamos un valor a la coordenada `x` en lugar de sumarle. Podemos hacer que vaya para arriba o para abajo si modificamos la coordenada `y`. ¡Más adelante moveremos los objetos del juego de maneras muy divertidas!

Nota: En el capítulo 4 veremos en mayor profundidad el manejo de imágenes. Por ahora nos alcanza con saber que el uso de la función `blit()` es `imgDestino.blit(imgOrigen, (destX, destY))`, en donde `imgDestino` es la imagen en la cual se dibuja, `imgOrigen` es la imagen que vamos a copiar a `imgDestino`, y `(destX, destY)` son las coordenadas en `imgDestino` donde vamos a dibujar. Estas coordenadas corresponden a dónde se coloca la esquina superior izquierda del rectángulo a copiar.

Agregando Otro Objeto al Juego

En este momento tenemos el juego con un cuadrado rojo que se mueve por la pantalla. Pero como los juegos no tienen solo un objeto, lo que haremos ahora es agregar al juego, otro cuadrado amarillo.

En el ejemplo que vimos anteriormente, usamos dos variables para describir la posición del cuadrado (`cuadradoX` y `cuadradoY`). También usamos una variable para contener la imagen del cuadrado rojo (`imgCuadrado`).

Para agregar otro cuadrado en el juego, lo que haremos es duplicar los pasos que hicimos anteriormente: crear el objeto y colocarlo en su posición inicial, y dentro del *game loop*, actualizaremos su posición y dibujaremos el cuadrado en su nueva posición. Para esto, duplicaremos las variables y les llamaremos: `cuadrado2X`, `cuadrado2Y` e `imgCuadrado2`.

Más adelante tendremos un *objeto* (una *clase*) con sus *atributos* internos para hacer esto más fácil, así el programa principal no se llena de variables y queda todo muy prolijo, además de que podremos reusar los objetos que hagamos en otros juegos y tendremos muchas ventajas más que ya veremos. En este ejemplo vamos a duplicar todas las variables, aunque ésta no es la mejor forma de agregar nuevos elementos al juego. Lo vamos a hacer de esta forma para entender más adelante lo bueno que es usar *objetos* (*clases*) en un juego, ya que usando objetos es mucho más fácil agregar elementos al juego, dado que el comportamiento básico de los objetos muchísimas veces se reutiliza. Ya veremos *clases* y *objetos* más adelante. Por ahora agregaremos otro cuadrado haciéndolo “manualmente”.

Vamos a agregar un cuadrado amarillo al último ejemplo, que comenzará situado a la derecha de la pantalla y se moverá hacia la izquierda.

Veamos el ejemplo ubicado en la carpeta `capitulo_03\002_mover_dos_cuadrados`. Abre el programa y ejecútalo.

El ejemplo es el mismo que el de la sección anterior, y se agregan las líneas relacionadas al segundo cuadrado. En el siguiente listado se encuentran resaltadas las líneas que tienen que ver con el segundo cuadrado: la inicialización antes del *game loop*, y luego dentro del *game loop*, su actualización y dibujo.

...

```
# Crear un cuadrado de 32x32.  
imgCuadrado = pygame.Surface((32, 32))  
imgCuadrado = imgCuadrado.convert()  
imgCuadrado.fill((255, 0, 0))
```

```
# Coordenadas del cuadrado.  
cuadradoX = 0  
cuadradoY = 180
```

```
# Crear otro cuadrado de 32x32.  
imgCuadrado2 = pygame.Surface((32, 32))  
imgCuadrado2 = imgCuadrado2.convert()  
imgCuadrado2.fill((255, 255, 0))
```

```
# Coordenadas del segundo cuadrado.  
cuadrado2X = 608  
cuadrado2Y = 212
```

```
# Inicializar las variables de control del game loop.  
clock = pygame.time.Clock()  
salir = False
```

```
# Loop principal (game loop) del juego.  
while not salir:
```

```

# Timer que controla el frame rate.
clock.tick(60)

# Procesar los eventos que llegan a la aplicación.
for event in pygame.event.get():
    ...

# Modificar la posición de los cuadrados (mover los cuadrados).
cuadradoX += 2
cuadrado2X -= 2

# Dibujar el fondo.
screen.blit(imgBackground, (0, 0))

# Dibujar los cuadrados en la nueva posición.
screen.blit(imgCuadrado, (cuadradoX, cuadradoY))
screen.blit(imgCuadrado2, (cuadrado2X, cuadrado2Y))

# Actualizar la pantalla.
pygame.display.flip()

# Cerrar Pygame y liberar los recursos que pidió el programa.
pygame.quit()

```

Nota: Cuando en el código veamos tres puntos (...), esto quiere decir que en ese lugar va el mismo código que había en el ejemplo anterior. Esto es a fines de ubicar mejor dónde van los cambios que hacemos desde el ejemplo anterior.

El segundo cuadrado lo hemos puesto en una posición inicial con coordenadas (608, 212), es decir, en el borde derecho de la pantalla y 32 píxeles más abajo que el primer cuadrado. ¿Porqué pusimos 608 como coordenada x? Pues bien, como queremos que el cuadrado comience recostado a la derecha de la pantalla, y el cuadrado mide 32 píxeles de ancho, tenemos que restar 640 (la coordenada del borde derecho de la pantalla) menos 32, lo cual da 608. Observa la siguiente figura.

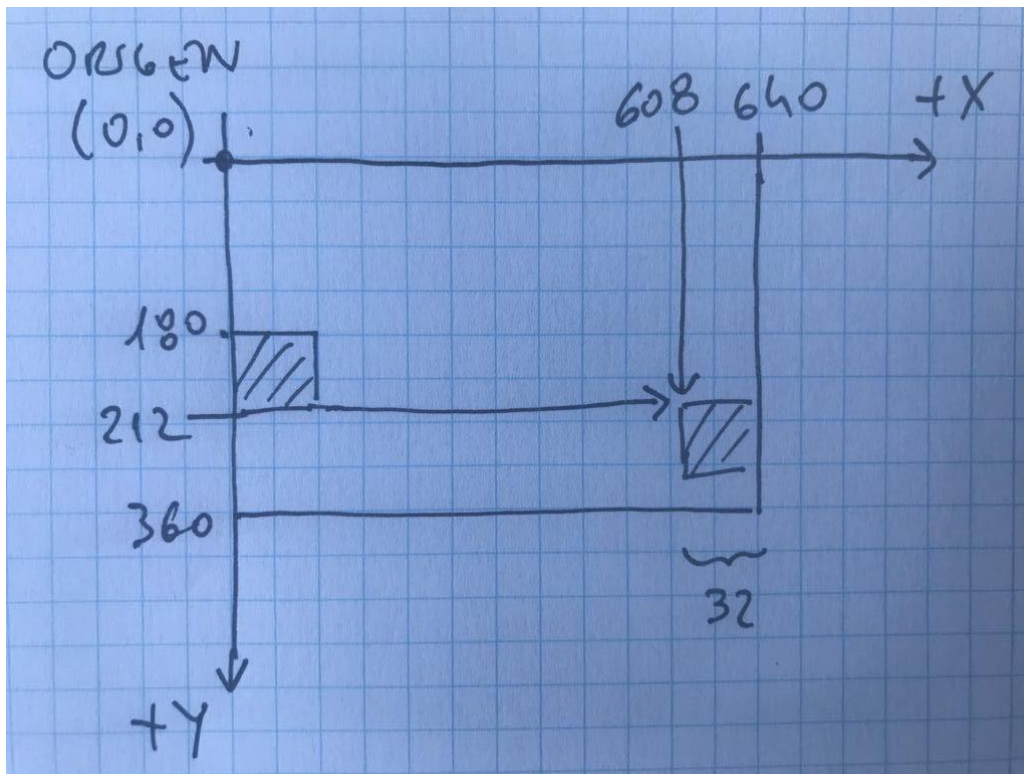


Figura 3-8: El segundo cuadrado en su posición inicial: (768, 400).

Si pusiéramos el cuadrado en una coordenada x inicial 640, el cuadrado aparecería fuera de la pantalla. Es importante tener en cuenta el ancho y el alto de los objetos cuando los inicializamos, de modo tal de que aparezcan exactamente donde queremos.

Para que el segundo cuadrado se mueva hacia la izquierda, lo que hacemos en el *game loop* es restarle 2 a la posición en x , en lugar de sumarle como hicimos con el primer cuadrado. De esta forma, si restamos, el cuadrado se mueve hacia la izquierda. Decimos entonces que este cuadrado se mueve con una velocidad de -2 píxeles por frame.

Cuando corremos el ejemplo vemos que los cuadrados se van de la pantalla. Esto ocurre porque continúan moviéndose en la dirección que llevan y no estamos haciendo nada para controlar que no se vayan más allá de los bordes de la pantalla. A continuación vamos a agregar código para asegurarnos de que los cuadrados siempre estén dentro de la pantalla.

Chequear los Bordes de la Pantalla

En el ejemplo de la sección anterior, los cuadrados se van de la pantalla porque no estamos controlando los bordes de la misma. Ahora vamos a hacer que los cuadrados siempre permanezcan dentro de la pantalla. Para esto, existen varias cosas que se pueden realizar, por ejemplo, que cuando un cuadrado se vaya por la izquierda, o bien que aparezca por la derecha, o bien que rebote, etc.

En general, cada vez que se cambia la posición de un objeto en el juego, hay que chequear si

pasan los bordes de la pantalla (en este caso la pantalla es todo el mundo del juego), y cuando se mueve un objeto también hay que chequear colisiones con otros elementos del juego (por ejemplo con las balas o con el jugador).

Lo que vamos a hacer ahora es simple. Haremos que si el cuadrado se va por un borde, que aparezca por el lado contrario, como hacen los asteroides en el famoso juego *Asteroids*. La figura 3-9 muestra una captura de este juego.

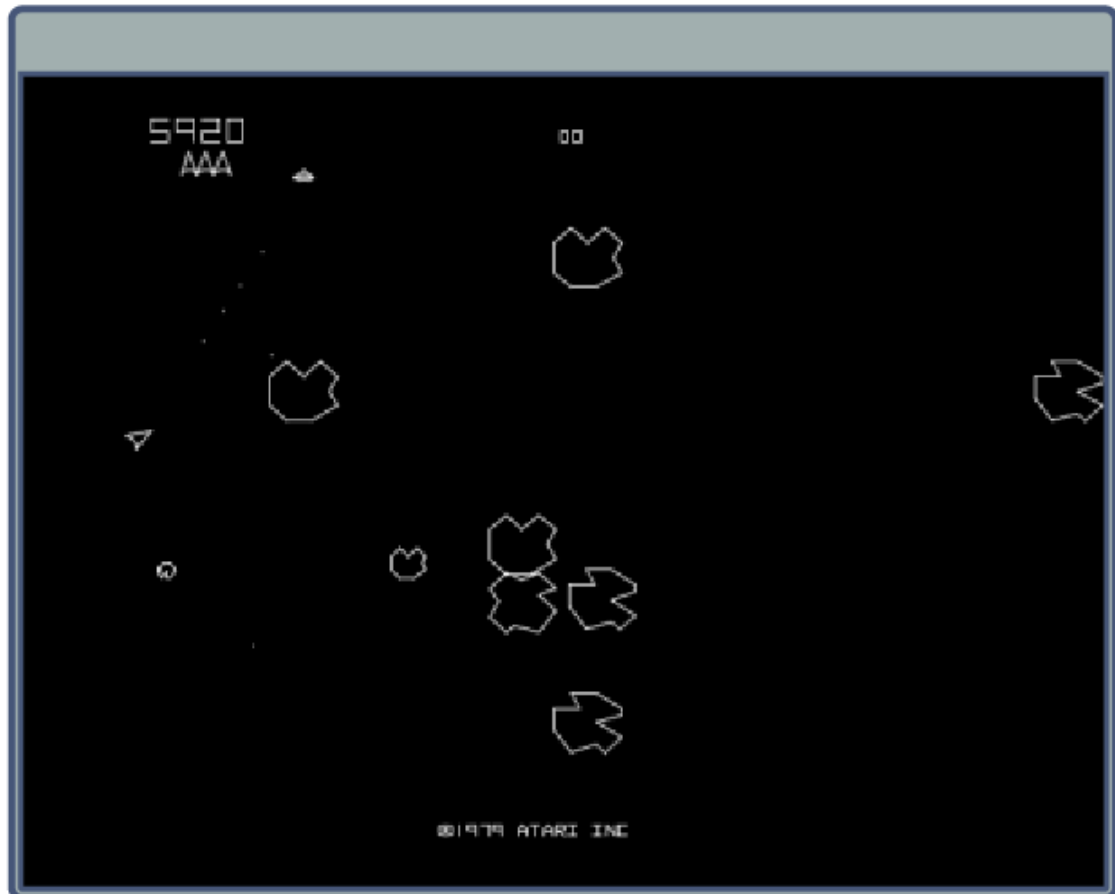


Figura 3-9: Asteroids © 1979 - Atari Inc.

Abre y ejecuta el ejemplo `capitulo_03\003_controlar_los_bordes`. En este ejemplo veremos que un cuadrado aparece desde la derecha y el otro desde la izquierda. Ambos se mueven en diagonal. Para lograr que los cuadrados se muevan en diagonal, lo que hacemos es modificar tanto la posición horizontal como la vertical, de la siguiente manera:

```
# Mover los cuadrados.  
cuadradoX += 2  
cuadradoY += 1
```

```
cuadrado2X -= 2
cuadrado2Y -= 1
```

De esta manera, el primer cuadrado (rojo) se moverá hacia la derecha 2 píxeles por frame, y hacia abajo 1 pixel por frame. El segundo cuadrado (amarillo) se moverá hacia la izquierda y hacia arriba (si tienes alguna duda sobre esto, mira el sistema de coordenadas para ver que restar en la vertical es “subir”). En la figura 3-10 vemos el movimiento en diagonal del primer cuadrado.

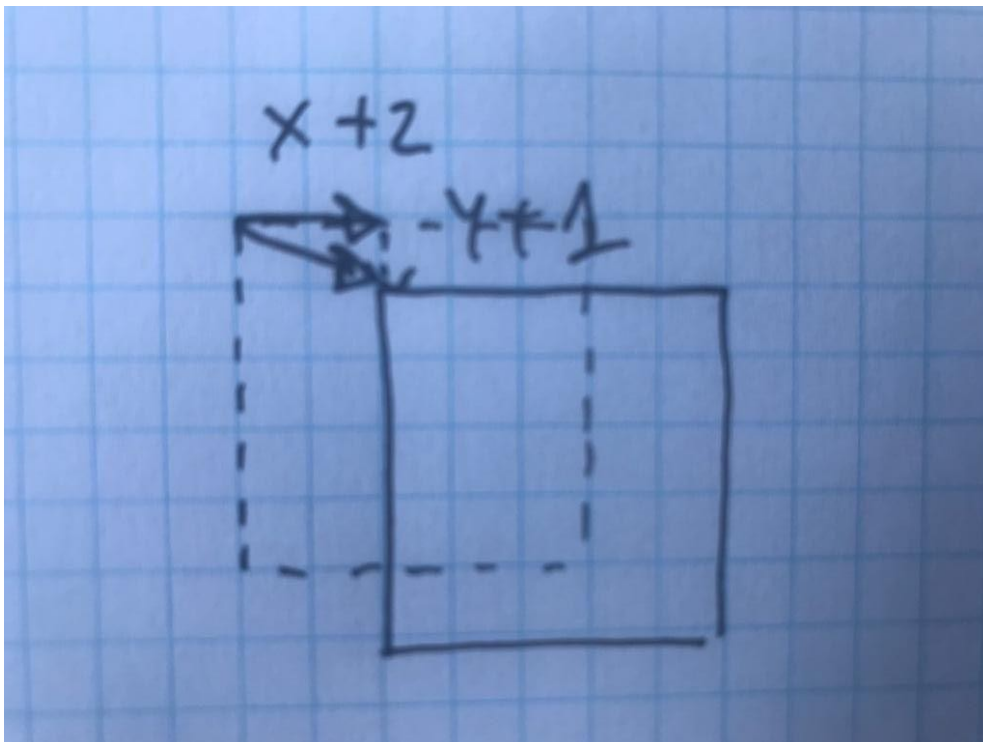


Figura 3-10: Movimiento del cuadrado en diagonal.

Cada vez que movemos los cuadrados, debemos chequear para ver si los mismos no han quedado ubicados fuera de la pantalla. En general, cada vez que un objeto del juego es movido, chequeamos las colisiones y controlamos que no se vayan de los bordes de la pantalla. Más adelante, para algunos objetos, preguntaremos si se fueron de los bordes para eliminarlos (por ejemplo, en el caso de las balas). Por ahora cuando un cuadrado se vaya de la pantalla, lo haremos aparecer desde el lado contrario.

Para controlar los bordes, agregamos en el *game loop*, el código de control de bordes, inmediatamente luego de modificar la posición de los cuadrados:

```
# Controlar los bordes del primer cuadrado.
if cuadradoX > screen.get_width():
    cuadradoX = 0
```

```

if cuadradoX < 0:
    cuadradoX = screen.get_width()
if cuadradoY > screen.get_height():
    cuadradoY = 0
if cuadradoY < 0:
    cuadradoY = screen.get_height()

# Controlar los bordes del segundo cuadrado.
if cuadrado2X > screen.get_width():
    cuadrado2X = 0
if cuadrado2X < 0:
    cuadrado2X = screen.get_width()
if cuadrado2Y > screen.get_height():
    cuadrado2Y = 0
if cuadrado2Y < 0:
    cuadrado2Y = screen.get_height()

```

Lo que hace este código es preguntar si la coordenada x del cuadrado es mayor que la coordenada x del borde derecho de la pantalla. Si ese es el caso, el cuadrado se ha salido de la pantalla y entonces lo colocamos en el borde izquierdo, o sea, en la coordenada $x = 0$. Esto es lo que causa que el objeto aparezca por el otro lado (aparecer por el otro lado de la pantalla se denomina *wrap*).

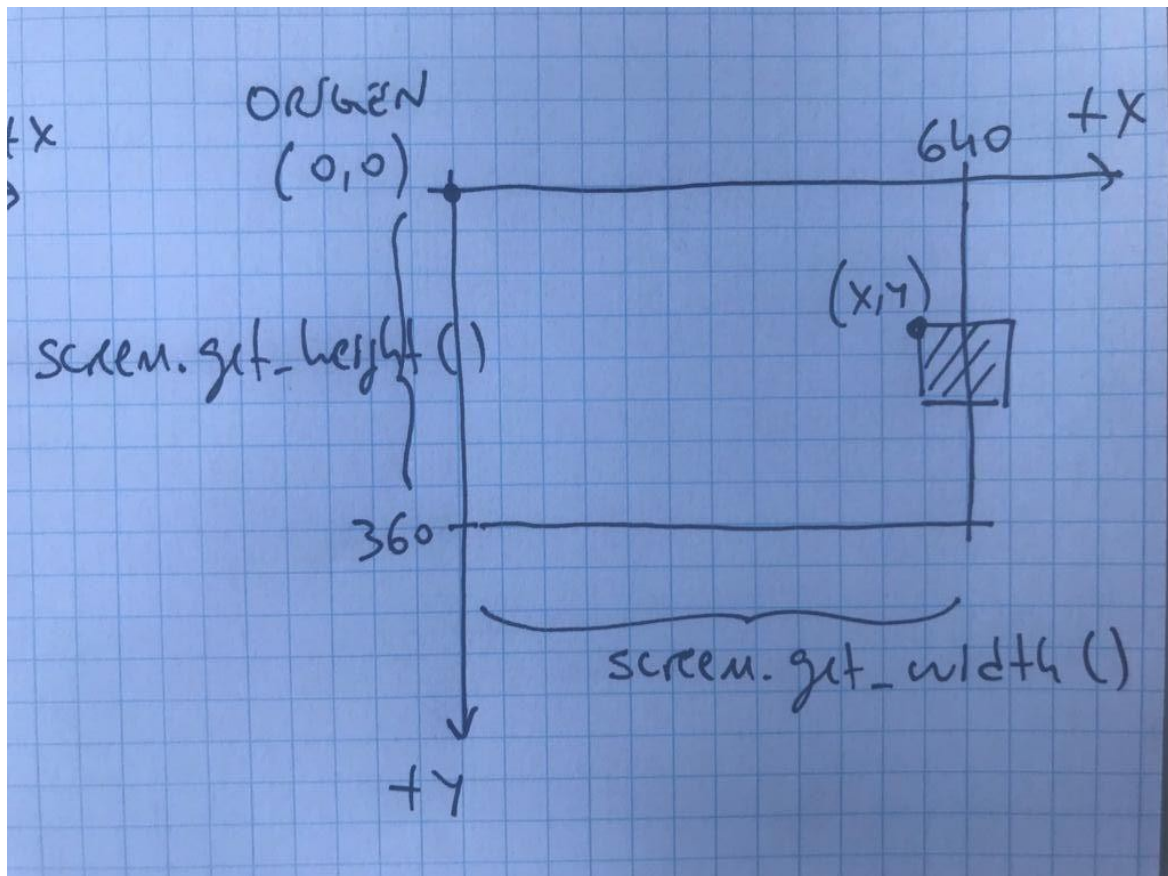


Figura 3-11: Chequeando los bordes de la pantalla.

En la figura 3-11 se muestra la situación que queremos controlar. La función `screen.get_width()` nos dice el ancho de la *ventana* (o *pantalla*) en píxeles. Este valor puede ser diferente según cómo hayamos inicializado la ventana (recordemos la función `pygame.display.set_mode()` a la cual le indicamos la resolución del juego). En esta figura el ancho de la pantalla es de 640 píxeles y el alto es de 360 píxeles.

```
# Controlar los bordes del primer cuadrado.  
if cuadradoX > screen.get_width():  
    cuadradoX = 0
```

Cuando se cumple la condición en la cual la coordenada x del cuadrado es superior a `screen.get_width()`, colocamos la coordenada x en cero, con lo cual el cuadrado se posiciona en el borde izquierdo de la pantalla. Esto se muestra en la figura 3-12.

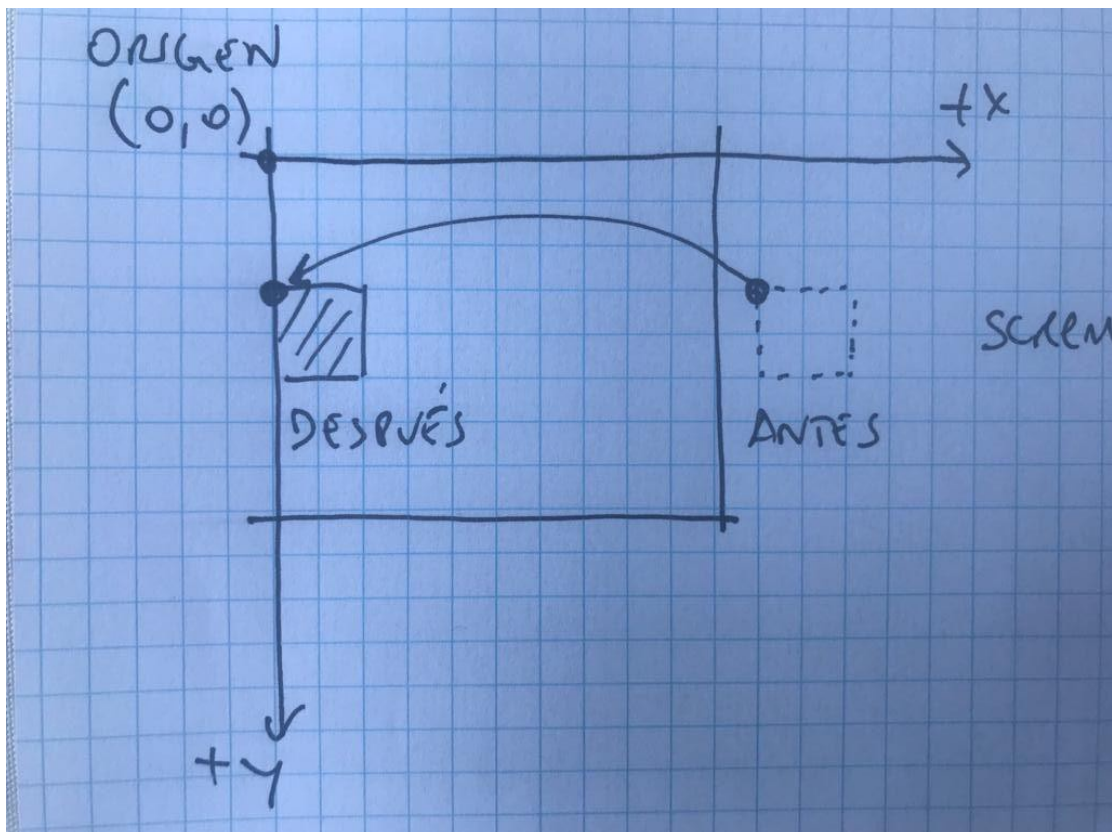


Figura 3-12: *Wrapping* por la derecha.

Del mismo modo, hacemos lo mismo para los bordes izquierdo, superior e inferior. Esto resulta en cuatro sentencias `if`, una para cada borde.

Cuando escribimos y ejecutamos el programa, debemos asegurarnos siempre de testear todos los casos. Para asegurar que los cuadrados hacen *wrap* en los cuatro bordes de la pantalla, debemos cambiar su movimiento en la horizontal y en la vertical, usando números positivos y luego negativos para que vaya para el otro lado (en las sentencias donde los cuadrados se mueven, en el *game loop*). Tenemos que probar bien todos los casos.

Si ejecutamos el programa, vemos que cuando el cuadrado va hacia la izquierda, apenas toca el borde izquierdo, aparece por el otro lado, pero cuando va hacia la derecha, el cuadrado se tiene que ir completamente de la pantalla para que aparezca por el otro lado. Esto es así por las condiciones que pusimos. Si quisiéramos que el cuadrado se vaya completamente de la pantalla, por ejemplo, cuando el cuadrado se va por la izquierda, debemos cambiar la condición para tener en cuenta el ancho del cuadrado (para dejar que se vaya completamente). Mira la figura 3-13 para entender lo que queremos hacer.

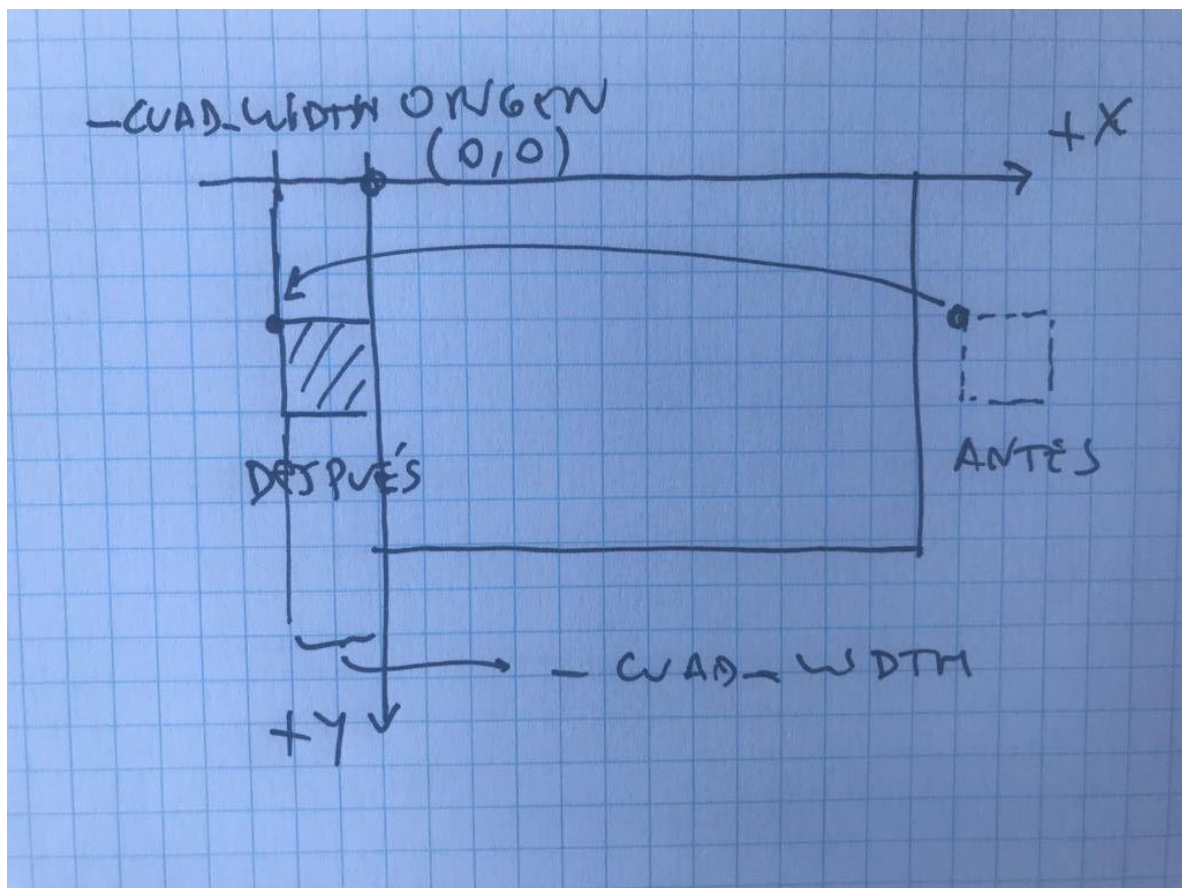


Figura 3-13: *Wrapping* de derecha a izquierda, corregido.

En el ejemplo `capitulo_03\004_controlar_los_bordes_v2` se encuentra hecha esta corrección. Mira el siguiente código:

```
# Ancho y alto del cuadrado.
CUAD_WIDTH = 32
CUAD_HEIGHT = 32
```

Primero definimos dos constantes con el ancho y el alto de los cuadrados. Esto es para no repetir los números en el código (recordemos que el uso de constantes deja más claro el código y permite entenderlo mejor).

```
# Controlar los bordes del primer cuadrado.
if cuadradoX > screen.get_width():
    cuadradoX = -CUAD_WIDTH
if cuadradoX < 0 - CUAD_WIDTH:
    cuadradoX = screen.get_width()
if cuadradoY > screen.get_height():
    cuadradoY = -CUAD_HEIGHT
if cuadradoY < 0 - CUAD_HEIGHT:
    cuadradoY = screen.get_height()

# Controlar los bordes del segundo cuadrado.
if cuadrado2X > screen.get_width():
    cuadrado2X = -CUAD_WIDTH
if cuadrado2X < 0 - CUAD_WIDTH:
    cuadrado2X = screen.get_width()
if cuadrado2Y > screen.get_height():
    cuadrado2Y = -CUAD_HEIGHT
if cuadrado2Y < 0 - CUAD_HEIGHT:
    cuadrado2Y = screen.get_height()
```

Este código es el código modificado para dejar que el cuadrado salga completamente de la pantalla antes de hacerlo entrar por el otro lado. Por ejemplo, cuando el cuadrado se va por el borde derecho, en lugar de ubicarlo en $x = 0$, lo que hacemos es posicionarlo en $x = -\text{CUAD_WIDTH}$, de forma tal que quede fuera de la pantalla, recostado contra el borde derecho, a punto de entrar a la pantalla.

Por ejemplo, cuando estamos chequeando si el cuadrado se va por el borde izquierdo de la pantalla, en lugar de preguntar si x es menor que cero, lo que hacemos es preguntar si x es menor que $-\text{CUAD_WIDTH}$, lo que significa chequear si el cuadrado se ha ido completamente de la pantalla.

Ejecuta nuevamente el ejemplo y mira si todo funciona correctamente. A continuación se lista el ejemplo completo con las líneas relacionadas a las constantes y al control de bordes resaltadas.

```
# -*- coding: utf-8 -*-
```

```
...
```

```

# Ancho y alto del cuadrado.
CUAD_WIDTH = 32
CUAD_HEIGHT = 32

# Crear un cuadrado de 32x32.
imgCuadrado = pygame.Surface((CUAD_WIDTH, CUAD_HEIGHT))
imgCuadrado = imgCuadrado.convert()
imgCuadrado.fill((255, 0, 0))

# Coordenadas del cuadrado.
cuadradoX = 0
cuadradoY = 180

# Crear otro cuadrado de 32x32.
imgCuadrado2 = pygame.Surface((CUAD_WIDTH, CUAD_HEIGHT))
imgCuadrado2 = imgCuadrado2.convert()
imgCuadrado2.fill((255, 255, 0))

# Coordenadas del segundo cuadrado.
cuadrado2X = 608
cuadrado2Y = 212

# Inicializar las variables de control del game loop.
clock = pygame.time.Clock()
salir = False

# Loop principal (game loop) del juego.
while not salir:

    # Timer que controla el frame rate.
    clock.tick(60)

    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():

        ...

    # Mover los cuadrados.
    cuadradoX += 2
    cuadradoY += 1

    cuadrado2X -= 2
    cuadrado2Y -= 1

    # Controlar los bordes del primer cuadrado.
    if cuadradoX > screen.get_width():
        cuadradoX = -CUAD_WIDTH
    if cuadradoX < 0 - CUAD_WIDTH:

```



```

        cuadradoX = screen.get_width()
    if cuadradoY > screen.get_height():
        cuadradoY = -CUAD_HEIGHT
    if cuadradoY < 0 - CUAD_HEIGHT:
        cuadradoY = screen.get_height()

    # Controlar los bordes del segundo cuadrado.
    if cuadrado2X > screen.get_width():
        cuadrado2X = -CUAD_WIDTH
    if cuadrado2X < 0 - CUAD_WIDTH:
        cuadrado2X = screen.get_width()
    if cuadrado2Y > screen.get_height():
        cuadrado2Y = -CUAD_HEIGHT
    if cuadrado2Y < 0 - CUAD_HEIGHT:
        cuadrado2Y = screen.get_height()

    # Dibujar el fondo.
    screen.blit(imgBackground, (0, 0))

    # Dibujar los cuadrados en la nueva posición.
    screen.blit(imgCuadrado, (cuadradoX, cuadradoY))
    screen.blit(imgCuadrado2, (cuadrado2X, cuadrado2Y))

    # Actualizar la pantalla.
    pygame.display.flip()

# Cerrar Pygame y liberar los recursos que pidió el programa.
pygame.quit()

```

Evitar Código Duplicado

Como podemos ver en el ejemplo, todo el código de control de bordes para el segundo cuadrado es idéntico al que tenemos para el primer cuadrado, salvo que usa las variables correspondientes al segundo cuadrado. Esta no es una buena práctica de programación. No es bueno que el código esté repetido. Lo hemos hecho así porque de alguna manera hay que empezar, pero ahora lo vamos a corregir.

Imagina si quisiéramos cambiar el control de bordes para los dos cuadrados. Deberíamos corregir el código dos veces. O peor aún, imagina si hubieran más cuadrados en el juego, el código en este caso quedaría muy extenso, dado que todo el bloque de control de bordes lo deberíamos repetir para cada cuadrado. Por este motivo, debemos evitar siempre tener código duplicado.

Lo que debemos hacer para evitar tener código duplicado, es programar una *clase*, a la que le llamaremos `CCuadrado`, que contendrá el código una vez sola, y también será más fácil de usar. Esto es lo que haremos en la siguiente sección.

Nota: A las clases le pondremos el prefijo “C” (por *Clase*) en el nombre para mostrar que es un nombre de *clase*. Esta es una nomenclatura bastante común en la *orientación a objetos* y la estaremos usando en este libro.

Escribiendo la Clase CCuadrado

Los videojuegos generalmente tienen varios objetos moviéndose en la pantalla. Hasta ahora en el juego tenemos dos cuadrados, y si vemos el código del ejemplo anterior, veremos que para cada cuadrado estamos usando tres variables: una variable con la imagen (superficie) y otras dos variables con las coordenadas (x, y).

Imagina si agregamos muchos cuadrados al juego. Si cada cuadrado tiene al menos tres variables, poner muchos cuadrados con tantas variables en el código haría muy largo el programa. Por esta razón (y por otras razones que ya veremos) es que debemos usar *objetos* y *clases* (técnica denominada *OOP: Programación Orientada a Objetos*). En este caso lo que vamos a hacer es una clase denominada `CCuadrado`. La idea detrás de esto es tener todo el comportamiento del cuadrado *encapsulado* (contenido) en una *clase* y luego utilizaremos una sola variable por cada cuadrado que haya en el juego, en lugar de tener todas las variables sueltas en el código. El cuadrado tendrá sus propias coordenadas dentro de él mismo (sus *propiedades* o *atributos* dentro de la *clase*), al igual que su comportamiento (en este caso, el comportamiento del cuadrado es moverse y chequear los bordes).

Ahora necesitamos comenzar a escribir la base de nuestra primera clase, en este caso, para representar un cuadrado. A la clase la llamaremos `CCuadrado`, e irá en un archivo `CCuadrado.py` (el nombre del archivo debe ser igual al nombre de la clase).

El cuadrado entonces tendrá como *atributos* (variables internas) las coordenadas x e y , y la imagen como hasta ahora. En la clase `CCuadrado` también pondremos atributos para tener las coordenadas de los límites de su movimiento, esto es, las coordenadas mínima y máxima en la horizontal y las coordenadas mínima y máxima en la vertical. También tendremos el ancho y el alto de la imagen (más adelante lo usaremos para detectar las colisiones) y el color que usamos para pintar el cuadrado. Todas estas variables internas del cuadrado son sus *propiedades*.

Abre el ejemplo ubicado en la carpeta `capitulo_03\005_la_clase_cuadrado` y ejecútalo. Verás el mismo comportamiento que en el ejemplo anterior, pero el programa ahora está armado utilizando una *clase* para representar el cuadrado.

Las funciones (denominadas *métodos*) que tendremos en la clase `CCuadrado` son: la función `__init__()` (denominada *constructor*), que se encarga de inicializar los atributos, la función `setXY()` para ubicar el objeto, la función `setBounds()` para definir los límites de su movimiento, la función `update()` que moverá el objeto (ejecuta su comportamiento) y por último la función `render()` que dibuja la imagen del cuadrado en la pantalla.

Entonces, la clase `CCuadrado` contiene la programación necesaria para implementar el *objeto* cuadrado. Esto supone definir sus propiedades e implementar sus métodos (escribir las funciones). La clase `CCuadrado` completa (en el archivo `CCuadrado.py`) se muestra en el siguiente listado:

```
# -*- coding: utf-8 -*-

#-----
# Clase CCuadrado.
# Simple cuadrado que se mueve por la pantalla chequeando los bordes.
#
# Autor: Fernando Sansberro - Batovi Games.
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

class CCuadrado(object):

    #-----
    # Constructor.
    # Parámetros:
    # aWidth: Ancho del cuadrado.
    # aHeight: Alto del cuadrado.
    # aColor: Color del cuadrado en formato RGB (r,g,b).
    #-----
    def __init__(self, aWidth, aHeight, aColor):

        # Coordenadas del cuadrado.
        self.mX = 0
        self.mY = 0

        # Imagen (superficie).
        self.mImg = None

        # Variables para controlar los bordes.
        self.mMinX = 0
        self.mMaxX = 0
        self.mMinY = 0
        self.mMaxY = 0

        # Ancho y alto.
        self.mWidth = aWidth
        self.mHeight = aHeight
```

```

# Color.
self.mColor = aColor

# Crear la superficie y llenarla con el color.
self.mImg = pygame.Surface((aWidth, aHeight))
self.mImg = self.mImg.convert()
self.mImg.fill(aColor)

# Establece la posición del objeto.
# Parámetros:
# aX, aY: Coordenadas x e y del objeto.
def setXY(self, aX, aY):

    self.mX = aX
    self.mY = aY

# Define los límites del movimiento del objeto.
# Parámetros:
# aMinX, aMinY: Coordenadas x e y mínimas del mundo.
# aMaxX, aMaxY: Coordenadas x e y máximas del mundo.
def setBounds(self, aMinX, aMinY, aMaxX, aMaxY):

    self.mMinX = aMinX
    self.mMaxX = aMaxX
    self.mMinY = aMinY
    self.mMaxY = aMaxY

# Mover el objeto.
# Parámetros:
# aIncX: Cantidad de píxeles que se mueve en la horizontal.
# aIncY: Cantidad de píxeles que se mueve en la vertical.
def update(self, aIncX, aIncY):

    # Mover el objeto.
    self.mX += aIncX
    self.mY += aIncY

    # Controlar los bordes haciendo wrap.
    if self.mX > self.mMaxX:
        self.mX = self.mMinX
    if self.mX < self.mMinX:
        self.mX = self.mMaxX
    if self.mY > self.mMaxY:
        self.mY = self.mMinY
    if self.mY < self.mMinY:
        self.mY = self.mMaxY

# Dibuja el objeto en la pantalla.

```

```
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    aScreen.blit(self.mImg, (self.mX, self.mY))
```

Nota: La función `__init__()` lleva dos caracteres *underscore* (`_`) de cada lado. Ten cuidado con esto porque puede no notarse bien en el texto. Si ponemos más o menos caracteres el programa dará un error.

Nota: Como vemos en el código, todos los parámetros de las funciones tienen una “a” como prefijo del nombre, y una “m” como prefijo de los atributos (a las variables dentro de una clase se le denomina variables *miembro* de la clase). Esta es una regla de *nomenclatura* del código (una forma de nombrar las cosas) que hace más legible el programa. Es la nomenclatura que usaremos en este libro.

Nota: En *Python*, todas las funciones de una clase deben tener como primer parámetro la referencia `self`, que es una *referencia* al objeto, y se debe utilizar siempre que se accedan a variables miembro (de lo contrario no se reconocerían).

Una vez que tenemos la clase `CCuadrado` pronta, desde la clase principal (`main.py`) es mucho más simple manejar los objetos. El código de `main.py` queda bastante más corto, debido a que el código que estaba duplicado ya se encuentra ahora una vez sola dentro de la clase `CCuadrado`.

Veamos a continuación el código de `main.py`. Se han resaltado las líneas que cambian.

```
...
```

```
# Importar Pygame.
import pygame
```

```
# Importar la clase CCuadrado.
from CCuadrado import *
```

```
# Inicializar Pygame.
pygame.init()
```

```
# Control del modo ventana o fullscreen.
RESOLUTION = (800, 600)
isFullscreen = False
```

```
# Poner el modo de video en ventana e indicar la resolución.
screen = pygame.display.set_mode(RESOLUTION)
# Poner el título de la ventana.
pygame.display.set_caption("Mi Juego")
```

```

# Crear la superficie del fondo o background.
imgBackground = pygame.Surface(screen.get_size())
imgBackground = imgBackground.convert()
imgBackground.fill((0, 0, 255))

# Ancho y alto del cuadrado.
CUAD_WIDTH = 32
CUAD_HEIGHT = 32

# Crear los cuadrados: ancho, alto, color (RGB).
c1 = CCuadrado(CUAD_WIDTH, CUAD_HEIGHT, (255, 0, 0))
c2 = CCuadrado(CUAD_WIDTH, CUAD_HEIGHT, (255, 255, 0))

# Colocar los cuadrados en su posición inicial.
c1.setXY(0, 300)
c2.setXY(768, 400)

# Marcar los límites del mundo.
c1.setBounds(-CUAD_WIDTH, -CUAD_HEIGHT, screen.get_width(),
             screen.get_height())
c2.setBounds(-CUAD_WIDTH, -CUAD_HEIGHT, screen.get_width(),
             screen.get_height())

# Inicializar las variables de control del game loop.
clock = pygame.time.Clock()
salir = False

# Loop principal (game loop) del juego.
while not salir:

    # Timer que controla el frame rate.
    clock.tick(30)

    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():

        ...

    # Mover los cuadrados y controlar los bordes.
    c1.update(2, 1)
    c2.update(-2, -1)

    # Dibujar el fondo.
    screen.blit(imgBackground, (0, 0))

    # Dibujar los cuadrados en la nueva posición.
    c1.render(screen)

```

```
c2.render(screen)
```

```
# Actualizar la pantalla.  
pygame.display.flip()
```

```
# Cerrar Pygame y liberar los recursos que pidió el programa.  
pygame.quit()
```

Como vemos en el código del programa principal, la primera línea importa la clase `CCuadrado`, lo cual siempre es necesario hacer si queremos usar la clase. Para esto usamos la sentencia:

```
# Importar la clase CCuadrado.  
from CCuadrado import *
```

Luego, en la parte de inicialización, se crean dos objetos de la clase `CCuadrado` (a esto se le denomina *instanciar*, o *crear* los objetos):

```
# Ancho y alto del cuadrado.  
CUAD_WIDTH = 32  
CUAD_HEIGHT = 32  
  
# Crear los cuadrados: ancho, alto, color (RGB).  
c1 = CCuadrado(CUAD_WIDTH, CUAD_HEIGHT, (255, 0, 0))  
c2 = CCuadrado(CUAD_WIDTH, CUAD_HEIGHT, (255, 255, 0))
```

Definimos dos constantes para tener el ancho y el alto de los cuadrados y esos son los parámetros que se le pasan a la función que crea los objetos. A esta función encargada de crear el objeto se le denomina *método constructor* del objeto (en este caso `CCuadrado()` que invoca a `__init__()` en la clase `CCuadrado`).

Los *parámetros* que se le pasan al constructor de `CCuadrado` son el ancho, el alto y el color, que se define como una tupla con los valores rojo, verde y azul (color *RGB*). Esto se puede hacer de la forma que se prefiera. Lo hemos hecho así para poder tener cuadrados de diferentes tamaños y colores.

Luego de creados los dos objetos, inicializamos su posición y establecemos los límites por donde se podrán mover:

```
# Colocar los cuadrados en su posición inicial.  
c1.setXY(0, 300)  
c2.setXY(768, 400)
```

```
# Marcar los límites del mundo.
c1.setBounds(-CUAD_WIDTH, -CUAD_HEIGHT, screen.get_width(),
             screen.get_height())
c2.setBounds(-CUAD_WIDTH, -CUAD_HEIGHT, screen.get_width(),
             screen.get_height())
```

Nota: Cuando a los objetos del juego le marcamos los límites para que no se vayan de la pantalla, en realidad estamos indicando los límites del mundo, esto es, las coordenadas mínima y máxima que el objeto puede tener, tanto en la vertical como en la horizontal. En los primeros juegos que hagamos, las coordenadas de pantalla y del mundo serán las mismas, pero esto no siempre es así (por ejemplo en los juegos con desplazamiento de pantalla, o *scrolling*, donde el mundo es más grande que la pantalla).

Finalmente, en el *game loop*, lo que se hace es invocar a las funciones `update()` y `render()` para cada cuadrado. La función `update()` mueve el objeto y la función `render()` lo dibuja en su nueva posición. El ciclo *update/render* lo utilizaremos para todos los objetos que tenga nuestro juego:

```
# Loop principal (game loop) del juego.
while not salir:

    ...

    # Mover los cuadrados y controlar los bordes.
    c1.update(2, 1)
    c2.update(-2, -1)

    # Dibujar el fondo.
    screen.blit(imgBackground, (0, 0))

    # Dibujar los cuadrados en la nueva posición.
    c1.render(screen)
    c2.render(screen)

    ...
```

Nota: La *Programación Orientada a Objetos (OOP)* es fundamental en el desarrollo de videojuegos. Si lo pensamos, es muy probable que cada elemento en un juego se corresponda con una *clase*. Cuando hagamos nuestro juego, tendremos una *clase* para el jugador, otra para los enemigos, otra para las balas, etc. Si tienes dudas sobre cómo escribir *clases* y utilizarlas, debes consultar los conceptos sobre programación *OOP* de *Python* o pedirle ayuda a un experto.

Nota: Si te fijas en la clase `CCuadrado`, para referirse a las propiedades de la clase,

utilizamos el operador `self`. Este operador quiere decir “yo mismo”. Si no utilizamos el operador `self` para referirnos a una *propiedad*, estaremos diciendo que es otra variable, lo que seguro ocasionará un error.

Con esto terminamos nuestro primer objeto que se mueve por la pantalla. Pero como los juegos no se basan en cuadrados, en el siguiente capítulo aprenderemos a manejar imágenes, de forma de poder poner gráficos y que nuestros objetos en el juego se vean lindos.

4. Manejo de Imágenes

Hasta ahora hemos utilizado cuadrados en lugar de imágenes, pero esto se puede ver como aburrido. Un videojuego siempre debe tener imágenes y de hecho debe tener un excelente arte. En este capítulo comenzaremos a incorporar imágenes a nuestro juego.

Nota: Cuando comenzamos a desarrollar un videojuego, no siempre tendremos el arte disponible, generalmente porque el artista está en proceso de hacerlo. El programador debe utilizar gráficos temporales para poder programar el juego, mientras no tiene los gráficos finales. A los gráficos o arte en general que es temporal, se le denomina *placeholder*, y es una práctica habitual que el programador desarrolle el juego con placeholders hasta que el arte final sea integrado en el juego.

Para manejar imágenes, utilizaremos el módulo `Image` de *Pygame*. Comenzaremos viendo qué son las superficies y cómo se crean, y luego veremos el manejo de colores y el manejo de las imágenes.

Usando Imágenes

Las imágenes son parte vital de un videojuego. Cuando estamos jugando un juego 2D, vemos imágenes representando al fondo (el escenario), al jugador, a los enemigos, a la interfaz del juego, etc. Si se tratara de un videojuego 3D, las imágenes son utilizadas, por ejemplo, en las texturas de los objetos para crear una escena.

Para crear las imágenes de un videojuego se utiliza cualquier software para la creación de gráficos. La imagen es almacenada en un archivo, que cargaremos en el programa y mostraremos en la pantalla.

Una imagen es almacenada en un archivo o en memoria, como una matriz de píxeles, cada uno de un color determinado. La forma en la que se guarda la información de los píxeles de la imagen se denomina *formato de la imagen*. Algunas imágenes pueden tener más información que otras para almacenar un píxel. Por ejemplo, una imagen puede tener un formato *RGB* para cada píxel, mientras que otra imagen puede tener un formato *RGBA* (*Red*, *Green*, *Blue*, *Alpha*), añadiendo un cuarto componente para la *transparencia* (*alpha*).

Durante la historia, se han desarrollado muchos formatos para guardar imágenes, cada formato con sus pros y sus contras. En el desarrollo de videojuegos, los más usados son el formato *JPEG* y *PNG*. Ambos formatos se pueden utilizar en la inmensa mayoría de los software de edición de gráficos.

- *PNG* (*Portable Network Graphics*): Los archivos *PNG* (tienen extensión `.png`) son los más utilizados en videojuegos porque tienen muy buena compresión (la imagen tiene un menor tamaño en bytes). El formato *PNG* puede tener transparencia y esto es muy usado en los

juegos. La única desventaja es que un archivo *PNG* suele ser más grande que un archivo *JPG*.

- *JPG (Joint Photographic Expert Group)*: Los archivos *JPG* (tienen extensión *.jpg* o *.jpeg*) fueron diseñados para almacenar imágenes fotográficas. Comprimen mucho más que los archivos *PNG*, por lo cual son de bastante menos tamaño, pero con la desventaja de que las imágenes se ven un poco peor de calidad, a causa de la compresión usada (generalmente la diferencia de calidad es muy sutil).

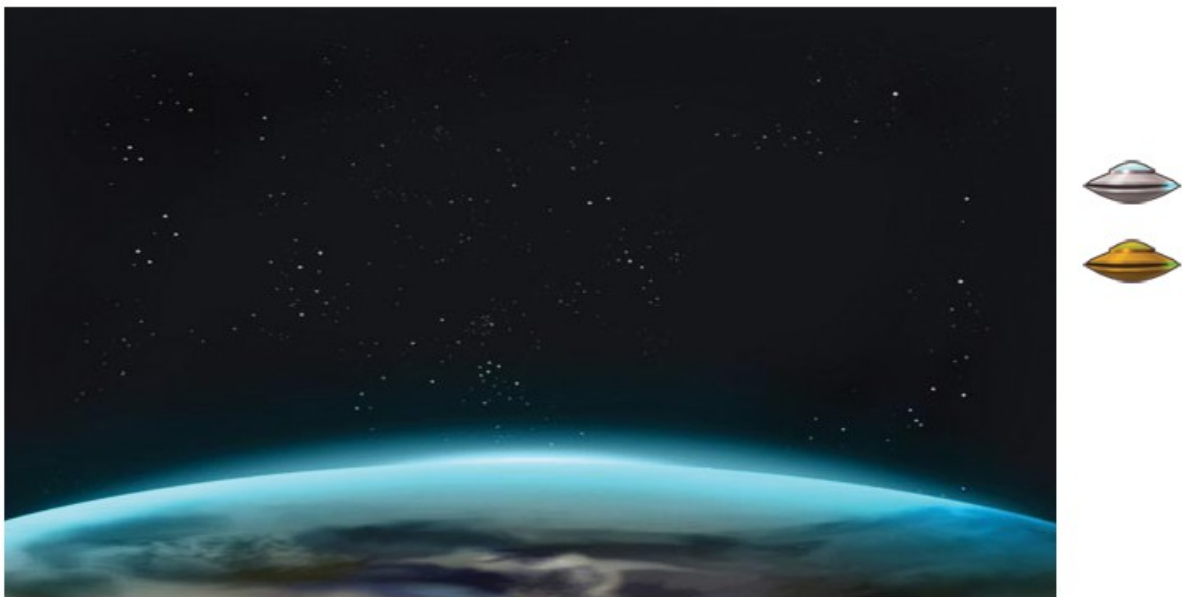


Figura 4-1: Imágenes que usaremos en nuestro juego.

Además de los formatos *PNG* y *JPG*, *Pygame* soporta otros formatos como *GIF*, *BMP*, *PCX*, *TGA*, *TIFF* etc.

En nuestros juegos, seguramente usemos imágenes en formato *JPG* para los fondos y para las imágenes que sean muy grandes. y utilizaremos imágenes en formato *PNG* para todos los objetos del juego como el personaje, los enemigos, etc. (generalmente denominados los sprites del juego).

Nota: El proyecto “*Hacete tu Videojuego*” del cual es parte este libro, contiene gráficos que puedes utilizar en tu propio juego. También es posible descargar arte de uso libre (*open source*) de los sitios listados en la carpeta que contiene estos assets y que viene con los materiales del libro.

Crear una Superficie

Una superficie es un espacio en la memoria de la computadora conteniendo una imagen.

Como todo lo que está en la memoria, una imagen es un área de bytes, conteniendo en este caso la información de los componentes de cada píxel de la imagen, un byte tras otro.

Podemos crear una superficie para luego dibujar en ella, o podemos cargar un archivo de imagen desde el *disco* o *drive*. Cuando cargamos una imagen, se crea una superficie en memoria conteniendo la información de la imagen que se lee desde el disco.

Como ya hemos visto cuando hicimos la clase `CCuadrado`, podemos crear una superficie (o sea, una imagen) haciendo lo siguiente:

```
self.mImg = pygame.Surface((aWidth, aHeight))
self.mImg = self.mImg.convert()
self.mImg.fill(aColor)
```

Lo que hace la función `pygame.Surface()` es crear una superficie con un tamaño dado por el ancho y el alto que le pasamos como parámetro en una tupla. Sin otros parámetros aparte del ancho y alto, esta función creará una superficie con el mismo formato del display, que generalmente esto es lo que queremos hacer, dado que es mucho más rápido de dibujar las imágenes que están en el mismo formato que el display.

Por ejemplo, el siguiente código crea una superficie de 256 x 256 píxeles (debemos pasarle como parámetro la tupla con el ancho y el alto):

```
imgTemp = pygame.Surface((256, 256))
imgTemp = imgTemp.convert()
```

Es posible crear superficies de cualquier tamaño, con tal que haya espacio suficiente en la memoria de la computadora para almacenarla. La función `convert()` lo que hace es convertir el formato de la imagen creada al formato que tenga el display. Siempre haremos esto porque es más rápido dibujar una imagen en otra si ambas tienen el mismo formato. Si no lo tuvieran, *Pygame* tiene que hacer la conversión cada vez que va a dibujar. Más adelante veremos más detalles sobre cómo convertir el formato de una imagen.

Con la función `fill()` se llena la superficie con un color. De esta forma hemos dibujado antes un cuadrado amarillo o rojo. A continuación, veremos cómo indicar los colores.

Los Colores en *Pygame*

Cuando creamos el cuadrado en los ejemplos anteriores, le pasamos una tupla con valores *RGB* a la función `fill()` para que llene la imagen con ese color. Ahora veremos cómo funcionan los colores en formato *RGB*.

Cuando tenemos que indicar un color, pasamos una tupla con tres valores enteros, uno para cada componente del color. Los tres componentes de un color son el rojo, el verde y el azul (*RGB: red, green, blue*). Cada uno de estos números puede estar entre 0 y 255, donde 0 significa ausencia del componente y 255 significa el máximo de ese componente (su máxima intensidad).

El color formado se da por la intensidad del componente rojo, el componente verde y el componente azul combinados. Cuanto más de un componente o de otro tenga la tupla, más rojo, verde o azul será el color (o la combinación de ellos).

En la siguiente tabla se ven los colores típicos:

Color	Red	Green	Blue	Tupla
 Negro - Black	0	0	0	(0, 0, 0)
 Azul - Blue	0	0	255	(0, 0, 255)
 Verde - Green	0	255	0	(0, 255, 0)
 Celeste - Cyan	0	255	255	(0, 255, 255)
 Rojo - Red	255	0	0	(255, 0, 0)
 Púrpura - Magenta	255	0	255	(255, 0, 255)
 Amarillo - Yellow	255	255	0	(255, 255, 0)
 Blanco - White	255	255	255	(255, 255, 255)

Figura 4-2: Tabla con los colores básicos.

Cuando vamos a establecer un color, debemos probar cambiando los valores *R*, *G* y *B* hasta conseguir el color deseado. Requiere práctica conseguir el color que deseamos y lo mejor es utilizar cualquier programa de dibujo que muestre los valores *R*, *G* y *B* mientras los vemos en un selector de colores. Una vez que tenemos los valores de los tres componentes, los copiamos y los pegamos en el código.

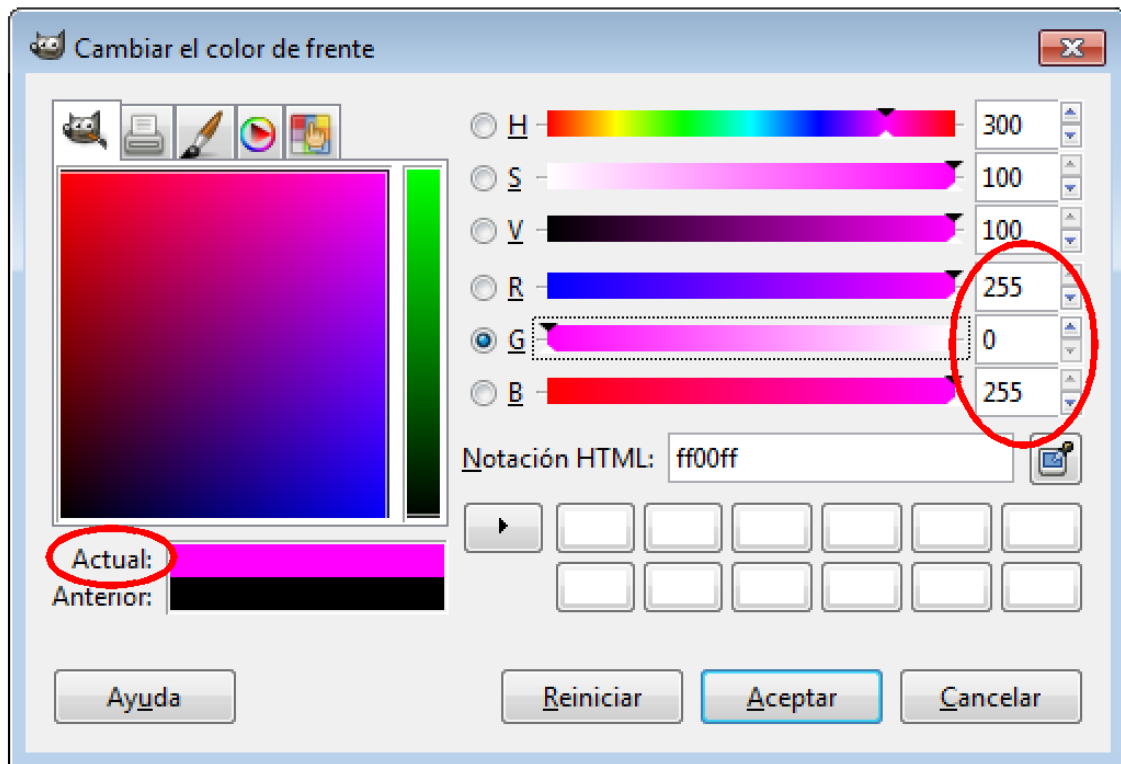


Figura 4-3: Selector de colores de GIMP.

En la figura 4-2 se muestra el selector de colores del programa Gimp, en el cual se muestra el color seleccionado actualmente y los tres valores de los componentes *RGB* (indicados en el dibujo). Esos tres valores son los que utilizaremos en el programa para indicar el color deseado.

Cargar una Imagen

Generalmente, cargaremos imágenes ya hechas por el artista y utilizaremos funciones de *Pygame* para cargar estas imágenes. Para mostrar imágenes en la pantalla, lo primero que haremos es cargarlas a la memoria desde un archivo *PNG* o *JPG*, y luego convertirlas al formato de la pantalla. Una vez hecho eso, las imágenes ya se podrán dibujar en la pantalla con la función `blit()`.

Cuando estamos desarrollando un juego, el artista nos da las imágenes de los elementos del juego y el programador se encarga de ponerlas en el proyecto y programar la lógica para que las cosas sucedan. Una vez que tenemos la imagen que nos entrega el artista, la colocamos en la carpeta (*folder*) correspondiente del proyecto para que el programa pueda acceder a ella.

Lo que haremos ahora es poner como fondo para nuestro juego una imagen del espacio. Mira el ejemplo que se encuentra en la carpeta `capitulo_04\001_cargar_imagen_de_fondo`. Al correr este ejemplo vemos que

aparece de fondo una imagen de fondo con la tierra y el espacio.

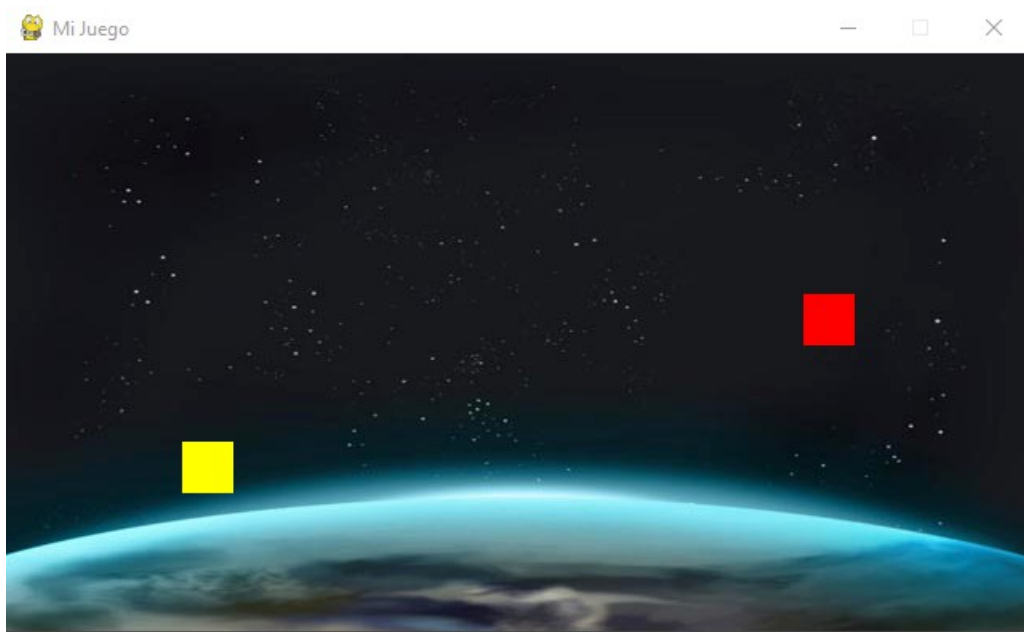


Figura 4-4: El juego con la imagen de fondo.

En la carpeta del ejemplo hay una imagen de nombre `space_640x360.jpg` que es la que el programa carga y muestra en la pantalla. El programa utiliza las siguientes sentencias para cargar la imagen:

```
# Cargar la imagen del fondo. La imagen es de 640 x 360.
imgSpace = pygame.image.load("space_640x360.jpg")
imgSpace = imgSpace.convert()
```

La función `pygame.image.load()` carga una imagen (que puede estar en cualquiera de los formatos comunes: *PNG*, *JPG*, *GIF* o *BMP*) y nos retorna una superficie con la imagen, cosa que guardamos en la variable correspondiente (denominada `imgSpace`). A la función le pasamos como parámetro la ruta donde se encuentra la imagen y el nombre del archivo. Si solamente pasamos el nombre de la imagen, ésta será buscada en la carpeta del programa.

La función `pygame.image.load()` lo que hace es cargar la imagen que está contenida en el archivo `space_640x360.jpg`. Como no le indicamos la ruta del archivo (solamente ponemos el nombre del archivo), lo toma desde la misma carpeta en la cual está el programa. Más adelante haremos carpetas para guardar imágenes, sonidos, niveles, y otros tipos de *assets*, pero por ahora como tenemos pocos archivos los pondremos todos juntos en la misma carpeta.

La segunda línea: `imgSpace = imgSpace.convert()` lo que hace es convertir la imagen al mismo formato que tiene la pantalla. Esto se debe hacer porque los archivos *JPG*, *GIF* o

PNG tienen compresión y *Pygame* necesita utilizarlos sin compresión cuando los va a mostrar en la pantalla. Si no convertimos la imagen cuando la cargamos, esto lo tendría que hacer *Pygame* cada vez que se va a dibujar, lo cual haría que todo funcione más lento. Por esta razón es que siempre luego de cargar o crear una imagen, se debe llamar a la función `convert()`.

Ahora, luego que tenemos cargada la imagen en memoria y convertida al formato de la pantalla, la podemos dibujar en cualquier momento.

Nota: La variable `imgSpace` es una variable que apunta al área de memoria de la máquina en donde se encuentra la imagen cargada. Internamente esa variable es una *referencia* a la imagen, o en otros lenguajes, como en el lenguaje C, se dice que es un *puntero* a la imagen. Esto significa que cada vez que usemos esta variable nos estamos refiriendo al área a la que apunta. En este caso, es una imagen.

Cuando cargamos la imagen del fondo, lo hacemos sin transparencia. Las imágenes que usamos de fondo no tienen transparencia, dado que no hay nada detrás de ellas.

Dibujar una Imagen

Una vez que tenemos la imagen cargada en memoria (y una variable representando o referenciando a la imagen), usamos la función `blit()` para mostrar la imagen. Esta función se utiliza aplicada a la imagen en donde vamos a dibujar (la imagen de destino).

```
# Dibujar la imagen cargada en la imagen de background.  
imgBackground.blit(imgSpace, (0, 0))
```

En este caso, la imagen de destino es la imagen de fondo. Por eso usamos `imgBackground.blit()`. La función se aplica a la imagen en donde queremos dibujar. Se le pasa como parámetro la imagen a dibujar y la coordenada donde se ubica la esquina superior izquierda. Como la imagen a dibujar tiene el mismo tamaño que la pantalla, dibujamos en la esquina superior izquierda de la pantalla, cuya coordenada es la tupla `(0, 0)`.

Blit significa *binary block transfer* y significa copiar píxel por píxel una imagen a una superficie. Es una técnica para copiar zonas rectangulares de una parte a otra, generalmente entre superficies o de una superficie a la pantalla. Esta operación viene incluida en todas las tarjetas de video. La acción de copiar una imagen en una nueva ubicación se denomina *blitting*.

Entonces, para dibujar una imagen (origen) en otra (destino), utilizamos la función `blit()`, que tiene la siguiente sintaxis: `(imgDestino.blit(imgOrigen, (x, y))`. La figura 4-5 muestra gráficamente cómo se copia píxel a píxel una imagen en otra.

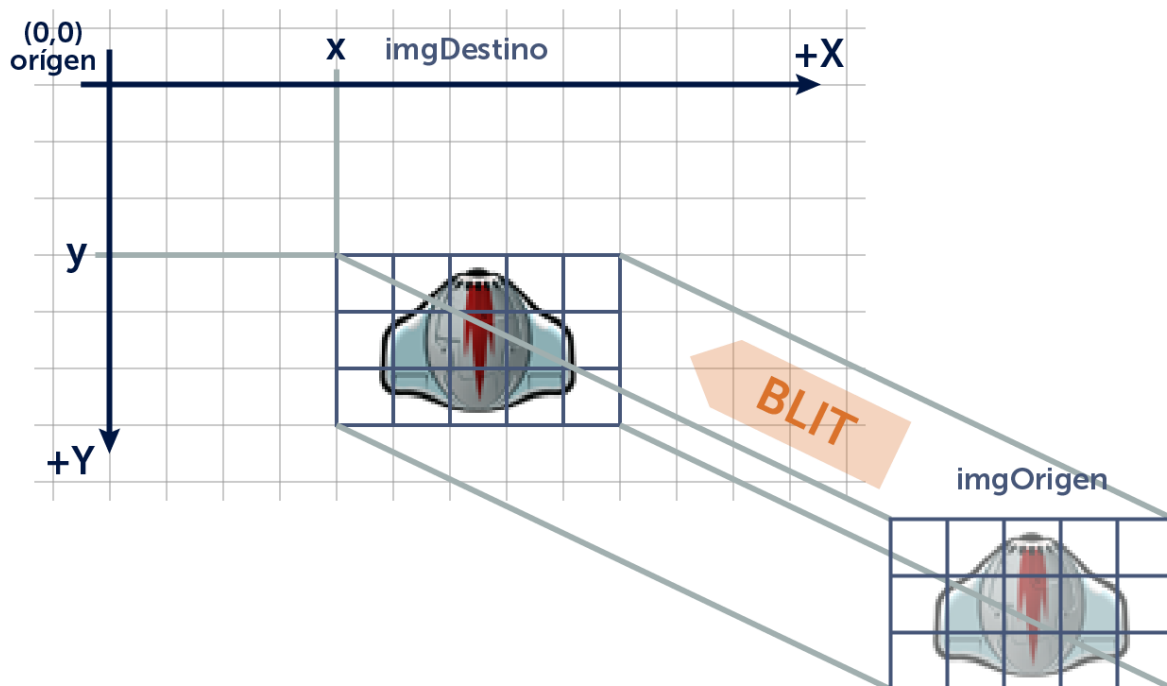


Figura 4-5: *Blit* de una imagen en otra.

Nota: Los ejemplos fueron escritos para funcionar en modo ventana en una resolución de 640 x 360 píxeles. Esto se ha definido así por dos motivos. Primero porque se utiliza la técnica *pixel art* y en esta técnica, las resoluciones son bajas. Segundo, porque es una resolución con formato de pantalla *wide*, que se podrá adaptar bien a los monitores modernos.

Nota: En los ejemplos recuerda que se puede poner en modo *fullscreen* (pantalla completa) pulsando la tecla *[F]*. Si en modo *fullscreen* la pantalla no se ve completa (en algunas tarjetas de video o monitores), no importa, en este punto debemos concentrarnos en aprender cómo programar videojuegos. Más adelante haremos que el juego funcione en todas las resoluciones posibles.

Lo siguiente que haremos ahora que tenemos el fondo del juego es agregar naves enemigas. Las naves del juego, como tienen partes transparentes, las debemos cargar con transparencia.

En realidad lo que haremos ahora será cambiar los cuadrados por naves enemigas. Para esto haremos otra clase denominada *CNave*, que será como la clase *CCuadrado*, pero en lugar de crear una superficie y llenarla de un color, ahora cargará una imagen. Esto es lo que haremos a continuación.

Cargar Imágenes con Transparencia

Una imagen con transparencia se usa cuando los objetos tienen partes transparentes, de

forma tal que se pueden dibujar sobre otras imágenes de fondo y se ve a través de partes de ella. Por ejemplo, cuando dibujamos una de las naves enemigas sobre el fondo del juego, si no se usa una imagen con transparencia para la nave, veríamos un feo rectángulo alrededor de ella. Si cargamos una imagen y ocurre esto, significa que no hemos cargado la imagen correctamente, o bien que la imagen no tiene transparencia.



Figura 4-6: Las naves del juego son imágenes con transparencia.

Nota: Si queremos usar una imagen con transparencia en el juego, debemos asegurarnos que la imagen a cargar ya tenga transparencia. Si este no fuera el caso, hay que utilizar un software gráfico como *GIMP* (www.gimp.org) y agregar el canal *alpha* (canal de transparencia) a la imagen.

Ahora lo que haremos es cambiar los cuadrados que tenemos en el ejemplo anterior, por naves espaciales. Para eso, ponemos los dos gráficos de las naves en la carpeta del proyecto. Los archivos tienen como nombres: `grey_ufo.png` y `yellow_ufo.png`.

Lo que haremos ahora es hacer una clase denominada `CNave` que será similar a la clase `CCuadrado`, salvo que en lugar de crear una superficie y llenarla de un color, va a cargar la imagen de la nave.

Mira el código del ejemplo que se encuentra en la carpeta `capitulo_04\002_clase_nave`, y ejecútalo. Verás dos naves enemigas moviéndose de igual forma que lo hacían los cuadrados.

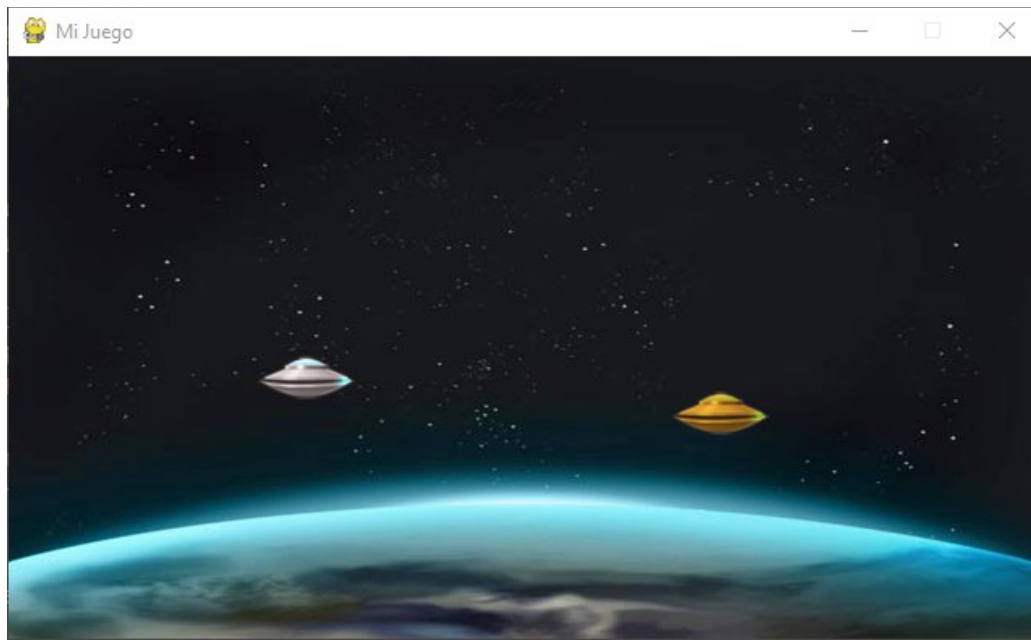


Figura 4-7: El juego ahora tiene imágenes para el fondo y para las naves.

La clase `CCuadrado` la copiamos y la nombramos como clase `CNave`. El código de la nave enemiga hace lo mismo que hacían los cuadrados. La diferencia está en el constructor, que carga la imagen de la nave (es una imagen con transparencia). El constructor de la clase `CNave` se muestra a continuación (el resto del código es igual que lo que había en la clase `CCuadrado`):

```
...

# -----
# Constructor.
# Parámetros:
# aImgFile: Nombre de la imagen a cargar.
# -----
def __init__(self, aImgFile):

    # Coordenadas de la nave.
    self.mX = 0
    self.mY = 0

    # Imagen (superficie).
    self.mImg = None

    # Variables para controlar los bordes.
    self.mMinX = -float("inf")
    self.mMaxX = float("inf")
    self.mMinY = -float("inf")
    self.mMaxY = float("inf")
```

```

# Cargar la imagen.
self.mImg = pygame.image.load(aImgFile)
self.mImg = self.mImg.convert_alpha()

# Guardar el ancho y el alto.
self.mWidth = self.mImg.get_width()
self.mHeight = self.mImg.get_height()

...

```

En el constructor de la clase `CNave` se recibe como parámetro el nombre del archivo que contiene la imagen de la nave y que se carga en memoria con la función `pygame.image.load()`. Luego se convierte la imagen al formato de la pantalla usando la función `convert_alpha()`, en lugar de usar `convert()` como hicimos antes. Esto hay que hacerlo así porque las imágenes de las naves tienen transparencia.

Si pruebas usando la función `convert()` en lugar de `convert_alpha()` verás que quedan las áreas que son transparentes como blancas y se ve el rectángulo alrededor de la nave. Cuando se carga una imagen que tiene transparencia, siempre hay que convertirla al formato de la pantalla usando `convert_alpha()` para que la transparencia funcione.

Por último, se guarda el ancho y el alto de la imagen utilizando las funciones `get_width()` y `get_height()` de la imagen. Ya no es necesario pasarle como parámetro el ancho y el alto como hacíamos con los cuadrados, porque ahora, el ancho y el alto se toman de la imagen misma.

En el programa principal (`main.py`), lo que hacemos es crear dos instancias de la clase `CNave`, en lugar de instancias de la clase `CCuadrado` como hacíamos en ejemplos anteriores. Para eso utilizamos dos variables, una para cada nave, que denominamos `n1` y `n2`.

El código del programa principal es igual al del ejemplo anterior, salvo por las siguientes sentencias que se encargan de crear (*instanciar*) las dos naves enemigas:

```

# Importar la clase CNave.
from CNave import *

...

# Definir ancho y alto de la pantalla
SCREEN_WIDTH = 640
SCREEN_HEIGHT = 360
RESOLUTION = (SCREEN_WIDTH, SCREEN_HEIGHT)
# Control del modo ventana o fullscreen.
isFullscreen = False

```

```

...

# Ancho y alto de la imagen de las naves.
NAVE_WIDTH = 60
NAVE_HEIGHT = 27

# Crear las naves: se le pasa como parámetro la imagen de la nave.
n1 = CNav("grey_ufo.png")
n2 = CNav("yellow_ufo.png")

# Colocar las naves en su posición inicial.
n1.setXY(0, 180)
n2.setXY(SCREEN_WIDTH - NAVE_WIDTH, 212)

# Marcar los límites del mundo.
n1.setBounds(-NAVE_WIDTH, -NAVE_HEIGHT, SCREEN_WIDTH, SCREEN_HEIGHT)
n2.setBounds(-NAVE_WIDTH, -NAVE_HEIGHT, SCREEN_WIDTH, SCREEN_HEIGHT)

...

```

Al principio del programa se encuentra la sentencia `from CNav import *`. Recuerda que para usar una clase siempre debemos importarla. De lo contrario nos dará un error al compilar.

Las constantes `NAVE_WIDTH` y `NAVE_HEIGHT` contienen el ancho y alto de las imágenes de las naves. Estos valores los conocemos porque son el ancho y alto de las imágenes que el artista ha creado para nosotros y no cambian en el juego (por eso le decimos constantes y se escriben sus nombres en mayúsculas). Utilizaremos estos valores para calcular los límites por donde se pueden mover las naves.

La sentencia `n1 = CNav("grey_ufo.png")` lo que hace es crear un objeto de la clase `CNav` (a lo que se le llama *instanciar un objeto*). Esta función, que tiene el mismo nombre que la clase, se le llama función *constructora*, y lo que hace es crear un objeto de esa clase (en este caso, crea un objeto de la clase `CNav` y lo asigna a la variable o referencia `n1`). El constructor de la nave recibe como parámetro el nombre de la imagen a cargar. De esta forma se crean dos naves en el programa.

Por último, posicionamos a las dos naves en los bordes de la pantalla y le damos los valores límites para su movimiento. Para esto, utilizamos las constantes `SCREEN_WIDTH` y `SCREEN_HEIGHT` que tienen el ancho y alto de la pantalla respectivamente, y las constantes `NAVE_WIDTH` y `NAVE_HEIGHT` que tienen el ancho y alto de las naves. Por ejemplo, para que la segunda nave quede recostada sobre la derecha de la pantalla, la coordenada `x` que debemos ponerle es `SCREEN_WIDTH - NAVE_WIDTH`.

Nota: Es una práctica profesional el usar constantes y nunca usar valores directamente en el

código. De esta forma, el programa continúa funcionando correctamente si luego cambia el ancho y alto de la imagen o de la pantalla. En tal caso solamente cambiamos los valores de las constantes. Imagina si cambia el ancho y alto de la pantalla, y hemos puesto por todas partes los números 640 y 360 en lugar de `SCREEN_WIDTH` y `SCREEN_HEIGHT`, deberíamos cambiar todos los números. Utilizando constantes el programa funciona igual si queremos una pantalla diferente, solamente cambiando las constantes.

Finalmente, en el game loop hemos cambiado las variables de los cuadrados (`c1` y `c2`) por las de las naves (`n1` y `n2`). Como siempre, invocamos a las funciones `update()` y `render()` en el game loop.

```
...

# Mover las naves y controlar los bordes.
n1.update(2, 1)
n2.update(-2, -1)

# Dibujar el fondo.
screen.blit(imgBackground, (0, 0))

# Dibujar las naves en la nueva posición.
n1.render(screen)
n2.render(screen)

...
```

Establecer Valores por Defecto

Es muy importante establecer valores por defecto que tengan sentido. Por ejemplo, para establecer los límites por donde se puede mover la nave, hemos puesto valores muy grandes (*-infinito* hasta *+infinito*) donde antes estaba en cero.

```
self.mMinX = -float("inf")
self.mMaxX = float("inf")
self.mMinY = -float("inf")
self.mMaxY = float("inf")
```

Si tuviéramos los bordes en cero, y nos olvidamos de llamar a `setBounds()` para establecer límites de movimiento, los límites por defecto serían cero, y por las sentencias `if` que hay al chequear bordes, siempre dejarían a la nave en la posición *(0,0)*, con lo cual no se movería.

Entonces, el comportamiento por defecto debería ser que si no llamamos a `setBounds()`

para establecer límites, entonces la nave no tiene límites, o lo que es lo mismo, los límites son desde *-infinito* a *+infinito*. Este número, “infinito”, es el número más grande que se puede representar, y en *Python* es `float("inf")`.

Siempre que establecemos valores por defectos, debemos establecer valores que tengan sentido y no depender de que llamemos a funciones para establecer valores que funcionen. Siempre hay que definir valores que por defecto permitan trabajar con el objeto en forma sencilla (en este caso, por defecto no hay límites de movimiento de la nave).

Con esto, terminamos este capítulo. A continuación veremos cómo es la estructura de un videojuego.

5. Estructura de un Videojuego

En este capítulo veremos cómo es la estructura de un videojuego a nivel de programación, e iremos armando la estructura del programa del juego en varios pasos. Haremos una versión del juego *Space Invaders*, en el cual manejamos una nave que dispara a los invasores extraterrestres. Durante este libro, haremos una versión de este juego clásico, que es un buen caso de un juego sencillo, para aprender a programar.

Primero comenzaremos viendo el ciclo del programa, el cual se compone de las funciones `init()`, `update() - render()` y `destroy()`. Luego veremos el *Game Object*, que será el objeto base de todos los objetos que se mueven en el juego. Por último veremos las carpetas que usaremos en el proyecto y la división en carpetas `api / game`, y finalmente veremos cómo deben ser los objetos para que su comportamiento quede encapsulado, o contenido en sí mismo.

Para cuando terminemos este capítulo, tendremos visualmente en la pantalla el mismo ejemplo que teníamos en el capítulo anterior, pero con la estructura de base armada para poder hacer videojuegos de calidad.

El Ciclo `init()` / `update()` - `render()` / `destroy()`

Como vimos en el capítulo 1, el programa se divide en básicamente cuatro funciones:

`init()`: Inicializa el sistema, los objetos del juego y carga los assets necesarios.

`update()`: Obtiene la entrada del jugador, actualiza todos los objetos del juego corriendo la lógica de cada uno y chequea las colisiones.

`render()`: Dibuja los objetos del juego y actualiza la pantalla.

`destroy()`: Libera toda la memoria utilizada por los objetos.

La estructura de un videojuego, ya se puede ver en los ejemplos que hemos realizado hasta ahora, y es la siguiente:

```
init()
while not salir:
    update()
    render()
destroy()
```

El programa comienza con su inicialización en la función `init()`, luego entra en el *game loop*, el cual se compone de las funciones `update()` y `render()`, y el programa permanece en ese loop hasta que el usuario decida salir del juego. En ese momento se sale del loop y

finalmente se invoca a la función `destroy()` para liberar la memoria utilizada.

Veamos el ejemplo de la carpeta `capitulo_05\001_init_update_render_destroy`. Al ejecutarlo podemos ver que hace lo mismo que el ejemplo del capítulo anterior. La diferencia es que el código se encuentra organizado en las funciones `init()`, `update()`, `render()` y `destroy()`. A continuación se muestra completo el programa principal (`main.py`):

```
# -*- coding: utf-8 -*-

#-----
#-----
# Naves enemigas moviéndose por la pantalla.
# Estructura init(), update() / render(), destroy().
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----
#-----

# Importar Pygame.
import pygame

# Importar la clase CNave.
from CNave import *

# Definir ancho y alto de la pantalla.
SCREEN_WIDTH = 640
SCREEN_HEIGHT = 360
RESOLUTION = (SCREEN_WIDTH, SCREEN_HEIGHT)
# Control del modo ventana o fullscreen.
isFullscreen = False

screen = None
imgBackground = None
imgSpace = None
clock = None
n1 = None
n2 = None

# Ancho y alto de la imagen de las naves.
NAVE_WIDTH = 60
NAVE_HEIGHT = 27

# Inicializar la variable de control del game loop.
salir = False
```

```

# Función de inicialización.
def init():

    global screen
    global imgBackground
    global imgSpace
    global n1
    global n2
    global clock

    # Inicializar Pygame.
    pygame.init()

    # Poner el modo de video en ventana e indicar la resolución.
    screen = pygame.display.set_mode(RESOLUTION)
    # Poner el título de la ventana.
    pygame.display.set_caption("Mi Juego")

    # Crear la superficie del fondo o background.
    imgBackground = pygame.Surface(screen.get_size())
    imgBackground = imgBackground.convert()

    # Cargar la imagen del fondo. La imagen es de 640 x 360.
    imgSpace = pygame.image.load("space_640x360.jpg")
    imgSpace = imgSpace.convert()

    # Dibujar la imagen cargada en la imagen de background.
    imgBackground.blit(imgSpace, (0, 0))

    # Crear las naves: se le pasa como parámetro la imagen de la nave.
    n1 = CNave("grey_ufo.png")
    n2 = CNave("yellow_ufo.png")

    # Colocar las naves en su posición inicial.
    n1.setXY(0, 180)
    n2.setXY(SCREEN_WIDTH - NAVE_WIDTH, 212)

    # Marcar los límites del mundo.
    n1.setBounds(-NAVE_WIDTH, -NAVE_HEIGHT, SCREEN_WIDTH,
SCREEN_HEIGHT)
    n2.setBounds(-NAVE_WIDTH, -NAVE_HEIGHT, SCREEN_WIDTH,
SCREEN_HEIGHT)

    # Inicializar el reloj.
    clock = pygame.time.Clock()

# Correr la lógica del juego.
def update():

```

```

global salir
global screen
global isFullscreen

# Timer que controla el frame rate.
clock.tick(30)

# Procesar los eventos que llegan a la aplicación.
for event in pygame.event.get():

    # Si se cierra la ventana se sale del programa.
    if event.type == pygame.QUIT:
        salir = True

    # Si se pulsa la tecla [Esc] se sale del programa.
    if event.type == pygame.KEYUP:
        if event.key == pygame.K_ESCAPE:
            salir = True

    # Si se pulsa la tecla [F], se cambia entre ventana y
    fullscreen.
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_f:
            isFullscreen = not isFullscreen
            if isFullscreen:
                screen = pygame.display.set_mode(RESOLUTION,
pygame.FULLSCREEN)
            else:
                screen = pygame.display.set_mode(RESOLUTION)

    # Mover las naves y controlar los bordes.
    n1.update(2, 1)
    n2.update(-2, -1)

# Dibujar el frame y actualizar la pantalla.
def render():
    # Dibujar el fondo.
    screen.blit(imgBackground, (0, 0))

    # Dibujar las naves en la nueva posición.
    n1.render(screen)
    n2.render(screen)

    # Actualizar la pantalla.
    pygame.display.flip()

# Función de destrucción.
def destroy():

```

```

global n1
global n2

# Destruir los objetos.
n1.destroy()
n1 = None
n2.destroy()
n2 = None

# Cerrar Pygame y liberar los recursos que pidió el programa.
pygame.quit()

# ===== Punto de entrada del programa. =====

# Inicializar los elementos necesarios del juego.
init()

# Loop principal del juego.
while not salir:
    # Actualizar los objetos.
    update()
    # Dibujar la pantalla.
    render()

# Liberar los recursos al final.
destroy()

```

Si miramos el bloque principal del programa, el cual se encuentra al final, vemos muy claramente que se invoca la función `init()` al principio, luego se entra al *game loop* invocando en cada frame a las funciones `update()` y `render()`, y al salir del *game loop* se llama a la función `destroy()`. El programa es el mismo que antes, pero lo hemos reestructurado en estas funciones.

Nota: Observa en el programa principal que hemos puesto las variables al comienzo, inicializándolas con `None` cuando se tratan de objetos.

Dentro de las funciones necesitamos declarar las variables con la palabra `global` para referirnos a la variable *global* (la variable definida al inicio del programa). Si no lo hiciéramos así, no podríamos acceder a la variable global desde dentro de una función y el programa no funcionaría.

Destrucción de Objetos

Para destruir un objeto en *Python*, debemos poner su referencia en `None`, que significa “*nada*”, o que ya no apunta a ningún objeto. Cuando hay un objeto en memoria que no tiene

ninguna variable apuntándole (se dice que no tiene *referencias*), el mismo es borrado definitivamente. Los lenguajes como *Python* implementan un sistema denominado Garbage Collector que realiza esta tarea de eliminar objetos que ya no son usados.

En general, tendremos una función `destroy()` en cada uno de nuestros objetos en el juego que se encargará de liberar toda la memoria que se haya utilizado en el objeto. Por ejemplo, como vemos en la función `destroy()` del programa principal, se llama a la función `destroy()` de las naves antes de eliminar el objeto.

En el caso de la nave (en la clase `CNave.py`), la función `destroy()` debe liberar la memoria pedida para la imagen que ha sido cargada. La función queda así:

```
# Liberar lo que haya creado el objeto.
def destroy(self):

    self.mImg = None
```

Nota: En general es necesario eliminar un objeto poniendo su referencia en `None`, cuando se trata de un objeto creado por nosotros, una imagen cargada en memoria, un array, etc.

Estructura de los Objetos del Juego

De la misma forma que el programa principal tiene una inicialización (la función `init()`), un *game loop* compuesto por la actualización y el dibujado (las funciones `update()` y `render()`) y un destructor (la función `destroy()`), todos los objetos que hagamos en el juego también tendrán estas cuatro funciones. Repasemos las funciones:

`init()` o el *constructor*: Inicializa el objeto, carga las imágenes necesarias, etc.

`update()`: Es invocada desde el *game loop*. Se encarga de mover el objeto y actualizar la animación de acuerdo a la lógica del objeto (lo que hace el objeto). Corre la lógica.

`render()`: Es invocada desde el *game loop*. Dibuja el objeto en su nueva posición en la pantalla.

`destroy()`: Destruye la memoria utilizada por el objeto. Se invoca al final de todo cuando se va a destruir el objeto.

Parece que toda esta reestructura hace al código más complicado, pero como vemos en el programa, el *game loop* queda muchísimo más chico y más claro. Tener esta estructura en el *game loop* y en los objetos con las funciones `init()` / `update()` - `render()` / `destroy()` nos permitirá una flexibilidad que necesitaremos más adelante para hacer juegos grandes.

El Game Object

Siguiendo con la estructura del código, lo que vamos a hacer ahora es tener un objeto base que será común a todos los objetos que se mueven en el juego. De esta forma, toda la programación del comportamiento base del objeto estará centralizada en un solo lugar, y todos los objetos del juego heredarán de él ese comportamiento. A este objeto base de todos los objetos que se mueven en el juego lo llamaremos *game object*.

En otras palabras, el *game object* es donde pondremos por ejemplo, el código de movimiento que ahora tenemos escrito en la clase de la nave. Imagina si ponemos en el juego otro objeto, aparte de la nave, y queremos que controle los bordes de la pantalla al igual que lo hace la nave. ¡Deberíamos duplicar el código de control de bordes en ambas clases! Eso sería una muy mala práctica de programación y no lo haremos.

Lo que haremos es tener un objeto de base llamado *game object* con el comportamiento de movimiento teniendo en cuenta los bordes de la pantalla y cuando creamos los otros objetos del juego, heredamos de ese *game object* y por lo tanto heredarán ese comportamiento. De esta forma, escribimos en el objeto base el comportamiento que reutilizaremos (heredamos) en los objetos que heredan.

Nota: La herencia es una de las principales propiedades de la *Programación Orientada a Objetos*. Cuando un objeto hereda de otro, automáticamente adquiere (hereda) sus variables y sus métodos, o sea, sus propiedades y su comportamiento.

Vamos a hacer una clase `CGameObject` básica, a la cual también se le denomina una *entidad* (o *game entity*). Este objeto no tendrá forma ninguna (por ejemplo, no tiene una imagen). Simplemente es un objeto abstracto que tendrá las cosas que comparten todos los objetos móviles del juego (por ejemplo: posición, velocidad, aceleración, etc). Todos los objetos móviles del juego heredarán de `CGameObject` y por lo tanto, adquirirán estas propiedades y sus métodos.

Por ahora, sólo haremos el objeto muy básico, y en los capítulos venideros le iremos agregando cosas, por ejemplo cuando hagamos movimientos con aceleración y que queramos que todos los objetos móviles lo tengan disponible por si lo queremos usar.

En el ejemplo que se encuentra en la carpeta `capitulo_05\002_game_object`, podemos ver los cambios en las clases `CNave` y `CGameObject`. En la clase `CNave` tendremos el código relacionado a la carga de la imagen de la nave y en la clase `CGameObject` tendremos el código del movimiento chequeando los bordes, comportamiento que heredarán todos los objetos del juego.

Cuando tenemos varias clases, unas heredando de otras, tenemos un *diseño orientado a objetos*. La relación de padres e hijos entre las clases se denomina *jerarquía de clases*, y se dibuja utilizando un *diagrama de clases*. El *diagrama de clases* del ejemplo se muestra en la figura 5-1.

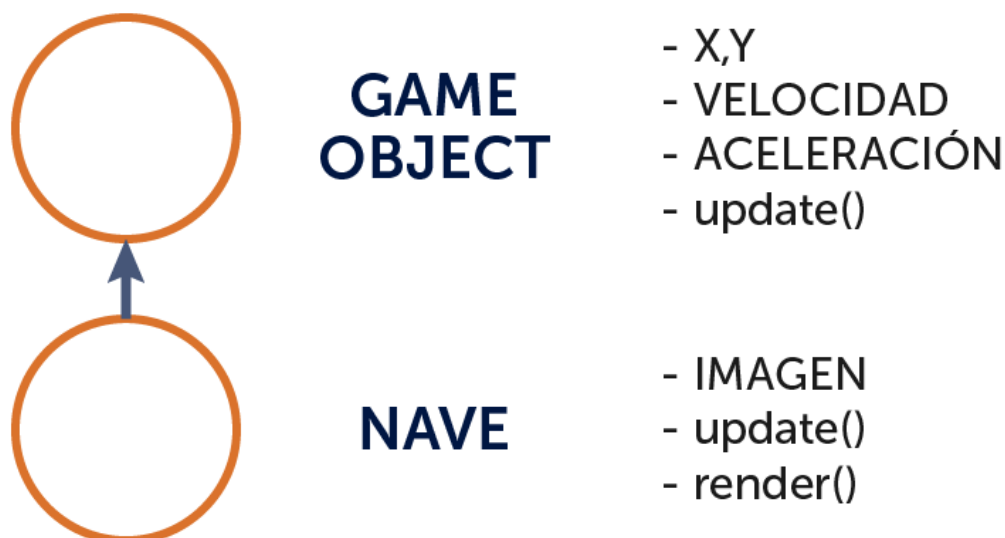


Figura 5-1: Diagrama de clases con CGameObject y CNav.

En este diagrama de clases, se ve que la clase CNav hereda de CGameObject. Esto significa que todos los *atributos* de CGameObject (*propiedades* o *variables*) y su comportamiento (*funciones* o *métodos*) serán heredados en la clase CNav.

Nota: Cuanto más arriba en la estructura de clases, más genérica es la clase. Cuanto más abajo en la estructura, más específica es la clase.

Nota que la clase CGameObject no tiene función `render()`. Esto es porque CGameObject es una clase que solamente lleva control de la posición y del movimiento del objeto, y no tiene forma ninguna (no tiene imagen). Todos los objetos del juego heredarán de la clase CGameObject y la imagen que se muestra será responsabilidad de cada clase. En este caso, cuando implementamos la clase CNav, esta clase se encarga de mostrar su propia imagen.

Al usar clases separadas, el código en cada una queda más simple y claro. Cada clase se encarga solamente de lo que le compete, y hereda el resto de su clase padre. Por ejemplo, veamos cómo queda la clase CGameObject, la cual se encarga del movimiento del objeto:

```

# -*- coding: utf-8 -*-

#-----
----
# Clase CGameObject.
# Clase base de todos los objetos que se mueven en el juego.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
  
```

```

#-----
----

class CGameObject(object):

    def __init__(self):

        # Coordenadas del objeto.
        self.mX = 0
        self.mY = 0

        # Variables para controlar los bordes.
        self.mMinX = -float("inf")
        self.mMaxX = float("inf")
        self.mMinY = -float("inf")
        self.mMaxY = float("inf")

        # Establece la posición del objeto.
        # Parámetros:
        # aX, aY: Coordenadas x e y del objeto.
        def setXY(self, aX, aY):

            self.mX = aX
            self.mY = aY

        # Define los límites del movimiento del objeto.
        # Parámetros:
        # aMinX, aMinY: Coordenadas x e y mínimas del mundo.
        # aMaxX, aMaxY: Coordenadas x e y máximas del mundo.
        def setBounds(self, aMinX, aMinY, aMaxX, aMaxY):

            self.mMinX = aMinX
            self.mMaxX = aMaxX
            self.mMinY = aMinY
            self.mMaxY = aMaxY

        # Mover el objeto.
        # Parámetros:
        # aIncX: Cantidad de píxeles que se mueve en la horizontal.
        # aIncY: Cantidad de píxeles que se mueve en la vertical.
        def update(self, aIncX, aIncY):

            # Mover el objeto.
            self.mX += aIncX
            self.mY += aIncY

            # Controlar los bordes haciendo wrap.
            if self.mX > self.mMaxX:

```



```

        self.mX = self.mMinX
    if self.mX < self.mMinX:
        self.mX = self.mMaxX
    if self.mY > self.mMaxY:
        self.mY = self.mMinY
    if self.mY < self.mMinY:
        self.mY = self.mMaxY

# Liberar lo que haya creado el objeto.
def destroy(self):

    pass

```

Como vemos, la clase `CGameObject` tiene como propiedades las coordenadas (x, y) de su posición, y las variables con los límites de movimiento. Luego sus métodos son `setXY()`, `setBounds()` y `update()`, relacionados al movimiento del objeto (de la misma forma en que se movían las naves antes). Por último, la función `destroy()`, que no hace nada, dado que no tiene nada que eliminar (nada ha sido creado por ahora), pero la implementamos porque es una buena práctica de programación y la vamos a invocar desde la clase `CNave`.

La clase `CNave`, heredará de `CGameObject`, y por lo tanto, heredará sus propiedades y el comportamiento de movimiento. El código de la clase `CNave` se muestra a continuación.

```

# -*- coding: utf-8 -*-

#
-----
# Clase CNave.
# Nave enemiga que se mueve por la pantalla chequeando los bordes.
#
# Autor: Fernando Sansberro - Batovi Games.
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#
-----

# Importar Pygame.
import pygame

# Importar la clase CGameObject.
from CGameObject import *

# La clase CNave hereda de CGameObject.
class CNave(CGameObject):

    # -----

```

```

# Constructor.
# Parámetros:
# aImgFile: Nombre de la imagen a cargar.
# -----
def __init__(self, aImgFile):

    # Invocar al constructor de la clase base.
    CGameObject.__init__(self)

    # Cargar la imagen.
    self.mImg = pygame.image.load(aImgFile)
    self.mImg = self.mImg.convert_alpha()

    # Guardar el ancho y el alto.
    self.mWidth = self.mImg.get_width()
    self.mHeight = self.mImg.get_height()

# Mover el objeto.
# Parámetros:
# aIncX: Cantidad de píxeles que se mueve en la horizontal.
# aIncY: Cantidad de píxeles que se mueve en la vertical.
def update(self, aIncX, aIncY):

    # Invocar a update() de la clase base para el movimiento.
    CGameObject.update(self, aIncX, aIncY)

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    aScreen.blit(self.mImg, (self.mX, self.mY))

# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar a destroy() de la clase base.
    CGameObject.destroy(self)
    self.mImg = None

```

Nota: La línea `class CNavе(CGameObject):` significa que estamos definiendo la clase `CNavе`, la cual hereda de `CGameObject` (se pone entre paréntesis la clase base, de la cual hereda).

La clase `CNavе` se encarga de cargar la imagen de la nave enemiga y de liberarla al final. Como el movimiento lo hereda de `CGameObject`, la función `update()` lo único que hace es invocar a la función `update()` de la clase base (`CGameObject`), que es donde se encuentra

el código del movimiento del objeto. Cuando hagamos objetos en el juego que tengan un comportamiento propio, la función `update()` tendrá más sentencias. Por ahora, como el movimiento de la nave usa el movimiento que está en la clase base, la función `update()` solamente invoca a la función `update()` en `CGameObject`.

Nota: En la función `update()`, vemos la siguiente línea: `CGameObject.update(self, aInc, aIncY)`. Esto significa invocar a la función `update()` en la clase base, la cual es `CGameObject`.

La función `render()` de `CNave`, lo que hace es dibujar la imagen en la pantalla y por último la función `destroy()` elimina la imagen utilizada e invoca a la función `destroy()` de `CGameObject`.

Nota: La idea de la *Programación Orientada a Objetos* es reutilizar en otros proyectos las clases que programemos. La clase `CGameObject` es la primera clase que estaremos reusando en varios de nuestros propios juegos.

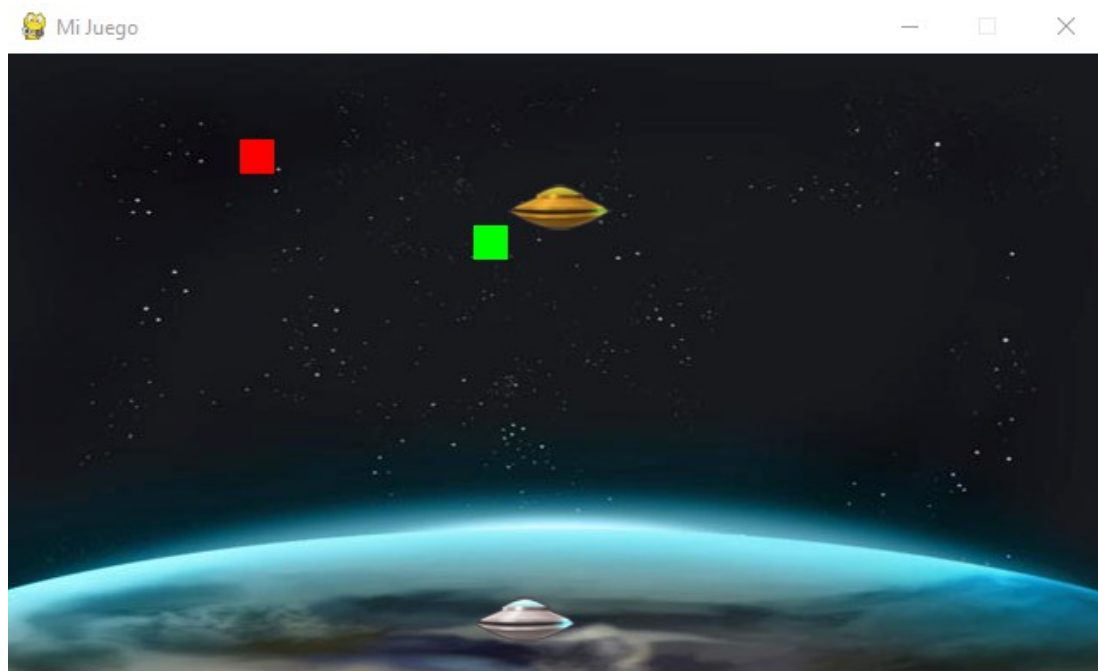


Figura 5-2: El juego ahora tiene naves y cuadrados heredando de `CGameObject`.

Packages api / game

Cuando programamos un videojuego (o cuando programamos cualquier otra cosa), generalmente resolvemos cosas que podremos reusar en futuros juegos. Por ejemplo, la clase `CGameObject` que estamos escribiendo, es una clase de base que sirve para todos los juegos que hagamos en el futuro.

Cuando comenzamos un nuevo juego, sería bueno ya poder contar con todas las clases que tenemos hechas y que nos puedan servir. De esta forma, desarrollaremos bastante más rápido y tendremos mucha funcionalidad disponible para usar en los juegos.

Para hacer esto, las clases deben estar bien organizadas, de modo de saber cuales son las clases genéricas (las que sirven para todos los juegos que hagamos) y cuales son las clases propias del juego actual (las que no sirven en otro juego directamente, sino que hay que adaptarlas para que funcionen).

Por otra parte, cuando un juego crece en complejidad, es muy conveniente organizar los archivos en carpetas y darle una estructura al proyecto, de modo de saber dónde se encuentra cada archivo sin tener que andar buscando mucho. Aprovecharemos que estamos creando la estructura del juego para hacer también carpetas para las imágenes que tenemos del juego. En la figura 5-3 se muestra la estructura de carpetas que utilizaremos.

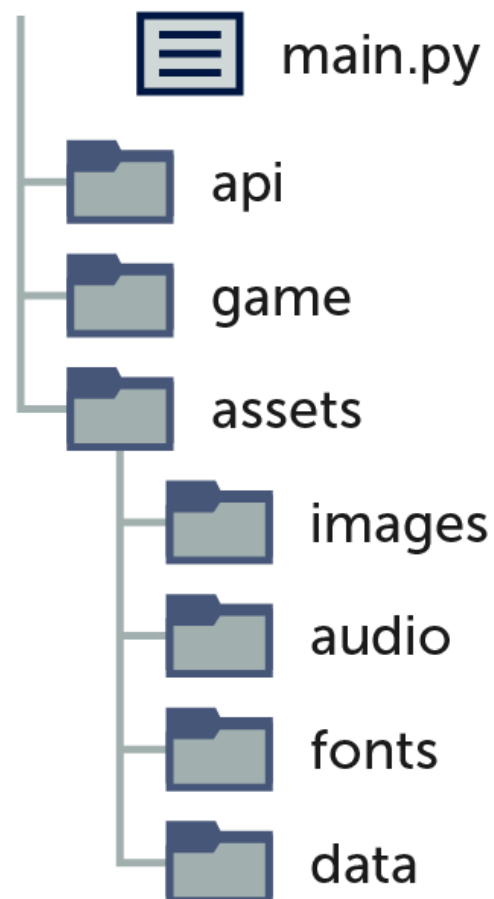


Figura 5-3: Estructura de carpetas y archivos del proyecto.

La estructura del proyecto se compone de las siguientes carpetas:

`api`: En esta carpeta irán las clases que se pueden utilizar en otros juegos.

`game`: En esta carpeta irán las clases que son propias del juego actual que estamos haciendo.

`assets`

`images`: Donde colocamos las imágenes utilizadas en el juego.

`audio`: Donde se encuentran los archivos de audio utilizados.

`fonts`: Donde colocamos los archivos de las fuentes.

`data`: Donde se encuentran los archivos con datos del juego como niveles, textos, etc.

En la raíz (en la base) del proyecto, tenemos el archivo `main.py`, que es el programa principal que se ejecuta.

Ahora veamos el ejemplo en la carpeta: `capitulo_05\003_estructura_api_game`. En este ejemplo hemos dividido el proyecto en carpetas como se muestra en la figura 5-3.

Luego de creadas las carpetas `api` y `game`, colocamos la clase `CGameObject` en la carpeta `api`, dado que es una clase que reusaremos entre juego y juego. Las clases `CNave` y `CCuadrado` son clases del juego que estamos haciendo en este momento, por lo que irá en la carpeta `game`.

Cuando las clases del programa se encuentran en una carpeta, a la carpeta se le denomina *package* (o paquete). En este caso, tendremos dos *packages*: `api` y `game`.

Nota: Para que *Python* trate a una carpeta como un *package* y reconozca las clases que hay dentro como clases, hay que poner un archivo `__init__.py`. Este archivo estará vacío, pero es obligatorio que exista en la carpeta, de lo contrario *Python* nos dará error al importar una clase desde esa carpeta. Por esta razón es que las carpetas `api` y `game` tienen este archivo dentro.

Cuando usamos carpetas, lo que tenemos que hacer es indicar su ruta cuando hacemos los *import* de las clases. Por ejemplo, en `main.py`, importamos la clase `CNave` y `CCuadrado` desde el *package* `game` de la siguiente manera:

```
# Importar la clase CNave.
from game.CNave import *

# Importar la clase Cuadrado.
from game.CCuadrado import *
```

En `CNave.py` y en `CCuadrado.py`, importamos la clase `CGameObject` del *package* `api` de la siguiente manera:

```
# Importar la clase CGameObject.
```

```
from api.CGameObject import *
```

Las imágenes las colocaremos en la carpeta `assets/images`. Luego, al momento de cargar las imágenes que hemos reubicado, debemos indicar su ruta cuando indicamos el nombre del archivo. Esto lo hacemos en `main.py`:

```
...

imgSpace = pygame.image.load("assets/images/space_640x360.jpg")

...

n1 = CNav("assets/images/grey_ufo.png")
n2 = CNav("assets/images/yellow_ufo.png")

...
```

Encapsulamiento de los Objetos

Para terminar con la estructuración del código, vamos a hacer una mejora relacionada con la nave enemiga que tenemos en el juego.

Si nos fijamos en el código del ejemplo anterior, en `main.py`, cuando se crean las naves, se les pasa como parámetro las imágenes a usar. Luego de esto se establecen los límites del movimiento. Esto se hace con las siguientes sentencias:

```
# Crear las naves: se le pasa como parámetro la imagen de la nave.
n1 = CNav("assets/images/grey_ufo.png")
n2 = CNav("assets/images/yellow_ufo.png")

# Colocar las naves en su posición inicial.
n1.setXY(0, 180)
n2.setXY(SCREEN_WIDTH - NAVE_WIDTH, 212)

# Marcar los límites del mundo.
n1.setBounds(-NAVE_WIDTH, -NAVE_HEIGHT, SCREEN_WIDTH, SCREEN_HEIGHT)
n2.setBounds(-NAVE_WIDTH, -NAVE_HEIGHT, SCREEN_WIDTH, SCREEN_HEIGHT)
```

Pues bien, no tienen nada de incorrecto esto, pero es mejor si creamos una nave y ésta misma toma la imagen y maneja su ancho y alto, de forma tal que la clase `CNav` sea bastante más simple de usar, y también para que la clase quede mucho más reusable.

Lo que haremos, es que que la clase `CNave` tenga un parámetro que será el tipo de nave del que se trate (dorada o plateada), y según el tipo de nave se cargará la imagen correspondiente, establecerá los límites según el tamaño de la imagen y de esta forma todas las naves enemigas del juego harán eso.

Abre y ejecuta el ejemplo que se encuentra en la carpeta `capitulo_05\004_clase_nave_encapsulada`. En este ejemplo, en `main.py` las naves se inicializan de la siguiente manera:

```
# Crear las naves: se le pasa como parámetro la imagen de la nave.
n1 = CNave(CNave.TYPE_PLATINUM)
n2 = CNave(CNave.TYPE_GOLD)

# Colocar las naves en su posición inicial.
n1.setXY(0, 180)
n2.setXY(SCREEN_WIDTH - n2.getWidth(), 212)

# Marcar los límites del mundo.
n1.setBounds(-n1.getWidth(), -n1.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)
n2.setBounds(-n2.getWidth(), -n2.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)
```

De esta manera, cuando creamos una nave, le pasamos el tipo de nave a crear y este tipo se guarda en la variable `self.mType` de la clase `CNave`. El tipo de nave es simplemente un número, (una *constante*), que se usará para diferenciar a las naves por su tipo.

En la clase `CNave`, en el constructor, se cargará la imagen correspondiente según se trate de la nave plateada o la dorada. En el código que se muestra arriba, también podemos observar que ya no tenemos constantes de ancho y alto de la nave. Las mismas se encuentran dentro de la clase `CNave`.

La clase `CNave` se modifica para tener un atributo con su tipo, y según el tipo, cargar la imagen correspondiente. A continuación se muestra la clase `CNave` y su constructor..

...

```
class CNave(CGameObject):
```

```
    # Tipos de naves.
    TYPE_PLATINUM = 0
    TYPE_GOLD = 1
```

```
    # -----
```

```

# Constructor. Recibe el tipo de nave (TYPE_PLATINUM o TYPE_GOLD).
# -----
def __init__(self, aType):

    # Invocar al constructor de la clase base.
    CGameObject.__init__(self)

    # Segun el tipo de la nave, la imagen que se carga.
    self.mType = aType
    if self.mType == CNavе.TYPE_PLATINUM:
        imgFile = "assets/images/grey_ufo.png"
    elif self.mType == CNavе.TYPE_GOLD:
        imgFile = "assets/images/yellow_ufo.png"

    # Cargar la imagen.
    self.mImg = pygame.image.load(imgFile)
    self.mImg = self.mImg.convert_alpha()

    # Guardar el ancho y el alto.
    self.mWidth = self.mImg.get_width()
    self.mHeight = self.mImg.get_height()

...

```

En la clase se definen dos constantes, `TYPE_PLATINUM` y `TYPE_GOLD`, que se usan para cargar la imagen que corresponda al tipo de nave. Al final, se cargan en las variables `self.mWidth` y `self.mHeight` el ancho y el alto de la imagen.

De esta forma, la clase `CNavе` se encarga de todos los detalles de la nave (su *tipo*, su *imagen* y sus propiedades como el *alto* y el *ancho*) y desde afuera de la clase se establecen parámetros de acuerdo a la lógica del juego.

En la clase `CNavе`, agregamos dos funciones, `getWidth()` y `getHeight()` para retornar el ancho y el alto de la nave respectivamente. Estas funciones se denominan *getters*, y se utilizan para consultar propiedades del objeto (en inglés “*get*” significa “obtener”):

```

# Retorna el ancho de la nave.
def getWidth(self):

    return self.mWidth

# Retorna el alto de la nave.
def getHeight(self):

    return self.mHeight

```


Tipos de Objetos

En los juegos es muy bueno tener variaciones en los objetos. En nuestro juego, tendremos naves espaciales de varios colores. Para eso hemos definido un tipo de nave (un número) que almacenamos en la variable `self.mType`, y según el tipo, de nave se cargará el gráfico que le corresponda.

De esta forma tendremos naves que se comportan en forma muy parecida pero que cambiarán los gráficos, o algún atributo como por ejemplo su velocidad. La alternativa a tener un tipo de objeto es tener diferentes *clases* de objetos. Si ambos objetos del juego comparten mucho código deberían ser la misma clase. Si los objetos del juego son muy diferentes entre sí, deberían estar en clases diferentes. Esta es la diferencia, por ejemplo, entre tener una sola clase `CNave` para las naves espaciales del juego con un tipo según el cual se cargan diferentes gráficos, versus tener una clase para cada tipo de nave: por ejemplo, clases `CNaveGris`, `CNaveRoja`, etc.

Bueno, hemos terminado de armar la estructura básica de un videojuego. Ahora pasaremos a la parte divertida, la física de movimiento.

6. Movimiento de Objetos

En este momento tenemos todo pronto para comenzar a programar el movimiento de los objetos. ¿Has visto la manera en que se mueven los objetos en un videojuego y te has preguntado cómo es posible lograrlo? Esto es lo que veremos en este capítulo.

Algunos objetos se mueven en forma constante, otros aceleran como un cohete o como un auto de carreras, otros caen por el efecto de la gravedad como ocurre al patear una pelota o disparar un proyectil, otros se deslizan sobre el hielo o se frenan con fricción en la tierra, etc. Todo lo que se nos pueda ocurrir al diseñar nuestro juego lo podremos implementar utilizando los conceptos fundamentales de la *física*.

Es inevitable escuchar la palabra *física* y tener algo de miedo, sentir que está lleno de fórmulas complicadas que tenemos que memorizarlas, pero nada más alejado de la realidad cuando se trata de videojuegos. Los conceptos de física, al verse aplicados en un videojuego, se convierten en conceptos intuitivos, fáciles de entender y, lo que es más importante, aplicables a nuestro videojuego para mover objetos de maneras sorprendentes.

En este capítulo vamos a ver los conceptos de *velocidad* y *aceleración*, y cómo modelamos el código para que lo que programemos lo podamos usar en todos nuestros juegos. Más adelante veremos los conceptos de *gravedad* y *fricción* que están relacionados a la *velocidad* y a la *aceleración*.

Introducción a la Física

La *física* es la ciencia que estudia las propiedades y el comportamiento de los objetos en la naturaleza (*energía, materia, tiempo, espacio* y las interacciones de estos cuatro conceptos entre sí).

Entre otras cosas, la física describe un objeto cayendo por la gravedad (como la manzana de *Newton*), las órbitas de los planetas, la electricidad, etc. Existen muchas áreas de especialización dentro de la física, pero la rama de la física que utilizaremos para mover objetos en los videojuegos es la *mecánica clásica*.

La mecánica clásica describe el movimiento de los objetos. Por ejemplo, si tenemos una manzana en la mano y la soltamos, es obvio que se va a caer por efecto de la gravedad. En muchas situaciones sabremos qué es lo que va a pasar, solamente utilizando el sentido común, pero en otras situaciones no será simple saber qué es lo que va a ocurrir.

Física Real vs. Física Simulada

Siempre deberemos hacer una distinción entre las fórmulas físicas reales y las fórmulas que simulan la física, o dicho de otra manera, son fórmulas suficientemente buenas como para ser utilizadas en los juegos. A veces, implementar en un juego algunas fórmulas o ecuaciones físicas es muy pesado como para usarse en lenguaje *Python*. Por eso es que a veces utilizaremos fórmulas que son sencillas de programar, y que son lo suficientemente buenas como para que el resultado sea muy similar a la realidad, ejecutando la menor cantidad posible de cálculos.

Además, como hemos visto en muchos videojuegos, la física de un videojuego se escribe para que el juego sea divertido de jugar, no para que las cosas se comporten como en la vida real (por ejemplo, basta ver qué tan alto salta un personaje en un juego de plataformas para entender que no se utiliza física simulando el mundo real).

Vectores

Antes de entrar con el tema de *velocidad*, necesitamos saber lo que es un *vector*. Si bien por ahora no los utilizaremos para programar nuestro primer juego, sí debemos saber que son. Más adelante, siempre programaremos juegos utilizando vectores, pues es algo imprescindible.

Un *vector* es un objeto matemático que contiene dos informaciones: una *magnitud* (es un número) y una *dirección* (generalmente se representa por el componente horizontal y el componente vertical). En la figura 6-1 se muestra gráficamente un *vector*.

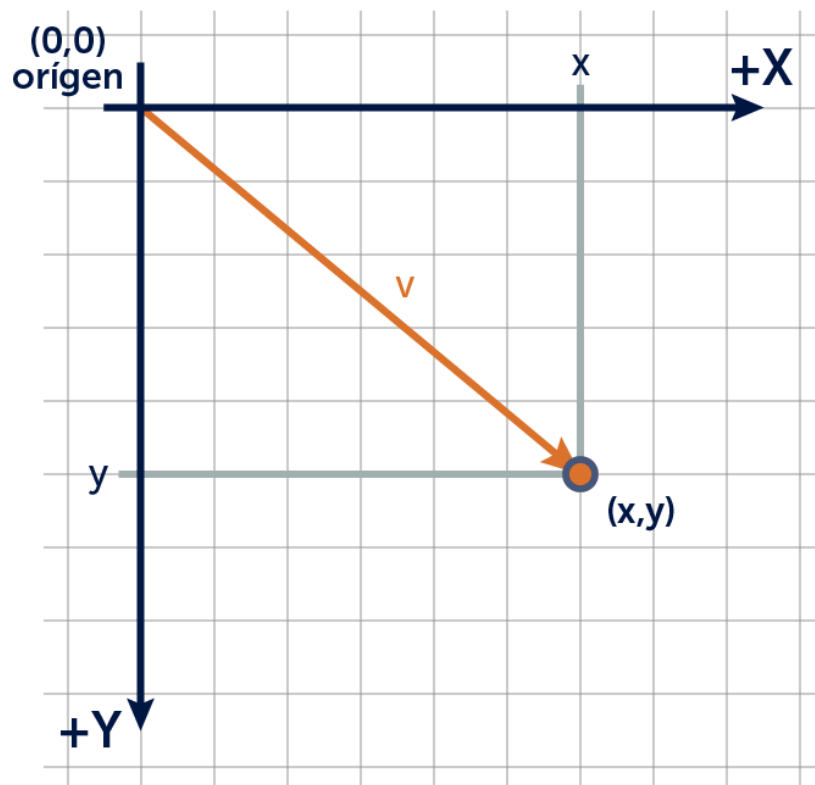


Figura 6-1: Un *vector* y sus coordenadas.

En este caso, el vector v tiene su punto de apoyo (*origen*) en las coordenadas $(0, 0)$, y termina en las coordenadas (x, y) . Los vectores se pueden descomponer siempre en dos vectores, según la componente x (horizontal) y la componente y (vertical), que es lo que usamos cuando, por ejemplo, a un objeto lo movemos cierta cantidad de unidades en la horizontal y cierta cantidad de unidades en la vertical.

Más adelante vamos a reescribir todo el código de movimiento para que use vectores, escribiendo para esto una clase `CVector` (lo haremos en el siguiente libro de la serie).

Por ahora, utilizaremos en forma separada las componentes horizontal y vertical para mover objetos. Esto significa que tendremos dos variables para la posición (las coordenadas x e y), y también dos variables para definir una velocidad (sus componentes x e y).

Velocidad y Rapidez

Supongamos que un auto se mueve por una carretera. Cuando el auto se mueve, tiene una cierta velocidad. La *velocidad* es un vector (indicando dirección), y la *rapidez* es la magnitud (también denominado *largo* o *módulo*) del vector velocidad. Por ejemplo, si decimos que el auto viaja a 60 km/h en dirección este, entonces tenemos:

Velocidad: Es el vector 60 km/h hacia el este. Es un vector.

Rapidez: 60 km/h (es la magnitud del vector velocidad). Es un número.

La velocidad es el vector que resulta de combinar la rapidez (el número) con la dirección (la flecha).

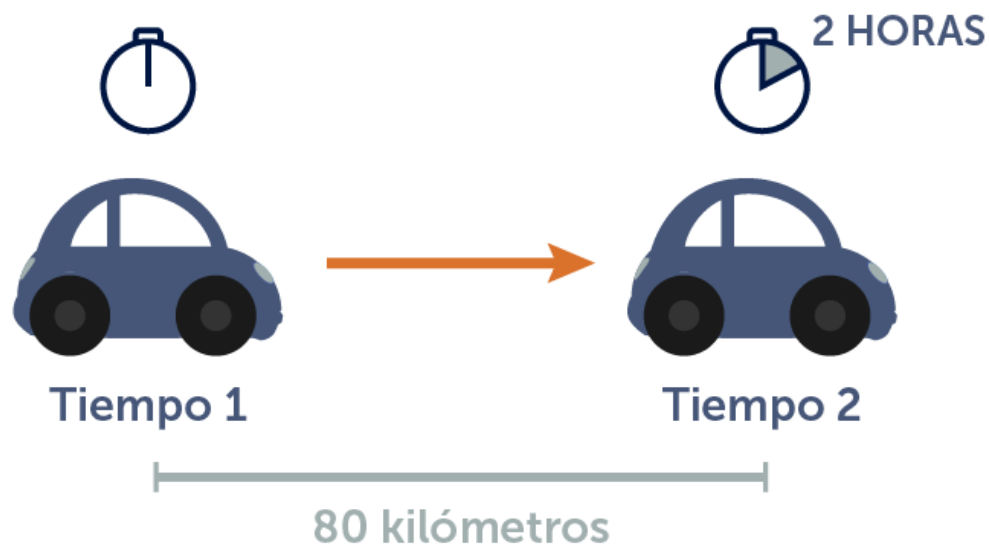
Nota: En inglés, la velocidad se dice “*velocity*” y se refiere al vector velocidad. Rapidez se dice “*speed*” y se refiere a la magnitud del vector velocidad.

Rapidez

La *rapidez* indica qué tanta distancia recorre un objeto en cierto tiempo. Matemáticamente esto corresponde a la ecuación:

$$\text{Rapidez} = \text{Distancia} / \text{Tiempo}$$

Por ejemplo, si un auto se mueve a 80 km/h, esto quiere decir que recorre una distancia de 80 kilómetros en una hora. Si el auto hiciera 80 km en 2 horas, esto significa que el auto recorre 40 kilómetros en una hora: $80 \text{ km} / 2 \text{ hs} = 40 \text{ km/h}$.



$$\text{Rapidez} = \frac{\text{Distancia}}{\text{Tiempo}} = \frac{80\text{km}}{2\text{h}} = 40\text{km/h}$$

Figura 6-2: El auto moviéndose a 40 km/h.

Como la rapidez es la distancia realizada en cierto tiempo, podemos calcular qué distancia recorrerá el auto, por ejemplo, en cuatro horas, despejando de la ecuación:

$$\text{Distancia} = \text{Rapidez} * \text{Tiempo}$$

Como la velocidad del auto es 80 km/h, entonces tenemos: $80 \text{ km/h} * 4 \text{ hs} = 320 \text{ kms}$. Con esta ecuación podemos calcular la distancia que se mueve un objeto si sabemos su rapidez y el tiempo que transcurre.

Del mismo modo, si sabemos la distancia que ha recorrido un auto y a qué rapidez iba, podemos saber el tiempo que le tomó:

$$\text{Tiempo} = \text{Distancia} / \text{Rapidez}$$

Si el auto recorrió 320 kms y llevaba una rapidez de 80 km/h, entonces $320 \text{ kms} / 80 \text{ km/h} = 4 \text{ hs}$. Al auto le toma 4 horas recorrer 320 kms a una rapidez de 80 km/h.

Tiempo y Frames

Cuando hablamos de la rapidez de un objeto, decimos que se mueve una determinada distancia en cierto tiempo. La distancia generalmente se mide en metros, o kilómetros cuando se trata de un auto, pero cuando se trata de un juego, y movemos un objeto, nos queda más natural dar la velocidad en píxeles. Cuando en el ejemplo de los cuadrados incrementamos en 2 la coordenada x para que se mueva, lo que estamos haciendo es moviéndolo 2 píxeles en cada frame. La rapidez entonces en ese caso es de $2 \text{ px} / \text{frame}$.

El tiempo que transcurre entre dos llamadas a `update()` es un frame. Si por ejemplo, el juego corre a 30 FPS (30 frames por segundo), el tiempo de un frame será equivalente a $1 / 30$ segundos (0,03333... segundos).

Si usamos *frame* como unidad de tiempo, podemos reemplazar en las ecuaciones al tiempo por frames. Entonces, como *Distancia = Rapidez * Tiempo*, la ecuación queda:

$$\text{Distancia} = \text{Rapidez} * \text{Frames}$$

De esta forma, podemos decir que una nave en un juego, por ejemplo, se mueve a una velocidad de diez píxeles por frame. La distancia recorrida por un objeto sigue siendo una

medida de distancia, excepto que en un juego generalmente es la cantidad de píxeles que se mueve en un frame. Entonces, en nuestros juegos, la distancia también la manejaremos en píxeles.

Nota: En los primeros juegos que hagamos, el mundo será la pantalla, por lo cual nuestras coordenadas y distancias estarán relacionadas a la pantalla. En futuros juegos, usaremos coordenadas y distancias del “*mundo*” (expresadas en nuestro mundo virtual), independientemente de cómo se ve el juego o de la resolución de la pantalla.

Nota: En algunos entornos o motores (como por ejemplo en *Unity en 3D*) es mejor trabajar en unidades virtuales, ya que la pantalla o el mundo se puede escalar y la distancia entre dos objetos no corresponde a la distancia en píxeles. Todo esto lo veremos a futuro. Por ahora nos concentramos en aprender a crear nuestro primer videojuego.

Movimiento a Velocidad Constante

Vamos a mover a tres naves espaciales en nuestro juego, cada una con una velocidad diferente. Pondremos las tres naves a la izquierda de la pantalla y éstas se moverán hacia la derecha. La nave de más arriba se moverá a 4 píxeles por frame, la nave del medio lo hará a 2 píxeles por frame y la nave de más abajo se moverá a 1 píxel por frame.

Abre el ejemplo de la carpeta: `capitulo_06\001_velocidad`. En este ejemplo los objetos tienen velocidad.

Para mover un objeto, en cada frame le sumamos a sus coordenadas la distancia que se mueve. Tendremos en `CGameObject` la velocidad del objeto en las variables `mVelX` y `mVelY`, siendo `mVelX` la velocidad en la componente horizontal (*x*) y `mVelY` la velocidad en la componente vertical (*y*). Las siguientes sentencias mueven el objeto de lugar.

```
# Mover el objeto.
def update(self):

    # Mover el objeto.
    self.mX += self.mVelX
    self.mY += self.mVelY

    ...
```

Si miramos en la función `update()` de `CGameObject`, hemos puesto estas dos líneas que mueven el objeto según estas dos variables de velocidad. Si cambiamos cualquiera de estos valores de velocidad, el objeto se moverá. La idea es decirle a un objeto, que tiene una velocidad determinada, y que al hacer eso el objeto se mueva solo (se moverá solo porque en su función `update()` tiene estas líneas de código que le agregamos).

Nota que también hemos quitado los parámetros de incremento de posición que antes recibía la función `update()` como parámetros. Ahora el objeto se moverá automáticamente cuando cambiemos su velocidad.

A partir de ahora vamos a crear muchas funciones en la clase `CGameObject`, de forma tal que nos sirvan para todos los objetos del juego.

En la clase principal (`main.py`), creamos las tres naves como siempre, las colocamos en la posición inicial y luego le damos la velocidad utilizando funciones que vamos a escribir en `CGameObject`:

```
# Crear las naves: se le pasa como parámetro la imagen de la nave.
n1 = CNavе(CNavе.TYPE_PLATINUM)
n2 = CNavе(CNavе.TYPE_GOLD)
n3 = CNavе(CNavе.TYPE_RED)

# Colocar las naves en su posición inicial.
n1.setXY(0, 100)
n2.setXY(0, 150)
n3.setXY(0, 200)

# Establecer la velocidad de las naves.
n1.setVelX(4)
n1.setVelY(0)
n2.setVelX(2)
n2.setVelY(0)
n3.setVelX(1)
n3.setVelY(0)
```

Como puedes ver, hemos agregado una nueva nave enemiga (una roja). Para eso en la clase `CNavе` agregamos una constante más (`CNavе.TYPE_RED`), y hemos puesto una imagen de la nave roja en la carpeta de imágenes dentro de la carpeta de `assets` (cópiala a tu propio proyecto y agrega la carga de la imagen según el tipo).

Lo que hace el código principal es simplemente asignar una velocidad a cada nave. Establecemos la velocidad asignando los componentes `x` e `y` de la velocidad. Por ejemplo, la primera nave se mueve de a 4 píxeles por frame en la horizontal, y no se mueve en la vertical. La idea detrás de esto es que una vez que le damos velocidad a un objeto, el objeto continuará moviéndose hasta que le digamos que se detenga o le digamos de hacer otra cosa.

En el `game loop`, se llama a la función `update()` de las naves. Hemos sacado los parámetros de incremento (`aIncX`, `aIncY`) que había en el ejemplo anterior. Ahora `update()` no lleva parámetros y simplemente suma la velocidad del objeto a su posición, como vimos antes. En el `game loop`, estas líneas mueven las naves:


```
# Lógica de las naves.  
n1.update()  
n2.update()  
n3.update()
```

En la clase `CGameObject`, agregamos dos atributos para llevar la velocidad del objeto: `mVelX` será la velocidad en la horizontal, y `mVelY` será la velocidad en la vertical:

```
class CGameObject(object):  
  
    def __init__(self):  
  
        # Coordenadas del objeto.  
        self.mX = 0  
        self.mY = 0  
  
        # Velocidad.  
        self.mVelX = 0  
        self.mVelY = 0  
  
        ...
```

La función `update()`, ahora suma la velocidad del objeto a su posición, como vimos antes:

```
# Update mueve el objeto según su velocidad.  
def update(self):  
  
    # Mover el objeto.  
    self.mX += self.mVelX  
    self.mY += self.mVelY  
  
    ...
```

Como hemos modificado la función `update()` de `CGameObject`, sacándole los parámetros que tenía antes, debemos de modificar también las funciones `update()` de las clases `CNave` y `CCuadrado`. Las funciones quedan así:

```
# Mover el objeto.  
def update(self):  
  
    # Invocar a update() de la clase base para el movimiento.
```

```
CGameObject.update(self)
```

Por último, en `CGameObject` agregamos las funciones que utilizamos en `main.py` para establecer los componentes `x` e `y` de la velocidad:

```
# Establece la velocidad X del objeto.
def setVelX(self, aVelX):

    self.mVelX = aVelX

# Establece la velocidad Y del objeto.
def setVelY(self, aVelY):

    self.mVelY = aVelY
```

En general, para cada variable que agreguemos en una clase, haremos sus funciones correspondientes para establecer los valores y recuperarlos. A estas funciones se les llama *setters* (para establecer valores) y *getters* (para obtener valores). Casi todas las clases tendrán estas funciones. Recordemos que en inglés “*set*” significa establecer y “*get*” significa obtener.

Al correr el ejemplo vemos que la nave de arriba (la primera) se mueve a 4 píxeles por frame, la segunda a 2 píxeles por frame y la de abajo (la tercera) a 1 pixel por frame.



Figura 6-3: Las tres naves moviéndose a diferente velocidad..

¿Qué Velocidad debe tener un Objeto en el Juego?

Al decidir qué velocidad tiene que tener un objeto, en lugar de ponerle una velocidad a los objetos y estar probando a ver si queda bien mediante prueba y error, lo mejor es calcular realmente la velocidad según lo que se desea hacer.

Como pusimos el game loop a 60 frames por segundo y la ventana tiene 640 píxeles de ancho, la nave que va a dos píxeles por frame recorrerá la pantalla entera en $640 \text{ px} / 2 \text{ px/frame} = 320 \text{ frames}$. Como en un segundo se ejecutan 60 ciclos del game loop, $320/60$ nos da 5,33 segundos. La nave que va a dos píxeles por segundo demora 5,33 segundos en atravesar la pantalla. Esto se da si realmente el game loop corre a 60 frames por segundo, porque si la computadora es lenta puede ser que en un segundo no se lleguen a hacer 60 frames.

Nota: En los primeros juegos que hagamos, las velocidades de los objetos dependen del framerate. En futuros juegos, haremos que la física de los objetos corra siempre igual, independiente del framerate.

Al establecer la velocidad de una nave, por ejemplo, si quisiéramos que la nave recorra la pantalla en 10 segundos, ¿qué velocidad debería tener?

Los datos que tenemos dados son:

Frame rate del juego: 60 frames por segundo

Ancho de la pantalla (distancia a recorrer): 640 píxeles

Entonces tenemos que:

$\text{velocidad (píxeles / frame)} = \text{distancia (píxeles)} / \text{tiempo (frames)}$

$\text{velocidad} = 640 \text{ pixels} / 6 \text{ segundos}$

$1 \text{ segundo} = 60 \text{ frames}$

Entonces:

$\text{velocidad} = 640 \text{ píxeles} / 60 * 6 \text{ frames} = 640 \text{ píxeles} / 360 \text{ frames} = 1,77 \text{ píxeles / frame}$

De esta forma, si a un objeto le ponemos 1,77 píxeles por frame de velocidad en la x, recorrerá la pantalla (640 píxeles) en 6 segundos (360 frames).

Movimiento en Diagonal

Si queremos mover un objeto en diagonal, lo que debemos hacer es moverlo en la horizontal y también en la vertical.

Abre el ejemplo de la carpeta: `capitulo_06\002_movimiento_diagonal`. En este ejemplo, en `main.py`, modificamos la velocidad de las naves para que se muevan en diagonal:

```
# Establecer la velocidad de las naves.
n1.setVelX(2)
n1.setVelY(2)
n2.setVelX(4)
n2.setVelY(-2)
n3.setVelX(1)
n3.setVelY(1)
```

La primera nave, por ejemplo, tendrá una velocidad horizontal de dos píxeles por frame y una velocidad vertical de dos píxeles por frame. La figura 6-4 muestra este movimiento.

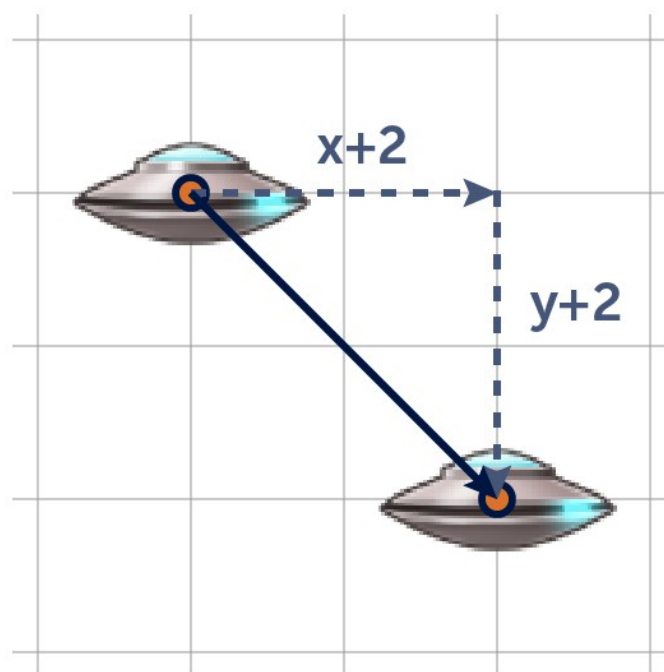


Figura 6-4: Movimiento de la nave en diagonal.

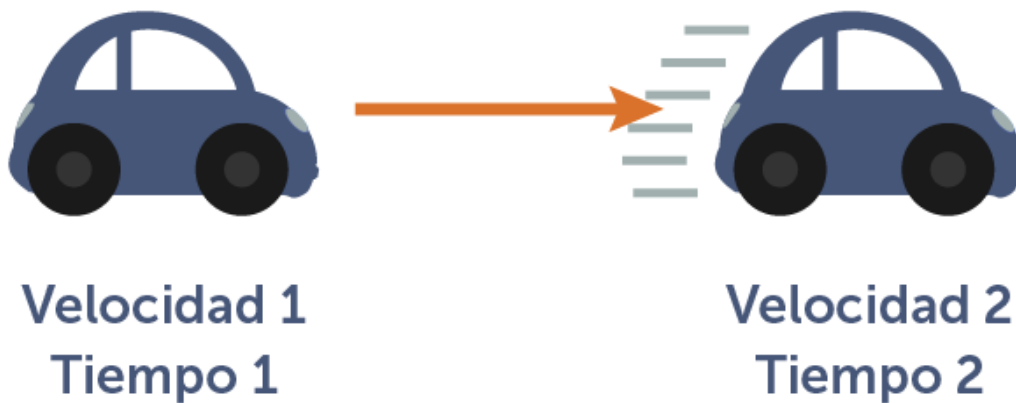
En este caso, la nave no se mueve con rapidez 2, sino que la rapidez es mayor (usando *Pitágoras* podemos calcular que la rapidez, en este caso, es la raíz cuadrada de 8, o sea aproximadamente 2,83). Esto implica que esta nave se mueve más rápido en la diagonal que en la horizontal, lo cual no es bueno para algunos juegos, por ejemplo, en los juegos *RPG* en los cuales movemos un personaje con las teclas de dirección, donde no queremos que el personaje camine más rápido en diagonal. Hay varios juegos donde esto pasa.

Más adelante cuando trabajemos con vectores, veremos como solucionar este problema que es común a muchos juegos cuando éstos son desarrollados por principiantes.

Aceleración

La *aceleración* produce un cambio de velocidad. Cuando un objeto cambia su velocidad es porque ha acelerado. Al igual que la velocidad, la aceleración es un vector, y como tal tiene una dirección y una magnitud.

La aceleración es un cambio de la velocidad en un cierto tiempo. Mira la figura 6-5.



$$\text{Aceleración} = \frac{(\text{velocidad2} - \text{velocidad1})}{(\text{tiempo2} - \text{tiempo1})}$$

Figura 6-5: El auto acelerando en un tiempo determinado.

En la figura 6-5 se muestra un auto acelerando. Un objeto tiene una velocidad inicial en un tiempo inicial y una velocidad final en un tiempo final. La aceleración es la diferencia del cambio de la velocidad en el tiempo transcurrido. La unidad de aceleración es *distancia/tiempo²*.

$$\text{Aceleración} = (\text{velocidad final} - \text{velocidad inicial}) / (\text{tiempo final} - \text{tiempo inicial})$$

¿Cómo saber dónde va a estar un objeto en el futuro?

En algunos juegos podemos querer saber dónde va a estar un objeto en el futuro (por ejemplo, dentro de tres segundos) para poder disparar una bala a ese punto y pegarle justo cuando el objeto llegue ahí.

Si sabemos la posición actual del objeto y su velocidad, podemos calcular su posición en un tiempo en el futuro (por ejemplo en esos tres segundos). Supongamos también que el objeto está acelerando. Calculamos la posición futura del objeto con la siguiente fórmula:

$$\text{Pos. futura} = \text{Pos. actual} + \text{Velocidad actual} * \text{tiempo} + \frac{1}{2} * (\text{Aceleración}) * \text{tiempo}^2$$

Como la velocidad y la aceleración son vectores, tenemos que resolver los vectores en sus componentes *x* e *y* para poder utilizarlos. Tendremos las variables `mVelX` y `mVelY` para representar el vector de velocidad, y las variables `mAccelX` y `mAccelY` para representar el vector de aceleración. Más adelante veremos en profundidad el manejo de vectores, haremos la clase `CVector` correspondiente y la utilizaremos, en lugar de tener dos variables para la velocidad y dos variables para la aceleración.

Entonces, por ahora, como aún no tenemos una clase para manejar un vector, usaremos los componentes *x* e *y* del vector, por separado:

$$x \text{ futura} = x \text{ actual} + \text{velX} * \text{tiempo} + \frac{1}{2} * (\text{accelX}) * \text{tiempo}^2$$

$$y \text{ futura} = y \text{ actual} + \text{velY} * \text{tiempo} + \frac{1}{2} * (\text{accelY}) * \text{tiempo}^2$$

Nota que cuando la aceleración es cero. Toda la multiplicación por cero, da cero, dejando a la ecuación como es cuando la velocidad es constante y no hay aceleración:

$$\text{Pos. futura} = \text{Pos. actual} + \text{Velocidad} * \text{tiempo}$$

(cuando no hay aceleración)

Simplificando las Ecuaciones en los Juegos

La ecuación que vimos anteriormente suma $\frac{1}{2} * (\text{accelX}) * \text{tiempo}^2$. Como en los juegos estamos corriendo la función `update()` en cada frame, el tiempo que transcurre entre un frame y otro es muy chico. Por ejemplo, si tenemos el juego a 60 frames por segundo, y un segundo son 1000 milisegundos, entonces el tiempo entre frame y frame es de $1000 / 60 = 16,66...$ milisegundos. Expresado en segundos, este tiempo es de $0,0166...$ segundos ($1 \text{ segundo} / 60 = 0,0166...$ segundos).

Si vemos la ecuación, estamos dividiendo por dos el tiempo al cuadrado, lo cual da 0.0013. Este valor es muy chico como para que la aceleración influya en el cambio de posición del objeto. Como estamos haciendo un videojuego 2D y no necesitamos una física realista,

podemos ignorar completamente esta parte de la ecuación, lo cual hará las fórmulas mucho más sencillas y correrán más rápido para el juego.

Nota: Hay diferentes formas de escribir estas ecuaciones de movimiento (se denominan *integraciones*, y tenemos integración de *Euler*, *Verlet*, *RK4*, etc.). La que utilizaremos es la más sencilla (*Euler*) que sirve para la mayoría de los juegos 2D. Si necesitáramos una física más precisa, como por ejemplo para el cálculo de proyectiles, o para tener mayor precisión porque el juego lo pide, debemos usar otra.

¿Cómo saber qué velocidad tendrá un objeto en el futuro?

Si sabemos la aceleración del objeto y su actual velocidad, podemos proyectar la velocidad en esos segundos en el futuro (el tiempo).

La ecuación entonces, es:

$$\text{Velocidad futura} = \text{Velocidad actual} + \text{Aceleración} * \text{tiempo}$$

Por ahora usaremos los componentes x e y por separado:

$$\text{velX futura} = \text{velX actual} + \text{accelX} * \text{tiempo}$$

$$\text{velY futura} = \text{velY actual} + \text{accelY} * \text{tiempo}$$

Como el tiempo que transcurre entre una llamada a `update()` y otra es de un frame cuando estamos trabajando a una framerate fijo, la ecuación queda simplificada (tiempo = 1):

$$\text{velX} = \text{velX} + \text{accelX}$$

$$\text{velY} = \text{velY} + \text{accelY}$$

Este es el código que cambia la velocidad según la aceleración, y es el código que pondremos en la función `update()` de `CGameObject` para que todos los objetos del juego soporten aceleración.

Implementando la Aceleración

Veamos el ejemplo que se encuentra en la carpeta: `capitulo_06\003_aceleracion`.

Al inicio de la clase `CGameObject`, agregamos las dos variables para llevar control de la aceleración del objeto en la horizontal y en la vertical. Ambas variables son los componentes del vector aceleración.

```
class CGameObject(object):

    def __init__(self):

        # Coordenadas del objeto.
        self.mX = 0
        self.mY = 0

        # Velocidad.
        self.mVelX = 0
        self.mVelY = 0

        # Aceleración.
        self.mAccelX = 0
        self.mAccelY = 0

        ...
```

Luego, escribimos dos funciones para asignar la aceleración del objeto como hacemos siempre cada vez que agregamos atributos en la clase (escribimos las funciones *setters* y *getters*).

```
# Establece la aceleración x del objeto.
def setAccelX(self, aAccelX):

    self.mAccelX = aAccelX

# Establece la aceleración y del objeto.
def setAccelY(self, aAccelY):

    self.mAccelY = aAccelY
```

Por último, en la función `update()` de `CGameObject` agregamos el código que cambia la velocidad del objeto según su aceleración.

```
# Update mueve el objeto según su velocidad.
def update(self):
```



```

# Modificar la velocidad según la aceleración.
self.mVelX += self.mAccelX
self.mVelY += self.mAccelY

# Mover el objeto.
self.mX += self.mVelX
self.mY += self.mVelY

...

```

Con esto, tenemos todo pronto para que un objeto en el juego tenga aceleración. En el programa principal (`main.py`), en la función `init()`, ponemos en cero las velocidades iniciales de las tres naves y le damos una aceleración diferente a cada una: 0.1 a la primera, 0.2 a la segunda y 0.3 a la tercera.

```

# Las naves comienzan detenidas.
n1.setVelX(0)
n1.setVelY(0)
n2.setVelX(0)
n2.setVelY(0)
n3.setVelX(0)
n3.setVelY(0)

# Acelerar las naves.
n1.setAccelX(0.1)
n1.setAccelY(0)
n2.setAccelX(0.2)
n2.setAccelY(0)
n3.setAccelX(0.3)
n3.setAccelY(0)

```

Al correr el ejemplo vemos que las naves aceleran cada vez más y luego llega un momento en que no se ven correctamente en la pantalla. Más adelante deberemos colocarle una velocidad máxima a los objetos para evitar problemas de este tipo.

Con esto ya tenemos la base para mover los objetos del juego. En el próximo capítulo veremos cómo agregar la nave del jugador y moverla con el teclado.

7. Controlando el Jugador con el Teclado

Hasta ahora, hemos aprendido a mover objetos en forma autónoma, pero la inmensa mayoría de los videojuegos para computadoras, necesitan tener un jugador que sea controlado por el jugador, con el teclado en este caso. En este capítulo vamos a agregar una nave en la parte de abajo de la pantalla que se podrá mover con las teclas y más adelante le podrá disparar a las naves enemigas. Haremos una clase `CPlayer`, que será la nave del jugador, y la misma se moverá a la derecha o a la izquierda con las teclas de dirección (las teclas de flechas). Antes de eso, tenemos que ver cómo se maneja el teclado. Para eso necesitamos ver cómo es el manejo de los eventos.

Manejo de Eventos

Como un videojuego es interactivo, siempre necesitaremos algún control: el *teclado*, el *mouse*, un *joystick*, la *pantalla táctil*, etc. para controlar algunas cosas, generalmente al jugador. Para hacer esto, debemos capturar los eventos que ocurren en la computadora.



Figura 7-1: Usaremos el teclado para nuestro primer juego.

Un *evento* es algo que sucede en la computadora (en el Sistema Operativo). Esto puede ser cuando se pulsa una tecla, cuando se presiona el mouse, cuando se cierra la ventana, etc. Cuando ocurre algo de esto, el sistema operativo genera un evento y se lo envía a la aplicación o actividad (se lo envía a nuestro programa). Entonces, un evento llega a la ventana del juego (que se llama *aplicación* en *Windows* o *actividad* en las computadoras XO, o puede tener otro nombre de acuerdo al sistema operativo).

Si capturamos un evento, podremos reaccionar al mismo (por ejemplo, si se pulsa la flecha derecha, movemos al jugador a la derecha).

El manejo de eventos se hace en el game loop, y ya lo venimos haciendo en los ejemplos

anteriores para detectar la tecla *[Esc]*. Por ejemplo, este código que se encuentra en el game loop es el encargado de capturar el evento de salir y el evento de tecla presionada: En el game loop de `main.py` tenemos:

```
# Correr la lógica del juego.
def update():

    ...

    # Timer que controla el frame rate.
    clock.tick(60)

    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():

        # Si se cierra la ventana se sale del programa.
        if event.type == pygame.QUIT:
            salir = True

        # Si se pulsa la tecla [Esc] se sale del programa.
        if event.type == pygame.KEYUP:
            if event.key == pygame.K_ESCAPE:
                salir = True

    ...
```

En cada frame, debemos procesar todos los eventos que se han producido desde el frame anterior y que en este frame han llegado a la ventana. Puede ser que no haya ocurrido ningún evento, o que sean varios eventos los que hayan ocurrido. *Pygame* pone todos los eventos ocurridos en una lista y nosotros debemos procesar esa lista (a esta tarea se le llama *capturar y procesar* los eventos). Cabe aclarar que si no procesamos los eventos, el juego no responderá a nada de lo que haga el usuario (por ejemplo, usar el mouse, el teclado, cerrar la ventana, minimizar la ventana, etc.)

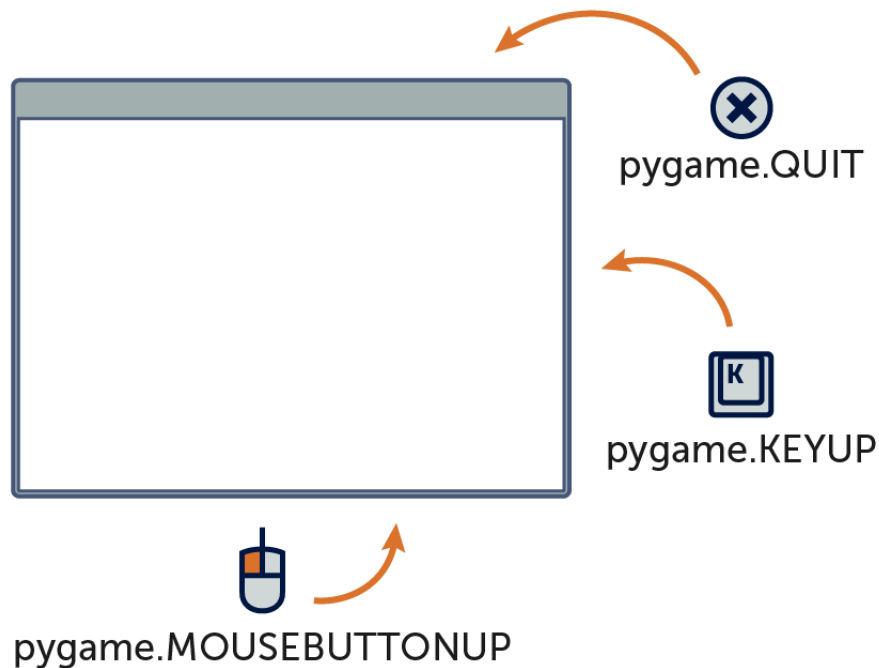


Figura 7-2: Varios eventos son enviados a la ventana de la aplicación en el frame actual.

Para procesar los eventos que han ocurrido en el frame actual, usamos una estructura `for` para recorrer la lista de eventos, que está dada por la función `pygame.event.get()`:

```
for event in pygame.event.get():
```

En cada iteración del `for`, la variable `event` contiene un objeto con la información del evento que estamos procesando.

Utilizando `event.type` podemos saber de qué tipo es el evento que ha ocurrido y según el tipo de evento podemos tener más información, como por ejemplo qué tecla se ha pulsado cuando el evento se trata de una pulsación de tecla. Esto lo veremos a continuación.

Chequear la Pulsación de Teclas

Cuando se pulsa una tecla, se recibe un evento en la ventana, y lo capturamos como dijimos antes. Esto ya lo estamos haciendo en el game loop. El código que tenemos ahora para capturar la pulsación de una tecla y luego ver si la tecla pulsada es `[Esc]`, es el siguiente:

En el game loop tenemos:

```
for event in pygame.event.get():  
  
    ...  
  
    if event.type == pygame.QUIT:
```

```

        salir = True

    if event.type == pygame.KEYUP:
        if event.key == pygame.K_ESCAPE:
            salir = True
    ...

```

Cuando se suelta una tecla, `event.type` equivale a `pygame.KEYUP`. Este evento se recibe cuando la tecla es soltada, no cuando es presionada. Una vez que tenemos un evento de teclado, `event.key` contiene el código de la tecla de la que se trata. Como en este caso queremos salir del programa si la tecla que se pulsó (en el momento en que se suelta) es `[Esc]`, para comparar esa tecla utilizamos su código, que es `pygame.K_ESCAPE` (esta es una constante, de valor 27, que es el código numérico que representa a esta tecla, y queda más claro usar la constante que usar el valor directamente).

Cada tecla tiene un código asociada a ella (se denomina *keycode*). Por ejemplo, el código de la tecla `[Esc]` es 27 (ese número es el valor de la constante `pygame.K_ESCAPE`).

Afortunadamente, no tenemos porqué acordarnos de los códigos de todas la teclas. Las teclas (como la barra espaciadora, escape o las flechas de dirección, etc.), tienen constantes especiales. Por ejemplo, en lugar de utilizar el valor 27 para la tecla `[Esc]`, lo que hacemos es utilizar la constante `pygame.K_ESCAPE` (constante definida en *Pygame*, que tiene el valor 27).

Cada tecla tiene su respectiva constante definida en *Pygame*. Las teclas que más utilizaremos en los juegos son:

Tecla	Constante	Descripción
<code>[Arriba]</code>	<code>pygame.K_UP</code>	Flecha arriba. Usada para subir, saltar o acelerar.
<code>[Abajo]</code>	<code>pygame.K_DOWN</code>	Flecha abajo. Usada para bajar, agacharse, etc.
<code>[Derecha]</code>	<code>pygame.K_RIGHT</code>	Flecha derecha. Usada para mover a la derecha.
<code>[Izquierda]</code>	<code>pygame.K_LEFT</code>	Flecha izquierda. Usada para mover a la izquierda.
<code>[Z]</code>	<code>pygame.K_z</code>	Generalmente usada para disparar (o saltar).
<code>[X]</code>	<code>pygame.K_x</code>	Generalmente usada para saltar (o disparar).
<code>[Espacio]</code>	<code>pygame.K_SPACE</code>	Usada para disparar o saltar.
<code>[Shift Izq.]</code>	<code>pygame.K_LSHIFT</code>	Shift (mayúsculas) izquierdo. Saltar o disparar.
<code>[Shift Der.]</code>	<code>pygame.K_RSHIFT</code>	Shift (mayúsculas) derecho. Saltar o disparar.
<code>[Ctrl Izq.]</code>	<code>pygame.K_LCTRL</code>	Ctrl (control) izquierdo. Saltar o disparar.
<code>[Ctrl Der.]</code>	<code>pygame.K_RCTRL</code>	Ctrl (control) derecho. Saltar o disparar.
<code>[Alt Izq.]</code>	<code>pygame.K_LALT</code>	Alt izquierdo. Saltar o disparar.
<code>[Alt Der.]</code>	<code>pygame.K_RALT</code>	Alt derecho. Saltar o disparar.
<code>[Enter]</code>	<code>pygame.K_RETURN</code>	Enter. Usada para seleccionar o aceptar.
<code>[Esc]</code>	<code>pygame.K_ESCAPE</code>	Escape. Usada para cancelar o salir del juego.

En el siguiente enlace se puede ver la lista completa de constantes de teclas definidas en Pygame:

<http://www.pygame.org/docs/ref/key.html>

Eventos del Teclado

En un juego necesitaremos saber el momento en el cual se pulsa una tecla y el momento en cuanto se levanta (se suelta) la tecla. Al mover la nave del jugador, la movemos apenas presionamos la tecla de dirección. La nave continúa moviéndose mientras la tecla se encuentre presionada. Cuando se suelte la tecla, la nave dejará de moverse.

Por otra parte, para disparar, debemos reconocer cuando la tecla de disparo es soltada, momento en el que se disparará una bala. Si se dispara la bala cuando la tecla de disparo es presionada, saldría una bala tras otra mientras la tecla de disparo no se suelte (saldría una bala en cada frame). Como no queremos que eso ocurra, dispararemos cuando la tecla se suelte. Más adelante mejoraremos esta forma de disparar.

Ahora veremos cómo capturar estos eventos de teclado. Cuando se pulsa una tecla, `event.type` será equivalente a `pygame.KEYDOWN` y cuando se suelte una tecla, `event.type` será equivalente a `pygame.KEYUP`. Luego debemos saber qué tecla es la que se presionó o la que se soltó, preguntando por el código de la tecla, que se encuentra en `event.key`.

Veamos el ejemplo ubicado en la carpeta: `capitulo_07\001_eventos_de_teclas`.

En este ejemplo, tenemos simplemente un game loop como el primero que hicimos y este programa muestra textos por la consola que dicen cuales teclas apretamos y cuales teclas soltamos. En este programa estamos capturando los eventos de teclado, y leemos las teclas que luego utilizaremos para manejar nuestra nave (usaremos las flechas izquierda y derecha). El siguiente listado muestra la parte del game loop en la cual capturamos los eventos del teclado y mostramos en la consola las teclas que son pulsadas y las teclas que son liberadas.

```
while not salir:
```

```
    clock.tick(60)
```

```
    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():
```

```
        # Evento cuando se suelta una tecla.
        if event.type == pygame.KEYUP:
            keyName = pygame.key.name(event.key)
            print("Tecla liberada:", keyName)
```

```

# Evento cuando se pulsa una tecla.
if event.type == pygame.KEYDOWN:
    keyName = pygame.key.name(event.key)
    print("Tecla pulsada:", keyName)
    if event.key == pygame.K_ESCAPE:
        print("[Esc] pulsada")
    if event.key == pygame.K_RIGHT:
        print("[Derecha] vamos a mover la nave a la derecha")
    if event.key == pygame.K_LEFT:
        print("[Izquierda] vamos a mover la nave a la
izquierda")

...

```

Con la función `pygame.key.name(event.key)` obtenemos un *string* con el nombre de la tecla pulsada. Por ejemplo, si se trata de la tecla *[Esc]*, el texto será "escape". Esto nos puede servir para *depurar* el programa (buscar errores) si tenemos algún problema con el teclado (sirve para mostrar en la consola las teclas que se presionan). En este programa se muestra el texto de cualquier tecla que se pulsa o se libera.

Si por ejemplo quisiéramos saber si se pulsa la tecla *[Derecha]* o *[Izquierda]* para mover la nave, comparamos `event.key` con las constantes `pygame.K_RIGHT` y `pygame.K_LEFT`, como se hace en el código anterior. Más adelante en este capítulo agregaremos la nave del jugador y la moveremos con el teclado.

La función `pygame.key.get_pressed()`

Una característica muy buena que tiene *Pygame*, es que podemos usar en cualquier momento (y desde cualquier clase) la función `pygame.key.get_pressed()` para saber qué teclas están siendo pulsadas y cuáles no. Esta función retorna una referencia al *array* (*tupla*) donde para cada tecla hay un valor *1* si la tecla está pulsada o *0* si la tecla no está pulsada.

De esta forma, podemos obtener el estado de todas las teclas en cualquier momento. Veamos el ejemplo ubicado en la carpeta: `capitulo_07\002_get_pressed`.

En este ejemplo simplemente guardamos en una variable la tupla que retorna la función `pygame.key.get_pressed()`. El código es el siguiente:

```

# Saber en el momento que teclas están pulsadas.
keys = pygame.key.get_pressed()

```

Esta tupla contiene un valor *booleano* (*verdadero* o *falso*) para cada tecla. El valor de cada elemento es *1* si la tecla está siendo pulsada y *0* si no lo está. Hay un valor para cada una de

las teclas.

Prueba pulsar y soltar teclas para ver como se prenden y apagan los valores booleanos de la tupla.

Para ver si una tecla está siendo pulsada, se accede a la tupla utilizando el nombre de la tecla. Por ejemplo, si queremos saber si el jugador toca la tecla derecha o izquierda se hace:

```
# Saber en el momento que teclas están pulsadas.
keys = pygame.key.get_pressed()

if keys[pygame.K_LEFT]:
    print("[Izquierda] vamos a mover la nave a la izquierda")
if keys[pygame.K_RIGHT]:
    print("[Derecha] vamos a mover la nave a la derecha")
```

Utilizando una sentencia print para mostrar la tupla, vemos los valores de cada tecla con claridad.

La pregunta que puede surgir ahora es, si esta forma de chequear las teclas funciona, ¿por qué debemos capturar el evento? La respuesta es que tenemos muchas cosas en *Pygame* que se hacen por nosotros, y que las podemos utilizar si nos hacen la programación más sencilla, pero debemos comprender cómo funciona todo el sistema.

Por ejemplo, si mañana quisiéramos desarrollar en otro lenguaje como Java o C++, no tendríamos una función como `pygame.key.get_pressed()` que nos facilite la programación, pero sabemos que las ventanas funcionan con un sistema de eventos y que en el game loop tenemos que capturar los eventos del teclado que llegan a la ventana. Luego, todo lo que venga dado en el lenguaje o motor que nos facilite la escritura mejor, pero es importante conocer completamente el funcionamiento del sistema para aprender correctamente los fundamentos. Por esta razón, esta no será la primera vez que se explique algo y se programe, solo para ver más adelante que *Pygame* ya lo tiene implementado para nosotros.

Por otra parte, más adelante en este capítulo vamos a hacer una clase `CKeyboard` donde pondremos funciones que no tiene *Pygame*, como por ejemplo una función `firstPress()` que nos dirá si una tecla ha sido pulsada por primera vez, cosa que más adelante veremos que es muy útil en los juegos para saltar o disparar.

Ahora que podemos leer las teclas, vamos a agregar la nave del jugador al juego y a moverla con las teclas izquierda y derecha.

Agregando el Jugador

Vamos a agregar la nave del jugador al juego, colocándola en la parte inferior de la pantalla. El jugador controlará la nave moviéndose a la izquierda y a la derecha con las teclas *[Izquierda]* y *[Derecha]* respectivamente. Más adelante también podremos disparar.

Veamos el ejemplo que se encuentra en la carpeta: `capitulo_07\003_agregar_el_jugador`.

La nave se encuentra en el archivo `player00.png`. La imagen tiene un tamaño de 56 x 42 píxeles. Como siempre, colocamos la imagen en la carpeta `assets/image`. En la siguiente figura se puede ver cómo luce la misma.



Figura 7-3: La nave que controla el jugador.

Lo que hacemos es escribir una clase `CPlayer`, que se encargará de manejar la lógica del jugador. Al igual que como hicimos con los enemigos, vamos a tener un tipo de jugador (para luego poder tener dos jugadores). Por ahora tendremos un solo jugador.

La clase `CPlayer` es muy parecida a la clase `CNave`. Por ahora no nos preocupamos de este problema de diseño (o sea, el tener clases muy parecidas). No es conveniente tener clases duplicadas o muy parecidas para diferentes tipos de objetos (la nave y el jugador en este caso) y que tengan el mismo código. La solución va a ser tener una clase `CSprite` de la cual heredarán todos los objetos del juego. Más adelante haremos esto.

La clase `CPlayer` va en el package `game` (porque es una clase propia del juego), y queda de la siguiente manera:

```
# -*- coding: utf-8 -*-

#-----
# Clase CPlayer.
# Nave que controla el jugador.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----
```

```

# Importar Pygame.
import pygame

# Importar la clase CGameObject.
from api.CGameObject import *

# La clase CPlayer hereda de CGameObject.
class CPlayer(CGameObject):

    # Tipos de jugador.
    TYPE_PLAYER_1 = 0
    TYPE_PLAYER_2 = 1

    # -----
    # Constructor. Recibe el tipo de nave (TYPE_PLAYER_1 o
TYPE_PLAYER_2).
    # -----
    def __init__(self, aType):

        # Invocar al constructor de la clase base.
        CGameObject.__init__(self)

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType
        if self.mType == CPlayer.TYPE_PLAYER_1:
            imgFile = "assets/images/player00.png"

        # Cargar la imagen.
        self.mImg = pygame.image.load(imgFile)
        self.mImg = self.mImg.convert_alpha()

        # Guardar el ancho y el alto.
        self.mWidth = self.mImg.get_width()
        self.mHeight = self.mImg.get_height()

    # Mover el objeto.
    def update(self):
        # Invocar a update() de la clase base para el movimiento.
        CGameObject.update(self)

    # Dibuja el objeto en la pantalla.
    # Parámetros:
    # aScreen: La superficie de la pantalla en donde dibujar.
    def render(self, aScreen):
        aScreen.blit(self.mImg, (self.mX, self.mY))

```

```

# Liberar lo que haya creado el objeto.
def destroy(self):
    # Invocar a destroy() de la clase base.
    CGameObject.destroy(self)
    self.mImg = None

# Retorna el ancho de la nave.
def getWidth(self):
    return self.mWidth

# Retorna el alto de la nave.
def getHeight(self):
    return self.mHeight

```

En el programa principal, `main.py`, importamos la clase `game.CPlayer` agregamos una variable `player` para el jugador, creamos el jugador y luego invocamos como siempre a sus funciones `update()` y `render()` en el game loop y `destroy()` al salir del juego. Se muestra a continuación el código, resaltado en negrita las líneas agregadas.

```

# -*- coding: utf-8 -*-

#-----
# Agregando la nave controlada por el jugador.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# Importar la clase CNave.
from game.CNave import *

# Importar la clase Cuadrado.
from game.CCuadrado import *

# Importar la clase CPlayer.
from game.CPlayer import *

# Definir ancho y alto de la pantalla.
SCREEN_WIDTH = 640
SCREEN_HEIGHT = 360
RESOLUTION = (SCREEN_WIDTH, SCREEN_HEIGHT)

```

```

# Control del modo ventana o fullscreen.
isFullscreen = False

screen = None
imgBackground = None
imgSpace = None
clock = None
n1 = None
n2 = None
n3 = None
c1 = None
c2 = None
c3 = None
player = None

# Inicializar la variable de control del game loop.
salir = False

# Función de inicialización.
def init():

    global screen
    global imgBackground
    global imgSpace
    global n1
    global n2
    global n3
    global c1
    global c2
    global c3
    global clock
    global player

    # Inicializar Pygame.
    pygame.init()

    # Poner el modo de video en ventana e indicar la resolución.
    screen = pygame.display.set_mode(RESOLUTION)
    # Poner el título de la ventana.
    pygame.display.set_caption("Mi Juego")

    # Crear la superficie del fondo o background.
    imgBackground = pygame.Surface(screen.get_size())
    imgBackground = imgBackground.convert()

    # Cargar la imagen del fondo. La imagen es de 640 x 360.
    imgSpace = pygame.image.load("assets/images/space_640x360.jpg")
    imgSpace = imgSpace.convert()

```

```

# Dibujar la imagen cargada en la imagen de background.
imgBackground.blit(imgSpace, (0, 0))

# Crear las naves: se le pasa como parámetro la imagen de la
nave.
n1 = CNavе(CNavе.TYPE_PLATINUM)
n2 = CNavе(CNavе.TYPE_GOLD)
n3 = CNavе(CNavе.TYPE_RED)

# Colocar las naves en su posición inicial.
n1.setXY(0, 100)
n2.setXY(0, 150)
n3.setXY(0, 200)

# Las naves comienzan detenidas.
n1.setVelX(0)
n1.setVelY(0)
n2.setVelX(0)
n2.setVelY(0)
n3.setVelX(0)
n3.setVelY(0)

# Acelerar las naves.
n1.setAccelX(0.1)
n1.setAccelY(0)
n2.setAccelX(0.2)
n2.setAccelY(0)
n3.setAccelX(0.3)
n3.setAccelY(0)

# Marcar los límites del mundo.
n1.setBounds(-n1.getWidth(), -n1.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)
n2.setBounds(-n2.getWidth(), -n2.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)
n3.setBounds(-n3.getWidth(), -n3.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)

c1 = CCuadrado(20, 20, (255,0,0))
c2 = CCuadrado(20, 20, (0, 255, 0))
c3 = CCuadrado(20, 20, (0, 0, 255))

c1.setXY(0, 50)
c2.setXY(0, 80)
c3.setXY(0, 110)

c1.setVelX(0.5)

```

```

c1.setVelY(0)
c2.setVelX(1.0)
c2.setVelY(0)
c3.setVelX(1.5)
c3.setVelY(0)

# Crear el jugador.
player = CPlayer(CPlayer.TYPE_PLAYER_1)
player.setXY(SCREEN_WIDTH / 2 - player.getWidth() / 2,
SCREEN_HEIGHT - player.getHeight())

# Establecer los límites de la pantalla.
player.setBounds(0, 0, SCREEN_WIDTH - player.getWidth(),
SCREEN_HEIGHT)

# Inicializar el reloj.
clock = pygame.time.Clock()

# Correr la lógica del juego.
def update():
    global salir
    global screen
    global isFullscreen

    # Timer que controla el frame rate.
    clock.tick(60)

    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():

        # Si se cierra la ventana se sale del programa.
        if event.type == pygame.QUIT:
            salir = True

        # Si se pulsa la tecla [Esc] se sale del programa.
        if event.type == pygame.KEYUP:
            if event.key == pygame.K_ESCAPE:
                salir = True

        # Si se pulsa la tecla [F], se cambia entre ventana y
        fullscreen.
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_f:
                isFullscreen = not isFullscreen
                if isFullscreen:
                    screen = pygame.display.set_mode(RESOLUTION,
pygame.FULLSCREEN)
                else:

```

```

        screen = pygame.display.set_mode(RESOLUTION)

        # Movimiento de la nave con las teclas.
        if event.type == pygame.KEYDOWN:
            if (event.key == pygame.K_RIGHT):
                player.setVelX(4)
            if (event.key == pygame.K_LEFT):
                player.setVelX(-4)

        if event.type == pygame.KEYUP:
            if (event.key == pygame.K_RIGHT):
                player.setVelX(0)
            if (event.key == pygame.K_LEFT):
                player.setVelX(0)

        # Lógica de las naves.
        n1.update()
        n2.update()
        n3.update()

        c1.update()
        c2.update()
        c3.update()

        # Lógica del jugador.
        player.update()

    # Dibujar el frame y actualizar la pantalla.
    def render():
        # Dibujar el fondo.
        screen.blit(imgBackground, (0, 0))

        # Dibujar las naves en la nueva posición.
        n1.render(screen)
        n2.render(screen)
        n3.render(screen)

        c1.render(screen)
        c2.render(screen)
        c3.render(screen)

        # Dibujar el jugador.
        player.render(screen)

        # Actualizar la pantalla.
        pygame.display.flip()

# Función de destrucción.

```

```

def destroy():
    global n1
    global n2
    global n3
    global c1
    global c2
    global c3
    global player

    # Destruir los objetos.
    n1.destroy()
    n1 = None
    n2.destroy()
    n2 = None
    n3.destroy()
    n3 = None
    c1.destroy()
    c1 = None
    c2.destroy()
    c2 = None
    c3.destroy()
    c3 = None

    # Destruir la nave del jugador.
    player.destroy()
    player = None

    # Cerrar Pygame y liberar los recursos que pidió el programa.
    pygame.quit()

# ===== Punto de entrada del programa. =====

# Inicializar los elementos necesarios del juego.
init()

# Loop principal del juego.
while not salir:
    # Actualizar los objetos.
    update()
    # Dibujar la pantalla.
    render()

# Liberar los recursos al final.
destroy()

```

Como vemos en el código anterior, en las partes resaltadas en negrita se muestra el código que crea y controla a la nave del jugador. Se importa la clase al inicio, se crea una variable

player, se crea el objeto y colocamos la nave en el medio de la pantalla y en la parte inferior.

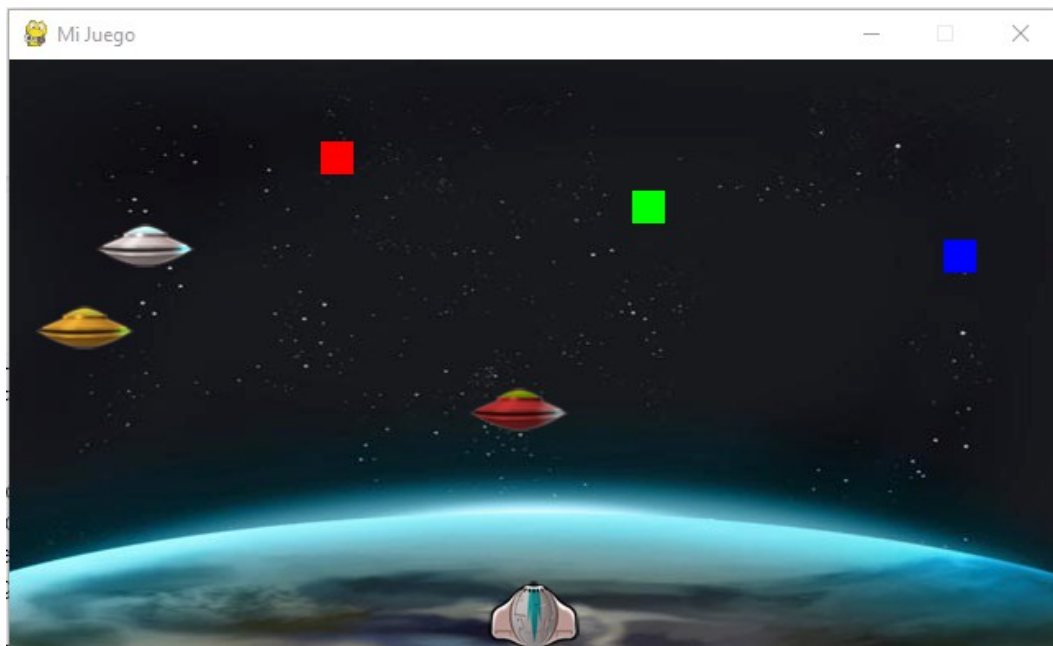


Figura 7-4: Nuestro juego ahora tiene la nave que controla el jugador.

El siguiente código es el que crea el jugador:

```
# Crear el jugador.  
player = CPlayer(CPlayer.TYPE_PLAYER_1)  
player.setXY(SCREEN_WIDTH/2 - player.getWidth()/2, SCREEN_HEIGHT -  
            player.getHeight())
```

Al colocar el jugador en su posición inicial, se toma en cuenta su ancho y alto para que la nave aparezca exactamente en el medio de la pantalla y en el piso, sobre la parte inferior de la pantalla (por eso se resta el alto de la nave al alto de la pantalla). Recuerda que el *punto de registro* (donde se posiciona la nave) es la esquina superior izquierda del dibujo.

Para mover al jugador, detectamos si se pulsa una tecla y si eso ocurre se le pone una velocidad al jugador. Cuando se suelta la tecla se pone la velocidad en cero, lo que hace que se detenga. A continuación se muestra el código que mueve la nave con el teclado:

```
def update():
```

```
...
```

```

# Procesar los eventos que llegan a la aplicación.
for event in pygame.event.get():

    ...

    # Movimiento de la nave con las teclas.
    if event.type == pygame.KEYDOWN:
        if (event.key == pygame.K_RIGHT):
            player.setVelX(4)
        if (event.key == pygame.K_LEFT):
            player.setVelX(-4)

    if event.type == pygame.KEYUP:
        if (event.key == pygame.K_RIGHT):
            player.setVelX(0)
        if (event.key == pygame.K_LEFT):
            player.setVelX(0)

    ...

# Lógica del jugador.
player.update()

```

No podemos olvidarnos de invocar a la función `update()` de la nave, para que se actualice su movimiento, ni olvidarnos de invocar a `render()` para dibujarla. En los primeros objetos que agreguemos al juego, es muy posible que olvidemos algo, y entonces la nave no se moverá o no se verá en pantalla. Cuando esto ocurra, vuelve al código de los ejemplos y comprueba línea a línea las diferencias.

Cuando ejecutamos el programa, notamos que el juego tiene varios problemas. El primer problema es que si pulsamos dos teclas a la vez, a veces la nave se tranca.

El segundo problema es que el código que controla al jugador se encuentra en el programa `main.py`, cuando debería estar en la clase `CPlayer` (es más natural que si luego queremos corregir cómo se mueve el jugador, vayamos a la clase `CPlayer`, no al código de `main.py`). Estos dos problemas los solucionaremos cuando hagamos una clase para la lectura del teclado. Ahora no podemos solucionarlo con lo que tenemos, por lo cual lo dejaremos así por el momento.

Existe un tercer problema, que es que si la nave se va de de la pantalla, aparecerá del otro lado de la misma. Esto es así porque el código que controla los bordes se encuentra en la clase `CGameObject`, y es el mismo comportamiento para todos los objetos del juego. Debemos poder indicar qué comportamiento queremos cuando se alcanza un borde, dado que el juego tendrá diferentes objetos que se comportan en diferentes formas. Esto lo vamos a corregir más adelante cuando agreguemos varios comportamientos al tocar los bordes y podamos elegir cuál usamos (algunos objetos al tocar el borde desaparecen, como las balas, otros se detendrán, como la nave del jugador, otros aparecerán por el lado contrario, etc.).

El último problema que existe es que en este momento las clases `CNave` y `CPlayer` son muy similares entre sí. Debemos hacer una clase `CSprite` que se encargue del manejo de las imágenes y de las animaciones. Esto también lo haremos más adelante.

A continuación arreglaremos el primer y segundo problema, que se solucionan haciendo una clase `CKeyboard` para manejar el teclado. Más adelante solucionaremos los problemas restantes cuando veamos el manejo de Sprites.

La Clase `CKeyboard`

Escribiremos una clase `CKeyboard` que contendrá en todo momento la información de qué teclas son pulsadas o soltadas. Esto es de suma utilidad para controlar objetos con el teclado fácilmente.

Cuando un evento de teclado llega a la ventana, el evento será capturado y será pasado a la clase `CKeyboard`, quien se encargará de guardar el estado de las teclas que necesitemos para el juego. Esta técnica se denomina *hardware polling* (hacer *polling* significa preguntar siempre por el estado de algo, como por ejemplo las teclas en este caso). En realidad, para mover un personaje en los juegos de acción, no nos interesa saber cuando se suelta o se pulsa una tecla, sino saber si la tecla está pulsada o no en un momento dado (en el frame actual).

De esta forma, podremos saber en cada momento si una tecla está pulsada o no, lo cual permitirá que cualquier objeto del juego pueda chequear teclas. Con la clase `CKeyboard` funcionando, el código que se encarga de manejar la nave del jugador no estará en `main.py`, sino en la clase `CPlayer`, como es natural pensar que ahí es donde debe estar. El código le pondrá velocidad al jugador cuando la tecla está pulsada y se detendrá cuando la tecla no esté pulsada. Para esto se consultarán las variables que tendremos en la clase `CKeyboard`.

Veamos el ejemplo que se encuentra en la carpeta: `capitulo_07\004_clase_keyboard`. Si corremos el ejemplo, veremos que la nave se mueve bien y no se tranca. Abre el programa `main.py`.

Lo primero que se hace es importar la clase:

```
# Importar la clase CKeyboard
from api.CKeyboard import *
```

Luego, en la función `update()`, se captura el evento de cuando se pulsa una tecla y de cuando se suelta y se llama a la función `keyDown()` y `keyUp()` respectivamente, para informarle a la clase `CKeyboard` que se pulsaron o soltaron teclas:

```

def update():

    ...

    for event in pygame.event.get():

        ...

        # Registrar cuando se apreta o suelta una tecla.
        if event.type == pygame.KEYDOWN:
            CKeyboard.inst().keyDown(event.key)
        if event.type == pygame.KEYUP:
            CKeyboard.inst().keyUp(event.key)

```

La clase `CKeyboard` se encuentra en el package `api`, dado que es una clase que se reutilizará entre distintos juegos. Esta clase contiene estas dos funciones, `keyDown()` y `keyUp()`, que se llaman desde el game loop cuando se pulsa una tecla y cuando se libera. Como vemos en este código, para usar la clase `CKeyboard` no es necesario crear ningún objeto (es una clase *singleton*). Más adelante hablaremos sobre esto. El código de la clase `CKeyboard` es el siguiente:

```

# -*- coding: utf-8 -*-

#-----
# Clase CKeyboard.
# Clase para manejar el estado de las teclas en el juego.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

class CKeyboard(object):

    mInstance = None
    mInitialized = False

    mLeftPressed = False
    mRightPressed = False

    def __new__(self, *args, **kwargs):
        if (CKeyboard.mInstance is None):

```

```

        CKeyboard.mInstance = object.__new__(self, *args, **kwargs)
        self.init(CKeyboard.mInstance)
    else:
        print("Cuidado: CKeyboard(): No se debería instanciar más
de una vez esta clase. Usar CKeyboard.inst().")
        return CKeyboard.mInstance

    @classmethod
    def inst(cls):
        if (not cls.mInstance):
            return cls()
        return cls.mInstance

    def init(self):
        if (CKeyboard.mInitialized):
            return
        CKeyboard.mInitialized = True

        CKeyboard.mLeftPressed = False
        CKeyboard.mRightPressed = False

    def keyDown(self, key):
        if (key == pygame.K_LEFT):
            CKeyboard.mLeftPressed = True
        if (key == pygame.K_RIGHT):
            CKeyboard.mRightPressed = True

    def keyUp(self, key):
        if (key == pygame.K_LEFT):
            CKeyboard.mLeftPressed = False
        if (key == pygame.K_RIGHT):
            CKeyboard.mRightPressed = False

    def leftPressed(self):
        return CKeyboard.mLeftPressed

    def rightPressed(self):
        return CKeyboard.mRightPressed

    def destroy(self):
        CKeyboard.mInstance = None

```

Esta clase lo que hace es almacenar dos variables con el estado de la tecla derecha e izquierda (hay una variable para cada tecla). Las variables `mRightPressed` y `mLeftPressed` contienen el estado de las teclas derecha e izquierda respectivamente. Estas variables contienen `True` si la tecla está siendo pulsada y `False` si no lo está.

Luego tenemos dos funciones para preguntar por el estado de estas teclas. La función `rightPressed()` retornará `True` si la tecla derecha está apretada y `False` si no lo está. De forma similar, la función `leftPressed()` retorna el estado de la tecla izquierda.

Ignora por ahora el comienzo de la clase `CKeyboard`, ya explicaremos qué hace esa parte. Para mover al jugador utilizando el teclado, en la clase `CPlayer`, en la función `update()`, se mueve el jugador según las teclas que se hayan pulsado. En la clase `CPlayer` tendremos:

```
...

# Importar la clase CKeyboard
from api.CKeyboard import *

...

def update(self):

    # Mover el jugador con el teclado.
    if not CKeyboard.inst().leftPressed() and not CKeyboard.inst().
rightPressed():
        self.setVelX(0)
    else:
        if CKeyboard.inst().leftPressed():
            self.setVelX(-4)
        elif CKeyboard.inst().rightPressed():
            self.setVelX(4)

    # Invocar a update() de la clase base para el movimiento.
    CGameObject.update(self)

...
```

Como podemos ver, en cualquier objeto y en cualquier momento podemos consultar si las teclas *[Derecha]* o *[Izquierda]* están pulsadas. Esto es posible simplemente utilizando `CKeyboard.inst().leftPressed()` que retorna `True` si la tecla izquierda está pulsada y `False` si no lo está.

Si corremos el ejemplo, podemos ver que la nave ya no se tranca. Esto es porque ahora tenemos la información correcta en la clase `CKeyboard`. Si la tecla derecha está siendo pulsada, se pone la velocidad positiva y la nave se mueve a la derecha. Si se está pulsando la tecla izquierda, se pone la velocidad negativa y la nave se mueve hacia la izquierda. Si ninguna de las dos teclas se pulsa, la velocidad se pone en cero, por lo cual la nave se queda quieta. No hay margen para el error en la lógica escrita de esta manera y es clave escribir los juegos sin errores.

Ahora bien, ¿por qué podemos usar la clase `CKeyboard` directamente de esta forma? (sin crear un objeto de la clase `CKeyboard`). La respuesta es que se ha utilizado una clase que es *Singleton*. Esto es lo que se explica a continuación.

Una Clase *Singleton*

En muchas ocasiones es preferible tener una sola instancia de una clase. Este es el caso de la clase `CKeyboard`, porque queremos que en el programa exista sólo una instancia de esta clase. No necesitamos dos o más clases para manejar el teclado, necesitamos una sola.

Por esta razón es que esta clase es un *singleton*. Un *singleton* es un *patrón de diseño* que se utiliza cuando es necesario tener una única instancia de una determinada clase. En este caso, queremos que exista un único objeto de la clase `CKeyboard`, o sea, una sola instancia de esta clase.

Para eso, tenemos un atributo de la clase, `mInstance`, el cual contendrá la referencia al único objeto creado de la clase (en este caso, el único objeto `CKeyboard` que existirá). Cuando accedemos a la clase y se intenta crear un objeto `CKeyboard`, éste se crea sólo la primera vez (cuando `mInstance` es `None`) en el constructor (ver la función `__new__`). En el programa, simplemente usamos `CKeyboard.inst()` para acceder a la clase `CKeyboard`. Si la instancia no existe, se crea, y si ya existiera, simplemente se retorna. De esta forma la clase es un singleton, y se asegura que solamente haya una instancia. A continuación se puede ver el código de `CKeyboard` que implementa el patrón *Singleton*.

...

```
class CKeyboard(object):

    mInstance = None
    mInitialized = False

    ...

    def __new__(self, *args, **kargs):
        if (CKeyboard.mInstance is None):
            CKeyboard.mInstance = object.__new__(self, *args,
**kargs)
            self.init(CKeyboard.mInstance)
        else:
            print("Cuidado: CKeyboard(): No se debería instanciar más
de una vez esta clase. Usar CKeyboard.inst().")
            return CKeyboard.mInstance

    @classmethod
```

```

def inst(cls):
    if (not cls.mInstance):
        return cls()
    return cls.mInstance

def init(self):
    if (CKeyboard.mInitialized):
        return
    CKeyboard.mInitialized = True

...

```

El patrón Singleton lo utilizaremos para todas las clases del juego que deban ser únicas, por ejemplo, la clase `CMouse` que se encarga del manejo del mouse (porque hay un solo mouse), la clase `CBulletManager` que se encargará de los disparos del juego (porque hay un solo manejador de balas), en general todos los managers que sean únicos, etc.

Al tener la clase `CKeyboard` definida como *singleton*, un sprite (u otra clase cualquiera del juego) en cualquier momento puede preguntar por el estado de las teclas. En general, haremos singleton a todas las clases del juego que deban existir una vez sola y para que cualquier clase pueda acceder a ellas (este patrón de diseño brinda un acceso global a la clase desde cualquier otra clase). Ejemplos de clases singleton son: `CKeyboard`, `CMouse`, los managers (listas) de enemigos o balas, el nivel, el sistema de partículas, etc.

Mejorando la Clase `CKeyboard`

Por ahora en la clase `CKeyboard` tenemos funciones para chequear el estado de las teclas de dirección para la izquierda y para la derecha, pero nos faltan muchas otras teclas que vamos a necesitar. Por ahora esto lo dejaremos así, y conforme vayamos haciendo el juego le iremos agregando funcionalidad a esta clase.

A veces es necesario programar varios juegos para saber todo lo que puede necesitar una clase de base como la clase `CKeyboard`. Por ahora dejaremos la clase así, y cuando agreguemos otro jugador, o disparemos, necesitaremos modificar la clase `CKeyboard`. Ya veremos esto cuando lleguemos a ese punto.

En el próximo capítulo veremos el manejo de sprites.

8. Movimiento de Sprites

En este capítulo vamos el manejo *sprites*. Los *sprites* son los elementos fundamentales de un videojuego. Son las imágenes que vemos en pantalla. Son los jugadores, los enemigos, las balas, etc. Vamos a escribir una clase base llamada `CSprite` de la cual heredarán todos los objetos visibles y móviles del juego. Luego de eso, escribiremos código para manejar los movimientos avanzados de los objetos, que ya nos quedarán disponibles para que cualquier elemento en el juego los use.

¿Qué es un Sprite?

Un *sprite* es simplemente algo que se mueve por la pantalla. El término *sprite* viene desde los orígenes de la programación de videojuegos y se usa como sinónimo de una representación en 2D que se muestra en pantalla. Originalmente se le llamaba *sprite* a una imagen, aunque luego, con el paso del tiempo, se usó el término *sprite* para denominar a un objeto en el juego. Un *sprite* entonces es cualquier objeto que se muestra en el juego, como por ejemplo el jugador, las naves espaciales, las balas que éstos disparan, las explosiones, etc.



Figura 8-1: Ejemplo de sprites en un juego.

¿Cuál es la diferencia entre un *Sprite* y un *Game Object*?

Esta es una pregunta que frecuentemente discuten los programadores de videojuegos. Hay muchas formas de hacer (programar) las cosas, pero básicamente el código que hace

determinadas cosas como por ejemplo dibujar o mover un objeto, en alguna parte del programa tiene que estar. Estas discusiones generalmente giran en torno a cómo organizar el código, y básicamente es un tema de gustos de cada programador. En nuestro caso se hará de manera simple, como se explica a continuación.

El *sprite* irá en una clase `CSprite` y está relacionado con cómo se ve el objeto, o sea, su forma. Generalmente esta clase se encarga de la imagen o de la animación, mientras que la clase `CGameObject` se encarga del movimiento. En general se piensa en `CSprite` como una imagen (o secuencia de imágenes) y en `CGameObject` como un elemento abstracto.

En nuestros juegos, si utilizamos este motor que estamos construyendo, todos los elementos que se mueven en el juego van a estar en su propia clase, la cual va a heredar de `CSprite` (que a su vez hereda de `CGameObject`). De esta forma, cuando heredamos de `CSprite`, obtendremos mucho código que ya está escrito en las clases superiores (en las clases base), lo cual hace que sea mucho más sencillo programar los juegos. A continuación definiremos y luego escribiremos la clase `CSprite`.

¿Qué información tiene un *Sprite*?

Todo *sprite* tiene una representación visual (una imagen o una animación) que es lo que se ve en la pantalla. Tiene una posición (un par de coordenadas (x,y) que define dónde está ubicado el *sprite*) y el tamaño (el ancho y el alto de la imagen que define su tamaño, y ésto es lo que se utiliza para chequear las colisiones).

Los *sprites* luego tienen varios comportamientos, algunos son genéricos para todos los *sprites* del juego y otros son específicos para cada objeto. Por ejemplo, los *sprites* se mueven por el mundo del juego y tienen la capacidad de saber si chocan con otro *sprite* (una bala, el jugador, etc.). A esto último se le llama *manejo de colisiones* y se verá más adelante cuando veamos el *manejador de sprites*. El *manejador de sprites* se encarga de la creación de *sprites*, eliminación de *sprites* y el chequeo de colisiones, entre otras cosas que hace.

Por supuesto, siempre habrá características y comportamientos especiales para cada *sprite*. Por ejemplo, la nave del jugador es controlada por el teclado, un enemigo se mueve autónomo y puede perseguir al jugador, una bala sigue derecho hasta que choca con algo, etc. Los *sprites* (los objetos en el juego) son autocontenidos, esto significa que cada uno maneja su propia posición (en la función `update()`), según las reglas que tiene el objeto en el juego, o dicho de otro modo, según su comportamiento o lógica.

Comenzaremos por hacer un *sprite* básico que sirva como base para todos y más adelante implementaremos el *manejador de sprites*.

La clase `CSprite`

Ahora construiremos la clase `CSprite`. Una vez que la tengamos funcionando, podremos

usarla para cualquier cosa que se mueva en el juego. Veamos el ejemplo que se encuentra en la carpeta: `capitulo_08\001_clase_sprite`.

Si ejecutamos este ejemplo, veremos que lo que hace es exactamente lo mismo que lo que hace el ejemplo anterior, salvo que se han reorganizado las clases. Ahora las clases `CNave` y `CPlayer` heredan de `CSprite` y a su vez `CSprite` hereda de `CGameObject`.

La clase `CSprite` se encarga de cargar la imagen y de dibujarla en la pantalla. El movimiento general del objeto se encuentra en la clase `CGameObject` de la cual hereda. La clase `CSprite` irá en la carpeta `api`, porque es una clase que utilizaremos en todos nuestros juegos.

El siguiente es el código de la clase `CSprite`:

```
# -*- coding: utf-8 -*-

#-----
# Clase CSprite.
# Simple sprite con una imagen. Hereda de CGameObject.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# Importar la clase CGameObject.
from api.CGameObject import *

class CSprite(CGameObject):

    # Constructor:
    # Parámetros: Nombre de la imagen a cargar.
    def __init__(self, aImgFile):

        CGameObject.__init__(self)

        # Imagen (superficie).
        self.mImg = None

        # Cargar la imagen.
        self.mImg = pygame.image.load(aImgFile)
        self.mImg = self.mImg.convert_alpha()

        # Guardar el ancho y el alto.
        self.mWidth = self.mImg.get_width()
```

```

        self.mHeight = self.mImg.get_height()

# Mover el objeto.
def update(self):

    # Invocar a update() de la clase base para el movimiento.
    CGameObject.update(self)

# Dibuja el objeto en la pantalla.
# Parámetros: La superficie de la pantalla donde dibujar la
imagen.
def render(self, aScreen):

    aScreen.blit(self.mImg, (self.getX(), self.getY()))

# Obtener el ancho del sprite.
def getWidth(self):

    return self.mWidth

# Obtener el alto del sprite.
def getHeight(self):

    return self.mHeight

# Liberar lo que haya creado el objeto.
def destroy(self):
    CGameObject.destroy(self)
    self.mImg = None

```

Como vemos, `CSprite` es una clase que en este momento tiene el código de la carga de la imagen y el código para dibujar la imagen en la pantalla que tenían las clases `CNave` y `CPlayer`.

Ahora, luego de implementada la clase `CSprite`, no existirá código duplicado del manejo de la imagen como había antes (en las clases `CNave` y `CPlayer`) y cada clase solamente se encarga de las tareas que le son pertinentes.

La función `render()`, hace acceso a las variables `(x,y)` que se encuentran en `CGameObject`. Estas variables, en `CGameObject`, se llaman `mX` y `mY` respectivamente. Como no es buena práctica de programación el tener que acordarse de cómo se llaman las variables internas de un objeto, hemos escrito dos funciones *getter* para obtener estos valores: `getX()` y `getY()`. En `CGameObject` se han agregados estas dos funciones:

...

```

class CGameObject(object):

    def __init__(self):

        ...

    # Obtener la coordenada X.
    def getX(self):
        return self.mX

    # Obtener la coordenada Y.
    def getY(self):
        return self.mY

    ...

```

Una vez que tenemos implementada la clase `CSprite`, heredamos `CNave` y `CPlayer` de `CSprite`. Todos los objetos móviles que hagamos en el juego, heredarán de la clase `CSprite`. El código de `CPlayer`, por ejemplo, quedará mucho más simple ahora, dado que el código de manejo de la imagen y dibujar la imagen en pantalla, que compartía con la clase `CNave`, se ha puesto ahora en la clase `CSprite`.

La clase `CNave`, ahora no hace nada específico o nada propio (por ahora). El código de `CNave` ahora queda así:

```

# -*- coding: utf-8 -*-

# -----
# Clase CNave.
# Nave enemiga que se mueve por la pantalla chequeando los bordes.
#
# Autor: Fernando Sansberro - Batovi Games.
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
# -----

# Importar Pygame.
import pygame

# Importar la clase CSprite.
from api.CSprite import *

# La clase CNave hereda de CSprite.
class CNave(CSprite):

    # Tipos de naves.

```

```

TYPE_PLATINUM = 0
TYPE_GOLD = 1
TYPE_RED = 2

# -----
# Constructor. Recibe el tipo de nave (TYPE_PLATINUM o TYPE_GOLD).
# -----
def __init__(self, aType):

    # Segun el tipo de la nave, la imagen que se carga.
    self.mType = aType
    if self.mType == CNavе.TYPE_PLATINUM:
        imgFile = "assets/images/grey_ufo.png"
    elif self.mType == CNavе.TYPE_GOLD:
        imgFile = "assets/images/yellow_ufo.png"
    elif self.mType == CNavе.TYPE_RED:
        imgFile = "assets/images/red_ufo.png"

    # Invocar al constructor de CSprite con la imagen a cargar.
    CSprite.__init__(self, imgFile)

# Mover el objeto.
def update(self):

    # Invocar update() de CSprite.
    CSprite.update(self)

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # Invocar render() de CSprite.
    CSprite.render(self, aScreen)

# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar destroy() de CSprite.
    CSprite.destroy(self)
    self.mImg = None

```

Por su parte, la clase `CPlayer`, ahora solamente contiene el código que controla el jugador con las teclas. El resto del código que manejaba la imagen y la mostraba en pantalla ya no se encuentra ahí, sino que se ha puesto en la clase base `CSprite`. Hacemos con esta clase lo mismo que hicimos con la otra. El código de `CPlayer` queda así:

```

# -*- coding: utf-8 -*-

#-----
# Clase CPlayer.
# Nave que controla el jugador.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# Importar la clase CSprite.
from api.CSprite import *

# Importar la clase CKeyboard
from api.CKeyboard import *

# La clase CPlayer hereda de CSprite.
class CPlayer(CSprite):

    # Tipos de jugador.
    TYPE_PLAYER_1 = 0
    TYPE_PLAYER_2 = 1

    # -----
    # Constructor. Recibe el tipo de nave (TYPE_PLAYER_1 o
TYPE_PLAYER_2).
    # -----
    def __init__(self, aType):

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType
        if self.mType == CPlayer.TYPE_PLAYER_1:
            imgFile = "assets/images/player00.png"

        # Invocar al constructor de CSprite con la imagen a cargar.
        CSprite.__init__(self, imgFile)

    # Mover el objeto.
    def update(self):

        # Mover el jugador con el teclado.
        if not CKeyboard.inst().leftPressed() and not

```

```

CKeyboard.inst().rightPressed():
    self.setVelX(0)
else:
    if CKeyboard.inst().leftPressed():
        self.setVelX(-4)
    elif CKeyboard.inst().rightPressed():
        self.setVelX(4)

    # Invocar update() de CSprite.
    CSprite.update(self)

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # Invocar render() de CSprite.
    CSprite.render(self, aScreen)

# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar destroy() de CSprite.
    CSprite.destroy(self)
    self.mImg = None

```

Como vemos, si comparamos las clases con como estaban antes, ahora están más simples porque el código que tenían para cargar la imagen y dibujarla se ha sacado de estas clases y se ha puesto en la clase base `CSprite`. Además, también hemos eliminado el código duplicado.

Chequeo de los Bordes de la Pantalla

Cuando un sprite se pasa de los bordes de la pantalla (o en general, cuando se pasa de los bordes del mundo), debemos determinar qué hacer. Los comportamientos pueden ser muchos, y depende del juego que estemos haciendo. Por ejemplo, podemos hacer que una nave aparezca por el otro lado, que rebote, que se quede detenida, etc.

Vamos a implementar los comportamientos básicos que se utilizan en la mayoría de los juegos. Como este comportamiento es general, lo escribiremos en la clase `CGameObject`, y todos los objetos del juego tendrán este comportamiento disponible (porque lo heredarán, recordemos que cualquier objeto del juego será una clase heredada de `CGameObject`). A cada comportamiento le daremos un nombre (definido por una constante). Los comportamientos básicos que implementaremos serán los siguientes:

NONE: Cuando el sprite llega al borde, le permitimos al sprite seguir su camino. De esta forma el sprite se puede ir de la pantalla. Este va a ser el comportamiento *por defecto* (si no le indicamos otro comportamiento al sprite, éste es el comportamiento que tendrá el sprite).

STOP: Si el sprite alcanza un borde, no se lo deja seguir y quedará detenido (queda como pegado al borde, y no puede pasar).

WRAP: Si el sprite alcanza el borde, aparecerá desde el otro lado como hacen los asteroides en el juego *Asteroids*. Los sprites entonces aparecen por el lado contrario al borde por el cual salieron.

BOUNCE: Cuando el sprite toca el borde, rebota, saliendo en sentido contrario, como por ejemplo una pelota que rebota por las paredes.

DIE: El objeto se muere al tocar el borde. Esto ocurre típicamente con las balas por ejemplo, que cuando tocan un borde se destruyen.

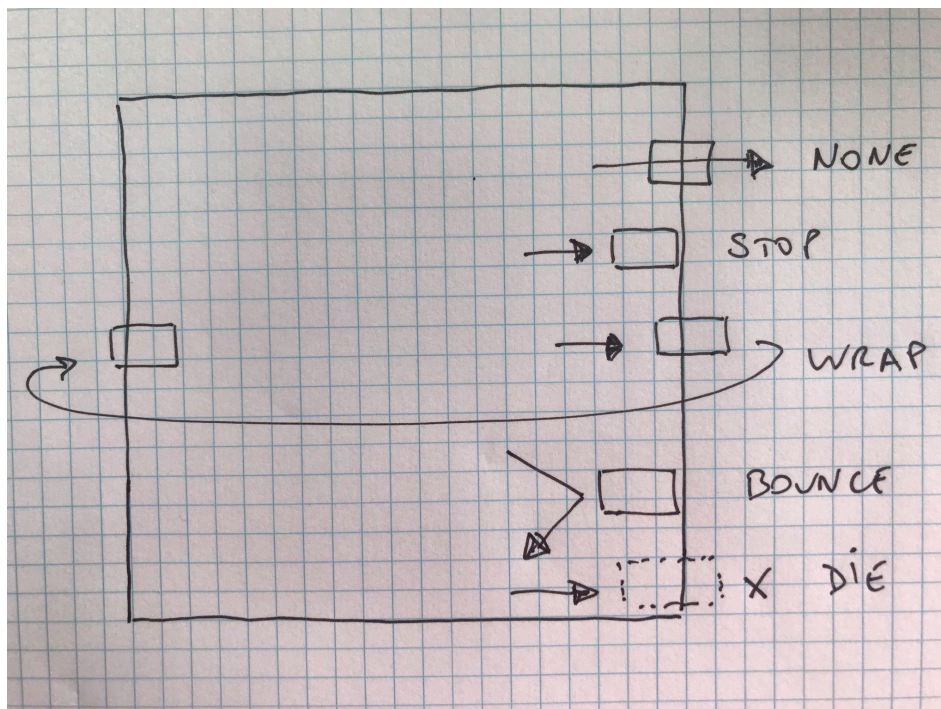


Figura 8-2: Diagrama con los comportamientos de bordes.

Es importante recordar que los objetos del juego (la clase `CGameObject`) tienen una función `setBounds()` que define los límites de movimiento. Esto lo hemos hecho desde que hicimos nuestro primer cuadrado y usamos estos valores cuando controlamos los bordes.

Estos límites pueden ser la pantalla o un área mayor que la pantalla (por ejemplo un mundo más grande que una pantalla, donde la pantalla es la cámara). Los comportamientos de borde se dan cuando el objeto se encuentra con uno de estos bordes que han sido definidos con la función `setBounds()`. Usamos estos valores y no el tamaño de la pantalla, así más adelante podemos tener objetos que se vayan de la pantalla y tengan sus bordes en el mundo, o sea, en un área mayor que la pantalla.

Ahora vamos a agregar estos comportamientos a los sprites. Colocaremos en el juego cinco naves espaciales y cada una tendrá uno de estos comportamientos cuando toque el borde. Al programa que ahora tiene tres naves, le colocamos dos naves más, creando las constantes de tipo de nave y las variables necesarias para eso e inicializando correctamente las naves.

Veamos el ejemplo que se encuentra en la carpeta: `capitulo_08\002_comportamiento_de_borde`.

En este ejemplo tenemos cinco naves, las tres que teníamos más dos que hemos agregado ahora. Una nave celeste (cyan) y la otra verde (green). Los comportamientos que tienen las naves cuando llegan a los bordes son:

Nave plateada (`CNave.TYPE_PLATINUM`): Al llegar al borde continúa su movimiento normalmente (`CGameObject.NONE`).

Nave dorada (`CNave.TYPE_GOLD`): Se detiene al llegar al borde (`CGameObject.STOP`).

Nave celeste (`CNave.TYPE_RED`). Al llegar al borde aparece por el lado contrario (`CGameObject.WRAP`).

Nave verde (`CNave.TYPE_GREEN`). Al llegar al borde rebota (`CGameObject.BOUNCE`).

Nave roja (`CNave.TYPE_CYAN`): Se elimina al llegar al borde (`CGameObject.DIE`).

Nota: En este ejemplo se han removido los cuadrados porque ya no los vamos a necesitar.

En el programa principal `main.py`, agregamos las líneas correspondientes a las dos naves que agregamos y en la función `init()`, cuando inicializamos las naves, usamos la función `setBoundAction()` para definir en cada nave un comportamiento diferente cuando alcance el borde:

```
# -*- coding: utf-8 -*-

#-----
# Ejemplo de los comportamientos de borde de los sprites.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# Importar la clase CNave.
from game.CNave import *

# Importar la clase Cuadrado.
from game.CCuadrado import *
```

```

# Importar la clase CPlayer.
from game.CPlayer import *

# Importar la clase CKeyboard
from api.CKeyboard import *

# Definir ancho y alto de la pantalla.
SCREEN_WIDTH = 640
SCREEN_HEIGHT = 360
RESOLUTION = (SCREEN_WIDTH, SCREEN_HEIGHT)
# Control del modo ventana o fullscreen.
isFullscreen = False

screen = None
imgBackground = None
imgSpace = None
clock = None
n1 = None
n2 = None
n3 = None
n4 = None
n5 = None
player = None

# Inicializar la variable de control del game loop.
salir = False

# Función de inicialización.
def init():

    global screen
    global imgBackground
    global imgSpace
    global n1
    global n2
    global n3
    global n4
    global n5
    global clock
    global player

    # Inicializar Pygame.
    pygame.init()

    # Poner el modo de video en ventana e indicar la resolución.
    screen = pygame.display.set_mode(RESOLUTION)
    # Poner el título de la ventana.
    pygame.display.set_caption("Mi Juego")

```

```

# Crear la superficie del fondo o background.
imgBackground = pygame.Surface(screen.get_size())
imgBackground = imgBackground.convert()

# Cargar la imagen del fondo. La imagen es de 640 x 360.
imgSpace = pygame.image.load("assets/images/space_640x360.jpg")
imgSpace = imgSpace.convert()

# Dibujar la imagen cargada en la imagen de background.
imgBackground.blit(imgSpace, (0, 0))

# Crear las naves: se le pasa como parámetro la imagen de la nave.
n1 = CNave(CNave.TYPE_PLATINUM)
n2 = CNave(CNave.TYPE_GOLD)
n3 = CNave(CNave.TYPE_RED)
n4 = CNave(CNave.TYPE_GREEN)
n5 = CNave(CNave.TYPE_CYAN)

# Colocar las naves en su posición inicial.
n1.setXY(0, 100)
n2.setXY(0, 150)
n3.setXY(0, 200)
n4.setXY(0, 250)
n5.setXY(0, 300)

# Las naves comienzan detenidas.
n1.setVelX(2)
n1.setVelY(0)
n2.setVelX(2)
n2.setVelY(0)
n3.setVelX(2)
n3.setVelY(0)
n4.setVelX(2)
n4.setVelY(2)
n5.setVelX(2)
n5.setVelY(0)

# Acelerar las naves.
n1.setAccelX(0)
n1.setAccelY(0)
n2.setAccelX(0)
n2.setAccelY(0)
n3.setAccelX(0)
n3.setAccelY(0)
n4.setAccelX(0)
n4.setAccelY(0)
n5.setAccelX(0)

```

```

n5.setAccely(0)

# Marcar los límites del mundo.
n1.setBounds(-n1.getWidth(), -n1.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)
n2.setBounds(-n2.getWidth(), -n2.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)
n3.setBounds(-n3.getWidth(), -n3.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)
n4.setBounds(-n3.getWidth(), -n3.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)
n5.setBounds(-n3.getWidth(), -n3.getHeight(), SCREEN_WIDTH,
SCREEN_HEIGHT)

# Poner los comportamientos con los bordes.
n1.setBoundAction(CGameObject.NONE)
n2.setBoundAction(CGameObject.STOP)
n3.setBoundAction(CGameObject.WRAP)
n4.setBoundAction(CGameObject.BOUNCE)
n5.setBoundAction(CGameObject.DIE)

# Crear el jugador.
player = CPlayer(CPlayer.TYPE_PLAYER_1)
player.setXY(SCREEN_WIDTH / 2 - player.getWidth() / 2,
SCREEN_HEIGHT - player.getHeight())

# Establecer los límites de la pantalla.
player.setBounds(0, 0, SCREEN_WIDTH - player.getWidth(),
SCREEN_HEIGHT)

# Inicializar el reloj.
clock = pygame.time.Clock()

# Correr la lógica del juego.
def update():
    global salir
    global screen
    global isFullscreen

    # Timer que controla el frame rate.
    clock.tick(60)

    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():

        # Si se cierra la ventana se sale del programa.
        if event.type == pygame.QUIT:
            salir = True

```

```

    # Si se pulsa la tecla [Esc] se sale del programa.
    if event.type == pygame.KEYUP:
        if event.key == pygame.K_ESCAPE:
            salir = True

    # Si se pulsa la tecla [F], se cambia entre ventana y
fullscreen.
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_f:
            isFullscreen = not isFullscreen
            if isFullscreen:
                screen = pygame.display.set_mode(RESOLUTION,
pygame.FULLSCREEN)
            else:
                screen = pygame.display.set_mode(RESOLUTION)

    # Registrar cuando se apreta o suelta una tecla.
    if event.type == pygame.KEYDOWN:
        CKeyboard.inst().keyDown(event.key)
    if event.type == pygame.KEYUP:
        CKeyboard.inst().keyUp(event.key)

# Lógica de las naves.
n1.update()
n2.update()
n3.update()
n4.update()
n5.update()

# Lógica del jugador.
player.update()

# Dibujar el frame y actualizar la pantalla.
def render():
    # Dibujar el fondo.
    screen.blit(imgBackground, (0, 0))

    # Dibujar las naves en la nueva posición.
    n1.render(screen)
    n2.render(screen)
    n3.render(screen)
    n4.render(screen)
    n5.render(screen)

    # Dibujar el jugador.
    player.render(screen)

```

```

    # Actualizar la pantalla.
    pygame.display.flip()

# Función de destrucción.
def destroy():
    global n1
    global n2
    global n3
    global n4
    global n5
    global player

    # Destruir los objetos.
    n1.destroy()
    n1 = None
    n2.destroy()
    n2 = None
    n3.destroy()
    n3 = None
    n4.destroy()
    n4 = None
    n5.destroy()
    n5 = None

    # Destruir la nave del jugador.
    player.destroy()
    player = None

    # Cerrar Pygame y liberar los recursos que pidió el programa.
    pygame.quit()

# ===== Punto de entrada del programa. =====

# Inicializar los elementos necesarios del juego.
init()

# Loop principal del juego.
while not salir:
    # Actualizar los objetos.
    update()
    # Dibujar la pantalla.
    render()

# Liberar los recursos al final.
destroy()

```

En la clase `CNave`, agregamos la parte que corresponde a tener dos tipos nuevos de nave.

Para eso ponemos las imágenes correspondientes a las nuevas naves (la verde y la celeste) en la carpeta de imágenes, y definimos las constantes para las nuevas naves. Copia las imágenes del programa de ejemplo a tu propio ejemplo.

```
# -*- coding: utf-8 -*-

# -----
# Clase CNavе.
# Nave enemiga que se mueve por la pantalla chequeando los bordes.
#
# Autor: Fernando Sansberro - Batovi Games.
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
# -----

# Importar Pygame.
import pygame

# Importar la clase CSprite.
from api.CSprite import *

# La clase CNavе hereda de CSprite.
class CNavе(CSprite):

    # Tipos de naves.
    TYPE_PLATINUM = 0
    TYPE_GOLD = 1
    TYPE_RED = 2
    TYPE_GREEN = 3
    TYPE_CYAN = 4

    # -----
    # Constructor. Recibe el tipo de nave (TYPE_PLATINUM o TYPE_GOLD).
    # -----
    def __init__(self, aType):

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType
        if self.mType == CNavе.TYPE_PLATINUM:
            imgFile = "assets/images/grey_ufo.png"
        elif self.mType == CNavе.TYPE_GOLD:
            imgFile = "assets/images/yellow_ufo.png"
        elif self.mType == CNavе.TYPE_RED:
            imgFile = "assets/images/red_ufo.png"
        elif self.mType == CNavе.TYPE_GREEN:
            imgFile = "assets/images/green_ufo.png"
        elif self.mType == CNavе.TYPE_CYAN:
```



```

        imgFile = "assets/images/cyan_ufo.png"

        # Invocar al constructor de CSprite con la imagen a cargar.
        CSprite.__init__(self, imgFile)

    # Mover el objeto.
    def update(self):

        # Invocar update() de CSprite.
        CSprite.update(self)

    # Dibuja el objeto en la pantalla.
    # Parámetros:
    # aScreen: La superficie de la pantalla en donde dibujar.
    def render(self, aScreen):

        # Invocar render() de CSprite.
        CSprite.render(self, aScreen)

    # Liberar lo que haya creado el objeto.
    def destroy(self):

        # Invocar destroy() de CSprite.
        CSprite.destroy(self)
        self.mImg = None

```

En la clase `CGameObject`, es donde implementamos el comportamiento que tendrá el objeto cuando alcanza uno de los bordes definidos para la pantalla o el mundo. Recordemos que los límites del mundo por el cual se mueve un objeto lo establecemos con la función `setBounds()`.

La clase `CGameObject`, queda de esta manera:

```

# -*- coding: utf-8 -*-

#-----
# Clase CGameObject.
# Clase base de todos los objetos que se mueven en el juego.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

class CGameObject(object):

    # Comportamientos del objeto al llegar a un borde.

```

```

NONE = 0      # No tiene ninguno, el objeto sigue de largo.
STOP = 1      # El objeto se detiene al alcanzar un borde.
WRAP = 2      # El objeto aparece por el lado contrario.
BOUNCE = 3    # El objeto rebota en el borde.
DIE = 4       # El objeto se marca para ser eliminado.

def __init__(self):

    # Coordenadas del objeto.
    self.mX = 0
    self.mY = 0

    # Velocidad.
    self.mVelX = 0
    self.mVelY = 0

    # Aceleración.
    self.mAccelX = 0
    self.mAccelY = 0

    # Variables para controlar los bordes.
    self.mMinX = -float("inf")
    self.mMaxX = float("inf")
    self.mMinY = -float("inf")
    self.mMaxY = float("inf")

    # Comportamiento de borde del objeto. Ponemos que no tenga
ninguno
    # por defecto, y el objeto seguirá de largo en los bordes.
    self.mBoundAction = CGameObject.NONE

    # Obtener la coordenada X.
    def getX(self):
        return self.mX

    # Obtener la coordenada Y.
    def getY(self):
        return self.mY

    # Establece la posición del objeto.
    # Parámetros:
    # aX, aY: Coordenadas x e y del objeto.
    def setXY(self, aX, aY):

        self.mX = aX
        self.mY = aY

    # Establece la velocidad X del objeto.

```

```

def setVelX(self, aVelX):

    self.mVelX = aVelX

# Establece la velocidad Y del objeto.
def setVelY(self, aVelY):

    self.mVelY = aVelY

# Establece la aceleración x del objeto.
def setAccelX(self, aAccelX):

    self.mAccelX = aAccelX

# Establece la aceleración y del objeto.
def setAccelY(self, aAccelY):

    self.mAccelY = aAccelY

# Define los límites del movimiento del objeto.
# Parámetros:
# aMinX, aMinY: Coordenadas x e y mínimas del mundo.
# aMaxX, aMaxY: Coordenadas x e y máximas del mundo.
def setBounds(self, aMinX, aMinY, aMaxX, aMaxY):

    self.mMinX = aMinX
    self.mMaxX = aMaxX
    self.mMinY = aMinY
    self.mMaxY = aMaxY

# Define el comportamiento al alcanzar los bordes del mundo.
def setBoundAction(self, aBoundAction):
    self.mBoundAction = aBoundAction

# Update mueve el objeto según su velocidad.
def update(self):

    # Modificar la velocidad según la aceleración.
    self.mVelX += self.mAccelX
    self.mVelY += self.mAccelY

    # Mover el objeto.
    self.mX += self.mVelX
    self.mY += self.mVelY

    # Comportamiento con el borde.
    self.checkBounds()

```

```

# Realiza el chequeo con los bordes y aplica el comportamiento que
# corresponda si el objeto toca alguno de los bordes.
# Esta función es invocada en update().
# La variable self.boundAction contiene el comportamiento:
# NONE: No tiene ninguno, el objeto sigue de largo.
# STOP: El objeto se detiene al alcanzar un borde.
# WRAP: El objeto aparece por el lado contrario.
# BOUNCE: El objeto rebota en el borde.
# DIE: El objeto se marca para ser eliminado.
def checkBounds(self):

```

```

    # Si el comportamiento es NONE no se chequea el borde.
    if self.mBoundAction == CGameObject.NONE:
        return

```

```

    # Saber qué bordes está tocando el objeto.
    left = (self.mX < self.mMinX)
    right = (self.mX > self.mMaxX)
    up = (self.mY < self.mMinY)
    down = (self.mY > self.mMaxY)

```

```

    # Si no toca ningún borde no hacemos nada.
    if not (left or right or up or down):
        return

```

```

    # Al llegar a este punto, el objeto está tocando un borde.
    # Hay que corregir la posición del objeto y luego modificar
    # su velocidad según el comportamiento que tenga.

```

```

    # Corregir la posición del objeto.
    # Si es WRAP, el objeto aparece desde el lado contrario.
    if (self.mBoundAction == CGameObject.WRAP):

```

```

        if (left):
            self.mX = self.mMaxX
        if (right):
            self.mX = self.mMinX
        if (up):
            self.mY = self.mMaxY
        if (down):
            self.mY = self.mMinY

```

```

    # Si es STOP, BOUNCE o DIE corregimos la posición porque sino

```

el

```

    # objeto queda con parte fuera de los límites.
    else:
        if (left):
            self.mX = self.mMinX
        if (right):
            self.mX = self.mMaxX

```

```

        if (up):
            self.mY = self.mMinY
        if (down):
            self.mY = self.mMaxY

        # Si el comportamiento es STOP o DIE, el objeto se detiene.
        if (self.mBoundAction == CGameObject.STOP or self.mBoundAction
== CGameObject.DIE):
            self.mVelX = 0
            self.mVelY = 0
            self.mAccelX = 0
            self.mAccelY = 0
        elif (self.mBoundAction == CGameObject.BOUNCE):
            if (right or left):
                self.mVelX *= -1
            if (up or down):
                self.mVelY *= -1

        # Si el comportamiento es que muera, no se hace más nada.
        # Esto se va a implementar más adelante.
        if (self.mBoundAction == CGameObject.DIE):
            print("Objeto debe morir")
            return

        # Liberar lo que haya creado el objeto.
        def destroy(self):

            pass

```

Como vemos en `CGameObject`, en la función `update()`, luego de mover el objeto, chequeamos si el mismo ha salido de los bordes definidos como los límites del mundo por el cual se puede mover. Si esto es así, se corrigen sus coordenadas según el comportamiento de borde que tenga definido el objeto.

Intenta seguir la lógica en la función `checkBounds()`, leyendo los comentarios, hasta entender completamente el funcionamiento de este código. Una vez programados los comportamientos de borde en `CGameObject`, cualquier objeto del juego, al heredar de esta clase, tendrá disponible esta funcionalidad.

Manejo de Colisiones con los Bordes

El ejemplo anterior funciona bien, pero con una pequeña falla. Las tres primeras naves se van de la pantalla y no se ven más. El comportamiento de la nave dorada es `CGameObject.STOP`, y esta nave se debería ver en pantalla cuando se detiene. No se ve,

porque queda fuera de la pantalla, cosa que corregiremos a continuación.

Miremos la siguiente imagen, que muestre el momento en el que la nave dorada (que tiene establecido el comportamiento `CGameObject.STOP`) se pasa de los límites:

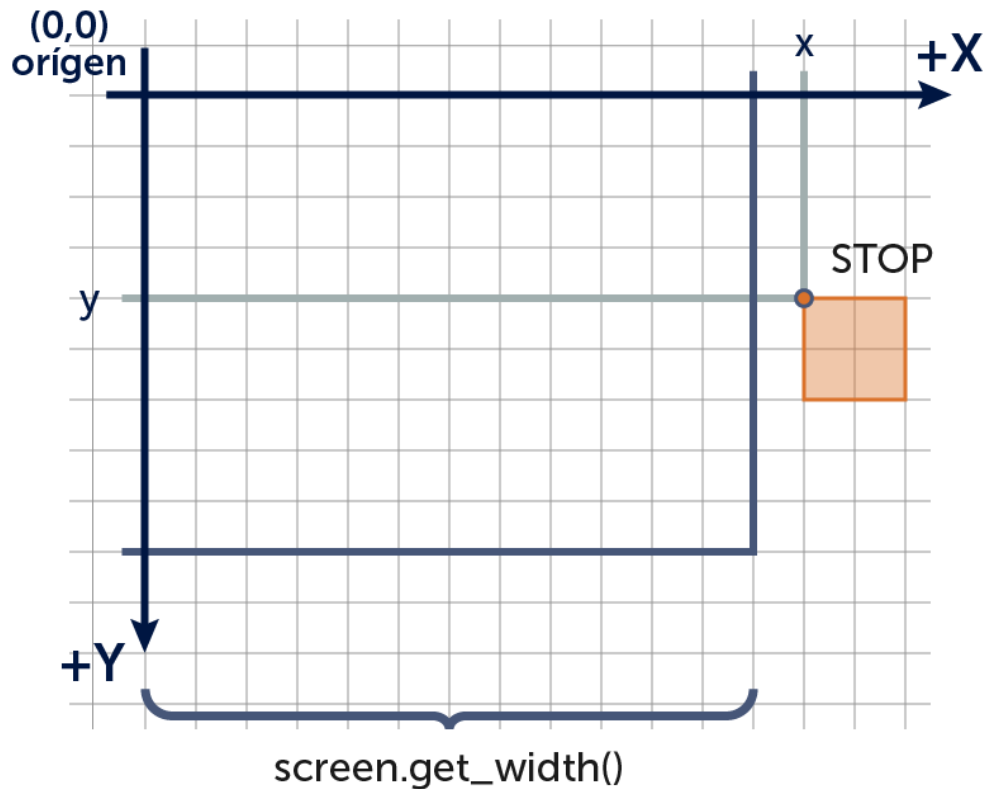


Figura 8-3: La nave con comportamiento *STOP* se detiene cuando se va de la pantalla.

Como vemos en el ejemplo anterior, la nave dorada desaparece de la pantalla y se detiene, cuando lo que queremos es que se detenga al llegar al borde (no al irse completamente y ya no verse más). Esto se debe a que o bien estamos chequeando incorrectamente los bordes, o bien no estamos tomando en cuenta el ancho y el alto de la nave.

La posición (x,y) de un sprite, está relacionada con cómo se dibuja el sprite. Por ahora, la posición (x,y) de un sprite es la posición de la coordenada superior izquierda del sprite, porque es en ese punto donde se dibuja la imagen. A ese punto de referencia, se le llama *pivot*, o *punto de registro*. Mira la siguiente figura.

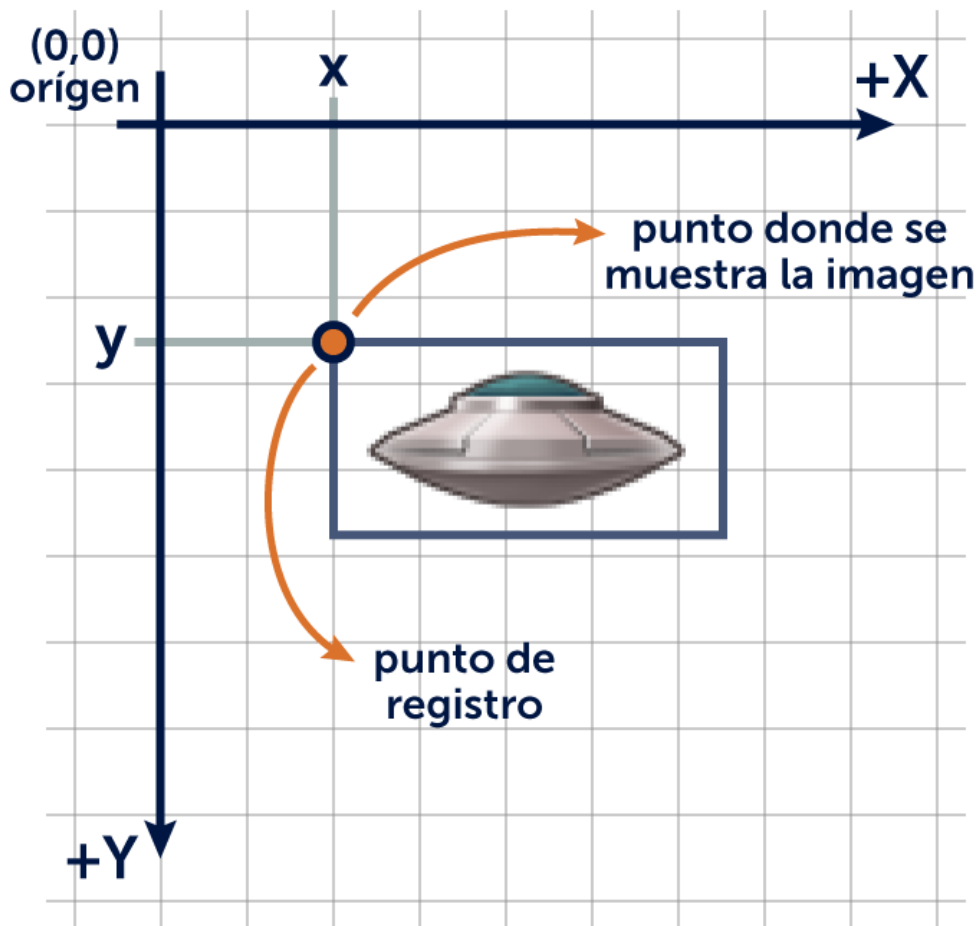


Figura 8-4: La posición (x,y) del sprite es la esquina superior izquierda del dibujo.

Entonces, para saber si los bordes de la nave se pasan de los límites del mundo, debemos tener en cuenta el ancho y el alto de la imagen para establecer los valores de bordes.

Veamos el ejemplo que se encuentra en la carpeta `capitulo_08\003_comportamiento_de_borde_corregido`.

Al ejecutar el ejemplo, vemos que la nave verde (que tiene comportamiento `CGameObject.BOUNCE`) rebota sin salirse de los bordes, y la nave roja (que tiene comportamiento `CGameObject.WRAP`) se va completamente de la pantalla antes de salir por el lado contrario. La clave para esto (y de hecho la única diferencia entre este ejemplo y el anterior), son los valores que hemos puesto en los límites de borde en la función `init()` en `main.py`:

```
# Función de inicialización.
def init():
    ...
```

```
# Marcar los límites del mundo.  
n1.setBounds(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT)  
n2.setBounds(0, 0, SCREEN_WIDTH - n2.getWidth(), SCREEN_HEIGHT -  
n2.getHeight())  
n3.setBounds(-n3.getWidth(), -n3.getHeight(), SCREEN_WIDTH,  
SCREEN_HEIGHT)  
n4.setBounds(0, 0, SCREEN_WIDTH - n4.getWidth(), SCREEN_HEIGHT -  
n4.getHeight())  
n5.setBounds(0, 0, SCREEN_WIDTH - n5.getWidth(), SCREEN_HEIGHT -  
n5.getHeight())
```

Cuando un objeto choca con una pared (como es el caso al chocar con el borde de la pantalla o del mundo), debemos asegurarnos de realizar dos cosas:

- 1o). Corregir la posición del objeto (para que el objeto no quede colisionando).
- 2o). Cambiar su velocidad si corresponde, según cómo se comporte con el borde (si rebota o no).

Observa la siguiente figura que muestra estos pasos a realizar cuando la nave se va del rango máximo (el límite derecho) y rebota contra el borde derecho:

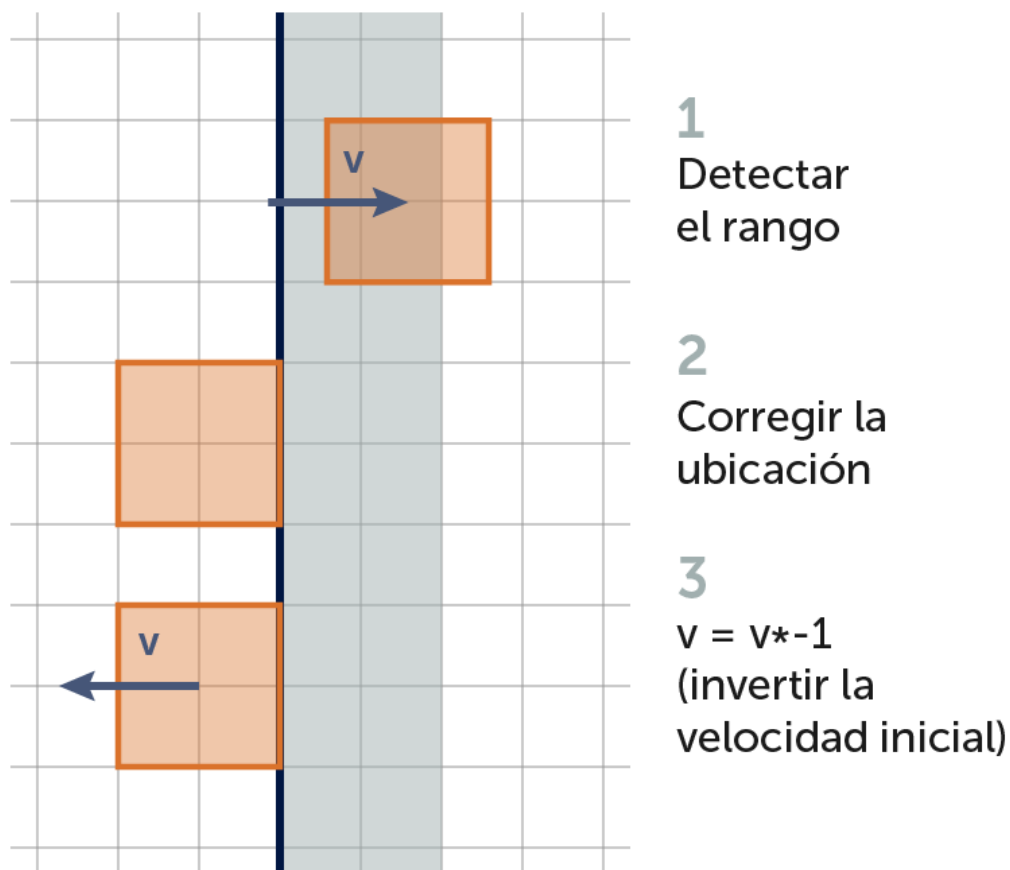


Figura 8-5: Rebote en el borde derecho de la pantalla.
Se invierte la velocidad horizontalmente.

Si observamos en el código de `CGameObject`, en la función `checkBounds()`, podemos ver cómo corregimos la posición del objeto cuando éste se va de los límites. Colocamos el objeto en el lado contrario si el comportamiento es `CGameObject.WRAP`, y colocamos el objeto contra el borde, e invertimos su velocidad si el comportamiento es `CGameObject.BOUNCE`.

Como hemos dicho, al estar estos comportamientos programados en la clase `CGameObject`, se encontrarán disponibles para todos los objetos que se muevan en el juego (los cuales heredan de `CGameObject`). De esta forma, la primera vez cuesta un poco implementar algo del estilo de los rebotes, pero luego acelera muchísimo el desarrollo de videojuegos, al tener clases con funcionalidades que podemos reutilizar en nuestros próximos videojuegos.

Con esto hemos finalizado el manejo básico de los sprites del juego. ¡En el siguiente capítulo veremos el manejador de sprites y cómo disparar!

9. Disparos y el Manager de Sprites

Un videojuego tipo arcade que se precie, como el que estamos haciendo (una versión del clásico *Space Invaders*), necesita que el jugador dispare muchas balas para matar a los enemigos. En este capítulo veremos cómo armar un array de balas y una clase para manejarlas, a lo que le llamaremos el *manager* de balas. Esto es, una clase que se encarga del manejo automático de todas las balas del juego.

Disparando Balas

Para lograr que la nave del jugador dispare balas, tenemos que realizar varias tareas, dado que es la primera vez que hacemos que un objeto dispare.

Primero, debemos crear una clase para la bala, a la que le llamaremos `CPlayerBullet`. Luego, como el juego tendrá muchas balas, deberemos pensar en alguna forma de manejar las balas, esto es, agregarlas a una lista, correrle la lógica y dibujar todas las balas que hay en el juego, eliminar una bala de la lista cuando se va de la pantalla, etc. Para esto, haremos una clase que llamaremos `CBulletManager`, la cual se encargará de estas tareas.

Una vez que tenemos la clase para la bala y el manager de balas, lo que haremos será que el jugador dispare con la tecla `[Space]`, creando en ese momento una bala y agregándola al manager de balas.

Corre el ejemplo que se encuentra en la carpeta `capitulo_09\001_manager_de_balas`. En este ejemplo, la nave dispara muchas balas cuando se pulsa la tecla `[Space]`. Todavía las balas no se eliminan cuando se van de la pantalla y como puedes observar, si dejamos la tecla de disparo pulsada, aparecen balas continuamente. Estas cuestiones las iremos corrigiendo a lo largo de este capítulo.

Primero veamos la clase para la bala, que se encuentra en `CPlayerBullet.py`. Esta clase no es nada particular. Simplemente es un sprite (hereda de `CSprite`) que carga la imagen de la bala. Esta clase se encuentra en la carpeta `game`, debido a que es un objeto del juego.

```
# -*- coding: utf-8 -*-
```

```
#-----  
# Clase CPlayerBullet.  
# Balas del jugador.  
#  
# Autor: Fernando Sansberro - Batovi Games Studio  
# Proyecto: Hacete tu Videojuego.  
# Licencia: Creative Commons. BY-NC-SA.  
#-----
```

```

# Importar Pygame.
import pygame

# Importar la clase CSprite.
from api.CSprite import *

# La clase CPlayerBullet hereda de CSprite.
class CPlayerBullet(CSprite):

    # Constructor.
    def __init__(self):
        CSprite.__init__(self, "assets/images/player_bullet.png")

    # Mover el objeto.
    def update(self):

        CSprite.update(self)

    # Dibuja el objeto en la pantalla.
    # Parámetros:
    # aScreen: La superficie de la pantalla en donde dibujar.
    def render(self, aScreen):

        CSprite.render(self, aScreen)

    # Liberar lo que haya creado el objeto.
    def destroy(self):

        CSprite.destroy(self)

```

La imagen que se usa para la bala es `player_bullet.png` y como siempre, la colocamos en la carpeta `assets/images`. Cópiala de los ejemplos a tu propio proyecto. El tamaño de la bala es de 7x7 píxeles.

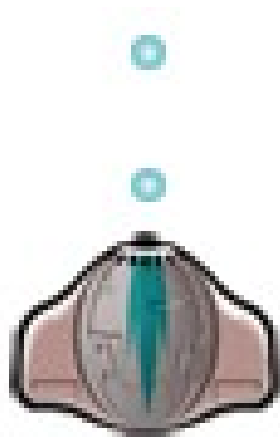


Figura 9-1: La bala que va a disparar nuestra nave.

En la clase `CPlayer`, en la función `update()`, agregamos el código que chequea si la tecla de disparo (`[Space]`) está pulsada. Si esto es así, se crea la bala.

```
# Mover el objeto.
def update(self):

    ...

    # Disparar con la tecla [Space].
    if CKeyboard.inst().spacePressed():
        print("FIRE")
        b = CPlayerBullet()
        b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() / 2,
self.getY())
        b.setVelX(0)
        b.setVelY(-10)
        b.setBounds(0, 0, 640, 360)
        b.setBoundAction(CGameObject.DIE)
        CBulletManager.inst().addBullet(b)

    # Invocar update() de CSprite.
    CSprite.update(self)
```

Como siempre, cuando vamos a usar una clase (en este caso, `CPlayerBullet` y `CBulletManager`), debemos importarlas al inicio de la clase:

```
# Importar el manager de balas.
from api.CBulletManager import *

# Importar la clase para la bala.
```

```
from game.CPlayerBullet import *
```

Como vemos en el código que dispara, primero se crea una bala, luego se establece su posición inicial (observar que se coloca la bala en la punta de la nave), y se establece su velocidad. Al darle una velocidad negativa en *y*, la bala saldrá hacia arriba.

Al pulsar la tecla *[Space]*, la nave dispara. Para poder detectar esta tecla, se ha agregado la función `spacePressed()` en la clase `CKeyboard`. Mira el código de `CKeyboard` para ver el cambio realizado.

```
...
```

```
class CKeyboard(object):
```

```
    mInstance = None
    mInitialized = False
```

```
    mLeftPressed = False
    mRightPressed = False
mSpacePressed = False
```

```
...
```

```
def init(self):
    if (CKeyboard.mInitialized):
        return
    CKeyboard.mInitialized = True

    CKeyboard.mLeftPressed = False
    CKeyboard.mRightPressed = False
CKeyboard.mSpacePressed = False
```

```
def keyDown(self, key):
    if (key == pygame.K_LEFT):
        CKeyboard.mLeftPressed = True
    if (key == pygame.K_RIGHT):
        CKeyboard.mRightPressed = True
if (key == pygame.K_SPACE):
    CKeyboard.mSpacePressed = True
```

```
def keyUp(self, key):
    if (key == pygame.K_LEFT):
        CKeyboard.mLeftPressed = False
    if (key == pygame.K_RIGHT):
        CKeyboard.mRightPressed = False
if (key == pygame.K_SPACE):
    CKeyboard.mSpacePressed = False
```

```

def leftPressed(self):
    return CKeyboard.mLeftPressed

def rightPressed(self):
    return CKeyboard.mRightPressed

def spacePressed(self):
    return CKeyboard.mSpacePressed

def destroy(self):
    CKeyboard.mInstance = None

```

Al disparar, con la función `setBounds()` le indicamos a la bala creada los límites de la pantalla, y le colocamos el comportamiento de borde `CGameObject.DIE` para que cuando la bala toque el borde de la pantalla sea eliminada. Este será el comportamiento típico para las balas

Nota: Es muy importante eliminar las balas que se van de la pantalla o de los bordes del mundo, porque en un juego de acción en el que disparamos mucho, si no se eliminan las balas, el juego se tornaría lento y usaría mucha memoria.

La última línea del disparo, `CBulletManager.inst().addBullet(b)`, agrega la bala al manager de bala. El manager de balas es la clase que se encargará de agregar las balas a una lista, se encargará de procesar las balas (invocar a su `update()`), de dibujarlas (invocar a su `render()`) y de eliminarla de la lista cuando la bala sea destruida. A continuación veremos esta clase.

El Manager de Balas

Como podemos ver en el programa `main.py`, tenemos una variable para controlar cada una de las naves enemigas. Imagina si el juego tuviera cincuenta naves enemigas. Eso quiere decir que deberíamos repetir el código con cincuenta variables diferentes, lo cual estaría muy mal.

Como ahora vamos a disparar muchas balas, no podemos tener una variable para cada una de las balas, sino que haremos un *array* o *lista*, en el cual cada elemento del array será una bala (un objeto de clase `CPlayerBullet`). Haremos una clase que contenga este array, y esta clase tendrá funciones para agregar una bala a la lista, eliminarla de la lista, procesar las balas, etc.

Cuando tenemos una clase que se encarga, como en este caso, del manejo de todas las balas, le llamamos *manager*. El manager de balas es una clase que contiene un array con las balas existentes en el juego, y el manager se encarga de actualizar y dibujar cada una de las balas. Más adelante haremos un manager para cada tipo de objetos (enemigos, partículas,

etc.).

En la siguiente figura vemos una representación gráfica del manager de balas. Cada elemento del array es una referencia a un sprite, en este caso, un objeto `CPlayerBullet`.

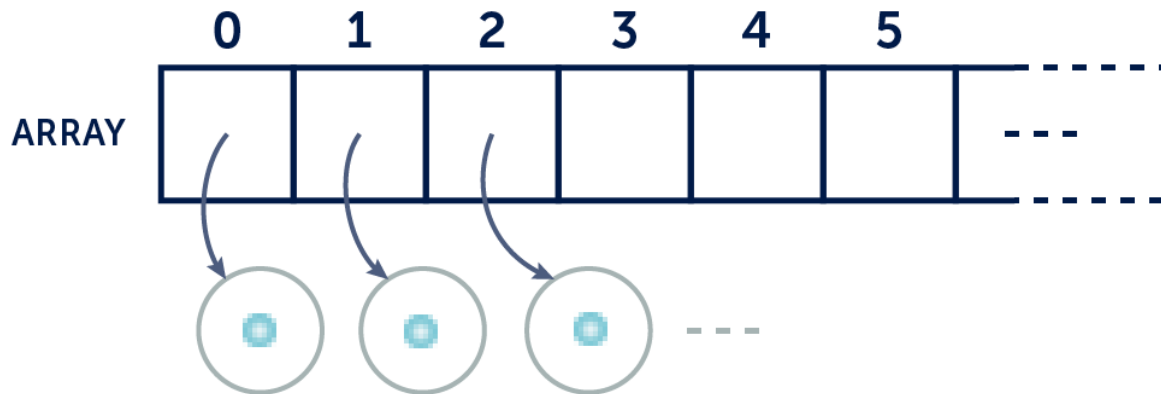


Figura 9-2: El manager es un array, donde cada elemento es una referencia a un objeto.

Del mismo modo, implementaremos más adelante un manager para manejar a los enemigos, momento en el que sacaremos todas las variables que ahora tenemos en `main.py`.

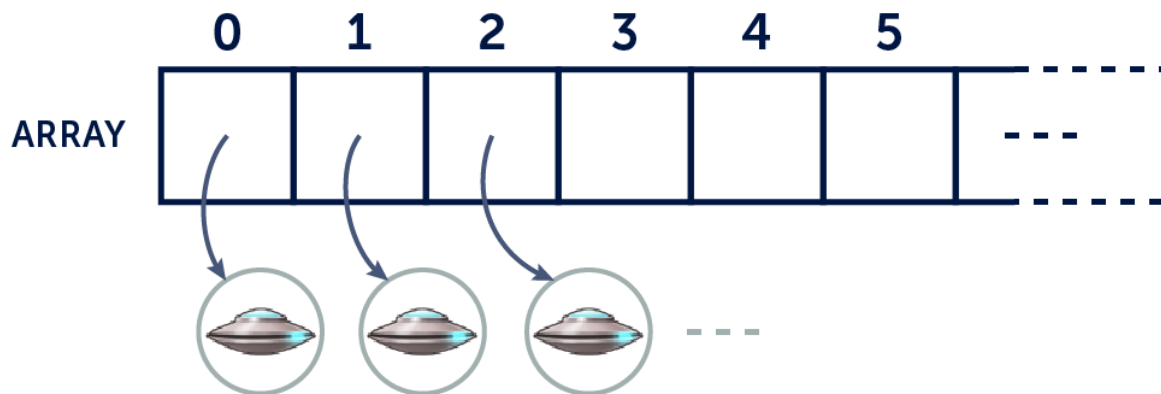


Figura 9-3: Más adelante tendremos un manager (un array) para los enemigos.

Veamos entonces el código de la clase `CBulletManager`, que hemos colocado en `\api` porque es una clase que reutilizaremos en todos nuestros juegos.

```
# -*- coding: utf-8 -*-
```

```
#-----
```

```
# Clase CBulletManager.
```

```
# Clase para manejar los disparos del juego.
```

```

#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

class CBulletManager(object):

    mInstance = None
    mInitialized = False

    # Lista de balas.
    mBullets = None

    def __new__(self, *args, **kwargs):
        if (CBulletManager.mInstance is None):
            CBulletManager.mInstance = object.__new__(self, *args,
**kwargs)
            self.init(CBulletManager.mInstance)
        else:
            print("Cuidado: CBulletManager(): No se debería instanciar
más de una vez esta clase. Usar CBulletManager.inst().")
            return self.mInstance

    @classmethod
    def inst(cls):
        if (not cls.mInstance):
            return cls()
        return cls.mInstance

    def init(self):
        if (CBulletManager.mInitialized):
            return
        CBulletManager.mInitialized = True

        # Crear la lista de balas.
        CBulletManager.mBullets = []

    # Procesar los objetos.
    def update(self):
        for b in CBulletManager.mBullets:
            b.update()

    # Dibujar los objetos.
    def render(self, aScreen):

```



```

        for b in CBulletManager.mBullets:
            b.render(aScreen)

# Agregar un objeto a la lista.
def addBullet(self, aBullet):
    CBulletManager.mBullets.append(aBullet)

# Destruir todos los objetos y removerlos de la lista.
# Destruir la lista.
def destroy(self):
    i = len(CBulletManager.mBullets)
    while i > 0:
        CBulletManager.mBullets[i - 1].destroy()
        CBulletManager.mBullets.pop(i - 1)
        i = i - 1

CBulletManager.mInstance = None

```

Como `CBulletManager` es una clase que se encarga de todas las balas en el juego, habrá una sola clase de este tipo. Por lo tanto, ha sido diseñada como *singleton*. Como siempre, la primera parte de un singleton se encarga de que no pueda crearse otra clase de ese tipo.

El manager contiene un array, llamado `mBullets`, que contendrá las balas en el juego (objetos de clase `CPlayerBullet`). La función `addBullet()` recibe como parámetro la bala creada al momento en el que el jugador dispara, y la agrega al array usando la función `append()`. De esta forma agregamos una bala a la lista, al final de la misma.

```

# Agregar un objeto a la lista.
def addBullet(self, aBullet):
    CBulletManager.mBullets.append(aBullet)

```

Ahora, tendremos todas las balas que se encuentran vivas, en el array `mBullets`. Para actualizar la lógica de cada una, debemos llamar a la función `update()` de cada una de las balas. Lo mismo haremos para dibujar cada bala, llamando a la función `render()` de todas las balas.

Entonces, se implementa en el manager una función `update()` y otra `render()` que lo que hacen es recorrer la lista de balas e invocan a la función `update()` para correr la lógica de cada bala y `render()` para dibujar cada bala.

```

# Procesar los objetos.
def update(self):
    for b in CBulletManager.mBullets:

```

```

        b.update()

# Dibujar los objetos.
def render(self, aScreen):
    for b in CBulletManager.mBullets:
        b.render(aScreen)

```

Este concepto es clave. La función `update()` de un manager se encarga de invocar a la función `update()` de cada uno de los elementos. Del mismo modo, la función `render()` del manager dibuja todos los elementos del array, invocando a la función `render()` para cada uno.

Por último, en el programa `main.py`, debemos invocar a las funciones `update()` y `render()` del manager de balas para actualizar y dibujar todas las balas.

Al final de todo, al terminar el programa, debemos siempre llamar a la función `destroy()` para que elimine todos los objetos que quedaron creados. Veamos la parte de `main.py` que invoca a funciones del manager:

```

...

# Importar el manager de balas.
from api.CBulletManager import *

...

# Correr la lógica del juego.
def update():

    ...

    # Mover las balas.
    CBulletManager.inst().update()

# Dibujar el frame y actualizar la pantalla.
def render():

    ...

    # Dibujar las balas.
    CBulletManager.inst().render(screen)

    ...

# Función de destrucción.
def destroy():

```

```

...

# Destruir las balas.
CBulletManager.inst().destroy()

# Cerrar Pygame y liberar los recursos que pidió el programa.
pygame.quit()

...

```

Cuando corremos el ejemplo, vemos que aún quedan tres cosas por corregir. Primero, cuando disparamos, las balas al llegar al borde de la pantalla no se eliminan. Más adelante en este capítulo arreglaremos este problema.

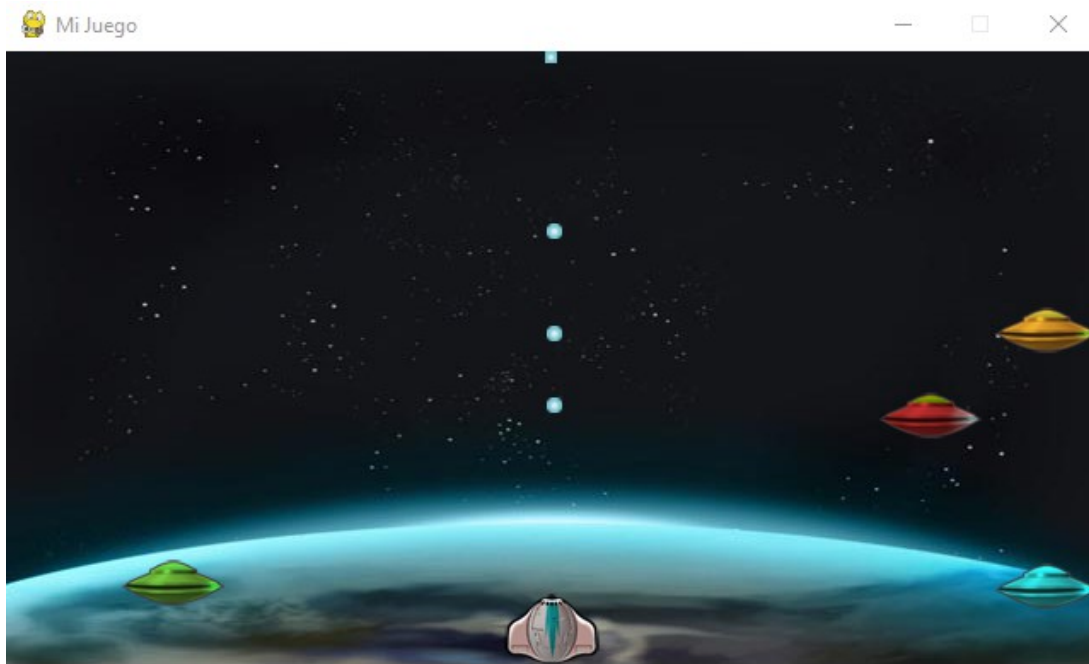


Figura 9-4: La nave del jugador ahora dispara balas.

El segundo problema que existe es un problema de diseño. Cuando en la clase `CPlayer` se dispara una bala, se le establecen los límites de la pantalla, pero desde la clase `CPlayer` no tenemos acceso al ancho y alto de la pantalla (esto está definido en el programa `main.py`). Este problema es bastante común al desarrollar videojuegos, que es que algunos datos los ponemos en una clase y luego no los tenemos disponibles para otras que lo necesitan. Esto lo solucionaremos más adelante en este capítulo y la solución es tener una clase donde guardaremos valores constantes para que todas las clases las puedan usar.

El tercer problema es que hemos utilizado la función `spacePressed()` de `CKeyboard`, que dice si la tecla `[Space]` está siendo pulsada o no. Pero esto no nos sirve, porque cuando

pulsamos *[Space]* para disparar, la tecla está siendo pulsada en varios frames consecutivos (recordemos que el juego corre a 60 frames por segundo). Es imposible apretar y soltar la tecla en un solo frame, y es por eso que durante varios frames la tecla está pulsada, motivo por el cual salen varias balas seguidas. Arreglaremos esto a continuación.

La función `fire()` de `CKeyboard`

Como vimos en el ejemplo anterior, salen muchas balas cuando dejamos pulsada la tecla de disparo. Queremos hacer que cuando el jugador pulse *[Space]* para disparar, salga una sola bala y no vuelva a salir otra bala hasta que el jugador no suelte la tecla y la vuelva a pulsar.

Para hacer esto, ya no usaremos la función `spacePressed()` de `CKeyboard`, sino que haremos una función que llamaremos `fire()`, que retornará `True` en el momento en que se toca la tecla *[Space]* por primera vez y no vuelve a retornar `True` hasta que se suelte la tecla y se vuelva a presionar.

Abre y ejecuta el ejemplo que se encuentra en la carpeta `capitulo_09\002_manager_de_balas_fire_keyboard`.

Aquí vemos que la bala sale solamente cuando se pulsa tecla *[Space]*, pero no vuelve a disparar hasta que no se levante la tecla y se vuelva a presionar. En la clase `CPlayer`, en la función `update()`, cambiamos `spacePressed()` por `fire()`:

```
# Disparar con la tecla [Space].
if CKeyboard.inst().fire():
    print("FIRE")
    b = CPlayerBullet()
    b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() / 2,
self.getY())
    b.setVelX(0)
    b.setVelY(-10)
    b.setBounds(0, 0, 640, 360)
    b.setBoundAction(CGameObject.DIE)
    CBulletManager.inst().addBullet(b)
```

En la clase `CKeyboard`, implementamos la función `fire()`, que debe retornar `True` solamente en el momento en que apretamos la tecla *[Space]* y si en el frame anterior no estaba pulsada. En cualquier otro caso esta función retornará `False`.

El siguiente es el código de `CKeyboard` y se han resaltado los cambios para implementar esto.

...

```
class CKeyboard(object):
```

```

mInstance = None
mInitialized = False

mLeftPressed = False
mRightPressed = False
# Estado de la tecla [Space] en el frame anterior.
mSpacePressedPreviousFrame = False
mSpacePressed = False

...

def init(self):
    if (CKeyboard.mInitialized):
        return
    CKeyboard.mInitialized = True

    CKeyboard.mLeftPressed = False
    CKeyboard.mRightPressed = False
    CKeyboard.mSpacePressedPreviousFrame = False
    CKeyboard.mSpacePressed = False

def keyDown(self, key):
    if (key == pygame.K_LEFT):
        CKeyboard.mLeftPressed = True
    if (key == pygame.K_RIGHT):
        CKeyboard.mRightPressed = True
    if (key == pygame.K_SPACE):
        CKeyboard.mSpacePressed = True

def keyUp(self, key):
    if (key == pygame.K_LEFT):
        CKeyboard.mLeftPressed = False
    if (key == pygame.K_RIGHT):
        CKeyboard.mRightPressed = False
    if (key == pygame.K_SPACE):
        CKeyboard.mSpacePressed = False

# Actualiza el estado de la tecla [Space].
def update(self):
    CKeyboard.mSpacePressedPreviousFrame = CKeyboard.mSpacePressed

def leftPressed(self):
    return CKeyboard.mLeftPressed

def rightPressed(self):
    return CKeyboard.mRightPressed

```

```

def spacePressed(self):
    return CKeyboard.mSpacePressed

# Función usada para disparar.
# Solo retorna True en el momento en que se apreta la tecla.
def fire(self):
    return CKeyboard.mSpacePressed == True and
CKeyboard.mSpacePressedPreviousFrame == False

def destroy(self):
    CKeyboard.mInstance = None

```

Agregamos una variable `mSpacePressedPreviousFrame`, que guarda el estado de la tecla `[Space]` para ser usado en el siguiente frame. Esto equivale a decir que esta variable almacena el estado de la tecla `[Space]` del frame anterior.

La clase `CKeyboard` ahora necesita una función `update()` dado que en cada frame guarda en esta variable el estado de la tecla `[Space]`.

Como vemos en la función `fire()`, la misma retorna `True` si la tecla `[Space]` está siendo pulsada y en el frame anterior no había sido pulsada. Solo en ese momento es que `fire()` retorna `True`. En ese momento podemos disparar la bala.

Para que esto funcione, debemos llamar a la función `update()` de `CKeyboard` desde `main.py`:

```

def update():
    global salir
    global screen
    global isFullscreen

    clock.tick(60)

    # Llamar a update() de CKeyboard.
    CKeyboard.inst().update()

    ...

```

Al ejecutar el programa, vemos que la nave ahora dispara bien. Ya no salen tantas balas juntas. Solamente sale una bala cada vez que se presiona la tecla de disparo.

A medida que vayamos desarrollando nuestro primer videojuego, y luego otros, iremos viendo que necesitaremos control de más teclas, control del mouse, etc. Necesitaremos ir mejorando la clase `CKeyboard` agregándole funcionalidad. Implementaremos esta clase en forma completa más adelante, así como también otras clases que formarán nuestra base para desarrollar videojuegos (denominada *API* o *framework*).

Eliminando Balas del Manager

Cuando una bala se va de la pantalla, ésta debe ser eliminada. Para que esto funcione, necesitamos dos partes. La primera es al momento de crear la bala, definir el comportamiento de borde `CGameObject.DIE`. Esto ya lo estamos haciendo, mira el código de `CPlayer` al disparar:

```
# Disparar con la tecla [Space].
    if CKeyboard.inst().fire():
        print("FIRE")
        b = CPlayerBullet()
        b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY())
        b.setVelX(0)
        b.setVelY(-10)
        b.setBounds(0, 0, 640, 360)
        b.setBoundAction(CGameObject.DIE)
        CBulletManager.inst().addBullet(b)
```

La segunda parte es cuando la bala toca el borde, como tiene el comportamiento `CGameObject.DIE`, tenemos que programar para que la bala sea eliminada. Hasta ahora habíamos puesto un mensaje “objeto debe morir”, y es hora de que realmente sea eliminado. Esto lo haremos en la clase `CGameObject`.

Si corremos el ejemplo que se encuentra en la carpeta `capitulo_09\003_manager_de_balas_eliminar_balas`, veremos que las balas que tocan el borde de la pantalla son eliminadas.

En la clase `CGameObject`, agregamos una variable `mIsDead` que comenzará en `False` (el objeto comienza vivo) y valdrá `True` en el momento que toque un borde (si tiene establecido el comportamiento de borde `CGameObject.DIE`).

```
...
```

```
class CGameObject(object):
```

```
    ...
```

```
    def __init__(self):
```

```
        ...
```

```

        # Indica si el objeto está vivo o muerto (debe morir).
        self.mIsDead = False

    ...

def checkBounds(self):

    ...

    # Si el comportamiento es que muera, se marca para morir.
    # El manager luego lo elimina.
    if (self.mBoundAction == CGameObject.DIE):
        self.mIsDead = True
        return

    # Indica si hay que eliminar el objeto o no.
    def isDead(self):
        return self.mIsDead

    ...

```

Como vemos en la función `checkBounds()`, cuando el objeto toca el borde y tiene el comportamiento `CGameObject.DIE`, se marca el objeto para morir, asignando `True` a la variable `mIsDead`. Luego utilizamos la función `isDead()` para saber si el objeto está marcado para morir o no.

Por sí solo, esto no hace nada. Es el manager (`CBulletManager.py`) quien se encarga de eliminar el objeto, cuando luego de correrle la lógica, el objeto quedó marcado para morir. En la función `update()` del manager, se invoca a la función `update()` de cada una de las balas mediante una sentencia `for`. El agregado, es que luego de hacer esto, se vuelve a recorrer todas las balas del manager, preguntando si la bala está marcada para morir, y si esto es así, se elimina:

```

# Procesar los objetos.
def update(self):
    for b in CBulletManager.mBullets:
        b.update()

    i = len(CBulletManager.mBullets)

    # Eliminar las balas marcadas para morir.
    while i > 0:
        if CBulletManager.mBullets[i - 1].isDead():
            CBulletManager.mBullets[i - 1].destroy()
            CBulletManager.mBullets.pop(i - 1)
            print("se elimina una bala")

```



```

i = i - 1

# Mostrar la cantidad de balas que hay en el manager.
print(len(CBulletManager.mBullets))

```

De esta forma cuando a una bala le colocamos el comportamiento de borde `CGameObject.DIE`, al llegar al borde de la pantalla se marca para morir, colocando la variable `mIsDead` en `True`. Esto automáticamente hace que el objeto sea eliminado, por el código que hemos agregado en la función `update()` del manager (el código que acabamos de ver).

Dicho de otra manera, hemos implementado un sistema sencillo que permite eliminar los objetos del juego automáticamente, simplemente prendiéndole una variable (poniéndola en `True`). El manager se ocupa del resto.

Para eliminar un elemento del array, utilizamos la función `pop()` de *arrays*, que recibe como parámetro el índice del elemento a borrar.

Como nota, si a `pop()` no le pasamos parámetro, elimina el último elemento del array. Debes consultar la documentación de *Python* cuando quieras hacer algo y no sepas con qué función se hace.

Nota: El array se recorre desde el final hacia el principio, porque utilizamos una función `pop()` que elimina el elemento dado y corre el array hacia atrás. De esta forma los elementos anteriores no se tocan y se puede seguir la recorrida del array en forma segura.

Destruyendo los Objetos

Como norma general, cada vez que creamos un objeto, al final debemos liberar la memoria que utiliza, lo que se conoce básicamente como destruir el objeto.

Por ejemplo, debemos agregar código a la función `destroy()` de `CBulletManager` para que elimine todos los elementos que quedan en el array:

```

# Destruir todos los objetos y removerlos de la lista.
# Destruir la lista.
def destroy(self):
    i = len(CBulletManager.mBullets)
    while i > 0:
        CBulletManager.mBullets[i - 1].destroy()
        CBulletManager.mBullets.pop(i - 1)
        i = i - 1

CBulletManager.mInstance = None

```

Es necesario eliminar todas las balas que quedan en el array, porque no sabemos en qué momento vamos a perder o ganar, y es muy probable que al momento de pasar de pantalla y eliminar los objetos que hay en el juego, todavía queden balas en el mundo.

Al eliminar un objeto, siempre le invocamos la función `destroy()` para que el objeto elimine lo que haya creado, y luego se usa la función `pop()` sobre el array para eliminar el elemento del array del manager.

La Clase `CGameConstants`

Ahora ya tenemos el manager completo, y cada bala que dispare el jugador, se eliminará al tocar un borde de la pantalla. Lo que nos queda hacer por ahora es mejorar un problema de diseño que tiene el juego.

Cuando el jugador dispara, en la clase `CPlayer`, hace lo siguiente:

```
# Disparar con la tecla [Space].
if CKeyboard.inst().fire():
    print("FIRE")
    b = CPlayerBullet()
    b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() / 2,
self.getY())
    b.setVelX(0)
    b.setVelY(-10)
    b.setBounds(0, 0, 640, 360)
    b.setBoundAction(CGameObject.DIE)
    CBulletManager.inst().addBullet(b)
```

Como vemos, le estamos estableciendo los límites de pantalla con dos números (*640 y 360*), pero no es una buena práctica de programación poner estos valores aquí, ya que de cambiar el tamaño de la pantalla, debemos cambiar todos estos valores en el código, en todas las partes donde los hayamos usado. En este momento lo tenemos aquí, y también lo tenemos en `main.py`, al crear la ventana. Si no tenemos cuidado, acabaremos con estos números repetidos en varias partes, y eso no es bueno.

Para corregir esto, haremos una clase en la cual pondremos todas las *constantes del juego* (los valores que no cambian). A esta clase la llamaremos `CGameConstants`, y como queremos poder acceder a estas constantes desde cualquier clase en el juego, esta clase será un *singleton*.

Veamos el código del ejemplo ubicado en la carpeta `capitulo_09\004_game_constants`. El código de la clase `CGameConstants` es simplemente una clase que tiene constantes:

```
# -*- coding: utf-8 -*-
```

```
#-----
# Clase CGameConstants.
# Constantes globales para el juego. Tiene valores estáticos.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----
```

```
import pygame
```

```
class CGameConstants(object):
```

```
    SCREEN_WIDTH = 640
    SCREEN_HEIGHT = 360
```

En `main.py`, en lugar de utilizar los valores 640 y 360 directamente, lo que hacemos es leerlos desde la clase de constantes:

```
# Importar la clase CGameConstants
from api.CGameConstants import *

# Definir ancho y alto de la pantalla.
SCREEN_WIDTH = CGameConstants.SCREEN_WIDTH
SCREEN_HEIGHT = CGameConstants.SCREEN_HEIGHT
```

En la clase `CPlayer`, en el momento de disparar, también se leen los valores desde la clase de constantes:

```
# Importar la clase CGameConstants
from api.CGameConstants import *

...

# Mover el objeto.
def update(self):

    ...

    # Disparar con la tecla [Space].
    if CKeyboard.inst().fire():
        print("FIRE")
        b = CPlayerBullet()
```

```

        b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY())
        b.setVelX(0)
        b.setVelY(-10)
        b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
        b.setBoundAction(CGameObject.DIE)
        CBulletManager.inst().addBullet(b)

```

Esto que hemos hecho se llama reorganizar el código, para hacerlo más prolijo, y minimizar los posibles errores en el futuro. Pero esto también es fundamental para hacer buenos juegos. Por ejemplo, si queremos probar tener una ventana de diferente tamaño, solo tendremos que cambiar los valores en la clase de constantes, y el juego seguirá funcionando.

De la manera en que estaba antes, deberíamos cambiar los valores en cada sitio del juego donde se usen, y eso lleva tiempo y es probable que nos olvidemos de alguno y luego haya problemas en el juego.

La clave para la flexibilidad que necesitan los juegos es programar limpio, claro y tener en todo momento el cien por ciento del control de lo que hace el programa.

De esta forma, hemos terminado con el manager de balas. En el siguiente capítulo implementaremos el manager de enemigos, para tener muchos enemigos en en juego.

10. El Manager de Enemigos

Así como en el capítulo anterior creamos balas y las controlamos con un *manager*, lo mismo haremos con los enemigos. Crearemos entonces un *manager* para manejar a los enemigos. De esta forma, el código del juego quedará mucho más sencillo de leer, y cada clase hará solamente lo que le corresponde. El *manager* de enemigos se encargará del control de la lista de enemigos, al igual que lo hacía el *manager* de balas con las balas.

El Manager de Enemigos

Comenzaremos agregando en la carpeta `api`, una clase denominada `CEnemyManager` para que contenga la lista (o array) de enemigos. Lo que haremos es copiar la clase `CBulletManager`, dado que lo que hace es lo mismo por ahora. Luego mejoraremos esto, porque no es bueno tener dos clases cuyo código sea similar, pero de alguna forma hay que empezar.

Veamos el código del ejemplo que se encuentra en la carpeta `capitulo_10\001_manager_de_enemigos`.

El código de la clase `CEnemyManager` se encuentra en la carpeta `api`, y es el siguiente:

```
# -*- coding: utf-8 -*-

#-----
# Clase CEnemyManager.
# Clase para manejar los enemigos del juego.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

class CEnemyManager(object):

    mInstance = None
    mInitialized = False

    # Lista de enemigos.
    mEnemies = None

    def __new__(self, *args, **kwargs):
```

```

        if (CEEnemyManager.mInstance is None):
            CEEnemyManager.mInstance = object.__new__(self, *args,
**kargs)
            self.init(CEEnemyManager.mInstance)
        else:
            print("Cuidado: CEEnemyManager(): No se debería instanciar
más de una vez esta clase. Usar CEEnemyManager.inst().")
            return self.mInstance

    @classmethod
    def inst(cls):
        if (not cls.mInstance):
            return cls()
        return cls.mInstance

    def init(self):
        if (CEEnemyManager.mInitialized):
            return
        CEEnemyManager.mInitialized = True

        # Crear la lista de enemigos.
        CEEnemyManager.mEnemies = []

    # Procesar los objetos.
    def update(self):
        for b in CEEnemyManager.mEnemies:
            b.update()

        i = len(CEEnemyManager.mEnemies)

        while i > 0:
            if CEEnemyManager.mEnemies[i-1].isDead():
                CEEnemyManager.mEnemies[i-1].destroy()
                CEEnemyManager.mEnemies.pop(i-1)
                print("se elimina un enemigo")
            i = i - 1

        # Mostrar la cantidad de balas que hay en el manager.
        print(len(CEEnemyManager.mEnemies))

    # Dibujar los objetos.
    def render(self, aScreen):
        for b in CEEnemyManager.mEnemies:
            b.render(aScreen)

    # Agregar un objeto a la lista.
    def addEnemy(self, aEnemy):
        CEEnemyManager.mEnemies.append(aEnemy)

```

```

# Destruir todos los objetos y removerlos de la lista.
# Destruir la lista.
def destroy(self):
    i = len(CEnemyManager.mEnemies)
    while i > 0:
        CEnemyManager.mEnemies[i-1].destroy()
        CEnemyManager.mEnemies.pop(i-1)
        i = i - 1

CEnemyManager.mInstance = None

```

Como vemos, el manager de enemigos es muy parecido al manager de balas que creamos en el capítulo anterior. De hecho, es el mismo código, salvo que cambia `CBulletManager` por `CEnemyManager`, el nombre del array `mBullets` cambia por `mEnemies` y la función `addBullet()` cambia por `addEnemy()`. Por ahora lo dejaremos así, y más adelante mejoraremos esto para no tener código duplicado.

Al igual que el manager de balas, el manager de enemigos es una clase que tiene un array con los enemigos que están actualmente en el juego. Usaremos la función `addEnemy()` para agregar un enemigo al manager, para que éste lo agregue a la lista de enemigos. Luego tendremos, como siempre, las funciones `update()`, `render()` y `destroy()` que se encargarán de recorrer el array de enemigos e invocar a la función `update()` y `render()` respectivamente para cada uno de los enemigos de la lista, e invocar a `destroy()` al eliminar el enemigo.

Entonces, la función `update()` del manager de enemigos corre la lógica de los enemigos, la función `render()` dibuja en pantalla todos los enemigos y la función `destroy()` que se llama al terminar el juego destruye a todos los enemigos que quedan en la lista. Todo esto es similar a lo que hicimos con las balas en el capítulo anterior.

Utilizando un manager, ya no tenemos necesidad de tener una variable para cada enemigo. El programa `main.py`, queda mucho más sencillo. Ya no tendremos las variables globales `n1`, `n2`, ...`n5` que teníamos para cada nave enemiga, sino que las usaremos solo como variables temporales al crear los enemigos en la función `init()`. Luego de crear las naves, las agregamos al manager y se manejan automáticamente.

Cuando creamos los enemigos en la función `init()` en `main.py`, debemos agregarlos al manager de enemigos. Hay que tener en cuenta que si nos olvidamos de hacer esto, los enemigos no se van a procesar ni a dibujar.

...

```

# Importar el manager de enemigos.
from api.CEnemyManager import *

```

```

...

# Función de inicialización.
def init():

    ...

    # Crear las naves: se le pasa como parámetro la imagen de la nave.
    n1 = CNave(CNave.TYPE_PLATINUM)
    n2 = CNave(CNave.TYPE_GOLD)
    n3 = CNave(CNave.TYPE_RED)
    n4 = CNave(CNave.TYPE_GREEN)
    n5 = CNave(CNave.TYPE_CYAN)

    # Colocar las naves en su posición inicial.
    n1.setXY(0, 100)
    n2.setXY(0, 150)
    n3.setXY(0, 200)
    n4.setXY(0, 250)
    n5.setXY(0, 300)

    ...

    # Agregar las naves al manager.
    CEnemyManager.inst().addEnemy(n1)
    CEnemyManager.inst().addEnemy(n2)
    CEnemyManager.inst().addEnemy(n3)
    CEnemyManager.inst().addEnemy(n4)
    CEnemyManager.inst().addEnemy(n5)

    ...

```

La función `update()` en `main.py`, ya no tiene una llamada a `update()` para cada enemigo (las variables `n1` a `n5` han sido eliminadas), sino que simplemente hace una llamada a la función `update()` del manager de enemigos, y éste es quien se encarga de invocar `update()` a cada enemigo:

```

def update():

    ...

    # Actualizar los enemigos
    CEnemyManager.inst().update()

    ...

```


Del mismo modo, en la función `render()` de `main.py`, se llama a la función `render()` del manager de enemigos y ya no usaremos una variable para cada enemigo:

```
def render():  
  
    ...  
  
    # Dibujar los enemigos.  
    CEnemyManager.inst().render(screen)  
  
    ...
```

Lo mismo ocurre en la función `destroy()`, en lugar de llamar a la función `destroy()` para cada una de las variables que había para cada enemigo, se llama a la función `destroy()` del manager de enemigos:

```
def destroy():  
  
    ...  
  
    # Destruir los enemigos.  
    CEnemyManager.inst().destroy()  
  
    ...
```

Como vemos, al haber eliminado muchas variables, el código queda bastante más simple, y el hecho de agregar enemigos ahora es mucho más sencillo, facilitando el desarrollo.

Disparo de los Enemigos

Como estamos construyendo un juego de disparos (un *shooter*) como *Space Invaders*, hagamos ahora que los enemigos disparen.

Lo primero que necesitaremos es una clase para la bala enemiga. A esta clase la llamaremos `CEnemyBullet`, y el código será igual al código de la clase de la bala del jugador, excepto que usa un gráfico de una bala roja, para indicar que es peligrosa. Como es una clase del juego, esta clase irá en la carpeta `game`. La imagen de la bala, como siempre, la colocamos en la carpeta `images` en `assets`. La imagen de la bala se encuentra en el archivo `enemy_bullet.png`. El tamaño es igual que la bala del jugador (7x7 píxeles). Cópiala del ejemplo a tu propio proyecto.



Figura 10-1: La bala que va a disparar las nave enemiga.

Veamos el ejemplo ubicado en la carpeta `capitulo_10\002_disparo_de_enemigos`. El código de la clase `CEnemyBullet` es muy simple, y lo único que hace es inicializar el sprite con la ruta de la imagen:

```
# -*- coding: utf-8 -*-

#-----
# Clase CEnemyBullet.
# Balas del enemigo.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# La clase CEnemyBullet hereda de CSprite
from api.CSprite import *

class CEnemyBullet(CSprite):

    # Constructor.
    def __init__(self):
        CSprite.__init__(self, "assets/images/enemy_bullet.png")

    # Mover el objeto.
```

```

def update(self):

    CSprite.update(self)

    # Dibuja el objeto en la pantalla.
    # Parámetros:
    # aScreen: La superficie de la pantalla en donde dibujar.
    def render(self, aScreen):

        CSprite.render(self, aScreen)

    # Liberar lo que haya creado el objeto.
    def destroy(self):

        CSprite.destroy(self)

```

Nota: Ahora, mientras hacemos las clases por primera vez, vemos que hay código muy parecido, como es el caso entre las balas del jugador y las balas enemigas. Esto en un juego mediano no es así, dado que cada clase tendrá sus propios comportamientos, animaciones o características que las distingan unas de otra. Conviene tener una clase para cada objeto del juego siempre y cuando haya diferencias sustanciales con otros objetos. Y siempre hay que tratar de evitar tener código duplicado o código muy similar.

Teniendo la imagen de la bala en la carpeta de imágenes y la clase `CEnemyBullet` pronta, lo que queda es hacer que la nave enemiga dispare una bala cada cierto tiempo. Para hacer esto simple, haremos que con cierta aleatoriedad (*random*), la nave dispara una bala.

Para disparar una bala, lo que hacemos es crear una instancia de la clase `CEnemyBullet`, le establecemos su posición inicial, su velocidad y la agregamos al manager de enemigos (porque es una bala enemiga). El resto se hace solo, dado que el manager luego se encarga de mover la bala y de dibujarla.

En la función `update()` de la clase `CNave`, tenemos el siguiente código para disparar una bala cada cierto tiempo:

```

...

# Importar las clases necesarias para disparar.
import random
from game.CEnemyBullet import *
from api.CGameConstants import *
from api.CEnemyManager import *

class CNave(Sprite):

    ...

```

```

# Mover el objeto.

def update(self):

    # Invocar update() de CSprite.
    CSprite.update(self)

    # Ver si la nave dispara.
    if random.randrange(1, 50) == 1:
        b = CEnemyBullet()
        b.setXY(self.getX() + self.getWidth()/2 - b.getWidth()/2,
self.getY() + self.getHeight())
        b.setVelX(0)
        b.setVelY(10)
        b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
        b.setBoundAction(CGameObject.DIE)
        CEnemyManager.inst().addEnemy(b)

    ...

```

Esta forma de disparar aleatoriamente no es la mejor forma de hacerlo y es temporal. En cada frame, si un número aleatorio entre 1 y 50 es igual a 1 (esa es una probabilidad de 1/50), se dispara una bala enemiga. En un futuro mejoraremos esto.

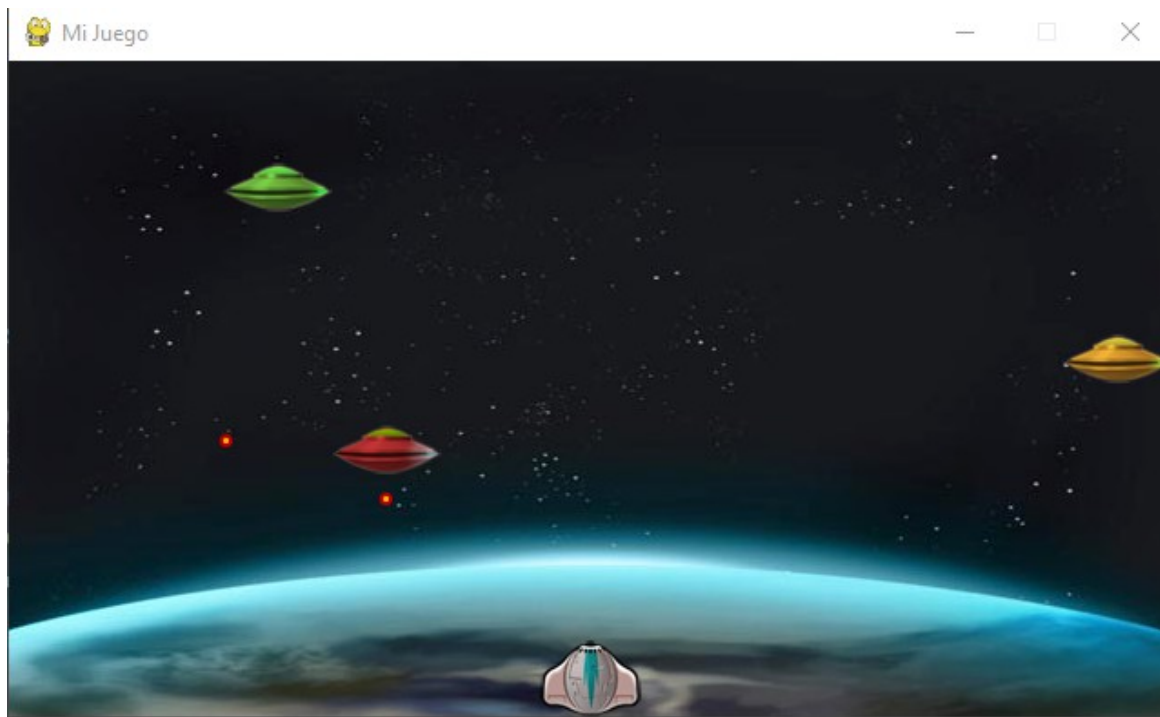


Figura 10-2: Las naves enemigas disparan y se vuelven peligrosas.

Al disparar, se crea una instancia de la clase `CEnemyBullet`, se coloca la bala en la posición inicial (veremos más sobre esto a continuación), se establece una velocidad vertical positiva para que la bala se mueva hacia abajo, se establecen los límites de la pantalla y se le establece la acción `CGameObject.DIE` para que cuando la bala toque un borde de la pantalla se muera. Por último, se agrega la bala al manager de enemigos.

Al igual que con las balas del jugador, cuando las balas de las naves tocan un borde, como fueron marcadas para morir, el manager las destruye y las elimina de la lista.

Para posicionar la bala, usamos la función `setXY()`, pero debemos hacer un cálculo, para saber la x y la y para colocar la bala debajo de la nave y en el medio. Mira la siguiente figura:

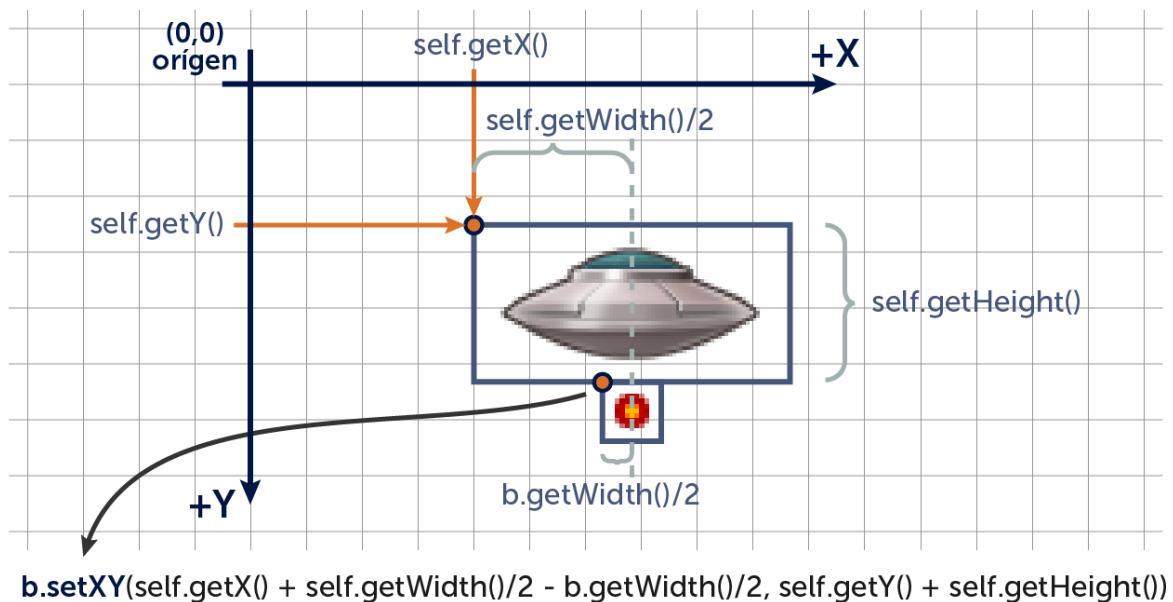


Figura 10-3: Colocando la bala en la posición inicial.

Como la coordenada (el *punto de registro*) de las imágenes es la esquina superior izquierda, debemos calcular que coordenadas (x, y) le ponemos a la bala para que aparezca en el medio de la nave y salga desde abajo de la misma. Como vemos, tenemos:

x = x de la nave + la mitad del ancho de la nave - la mitad del ancho de la bala.
y = y de la nave + el alto de la nave .

Esto, traducido al programa queda:

```
b.setXY(self.getX() + self.getWidth()/2 - b.getWidth()/2,  
        self.getY() + self.getHeight())
```

Nota: En los videojuegos es muy común este tipo de operaciones para que las cosas aparezcan en el lugar correcto. Lleva tiempo programar muy prolijo como es este caso, pero

programar prolijo y atendiendo a los detalles, hace que los juegos queden muy pulidos y se puedan adaptar rápidamente a los cambios para mejoras.

Inicialización de los Enemigos - La Formación

En esta sección vamos a implementar la formación inicial de los enemigos, es decir, al inicializar a los enemigos, los colocaremos en una posición determinada, creando una formación de naves enemigas. Veamos la siguiente figura.



Figura 10.4: La formación inicial de los enemigos.

Como ya tenemos implementado el manager de enemigos, lo único que debemos hacer es crear a cada enemigo y establecer su posición inicial, velocidad y tipo de nave. Esto lo hacemos en la función `init()`, en `main.py`, que es en donde ahora estamos creando a los enemigos.

Veamos el ejemplo que se encuentra ubicado en la carpeta `capitulo_10\003_formacion_de_enemigos`.

Como tenemos que colocar a cada enemigo en una posición en una grilla (porque la formación tiene forma de grilla), usaremos una estructura `while` dentro de otra, la primera corresponderá a la *fila* y la segunda a la *columna* (usamos la variables `f` para la fila y la variable `c` para la columna). Veamos el código que crea a los enemigos en formación:

```
def init():
```

```
...
```

```

# Crear la formación inicial.
f = 0
while f <= 4:
    c = 0
    while c <= 4:
        n = CNav(f)
        n.setXY(100 + (100 * c), 50 + (40 * f))
        n.setVelX(4)
        n.setVelY(0)
        n.setBounds(0, 0, SCREEN_WIDTH - n.getWidth(),
                    SCREEN_HEIGHT - n.getHeight())
        n.setBoundAction(CGameObject.BOUNCE)
        CEnemyManager.inst().addEnemy(n)
        c = c + 1
    f = f + 1

...

```

En la formación tendremos cinco filas y cinco columnas, y crearemos a los enemigos desde arriba hacia abajo y de izquierda a derecha. Entonces, el primer `while` corresponde a las filas (se usa la variable `f` por “fila”) y el segundo `while` corresponde a la columna (usando la variable `c` por “columna”). Para cada fila, recorreremos las columnas.

Como la formación tiene cinco filas, la variable `f` irá desde cero a cuatro (son cinco iteraciones). Del mismo modo, para cada fila, la variable `c` irá desde cero a cuatro (siendo cinco iteraciones).

Al momento de crear la nave, se le pasa como parámetro el tipo de nave. Este valor coincide con el número de fila (ver los valores de las constantes en la clase `CNav`, donde `CNav.TYPE_PLATINUM` es 0, `CNav.TYPE_GOLD` es 1, etc).

Luego viene la sentencia `n.setXY(100 + (70 * c), 50 + (35 * f))` para colocar a cada nave en su posición inicial. Aquí usamos varios números operados para calcular las coordenadas iniciales de la nave. Como vemos en el cálculo de la sentencia `setXY()`, tenemos varios valores a tener en cuenta: el margen horizontal desde el borde izquierdo de la pantalla (`100 + ...`) (denominado *offset horizontal*), el margen vertical desde la parte superior de la pantalla (`50 + ...`) (denominado *offset vertical*), el espacio horizontal entre las naves (`70`) y el espacio vertical entre las naves (`35`).

Todos estos valores se tienen en cuenta, junto con la columna y fila de la cual se trate, para calcular la posición inicial de las naves. Para calcular la coordenada `x`, se multiplica la columna por la separación entre naves y a eso se le suma el *offset horizontal*. Del mismo modo, para calcular la coordenada `y`, se multiplica la fila por la separación entre naves y a eso se le suma el *offset vertical*.

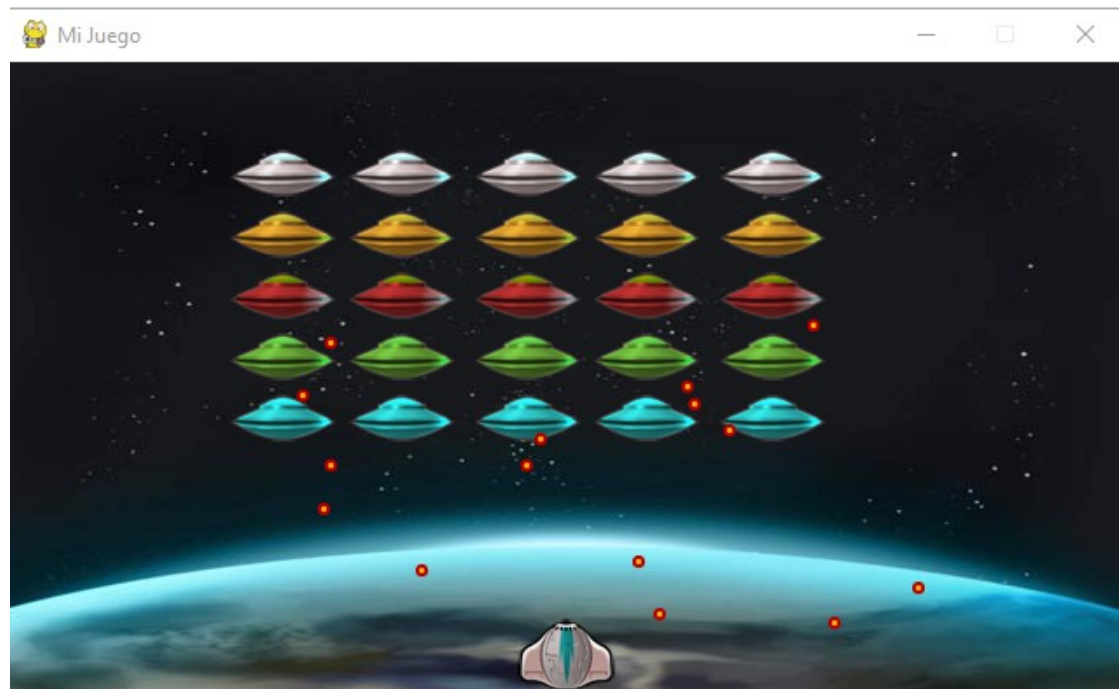


Figura 10.5: Ahora el juego tiene muchos enemigos ya formados.

Por último, a la nave le establecemos los límites de movimiento, le ponemos el comportamiento para rebotar como siempre, y agregamos el enemigo al manager. Es importante notar que el código de inicialización de los enemigos ha quedado ahora mucho más reducido, dado que las cosas se calculan y no se establecen los valores de a uno para cada nave, como se hacía antes.

Al correr el ejemplo vemos que la formación rebota y no baja toda junta como en el juego *Space Invaders*. Eso lo haremos más adelante.

Agregando otro Jugador - Modo Dos Players

Los videojuegos son mucho más divertidos cuando se juega con varios jugadores, por eso es que agregaremos un jugador más al juego. Implementaremos un modo de dos jugadores, en el cual uno controla una nave con color azul, utilizando las flechas para moverla y *[Space]* para disparar, y el segundo jugador manejará una nave de color rojo, utilizando las teclas *[A]* y *[D]* para moverse, y *[Q]* para disparar.



Figura 10-6: La nave azul es el primer jugador y la nave roja es el segundo jugador.

Veamos el ejemplo que se encuentra ubicado en la carpeta `capitulo_10\004_dos_jugadores`.

Lo primero que hacemos es colocar la imagen de la nave roja en la carpeta `assets/images`. En el programa `main.py`, vamos a renombrar la variable `player` por `player1`, y agregamos una variable `player2` para controlar al segundo jugador, y agregamos el código que lo crea e inicializa, luego se le corre `update()` y `render()` en el game loop, y se destruye al final con `destroy()`. En el siguiente código se muestra esto:

```
...

player1 = None
player2 = None

...

# Función de inicialización.
def init():

    ...

    global player1
    global player2

    ...

    # Crear el jugador 1.
    player1 = CPlayer(CPlayer.TYPE_PLAYER_1)
    player1.setXY(SCREEN_WIDTH / 4 - player1.getWidth() / 2,
SCREEN_HEIGHT - player1.getHeight())
    player1.setBounds(0, 0, SCREEN_WIDTH - player1.getWidth(),
SCREEN_HEIGHT)

    # Crear el jugador 2.
```

```

    player2 = CPlayer(CPlayer.TYPE_PLAYER_2)
    player2.setXY(SCREEN_WIDTH / 4 * 3 - player2.getWidth() / 2,
SCREEN_HEIGHT - player2.getHeight())
    player2.setBounds(0, 0, SCREEN_WIDTH - player2.getWidth(),
SCREEN_HEIGHT)

    ...

# Correr la lógica del juego.
def update():

    ...

    # Lógica de los jugadores.
    player1.update()
    player2.update()

    ...

# Dibujar el frame y actualizar la pantalla.
def render():

    ...

    # Dibujar los jugadores.
    player1.render(screen)
    player2.render(screen)

    ...

# Función de destrucción.
def destroy():
    global player1
    global player2

    # Destruir la nave de los jugadores.
    player1.destroy()
    player1 = None
    player2.destroy()
    player2 = None

    ...

```

Como vemos en la creación de los jugadores, le estamos pasando como parámetro el tipo de jugador del que se trata (`CPlayer.TYPE_PLAYER_1` o `CPlayer.TYPE_PLAYER_2`). En la clase `CPlayer`, guardamos este valor en una variable `mType`, y según el valor que tenga manejaremos el jugador con un conjunto de teclas u otro. También según el tipo de jugador

del que se trate, se cargará una imagen u otra.

A continuación se muestran los cambios realizados en la clase `CPlayer`.

...

La clase `CPlayer` hereda de `CSprite`.

```
class CPlayer(CSprite):
```

```
    # Tipos de jugador.
```

```
    TYPE_PLAYER_1 = 0
```

```
    TYPE_PLAYER_2 = 1
```

...

```
def __init__(self, aType):
```

```
    # Segun el tipo de la nave, la imagen que se carga.
```

```
    self.mType = aType
```

```
    if self.mType == CPlayer.TYPE_PLAYER_1:
```

```
        imgFile = "assets/images/player00.png"
```

```
    elif self.mType == CPlayer.TYPE_PLAYER_2:
```

```
        imgFile = "assets/images/player10.png"
```

```
    # Invocar al constructor de CSprite con la imagen a cargar.
```

```
    CSprite.__init__(self, imgFile)
```

```
    # Mover el objeto.
```

```
def update(self):
```

```
    # Obtener los controles.
```

```
    if self.mType == CPlayer.TYPE_PLAYER_1:
```

```
        left = CKeyboard.inst().leftPressed()
```

```
        right = CKeyboard.inst().rightPressed()
```

```
        fire = CKeyboard.inst().fire()
```

```
    else:
```

```
        left = CKeyboard.inst().APressed()
```

```
        right = CKeyboard.inst().DPressed()
```

```
        fire = CKeyboard.inst().fire2()
```

```
    # Mover la nave según las teclas.
```

```
    if (not left and not right):
```

```
        self.setVelX(0)
```

```
    else:
```

```
        if left:
```

```
            self.setVelX(-4)
```

```
        elif right:
```

```

        self.setVelX(4)

    # Disparar.
    if fire:
        print("FIRE")
        b = CPlayerBullet()
        b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY())
        b.setVelX(0)
        b.setVelY(-10)
        b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
        b.setBoundAction(CGameObject.DIE)
        CBulletManager.inst().addBullet(b)

    ...

```

Como vemos en el código de `update()` de `CPlayer`, según el tipo de nave del que se trate, usamos controles diferentes para mover el jugador. Como ahora necesitamos leer más teclas, tenemos que agregar el control de estas teclas en la clase `CKeyboard`.

En la clase `CKeyboard` agregamos variables para leer las teclas de movimiento que necesitamos. Más adelante debemos terminar de escribir la clase `CKeyboard` para que sirva para poder saber si cualquier tecla del teclado está siendo pulsada, si está suelta, etc.

Aprovechamos y agregamos funciones para detectar las teclas de direcciones usando `[Izquierda]`, `[Derecha]`, `[Arriba]`, `[Abajo]` y `[W]`, `[A]`, `[S]`, `[D]`. Para disparar, se usan las funciones `fire1()` y `fire2()`, para el jugador uno y dos respectivamente, que chequean cuando las teclas de disparo se pulsan por primera vez. Para disparar usamos `[Space]` para el jugador uno y `[Q]` para el jugador dos.

Examina en el programa, el código de la clase `CKeyboard` para ver estas funciones, que son similares a las que ya habían sido implementadas.

```

# -*- coding: utf-8 -*-

#-----
# Clase CKeyboard.
# Clase para manejar el estado de las teclas en el juego.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

```

```

class CKeyboard(object):

    mInstance = None
    mInitialized = False

    mLeftPressed = False
    mRightPressed = False
    mUpPressed = False
    mDownPressed = False
    mAPressed = False
    mDPressed = False
    mWPressed = False
    mSPressed = False

    # Estado de la tecla [Space] en el frame anterior.
    mSpacePressedPreviousFrame = False
    mSpacePressed = False
    # Estado de la tecla [Q] en el frame anterior.
    mQPressedPreviousFrame = False
    mQPressed = False

    def __new__(self, *args, **kwargs):
        if (CKeyboard.mInstance is None):
            CKeyboard.mInstance = object.__new__(self, *args, **kwargs)
            self.init(CKeyboard.mInstance)
        else:
            print("Cuidado: CKeyboard(): No se debería instanciar más
de una vez esta clase. Usar CKeyboard.inst().")
            return CKeyboard.mInstance

    @classmethod
    def inst(cls):
        if (not cls.mInstance):
            return cls()
        return cls.mInstance

    def init(self):
        if (CKeyboard.mInitialized):
            return
        CKeyboard.mInitialized = True

        CKeyboard.mLeftPressed = False
        CKeyboard.mRightPressed = False
        CKeyboard.mUpPressed = False
        CKeyboard.mDownPressed = False
        CKeyboard.mAPressed = False
        CKeyboard.mDPressed = False
        CKeyboard.mWPressed = False

```

```

CKeyboard.mSPressed = False

CKeyboard.mSpacePressedPreviousFrame = False
CKeyboard.mSpacePressed = False
CKeyboard.mQPressedPreviousFrame = False
CKeyboard.mQPressed = False

def keyDown(self, key):
    if (key == pygame.K_LEFT):
        CKeyboard.mLeftPressed = True
    if (key == pygame.K_RIGHT):
        CKeyboard.mRightPressed = True
    if (key == pygame.K_UP):
        CKeyboard.mUpPressed = True
    if (key == pygame.K_DOWN):
        CKeyboard.mDownPressed = True
    if (key == pygame.K_SPACE):
        CKeyboard.mSpacePressed = True
    if (key == pygame.K_a):
        CKeyboard.mAPressed = True
    if (key == pygame.K_d):
        CKeyboard.mDPressed = True
    if (key == pygame.K_w):
        CKeyboard.mWPressed = True
    if (key == pygame.K_s):
        CKeyboard.mSPressed = True
    if (key == pygame.K_q):
        CKeyboard.mQPressed = True

def keyUp(self, key):
    if (key == pygame.K_LEFT):
        CKeyboard.mLeftPressed = False
    if (key == pygame.K_RIGHT):
        CKeyboard.mRightPressed = False
    if (key == pygame.K_UP):
        CKeyboard.mUpPressed = False
    if (key == pygame.K_DOWN):
        CKeyboard.mDownPressed = False
    if (key == pygame.K_SPACE):
        CKeyboard.mSpacePressed = False
    if (key == pygame.K_a):
        CKeyboard.mAPressed = False
    if (key == pygame.K_d):
        CKeyboard.mDPressed = False
    if (key == pygame.K_w):
        CKeyboard.mWPressed = False
    if (key == pygame.K_s):
        CKeyboard.mSPressed = False

```

```

        if (key == pygame.K_q):
            CKeyboard.mQPressed = False

# Actualiza el estado de la tecla [Space].
def update(self):
    CKeyboard.mSpacePressedPreviousFrame = CKeyboard.mSpacePressed
    CKeyboard.mQPressedPreviousFrame = CKeyboard.mQPressed

def leftPressed(self):
    return CKeyboard.mLeftPressed

def rightPressed(self):
    return CKeyboard.mRightPressed

def upPressed(self):
    return CKeyboard.mUpPressed

def downPressed(self):
    return CKeyboard.mDownPressed

def spacePressed(self):
    return CKeyboard.mSpacePressed

def APressed(self):
    return CKeyboard.mAPressed

def DPressed(self):
    return CKeyboard.mDPressed

def WPressed(self):
    return CKeyboard.mWPressed

def SPressed(self):
    return CKeyboard.mSPressed

def QPressed(self):
    return CKeyboard.mQPressed

# Función usada para disparar.
# Solo retorna True en el momento en que se apreta la tecla.
def fire(self):
    return CKeyboard.mSpacePressed == True and
CKeyboard.mSpacePressedPreviousFrame == False

# Función usada para disparar.
# Solo retorna True en el momento en que se apreta la tecla.
def fire2(self):
    return CKeyboard.mQPressed == True and

```

```
CKeyboard.mQPressedPreviousFrame == False

def destroy(self):
    CKeyboard.mInstance = None
```

Por último, como un detalle, en la clase `CNave`, hemos cambiado la frecuencia (la probabilidad) con la que disparan las naves enemigas, porque era muy difícil esquivar las balas. Cambiamos 50 por 500 para que tenga menos chance de disparar.

Unificando los Managers

En este momento en el proyecto tenemos dos *managers* que son prácticamente iguales (el manager de enemigos y el manager de balas), y esto no es una buena práctica de diseño, dado que lleva a tener dos clases con el código repetido o muy similar. Debemos siempre evitar este tipo de cosas, porque hacen al código menos mantenible. Por ejemplo, si quisiéramos modificar el comportamiento de una clase, seguramente tengamos que modificar en más de un lugar a la vez.

Nota: Un videojuego es un sistema que requiere de cambios frecuentes en la funcionalidad. Muchas cosas que programamos no sabremos si funcionan bien hasta no probarlas en el juego. Otras cosas se planifican de una manera pero en el juego funcionan mejor de otra forma, y esto se va mejorando en sucesivas versiones del juego, a las que se le denomina *iteraciones* del desarrollo.

Lo que haremos es poner el código común en una clase base, denominada `CManager`, y luego tener dos clases, una para cada manager: `CEnemyManager` y `CBulletManager`. Podríamos tener una sola clase, pero como es probable que más adelante tengamos código específico para el manager de los enemigos y código específico para el manager de las balas, mejor haremos dos clases aunque por ahora queden casi vacías.

Examina el ejemplo de la carpeta `capitulo_10\005_managers_enemigos_disparos_unificados`.

Este ejemplo es igual al anterior, pero se ha agregado una clase `CManager` que tiene un array de elementos (balas o enemigos según el caso) y las funciones `update()`, `render()` y `destroy()` como es habitual. Luego, las clases `CEnemyManager` y `CBulletManager` son clases singleton que heredan de la clase `CManager`.

El código de la case `CManager` se lista a continuación:

```
# -*- coding: utf-8 -*-

#-----
# Clase Manager.
# Clase para manejar una lista con un tipo de objeto.
```



```

#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

class CManager(object):

    def __init__(self):

        # Lista de objetos.
        self.mArray = []

        # Procesar los objetos.
        def update(self):
            for e in self.mArray:
                e.update()

        i = len(self.mArray)
        while i > 0:
            if self.mArray[i-1].isDead():
                self.mArray[i-1].destroy()
                self.mArray.pop(i-1)
            i = i - 1

        # Dibujar los objetos.
        def render(self, aScreen):
            for e in self.mArray:
                e.render(aScreen)

        # Agregar un objeto a la lista.
        def add(self, aElement):
            self.mArray.append(aElement)

        # Destruir todos los objetos y removerlos de la lista.
        # Destruir la lista.
        def destroy(self):
            i = len(self.mArray)
            while i > 0:
                self.mArray[i-1].destroy()
                self.mArray.pop(i-1)
                i = i - 1

            self.mArray = None

```

De esta forma, el código que maneja el array ya no se encuentra duplicado en ambas clases como antes, sino que se encuentra centralizado ahora en la clase `CManager`. Antes una clase tenía un array llamado `mBullets` y la otra clase tenía un array llamado `mEnemies`. Ahora existe un solo código centralizado y el array se llama `mArray`. Veamos el código de ambos manager y veremos que queda mucho más simple. El código de `CEnemyManager` queda de la siguiente manera:

```
# -*- coding: utf-8 -*-

#-----
# Clase CEnemyManager.
# Clase para manejar los enemigos del juego.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# Importar la clase base del manager.
from api.CManager import *

class CEnemyManager(CManager):

    mInstance = None
    mInitialized = False

    def __new__(self, *args, **kwargs):
        if (CEnemyManager.mInstance is None):
            CEnemyManager.mInstance = object.__new__(self, *args,
**kwargs)
            self.init(CEnemyManager.mInstance)
        else:
            print("Cuidado: CEnemyManager(): No se debería instanciar
más de una vez esta clase. Usar CEnemyManager.inst().")
            return self.mInstance

    @classmethod
    def inst(cls):
        if (not cls.mInstance):
            return cls()
        return cls.mInstance

    def init(self):
```

```

        if (CEnemyManager.mInitialized):
            return
        CEnemyManager.mInitialized = True

        # Invocar al constructor de la clase base que crea la lista.
        CManager.__init__(self)

    # Procesar los objetos.
    def update(self):
        CManager.update(self)

    # Dibujar los objetos.
    def render(self, aScreen):
        CManager.render(self, aScreen)

    # Agregar un objeto a la lista.
    def add(self, aEnemy):
        CManager.add(self, aEnemy)

    # Destruir todos los objetos y removerlos de la lista.
    # Destruir la lista.
    def destroy(self):
        CManager.destroy(self)

    CEnemyManager.mInstance = None

```

El código de CBulletManager queda de la siguiente manera:

```

# -*- coding: utf-8 -*-

#-----
# Clase CBulletManager.
# Clase para manejar los disparos del juego.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# Importar la clase base del manager.
from api.CManager import *

class CBulletManager(CManager):

```

```

mInstance = None
mInitialized = False

def __new__(self, *args, **kwargs):
    if (CBulletManager.mInstance is None):
        CBulletManager.mInstance = object.__new__(self, *args,
**kwargs)
        self.init(CBulletManager.mInstance)
    else:
        print("Cuidado: CBulletManager(): No se debería instanciar
más de una vez esta clase. Usar CBulletManager.inst().")
        return self.mInstance

@classmethod
def inst(cls):
    if (not cls.mInstance):
        return cls()
    return cls.mInstance

def init(self):
    if (CBulletManager.mInitialized):
        return
    CBulletManager.mInitialized = True

    # Invocar al constructor de la clase base que crea la lista.
    CManager.__init__(self)

# Procesar los objetos.
def update(self):
    CManager.update(self)

# Dibujar los objetos.
def render(self, aScreen):
    CManager.render(self, aScreen)

# Agregar un objeto a la lista.
def add(self, aBullet):
    CManager.add(self, aBullet)

# Destruir todos los objetos y removerlos de la lista.
# Destruir la lista.
def destroy(self):
    CManager.destroy(self)

    CBulletManager.mInstance = None

```

Para agregar naves al manager de enemigos (en `main.py`), y para agregar balas al manager de balas, tanto al disparar en el jugador como en las naves enemigas (clases `CPlayer` y `CNave` respectivamente), usamos la función `add()` del manager, en lugar de tener que usar `addEnemy()` o `addBullet()`. De esta forma el código también queda más sencillo. Debes cambiar en estas clases estas llamadas por `add()`.

Recuerda que no es bueno tener código duplicado, porque esto hace menos mantenible el proyecto. Esta es la razón principal detrás de este cambio que hemos realizado.

Con esto finalizamos el manejo del manager de enemigos y del manager general. Ahora como vemos al ejecutar el juego, hay muchas balas pero ninguna hace efecto. En el siguiente capítulo veremos cómo hacer las colisiones entre objetos.

11. Detectando las Colisiones

Las colisiones son un aspecto muy importante en los juegos, sobre todo en los juegos arcade, porque las cosas interesantes suceden cuando dos objetos chocan. En este capítulo vamos a ver cómo detectar colisiones entre dos sprites y luego veremos cómo detectar colisiones entre un sprite y todos los enemigos o todas las balas.

Colisiones entre Dos Sprites

En el juego que estamos construyendo, debemos detectar cuando una bala del jugador choca con un enemigo para que éste explote. De la misma manera, las balas de los enemigos deben chequear colisiones con el jugador para saber si le pegan, y si esto ocurre, el jugador deberá morir. También, dependiendo del juego del que se trate, será necesario chequear las colisiones entre el jugador y los enemigos, entre los enemigos entre sí, o entre las balas entre sí, etc.

Comenzaremos con las colisiones más fáciles de detectar, y luego, más adelante avanzaremos a colisiones más complicadas. Por ahora lo que necesitamos saber básicamente es cuando un sprite colisiona con otro.

Sprites Rectangulares

Los sprites son imágenes, y siempre hay que tener en cuenta que los sprites son rectangulares. Al menos la imagen que se muestra en la pantalla es rectangular, aunque la forma real del sprite (el dibujo) no lo sea, porque tiene áreas con transparencia alrededor. Por ejemplo, veamos el sprite de la nave enemiga:



Figura 11-1: Imagen del sprite con áreas en blanco (transparente).

Al rectángulo que utilizamos para chequear la colisión de un sprite, lo denominamos *bounding box*, y es un rectángulo que encierra el área colisionable de un sprite. El *bounding box* es el rectángulo más chico que se pueda dibujar alrededor del sprite, lo que representa el área colisionable. Puede quedar algo del dibujo fuera del *bounding box*, pero no debería ser mucho porque en esa parte no se detecta la colisión.

Por ahora, utilizaremos como bounding box la misma área de la imagen, por lo cual no debería haber grandes áreas transparentes en el sprite. Más adelante, en otro capítulo, programaremos en la clase `CSprite` la posibilidad de tener un bounding box diferente al rectángulo de la imagen. El bounding box, entonces, es el rectángulo que usaremos para detectar la colisión y por ahora será del mismo tamaño que la imagen.

Lo ideal sería chequear si los píxeles de dos sprites se tocan, lo cual se llama *perfect pixel collision*, pero rara vez se usa, debido a que es muy lento determinar si los píxeles se tocan realmente. Esto haría que el juego funcione más lento. Por esta razón es que para detectar colisiones, se utiliza la comparación entre rectángulos, porque es muy rápida en comparación con cualquier otro método.

Al chequear las colisiones entre rectángulos, debemos tener cuidado con los casos donde las imágenes son irregulares y no corresponden a rectángulos, lo cual puede tener efectos como los que se muestran a continuación.

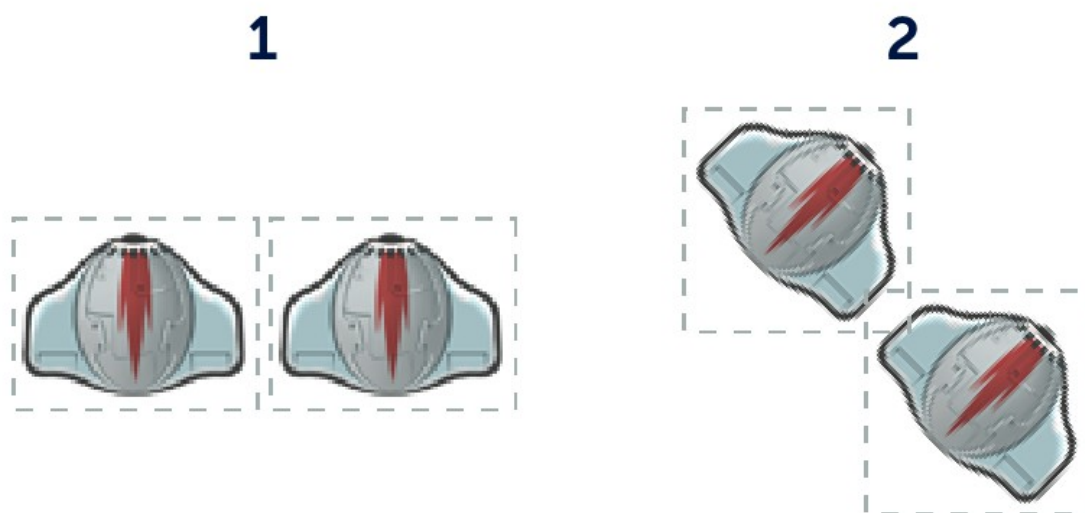


Figura 11-2: Las naves rotadas tienen áreas de colisión con partes en blanco.

En esta imagen, las naves de la primera posición no se están tocando, y esto se ve claramente en el dibujo. Los rectángulos de colisión (*bounding box*) no se tocan. Sin embargo, al rotar las naves (ver la segunda imagen), las naves no se tocan en la imagen, pero sus rectángulos de colisión sí se tocan.

Más adelante implementaremos el bounding box en los sprites para solucionar estos problemas. Por ahora tendremos que utilizar sprites cuyas imágenes sean aproximadamente

más rectangulares en su forma, para que las colisiones funcionen mejor.

También es posible hacer colisiones entre círculos si los sprites son de aspecto más circulares que rectangulares. Este tipo de colisión es muy sencillo de implementar y ésto lo veremos más adelante.

Colisiones entre Rectángulos

Comenzaremos el trabajo de colisiones detectando si los dos jugadores se chocan. Para saber si dos sprites chocan, debemos revisar si sus rectángulos se están tocando. La siguiente imagen muestra esto:

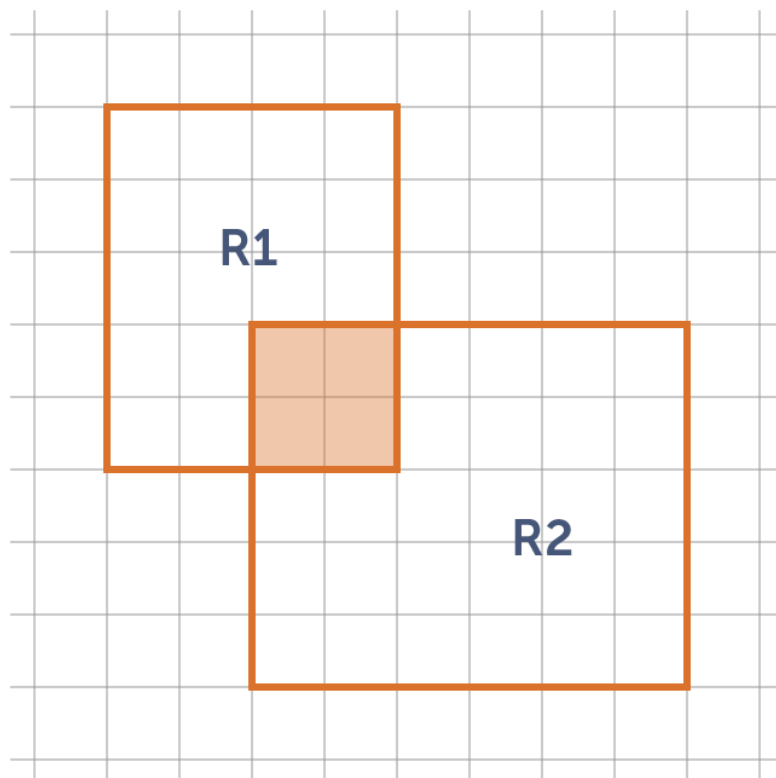


Figura 11-3: Colisión entre dos rectángulos.

Cada `sprite` en nuestro programa, tiene una función `getX()`, `getY()`, `getWidth()` y `getHeight()` que retorna la posición `x` e `y`, el *ancho* y el *alto* del `sprite` respectivamente. Con estas funciones, podemos calcular si los rectángulos colisionan (determinar si se intersectan).

Para detectar colisiones, haremos una función en la clase `CSprite` que reciba otro `sprite` como parámetro y se fije si los rectángulos de las imágenes colisionan. La función `collides()` de la clase `CSprite` es la siguiente:


```
# Función para detectar colisión contra otro sprite.
def collides(self, aSprite):
    x1 = self.getX()
    y1 = self.getY()
    w1 = self.getWidth()
    h1 = self.getHeight()
    x2 = aSprite.getX()
    y2 = aSprite.getY()
    w2 = aSprite.getWidth()
    h2 = aSprite.getHeight()

    if (((x1 + w1) > x2) and (x1 < (x2 + w2))) and
        (((y1 + h1) > y2) and (y1 < (y2 + h2))):
        return True
    else:
        return False
```

Veamos el ejemplo ubicado en la carpeta `capitulo_11\001_colisiones_entre_rectangulos`. En la clase `CSprite` se puede ver implementada esta función. Conviene hacer un dibujo en papel para entender lo que pregunta la función para saber si dos rectángulos colisionan (se pueden buscar demos en Internet, este algoritmo se denomina *AABB collision detection*). La ventaja es que una vez que se entiende esta función, se puede reutilizar en cualquier juego.

En este ejemplo movemos a las dos naves, cada una con sus teclas, y en la consola podemos ver que sale un mensaje en cada frame indicando si las naves están colisionando o no. Para esto, se ha puesto un chequeo en el programa `main.py`, en `update()`:

```
# Detectar la colision entre los dos jugadores.
if player1.collides(player2):
    print("COLISION ENTRE LOS JUGADORES")
else:
    print("...")
```

De esta forma, ahora podremos detectar colisiones entre dos sprites del juego. Lo siguiente será reaccionar a esas colisiones (explosiones, dañar una nave o matarla, etc), pero primero, debemos aprender cómo implementar colisiones contra un manager, o dicho de otra forma, detectar la colisión de un sprite contra todos los sprites de una lista de sprites (los sprites del manager).

Colisiones de un Sprite contra una Lista de Sprites (Manager)

En los juegos arcade o de acción como el que estamos haciendo, necesitamos saber si un sprite choca contra alguno de los sprites de una lista. Esto lo usaremos en varios casos. Por ejemplo, cuando queremos saber si una bala del jugador toca a alguno de los enemigos, o cuando el jugador mismo es quien toca a alguno de los enemigos. Ahora vamos a detectar cuando el jugador está chocando contra un enemigo (una nave o una bala, recuerda que tanto las naves como las balas enemigas se encuentran en el manager de enemigos).

Para chequear la colisión del jugador contra los sprites de la lista (manager) de enemigos, debemos chequear la colisión del rectángulo del jugador contra cada uno de los enemigos que exista en la lista de enemigos. La lista de enemigos se encuentra en la clase `CEnemyManager`. Hay que tener en cuenta que en esta lista también se encuentran las balas enemigas, las cuales se cuentan como un enemigo más (en realidad las balas enemigas son un tipo de enemigo).

Veamos el ejemplo que se encuentra en la carpeta `capitulo_11\002_colisiones_sprite_contra_lista`.

En este ejemplo, se muestra un texto por consola cuando uno de los jugadores choca contra alguno de los enemigos. Para esto, agregamos en la clase `CManager` una función que recibe un sprite como parámetro (el jugador en este caso) y recorre la lista chequeando la colisión con cada sprite de la lista. Si el sprite pasado como parámetro colisiona con algún sprite de la lista, la función retorna `True` y si al final de recorrer la lista no ha colisionado con ninguno retorna `False`. La función recorre la lista y usa la función `collides()` de `CSprite` escrita anteriormente para chequear la colisión. Veamos la función en la clase `CManager`:

```
# Determina si el sprite que se recibe como parámetro choca
# con alguno de los sprites de la lista.
# Retorna True si colisiona con alguno y False si no colisiona
# con ninguno.
def collides(self, aSprite):
    i = 0
    while i < len(self.mArray):
        if aSprite.collides(self.mArray[i]):
            return True
        i = i + 1
    return False
```

En `main.py` chequeamos la colisión entre el jugador y todos los sprites enemigos:

```
# Detectar la colisión entre los jugadores y los enemigos.
if CEnemyManager.inst().collides(player1):
    print("COLISION ENTRE PLAYER1 Y ENEMIGO")
if CEnemyManager.inst().collides(player2):
    print("COLISION ENTRE PLAYER2 Y ENEMIGO")
```

Por ahora, solo mostraremos en la consola si un jugador choca o no contra un enemigo, pero no importa cual. Esto lo hacemos así para comprobar que todo funciona bien. Más adelante, necesitaremos que la función `collides()` del manager retorne el sprite con el cual choca, para aplicarle alguna acción (por ejemplo, que explote o salte, o que tenga algún efecto).

Ahora que sabemos cómo chequear colisiones entre un sprite y una lista de sprites, seguiremos con la detección de colisiones que son necesarias para este juego.

Colisiones de las Balas del Jugador contra los Enemigos

Ahora vamos a detectar colisiones entre todas las balas del jugador contra todos los enemigos. Cuando la bala del jugador choque contra un enemigo, éste último morirá (y la bala también). Por ahora haremos que se muera el enemigo en forma inmediata, por lo cual desaparecerá instantáneamente. Más adelante haremos animaciones de una explosión cuando veamos cómo hacer animaciones y cuando sepamos cómo trabajar con los estados de los objetos (para hacerlo explotar).

Para ver si una bala choca a algún enemigo, debemos determinar si la bala choca con cada uno de los enemigos que hay en la lista de enemigos, preguntando uno a uno. Para eso, en la bala del jugador (en la clase `CPlayerBullet`), debemos recorrer la lista de enemigos y uno a uno ver si hay colisión o no. Esto es lo que hace la función `collides()` del manager, que hicimos en la sección anterior, así que la usaremos aquí. El código de `CPlayerBullet` queda de la siguiente manera:

```
...

# Importar el manager de enemigos para detectar colisiones.
from api.CEnemyManager import *

# La clase CPlayerBullet hereda de CSprite.
class CPlayerBullet(CSprite):

    ...

    # Mover el objeto.
    def update(self):

        CSprite.update(self)

        # Detectar choque contra los enemigos.
        if CEnemyManager.inst().collides(self):
            print("*****COLISION ENTRE BALA DEL JUGADOR Y ENEMIGO")

    ...
```

Abre el proyecto que se encuentra en la carpeta `capitulo_11\003_colisiones_balas_contra_enemigos` para ver este código en la clase `CPlayerBullet`. Ahora podemos detectar las colisiones de las balas del jugador y ya tenemos casi todo pronto para que las balas hagan algo.

Al chequear cada bala que se dispara contra todos los enemigos, vemos que el chequeo de colisiones crece exponencialmente con la cantidad de sprites que haya en el juego. Esta es un área que siempre deberemos optimizar.

Por ejemplo, si el juego tiene cinco sprites, y cada uno puede colisionar con los restantes (cuatro), la cantidad de colisiones que hay que detectar es $5 \times 4 = 20$ chequeos de colisión. Si N es la cantidad de sprites en el juego, la cantidad de colisiones es $N * (N-1)$. Esto es si cada sprite chequea colisiones contra todos los demás.

Como la acción de realizar cada chequeo de colisión lleva tiempo, en el futuro esto se debe optimizar para que el juego no se torne lento con los cálculos si hay muchas cosas en pantalla. Realizar muchas colisiones es algo que puede enlentecer el juego. Para evitar esto, por ejemplo, podemos no detectar las colisiones entre los enemigos entre sí. O no chequear la colisión entre dos sprites dos veces. Dependiendo del juego del que se trate por supuesto, esto ayuda a reducir la cantidad de colisiones a detectar.

Reaccionando a las Colisiones

Ahora que sabemos detectar colisiones, y que ya tenemos colocado el chequeo de colisiones en el código, vamos a hacer que cuando dos objetos se toquen, mueran. Más adelante los haremos explotar, pero por ahora haremos que mueran (en realidad en algún momento van a morir, luego de explotar, así que esto hay que hacerlo igual).

Para hacer que un objeto muera, haremos una función `die()` en la clase `CGameObject` que lo que hace es establecer la variable `mIsDead` en `True`.

Como hemos visto antes, y ya está programado en el motor, cuando esta variable se vuelve `True`, el manager destruye el objeto y lo elimina de la lista. Entonces, en el momento que detectamos colisiones, invocamos a la función `die()` en los objetos que queremos y los mismos serán eliminados. Más adelante en este punto del programa dispararemos sonidos, partículas, incrementaremos el puntaje, etc.

Lo primero que hay que hacer es cambiar la función de chequeo de colisiones que se encuentra en la clase `CManager`, para que en lugar de retornar sólo `True` o `False` como está ahora, que retorne el sprite con el cual se colisiona, o `None` si no se colisiona con ninguno. De esta forma podemos hacer algo con la referencia al sprite.

Veamos como queda la función `collides()` en la clase `CManager`, en el ejemplo que se encuentra en la carpeta `capitulo_11\004_matando_enemigos`:

```

# Determina si el sprite que se recibe como parámetro choca
# con alguno de los sprites de la lista.
# Retorna True si colisiona con alguno y False si no colisiona
# con ninguno.
def collides(self, aSprite):
    i = 0
    while i < len(self.mArray):
        if aSprite.collides(self.mArray[i]):
            return self.mArray[i]
        i = i + 1
    return None

```

En la clase CPlayerBullet, chequeamos la colisión de la bala contra los enemigos. Para eso guardamos en una variable `enemy` la referencia al primer sprite con el que choca la bala, o `None` si no choca con ninguno.

...

```

# La clase CPlayerBullet hereda de CSprite.
class CPlayerBullet(CSprite):

```

...

```

# Mover el objeto.
def update(self):

```

```

    CSprite.update(self)

```

```

    # Detectar choque contra los enemigos.

```

```

    # Si la bala choca un enemigo, mueren ambos.

```

```

    enemy = CEnemyManager.inst().collides(self)

```

```

    if enemy != None:

```

```

        print("*****COLISION ENTRE BALA DEL JUGADOR Y ENEMIGO")

```

```

        enemy.die()

```

```

        self.die()

```

...

Cuando la bala toca a un enemigo, tanto la bala como el enemigo mueren. Luego en este punto le agregaremos explosiones, sonidos, partículas, etc. Por ahora invocamos a la función `die()` para que mueran ambos objetos. En la clase `CGameObject`, implementamos esta función:

```
# Marcar al objeto para morir. El manager lo elimina.  
def die(self):  
    self.mIsDead = True
```

Al llamar a esta función, simplemente se marca el objeto para morir, y luego de ejecutarse su función `update()`, el manager se encarga de eliminarlo. Este comportamiento ya lo tenemos programado desde que hicimos el manager.

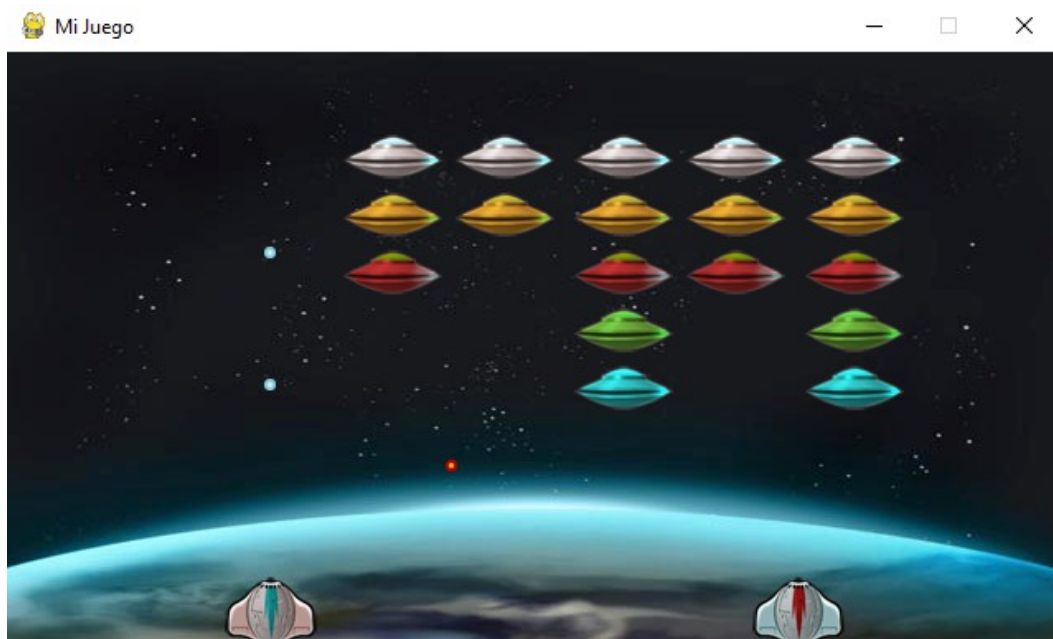


Figura 11-4: Ahora podemos disparar y eliminar enemigos.

Al correr el ejemplo, si le disparamos a todos los enemigos, nos quedaremos con la pantalla vacía. Más adelante programaremos la condición para ganar el juego, que es cuando no queden más enemigos en el manager. Ahora pasaremos a implementar los sprites con animaciones, efectos de sonidos, etc. y luego trabajaremos en el armado completo del juego.

En el siguiente capítulo veremos cómo implementar animaciones de sprites.

12. Animación de Sprites

En este capítulo veremos cómo hacer *animaciones* de sprites. Las animaciones son parte clave de los videojuegos, y hacen que los elementos del juego cobren vida. Para animar un sprite, necesitamos tener varias imágenes, que iremos mostrando sucesivamente para crear la ilusión de movimiento (*animación*).

Los Cuadros de una Animación

Hasta ahora nuestros sprites tienen solamente una imagen, pero un sprite puede tener más de una imagen si se la cambiamos cada cierto tiempo, para crear una animación. Comenzaremos haciendo que las naves enemigas tengan animación en lugar de tener una única imagen fija. Para esto, necesitamos tener una secuencia de imágenes correspondientes a la animación.



Figura 12-1: Cuadros (*frames*) de la nave enemiga.

A cada una de las imágenes pertenecientes a la animación, se le denomina *cuadro* o *frame* de animación. En el caso de la nave enemiga, como se muestra en la figura 12-1, la animación tiene nueve cuadros.

Estas imágenes deben estar nombradas secuencialmente con un número al final, por ejemplo, `grey_ufo_00.png`, `grey_ufo_01.png`, `grey_ufo_02.png` hasta `grey_ufo_08.png`, de forma tal que sea más sencillo cargar la secuencia de imágenes en forma consecutiva desde la programación, como veremos a continuación.

Un sprite animado podrá tener muchas imágenes en su animación, pero solo una será la que se muestra en el frame actual. Luego, la imagen se va cambiando a medida que pasan los frames, haciendo que se vayan mostrando una a una toda la secuencia de imágenes de la animación.

La Clase `CAnimatedSprite`

Para implementar un sprite con animación, haremos una clase denominada `CAnimatedSprite` que heredará de `CSprite`. Cuando necesitemos un sprite animado, haremos una clase (como por ejemplo la clase `CNave`) que heredará de `CAnimatedSprite` y ésta tendrá funciones relacionadas a la animación, heredando todo lo que ya tenemos de la clase `CSprite`. El diseño de estas dos clases queda de la siguiente manera:

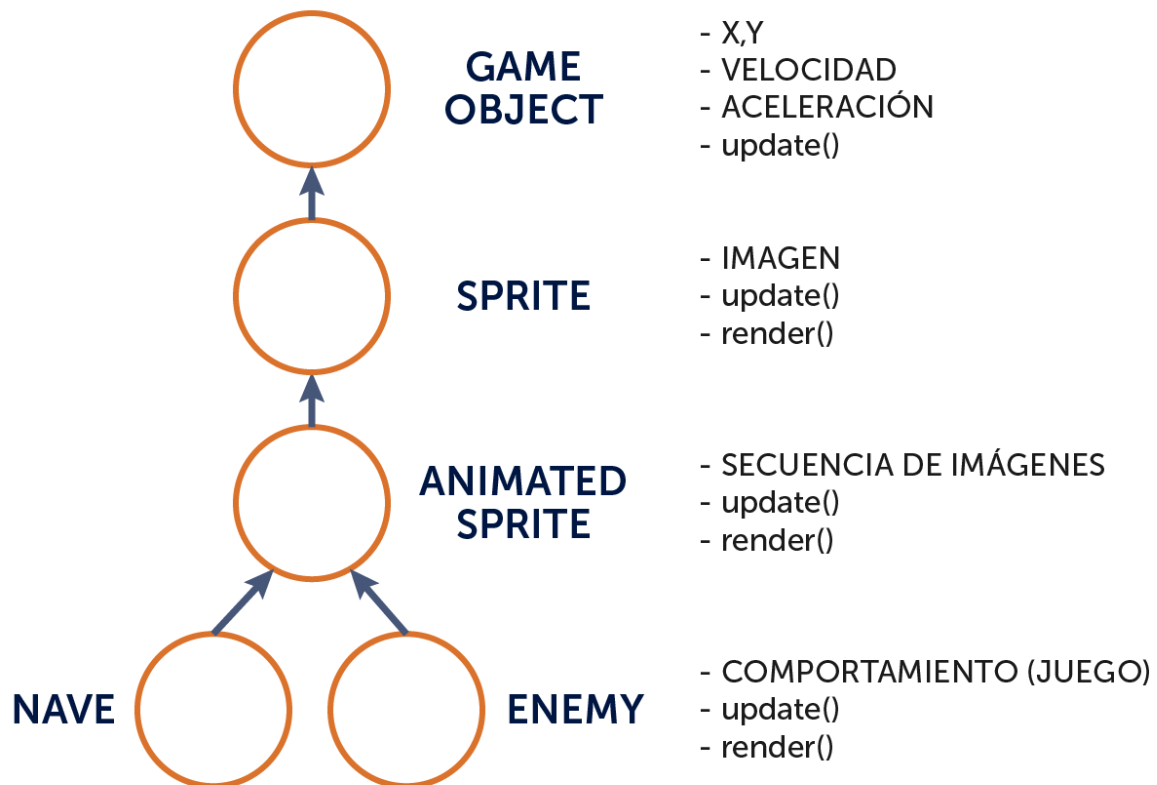


Figura 12-2: Diagrama de clases con la clase `CAnimatedSprite`.

Como vemos en el diagrama, la clase `CSprite` contiene la imagen que se muestra en la pantalla, y la clase `CAnimatedSprite` la cambia cada cierto tiempo para hacer la animación. Antes de comenzar a implementar la clase `CAnimatedSprite`, debemos preparar a la clase `CSprite` para poder cambiar su imagen.

La Función `setImage()` de la Clase `CSprite`

En el último ejemplo que realizamos, la clase `CSprite` está cargando la imagen del sprite en la función `init()`. Esto ahora ya no nos sirve, porque esta imagen va a cambiar continuamente si el sprite es animado. Lo que haremos es que cada clase (cada objeto del juego) cargue las imágenes que vaya a utilizar y simplemente se indique cual es la imagen que se va a dibujar, utilizando una función denominada `setImage()`, que haremos a continuación, y que de esta forma no se vuelva a cargar una imagen desde el archivo, sino que simplemente la cambiamos.

Veamos el ejemplo de la carpeta `capitulo_12\001_set_image_clase_sprite`. Este ejemplo hace lo mismo que el último ejemplo que realizamos, pero se ha modificado la clase `CSprite` para que ya no reciba una imagen como parámetro y cargarla, sino que cargar la imagen sea responsabilidad de la clase superior. Se agrega la función `setImage()` en la clase `CSprite`, para cargar una imagen. El código del constructor de `CSprite` y la función `setImage()` quedan así:

...

```
class CSprite(CGameObject):

    # Constructor:
    def __init__(self):

        CGameObject.__init__(self)

        # Imagen (superficie) del sprite.
        # Usar setImage() en la clase superior para establecer una
imagen.
        self.mImg = None

        # Ancho y alto de la imagen. La función setImage() los
actualiza.
        self.mWidth = 0
        self.mHeight = 0

        # Establecer la imagen del sprite.
        def setImage(self, aImg):

            # Establecer la imagen.
            self.mImg = aImg

            # Guardar el ancho y el alto.
            self.mWidth = self.mImg.get_width()
            self.mHeight = self.mImg.get_height()

    ...
```

Nota que el constructor de `CSprite` ya no recibe la imagen como parámetro.

Cuando hagamos un objeto del juego (como por ejemplo las clases `CEnemyBullet`, `CPlayerBullet`, `CNave` y `CPlayer`), haremos una clase que heredará de `CSprite`, cargando la imagen que corresponda y estableciéndola usando la función `setImage()` en `CSprite`. La función `setImage()` además, establece el ancho y el alto de la imagen, lo cual se utiliza en las colisiones.

Las clases `CEnemyBullet`, `CPlayerBullet`, `CNave` y `CPlayer` ahora modifican su constructor, para cargar la imagen y establecerla con `setImage()`. Estas clases quedan como se muestra a continuación.

```
...
```

```
class CEnemyBullet(CSprite):
```

```
    # Constructor.
```

```
    def __init__(self):
```

```
        CSprite.__init__(self)
```

```
        img = pygame.image.load("assets/images/enemy_bullet.png")
```

```
        img = img.convert_alpha()
```

```
        self.setImage(img)
```

```
...
```

```
...
```

```
# La clase CPlayerBullet hereda de CSprite.
```

```
class CPlayerBullet(CSprite):
```

```
    # Constructor.
```

```
    def __init__(self):
```

```
        CSprite.__init__(self)
```

```
        img = pygame.image.load("assets/images/player_bullet.png")
```

```
        img = img.convert_alpha()
```

```
        self.setImage(img)
```

```
...
```

```
...
```

```
# La clase CPlayer hereda de CSprite.
```

```
class CPlayer(CSprite):
```

```
    ...
```

```
    def __init__(self, aType):
```

```
        CSprite.__init__(self)
```

```
        # Segun el tipo de la nave, la imagen que se carga.
```

```

self.mType = aType
if self.mType == CPlayer.TYPE_PLAYER_1:
    imgFile = "assets/images/player00.png"
elif self.mType == CPlayer.TYPE_PLAYER_2:
    imgFile = "assets/images/player10.png"

img = pygame.image.load(imgFile)
img = img.convert_alpha()
self.setImage(img)

...

...

# La clase CNavе hereda de CSprite.
class CNavе(CSprite):

    ...

    def __init__(self, aType):
        CSprite.__init__(self)

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType
        if self.mType == CNavе.TYPE_PLATINUM:
            imgFile = "assets/images/grey_ufo.png"
        elif self.mType == CNavе.TYPE_GOLD:
            imgFile = "assets/images/yellow_ufo.png"
        elif self.mType == CNavе.TYPE_RED:
            imgFile = "assets/images/red_ufo.png"
        elif self.mType == CNavе.TYPE_GREEN:
            imgFile = "assets/images/green_ufo.png"
        elif self.mType == CNavе.TYPE_CYAN:
            imgFile = "assets/images/cyan_ufo.png"

        img = pygame.image.load(imgFile)
        img = img.convert_alpha()
        self.setImage(img)

    ...

```

Como ahora cada clase del juego es responsable de cargar la imagen y establecerla con `setImage()` en el sprite, puede ocurrir que nos olvidemos de establecerla. Entonces, en la clase `CSprite` ponemos un chequeo al dibujar por si nos olvidamos de establecer la imagen, para evitar que de error al querer dibujar una imagen que tiene el valor `None` (esto ocurre

cuando no se ha inicializado la variable). La función `render()` de `CSprite` queda así:

```
# Dibuja el objeto en la pantalla.  
# Parámetros: La superficie de la pantalla donde dibujar la imagen.  
def render(self, aScreen):  
  
    if (self.mImg != None):  
        aScreen.blit(self.mImg, (self.getX(), self.getY()))
```

De esta manera, si en una clase que hereda de `CSprite`, nos olvidamos de establecer una imagen, no dará error, aunque la imagen no se verá en la pantalla. Recordar entonces que al crear un sprite, hay que cargar la imagen y usar `setImage()` para establecer la imagen cargada.

Animando un Sprite

Con el código de `CSprite` que simplemente maneja una imagen y que se puede cambiar con la función `setImage()`, es simple escribir una clase por encima para que el sprite sea animado. Esta clase se llamará `CAnimatedSprite` y heredará de `CSprite`, la cual ya maneja el dibujo de una imagen y su colisión.

Veamos el ejemplo ubicado en la carpeta `capitulo_12\002_animacion_de_sprites`. En este ejemplo, las naves enemigas se encuentran animadas. Cuesta verlo al inicio, porque la animación es muy rápida. Veamos a continuación los pasos necesarios para crear nuestro primer sprite animado.

Cargando las Imágenes

Las naves enemigas (mira el código en la clase `CNave`), en lugar de utilizar una sola imagen estática como estábamos haciendo hasta ahora, vamos a usar una secuencia de imágenes. Mira en el proyecto la carpeta `assets\images`, y verás que hemos puesto los nueve cuadros de animación para cada una de las naves. Debes copiar todas estas imágenes a tu propio proyecto.

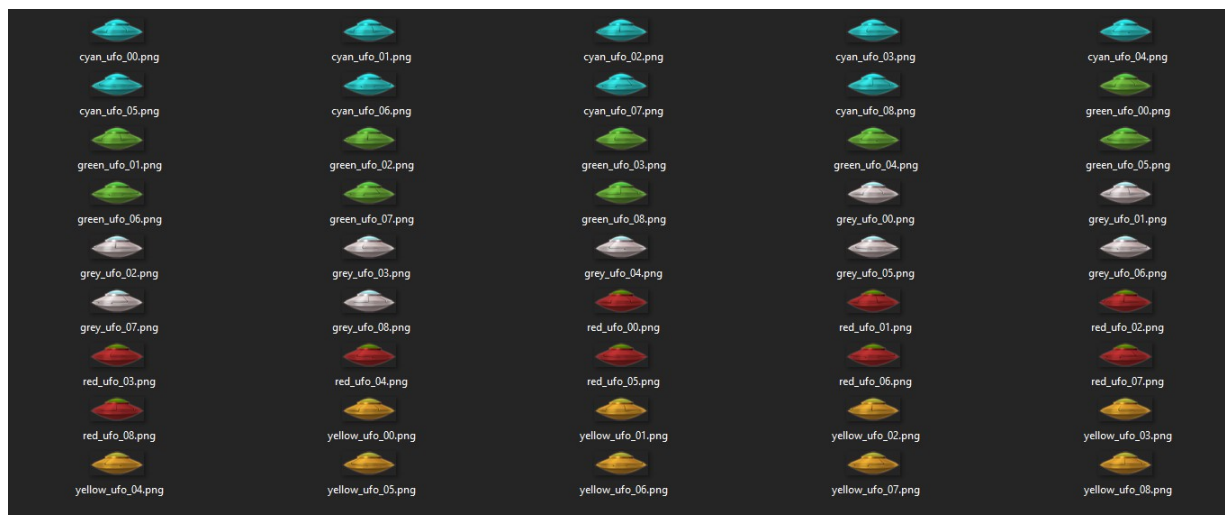


Figura 12-3: Las imágenes de los frames de las naves animadas.

Los archivos los hemos nombrado de la siguiente manera: `<color>_ufo_01` a `<color>_ufo_08`. Donde dice `<color>`, significa que en esa parte del nombre va el color correspondiente al tipo de nave. Ahora tenemos cinco tipos de naves enemigas y cada nave tiene nueve frames de animación.

Según el tipo de nave, ahora deberemos cargar la secuencia de imágenes correspondiente. Observa cómo se cargan las imágenes en el constructor de la clase `CNave`. La clase se lista a continuación.

```
# -*- coding: utf-8 -*-

# -----
# Clase CNave.
# Nave enemiga que se mueve por la pantalla chequeando los bordes.
#
# Autor: Fernando Sansberro - Batovi Games.
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
# -----

# Importar Pygame.
import pygame

# Importar la clase para sprites animados.
from api.CAnimatedSprite import *

# Importar las clases necesarias para disparar.
import random
from game.CEnemyBullet import *
from api.CGameConstants import *
from api.CEnemyManager import *
```

```

# La clase CNavé hereda de CAnimatedSprite.
class CNavé(CAnimatedSprite):

    # Tipos de naves.
    TYPE_PLATINUM = 0
    TYPE_GOLD = 1
    TYPE_RED = 2
    TYPE_GREEN = 3
    TYPE_CYAN = 4

    # -----
    # Constructor. Recibe el tipo de nave (TYPE_PLATINUM o TYPE_GOLD).
    # -----
    def __init__(self, aType):
        CAnimatedSprite.__init__(self)

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType

        if self.mType == CNavé.TYPE_PLATINUM:
            imgFile = "assets/images/grey_ufo_0"
        elif self.mType == CNavé.TYPE_GOLD:
            imgFile = "assets/images/yellow_ufo_0"
        elif self.mType == CNavé.TYPE_RED:
            imgFile = "assets/images/red_ufo_0"
        elif self.mType == CNavé.TYPE_CYAN:
            imgFile = "assets/images/cyan_ufo_0"
        elif self.mType == CNavé.TYPE_GREEN:
            imgFile = "assets/images/green_ufo_0"

        # Cargar la secuencia de imágenes.
        self.mFrames = []
        i = 0
        while (i <= 8):
            tmpImg = pygame.image.load(imgFile + str(i) + ".png")
            tmpImg = tmpImg.convert_alpha()
            self.mFrames.append(tmpImg)
            i = i + 1

        # Inicializar la animación.
        # Se pasa el array de imágenes y el primer frame.
        self.initAnimation(self.mFrames, 0)

    # Mover el objeto.
    def update(self):

        # Invocar update() de CAnimatedSprite.

```

```

CAnimatedSprite.update(self)

# Ver si la nave dispara.
if random.randrange(1, 500) == 1:
    b = CEnemyBullet()
    b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY() + self.getHeight())
    b.setVelX(0)
    b.setVelY(10)
    b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
    b.setBoundAction(CGameObject.DIE)
    CEnemyManager.inst().add(b)

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # Invocar render() de CAnimatedSprite.
    CAnimatedSprite.render(self, aScreen)

# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar destroy() de CAnimatedSprite.
    CAnimatedSprite.destroy(self)

# Eliminar todos los frames creados.
i = len(self.mFrame)
while i > 0:
    self.mFrame[i - 1] = None
    self.mFrame.pop(i - 1)
    i = i - 1
# Eliminar el array.
self.mFrames = None

```

La clase CNave ahora hereda de la clase CAnimatedSprite. En la función `init()`, cargamos una secuencia de imágenes que las guardaremos en un array llamado `self.mFrames`.

Al inicio, según el tipo de nave, se crea un string con la primera parte del nombre. Luego se crea el array que va a tener las imágenes de la animación. Se crea el array `self.mFrames` como un array vacío (determinado por los dos corchetes: `[]`).

Luego, se recorre desde cero hasta el último número de imagen que corresponda a la animación. En el caso de la nave gris, los nombres van desde "grey_ufo_00.png" a

"grey_ufo_08.png", siendo en total nueve cuadros para esta animación de la nave.

En cada paso del loop, se carga la imagen correspondiente al número de frame actual (la variable `i`) en una variable temporal `tmpImg` y como siempre, se convierte al formato de *Pygame*. Por último se agrega al array de imágenes usando la función `append()`.

Nota que para armar el nombre del archivo de imagen a cargar se usa una concatenación de strings, para colocar el número entre el nombre y la extensión del archivo (".png").

Al final, llamamos a la función `initAnimation()` de `CAnimatedSprite`, que lo que hace es establecer el array de frames, decir cual es el frame actual y cargar la primera imagen a mostrar.

A continuación veremos la clase `CAnimatedSprite`, pero para terminar con la clase `CNave`, es necesario notar que ahora en las funciones `update()`, `render()` y `destroy()`, se invocan a las respectivas funciones de `CAnimatedSprite` (antes era `CSprite` porque la clase `CNave` heredaba de `CSprite`, pero ahora hereda de `CAnimatedSprite`). Al final, en la función `destroy()`, se liberan de memoria las imágenes cargadas y se eliminará el array de frames.

Si repasamos la clase `CNave`, vemos que solamente carga las imágenes y luego invoca a `initAnimation()` (de la clase `CAnimatedSprite`). Luego, como `update()` y `render()` invocan a las respectivas funciones en `CAnimatedSprite`, vemos que el código de la animación se encuentra en `CAnimatedSprite`. Veamos esta clase a continuación.

Inicializando la Animación

La clase `CAnimatedSprite` contiene una referencia al array con las imágenes de la animación (`self.mFrame`) y un atributo `self.mCurrentFrame` que lleva control del cuadro actual que hay que dibujar. Veamos la clase `CAnimatedSprite` completa:

```
# -*- coding: utf-8 -*-

#-----
# Clase CAnimatedSprite.
# Sprite con animación. Hereda de Sprite.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

import pygame
```



```

# Importar la clase CSprite.
from api.CSprite import *

class CAnimatedSprite(CSprite):

    def __init__(self):

        CSprite.__init__(self)

        # Array con los frames de la animación (las imágenes).
        self.mFrame = None

        # Frame actual de la animación.
        self.mCurrentFrame = 0

        # Inicializa la animación del sprite. Establece el array de
        imágenes
        # de la animación y el frame actual.
        def initAnimation(self, aFramesArray, aStartFrame):
            self.mFrame = aFramesArray
            self.mCurrentFrame = aStartFrame
            self.setImage(self.mFrame[self.mCurrentFrame])

        def update(self):

            CSprite.update(self)

            # Actualizar en cuadro de animación.
            self.mCurrentFrame = self.mCurrentFrame + 1
            if self.mCurrentFrame >= len(self.mFrame):
                self.mCurrentFrame = 0
            self.setImage(self.mFrame[self.mCurrentFrame])

        def render(self, aScreen):

            CSprite.render(self, aScreen)

        def destroy(self):

            CSprite.destroy(self)

            i = len(self.mFrame)
            while i > 0:
                self.mFrame[i-1] = None
                self.mFrame.pop(i-1)
                i = i - 1

```

En el constructor de `CNave`, luego de cargar la secuencia de imágenes, debemos invocar a la función `initAnimation()`, pasándole como parámetro el array de imágenes y el frame inicial a mostrar. Si no hacemos esto, no aparecerá nada en la pantalla (recuerda que no dará error si no inicializamos la animación porque pusimos un control en la función `render()` de la clase `CSprite`, para que no intente dibujar una imagen que no ha sido inicializada, aunque no veremos nada en pantalla si nos olvidamos de inicializar la animación en la clase `CNave`, o en general en cualquier clase que herede de `CAnimatedSprite`).

La función `initAnimation()` guarda el array de imágenes de la animación y el frame actual, y llama a `setImage()` en `CSprite` para establecer la primera imagen a utilizar (recuerda que esta función establece el ancho y el alto de la imagen, que luego se usa en las colisiones).

Actualizando la Animación en `update()`

La función `update()` de `CAnimatedSprite` tiene código para actualizar el frame actual de la animación, incrementándose en uno. Cuando el frame actual llega a la cantidad de frames totales, se vuelve a cero para que la animación comience nuevamente desde el principio. Recuerda que si se accede a un array fuera de rango, aparecerá un error y no queremos que eso ocurra.

```
def update(self):

    CSprite.update(self)

    # Actualizar en cuadro de animación.
    self.mCurrentFrame = self.mCurrentFrame + 1
    if self.mCurrentFrame >= len(self.mFrame):
        self.mCurrentFrame = 0
    self.setImage(self.mFrame[self.mCurrentFrame])
```

Cuando se cambia de frame, se invoca a la función `setImage()` para cambiar de imagen. Más adelante haremos otros tipos de animaciones que no sean cíclicas como esta. Por ahora las animaciones serán todas cíclicas (denominada animación en *loop*).

Por último, en la función `render()`, en lugar de mostrar en pantalla siempre la misma imagen como se hacía con sprites estáticos, se muestra la imagen correspondiente al frame actual (esto va cambiando la imagen que se muestra en pantalla y avanzando en la secuencia de frames).

Al ejecutar el juego, vemos que la animación de la nave cambia de cuadro en cada frame, y por eso no luce del todo bien (los cuadros cambian muy rápido). Por ahora no estamos controlando el *frame rate* de la animación, y la animación ocurre muy rápido. A continuación

veremos cómo controlar la velocidad de la animación.

Controlando la Velocidad de la Animación del Sprite

En el ejemplo anterior, en cada frame del juego se está cambiando el cuadro de la animación. Como el juego corre a 60 frames por segundo, eso es muy rápido para que la mayoría de las animaciones se vean bien. Lo que haremos es establecer una demora (denominado *delay*) entre cuadro y cuadro de animación.

Veamos el ejemplo ubicado en la carpeta `capitulo_12\003_animacion_controlada`. En este ejemplo, agregamos a la clase `CAnimatedSprite` el código para tener una animación controlada. Si ejecutamos el juego vemos que las naves enemigas tienen más lenta la animación.

En la clase `CNave`, inicializamos la animación pasando un tercer parámetro que indica el *delay* de la animación. En este caso le pasamos 8, que indica que va a estar ocho cuadros con la misma imagen, antes de cambiar de cuadro. Veamos este cambio de la clase `CNave`:

```
...

# La clase CNave hereda de CAnimatedSprite.
class CNave(CAnimatedSprite):

    ...

    def __init__(self, aType):

        CAnimatedSprite.__init__(self)

        ...

        # Inicializar la animación.
        # Se pasa el array de imágenes y el primer frame.
        self.initAnimation(self.mFrames, 0, 8)

...
```

Lo que haremos entonces es enlentecer la animación. Para esto, en lugar de cambiar de cuadro en cada frame, lo haremos esperar que pasen una cierta cantidad de frames del juego (ocho en este caso) antes de cambiar de frame en la animación.

Para hacer esto, programaremos este comportamiento en la clase `CAnimatedSprite`. Veamos esta clase.

...

```
class CAnimatedSprite(CSprite):

    def __init__(self):

        CSprite.__init__(self)

        # Array con los frames de la animación (las imágenes).
        self.mFrame = None

        # Frame actual de la animación.
        self.mCurrentFrame = 0

        # Control de cuando pasar de frame.
        self.mTimeFrame = 0
        # Cuantos frames deben pasar antes de pasar de imagen.
        self.mDelay = 0

        # Inicializa la animación del sprite. Establece el array de
imágenes
        # de la animación, el frame actual y el delay de la animación.
        def initAnimation(self, aFramesArray, aStartFrame, aDelay):
            self.mFrame = aFramesArray
            self.mCurrentFrame = aStartFrame
            self.mTimeFrame = 0
            self.mDelay = aDelay
            self.setImage(self.mFrame[self.mCurrentFrame])

        def update(self):

            CSprite.update(self)

            # Ver si hay que cambiar de frame.
            self.mTimeFrame = self.mTimeFrame + 1
            if (self.mTimeFrame > self.mDelay):
                # Resetear el tiempo.
                self.mTimeFrame = 0

            # Actualizar en cuadro de animación.
            self.mCurrentFrame = self.mCurrentFrame + 1
            if self.mCurrentFrame >= len(self.mFrame):
                self.mCurrentFrame = 0
            self.setImage(self.mFrame[self.mCurrentFrame])

    ...
```

Se han agregado dos variables en la clase `CAnimatedSprite`. La variable `self.mTimeFrame` comienza en cero y lleva el control del tiempo (la cantidad de frames) que la animación se encuentra en el cuadro actual. La variable `self.mDelay` contiene la cantidad de tiempo (*frames*) que deben pasar antes de avanzar al siguiente cuadro de animación.

Como se ve en el código de `update()`, lo que hemos agregado es código que le suma uno a `self.mTimeFrame` y no cambia de cuadro hasta que este valor alcance el indicado por `self.mDelay`. Cuando esto ocurre, se avanza de frame y se vuelve a poner `self.mTimeFrame` en cero para comenzar a contar el delay desde cero para el siguiente cuadro.

Siendo que el juego está corriendo a 60 frames por segundo, los delays que podemos usar son: 0 = 60 FPS, 1 = 30 FPS, 2 = 20 FPS, 3 = 15 FPS, 4 = 12 FPS, ... 59 = 1 FPS.

Prueba a cambiar el parámetro del *delay* en la clase `CNave`, para ver cómo queda la velocidad de animación. Si ponemos 59 de delay, veremos que la animación cambia de a un cuadro por segundo.

Animando la Nave del Jugador

Lo que haremos ahora es animar a la nave del jugador. La animación será diferente a lo que hemos hecho con la nave enemiga, debido a que esta nave usará otro tipo de animación. La animación de la nave enemiga era cíclica porque es un plato volador girando (un *loop de animación*), mientras que la animación de la nave del jugador tiene un cuadro para cuando no se mueve, un cuadro para cuando se mueve a la derecha y un cuadro para cuando se mueve a la izquierda. En la siguiente imagen se muestran los cuadros de la nave del jugador.



Figura 12-4: Cuadros de la animación de la nave del jugador.

Veamos el ejemplo ubicado en la carpeta carpeta

capitulo_12\004_animacion_jugador. Como siempre, debes copiar las imágenes a utilizar, a tu propio proyecto.

En la función `init()` de la clase `CPlayer`, cargamos todas las imágenes de los cuadros de animación, de forma similar a como lo hicimos para la nave enemiga. Veamos el código:

```
...

# La clase CPlayer hereda de CSprite.
class CPlayer(CSprite):

    ...

    # -----
    # Constructor. Recibe el tipo de nave (TYPE_PLAYER_1 o
    TYPE_PLAYER_2).
    # -----
    def __init__(self, aType):
        CSprite.__init__(self)

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType

        if self.mType == CPlayer.TYPE_PLAYER_1:
            imgFile = "assets/images/player0"
        elif self.mType == CPlayer.TYPE_PLAYER_2:
            imgFile = "assets/images/player1"

        self.mFrames = []
        i = 0
        while (i <= 2):
            tmpImg = pygame.image.load(imgFile + str(i) + ".png")
            tmpImg = tmpImg.convert_alpha()
            self.mFrames.append(tmpImg)
            i = i + 1

        self.setImage(self.mFrames[0])

...
```

Como vemos en las imágenes, el índice *cero* del array tendrá la imagen correspondiente a la nave quieta, el índice *uno* corresponde a la nave moviéndose a la izquierda y el índice *dos* corresponde a la nave moviéndose a la derecha. Mira los nombres de las imágenes y el código para entender que esto es así.

Entonces, en la función `update()`, cuando la nave se mueve a la izquierda, se establece

como imagen a `self.mFrames[1]`, y cuando se mueve a la derecha se establece la imagen `self.mFrames[2]`. Cuando la nave se queda quieta, se establece la imagen `self.mFrames[0]`. El código de la función `update()` que implementa esta animación del jugador se muestra a continuación.

...

```
def update(self):

    # Obtener los controles.
    if self.mType == CPlayer.TYPE_PLAYER_1:
        left = CKeyboard.inst().leftPressed()
        right = CKeyboard.inst().rightPressed()
        fire = CKeyboard.inst().fire()
    else:
        left = CKeyboard.inst().APressed()
        right = CKeyboard.inst().DPressed()
        fire = CKeyboard.inst().fire2()

    # Mover la nave según las teclas.
    if (not left and not right):
        self.setVelX(0)
        self.setImage(self.mFrames[0])
    else:
        if left:
            self.setVelX(-4)
            self.setImage(self.mFrames[1])
        elif right:
            self.setVelX(4)
            self.setImage(self.mFrames[2])
    ...
```

Notemos que este tipo de animación es diferente al de la nave enemiga. En esta animación, en cada `update()` establecemos la imagen que corresponda a la dirección de movimiento del jugador, mientras que las naves enemigas tienen una animación continua.

También debemos notar que la clase `CPlayer` hereda de `CSprite`. Esto es así porque no tenemos una animación automática, sino que simplemente cambiamos la imagen a usar.

Nota: Un videojuego suele tener tanto *sprites* sin animaciones, como *sprites* con animaciones continuas y *sprites* con animaciones controladas de diversas formas. Una vez que hacemos algunas, el resto son parecidas. La primera vez será más trabajo implementarlas, pero luego es mucho más sencillo.

Optimizando la Memoria

Una cuestión importante a tener en cuenta es el manejo de memoria. Al principio cuando el juego es chico no hay problema, pero a medida que el juego crece, pueden aparecer problemas de falta de memoria si no tenemos cuidado con el uso de la memoria.

Por ejemplo, si observamos la clase `CNave`, vemos que al inicio se crea un array de imágenes, cargando las imágenes correspondientes a la animación de la nave. Esto está bien, pero, ¡esta carga de imágenes se hace para cada una de las naves del juego!

Esto hace que se desperdicie mucha memoria, dado que habrán varias naves que utilizarán los mismos gráficos, sin embargo, se están cargando las imágenes por cada nave. Lo mismo ocurre con las balas del jugador, con las balas enemigas, y en general con cualquier gráfico que se use en dos o más clases.

No es sencillo al principio solucionar esto, pero con el tiempo nos iremos dando cuenta de cuáles son las posibles optimizaciones y las iremos realizando.

Esto lo arreglaremos más adelante cuando implementemos un *manager de assets*. Esto es, una clase que se encargará de cargar todas las imágenes al inicio del juego (cuando se está mostrando una *pantalla de carga*). De esta forma, cada clase pedirá los gráficos necesarios para dibujar, pero estos gráficos se cargarán una única vez. Ningún gráfico del juego debe cargarse más de una vez. Lo mismo que hacemos con las imágenes se aplicará a todos los *assets* en general (imágenes, sonidos y fuentes).

Nota: Siempre es mejor implementar la funcionalidad de una parte determinada del juego antes de ponerse a optimizar. Con el tiempo se adquiere experiencia y ya se pueden prever ciertas situaciones, pero por ahora está bien programar sin pensar en la optimización y más adelante optimizar.

También recuerda que en la función `destroy()` debemos liberar de memoria todo lo que se ha creado en la función constructora. Por ejemplo, la clase `CPlayer` carga las imágenes en el array de frames, y la clase `destroy()` las elimina. Veamos esta función:

```
# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar destroy() de CSprite.
    CSprite.destroy(self)

    i = len(self.mFrames)
    while i > 0:
        self.mFrames[i - 1] = None
        self.mFrames.pop(i - 1)
        i = i - 1
    mFrames = None
```


Como vemos, se recorre el array `mFrames` de imágenes, poniendo en `None` esa imagen, eliminando el elemento del array y por último poniendo la referencia del array en `None`. Con esto queda todo eliminado.

De esta forma terminamos el capítulo de animación de sprites. Ahora ya podemos implementar sprites animados. Pero para poder crear sprites con muchas animaciones y comportamientos, debemos aprender a programar los estados de los objetos del juego. Esto lo veremos en el siguiente capítulo.

13. Máquinas de Estados de los Objetos

En este capítulo se introduce el concepto de *máquina de estados*, que viene a ser el conjunto de *estados* (*acciones*) posibles que puede tener un objeto en el juego (por ejemplo, saltando, cayendo, pegando, disparando, etc) y cómo se pasa de un estado a otro. Este es uno de los conceptos claves y más importantes a la hora de programar videojuegos.

Por lo general, la *máquina de estados* es un tema que se ignora en las carreras de informática o se ve en forma superficial. Para el desarrollo de videojuegos, el concepto de máquina de estados es uno de los secretos para desarrollar videojuegos de calidad, que junto al *game loop* y al uso de la matemática, hace posible programar cualquier tipo de juego.

Este capítulo muestra cómo se implementan los comportamientos de los objetos del juego, lo cual se hace siempre teniendo una máquina de estados. Comenzaremos a implementar varios comportamientos en el jugador.

¿Qué es una Máquina de Estados?

Una *máquina de estados finita* (*finite state machine*, o *FSM*) es una máquina abstracta (sin forma) que realiza o soluciona un tipo de problema determinado. Las máquinas de estados son un programa (código) que realiza una tarea sencilla. Las vemos en todas partes. Por ejemplo, desde algoritmos simples como los de un ascensor, un semáforo, máquinas expendedoras de bebidas, etc. a cosas con lógica más avanzada como los sistemas de inteligencia artificial.

Una máquina de estados tiene dos componentes fundamentales. Primero, contiene *estados*. La máquina tiene una cantidad limitada de estados en los que puede estar. Segundo, contiene *transiciones*. Cada estado tiene ciertas condiciones que cuando se cumplen hacen pasar a la máquina de un estado a otro. A este pasaje se le denomina *transición*.

La máquina de estados comienza en un estado determinado, al que se le llama el *estado inicial*. Luego, según las condiciones que se den, se puede pasar de un estado a otro, y así hasta que termine el juego, o exista un estado final donde el objeto se destruya al alcanzarlo (por ejemplo, el estado “morir”).

¿Cómo Luce una Máquina de Estados?

Una máquina de estados se puede dibujar con un *diagrama de estados y transiciones*, que puede ser en forma de tabla o en forma de diagrama. En este diagrama, tendremos la lista de los estados posibles que puede tener el objeto, y las transiciones (flechas) entre un estado y otro.

Como ejemplo práctico, lo que haremos ahora es mejorar el comportamiento de la nave del jugador. Para eso, lo primero que hacemos es definir qué queremos que haga la nave del jugador. Escribimos estas notas:

- Si el jugador colisiona con un enemigo o una bala enemiga, se muere.
- Luego de un segundo, la nave aparece de nuevo (es una nueva vida).

Como vemos, tenemos dos estados. Al primer estado le llamamos `NORMAL`, y es el estado en el cual el jugador se controla normalmente. Cuando lo toca una bala o un enemigo, se pasa al estado `DYING` (muriendo). En este estado, cuando pase más de un segundo, volverá a pasar al estado `NORMAL`. La máquina de estados en forma de tabla (o sea, los estados y las transiciones), es la siguiente:

Estado	Condición	Transición
NORMAL	Le pega una bala	Pasa a estado DYING
NORMAL	Le pega un enemigo	Pasa a estado DYING
DYING	Pasa más de un segundo	Pasa a estado NORMAL

Figura 13-1: La máquina de estados del jugador en forma de tabla.

Esta tabla, es más sencilla de ver si la dibujamos con forma de *diagrama*, en la cual cada estado es un círculo y cada transición es una flecha que va de un estado a otro. Encima de la transición se pone un texto con la condición que se debe cumplir para que se cambie de estado. Veamos como queda la tabla dibujada como un diagrama:

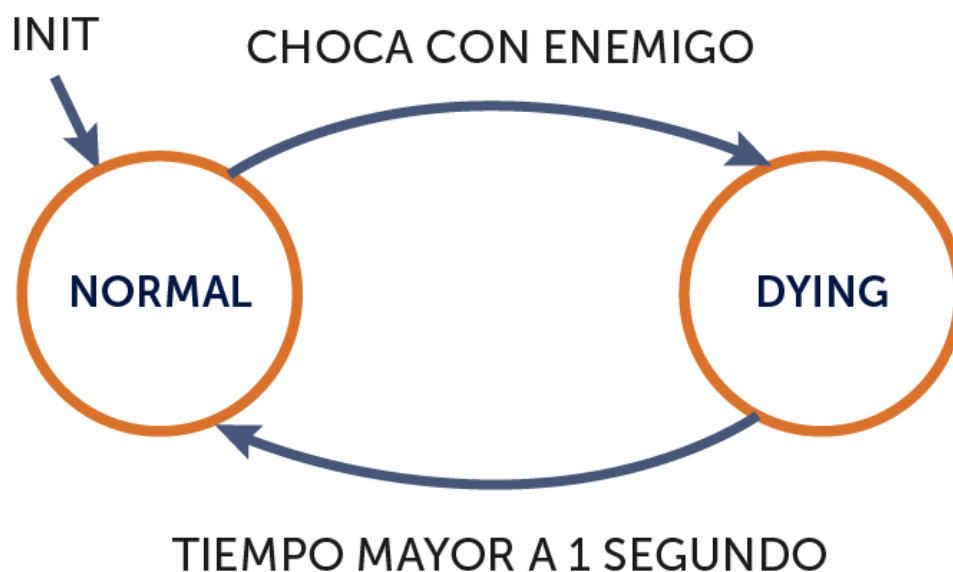


Figura 13-2: La máquina de estados del jugador en forma de diagrama.

Implementando los Estados NORMAL y DYING

Ahora vamos a implementar en la programación la máquina de estados que definimos para el jugador. La misma va a tener los estados `NORMAL` y `DYING` como muestra la figura 13-2.

La forma más común de implementar una máquina de estados es usando una sentencia `switch` (o varios `if` anidados).

Lo que haremos es que cuando la nave choque con un enemigo muera (cuando toque una bala enemiga o una nave enemiga). Mientras muere, el jugador va a parpadear y no se va a poder controlar durante un segundo. Luego de transcurrido ese tiempo la nave volverá a estar controlable (es cuando se usa una nueva vida).

Abre el ejemplo que se encuentra en la carpeta `capitulo_13\001_maquina_de_estados_jugador`.

La clase `CPlayer` ahora contiene una máquina de estados para implementar el comportamiento según el diseño que hemos realizado (ver el diagrama anterior).

El código de la clase `CPlayer` es el siguiente y se explicará a continuación.

```
# -*- coding: utf-8 -*-

#-----
# Clase CPlayer.
# Nave que controla el jugador.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# Importar la clase CSprite.
from api.CSprite import *

# Importar la clase CKeyboard
from api.CKeyboard import *

# Importar el manager de balas.
from api.CBulletManager import *

# Importar la clase para la bala.
from game.CPlayerBullet import *

# Importar la clase CGameConstants
```

```

from api.CGameConstants import *

# La clase CPlayer hereda de CSprite.
class CPlayer(CSprite):

    # Tipos de jugador.
    TYPE_PLAYER_1 = 0
    TYPE_PLAYER_2 = 1

    # Máquina de estados.
    NORMAL = 0
    DYING = 1

    # Tiempo que dura la muerte.
    TIME_DYING = 60

    # -----
    # Constructor. Recibe el tipo de nave (TYPE_PLAYER_1 o
TYPE_PLAYER_2).
    # -----
    def __init__(self, aType):
        CSprite.__init__(self)

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType

        if self.mType == CPlayer.TYPE_PLAYER_1:
            imgFile = "assets/images/player0"
        elif self.mType == CPlayer.TYPE_PLAYER_2:
            imgFile = "assets/images/player1"

        self.mFrames = []
        i = 0
        while (i <= 2):
            tmpImg = pygame.image.load(imgFile + str(i) + ".png")
            tmpImg = tmpImg.convert_alpha()
            self.mFrames.append(tmpImg)
            i = i + 1

        self.setImage(self.mFrames[0])

        # Estado inicial.
        self.mState = CPlayer.NORMAL
        self.setState(CPlayer.NORMAL)

        # Control del tiempo cuando se muere.
        self.mTimeDying = 0

```

```

        # Variable para hacer parpadear el sprite.
        self.mCounter = 0

# Mover el objeto.
def update(self):

    # Invocar update() de CSprite.
    CSprite.update(self)

    # Actualizar el contador para parpadear.
    self.mCounter = self.mCounter + 1

    # Lógica del estado normal.
    if self.mState == CPlayer.NORMAL:
        if CEnemyManager.inst().collides(self):
            self.setState(CPlayer.DYING)
            return
        self.move()

    # Lógica del estado muriendo.
    elif self.mState == CPlayer.DYING:
        self.mTimeDying = self.mTimeDying + 1
        if (self.mTimeDying > CPlayer.TIME_DYING):
            self.setState(CPlayer.NORMAL)
            return

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # Si es estado normal, dibujar normalmente.
    if self.mState == CPlayer.NORMAL:
        CSprite.render(self, aScreen)
    # Si es estado muriendo, dibujar cuadro por medio.
    elif self.mState == CPlayer.DYING:
        if self.mCounter % 2 == 0:
            CSprite.render(self, aScreen)

# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar destroy() de CSprite.
    CSprite.destroy(self)

    i = len(self.mFrames)
    while i > 0:
        self.mFrames[i - 1] = None

```

```

        self.mFrames.pop(i - 1)
        i = i - 1
    mFrames = None

# Establece el estado actual e inicializa
# las variables correspondientes al estado.
def setState(self, aState):

    self.mState = aState

    if self.mState == CPlayer.NORMAL:
        pass
    elif self.mState == CPlayer.DYING:
        self.mTimeDying = 0
        self.mCounter = 0
        self.stopMove()

# Movimiento de la nave.
def move(self):
    # Obtener los controles.
    if self.mType == CPlayer.TYPE_PLAYER_1:
        left = CKeyboard.inst().leftPressed()
        right = CKeyboard.inst().rightPressed()
        fire = CKeyboard.inst().fire()
    else:
        left = CKeyboard.inst().APressed()
        right = CKeyboard.inst().DPressed()
        fire = CKeyboard.inst().fire2()

    # Mover la nave según las teclas.
    if (not left and not right):
        self.setVelX(0)
        self.setImage(self.mFrames[0])
    else:
        if left:
            self.setVelX(-4)
            self.setImage(self.mFrames[1])
        elif right:
            self.setVelX(4)
            self.setImage(self.mFrames[2])

    # Disparar.
    if fire:
        print("FIRE")
        b = CPlayerBullet()
        b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY())
        b.setVelX(0)

```

```

        b.setVely(-10)
        b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
        b.setBoundAction(CGameObject.DIE)
        CBulletManager.inst().add(b)

```

Como vemos en el código, la máquina de estados consiste en definir primero una constante para cada estado e inicializar el estado actual. La variable `self.mState` tendrá el estado actual del objeto (el jugador en este caso).

...

```

# La clase CPlayer hereda de CSprite.
class CPlayer(CSprite):

```

...

```

# Máquina de estados.
NORMAL = 0
DYING = 1

```

```

# Tiempo que dura la muerte.
TIME_DYING = 60

```

...

```

def __init__(self, aType):
    CSprite.__init__(self)

```

...

```

# Estado inicial.
self.mState = CPlayer.NORMAL
self.setState(CPlayer.NORMAL)

```

```

# Control del tiempo cuando se muere.
self.mTimeDying = 0

```

```

# Variable para hacer parpadear el sprite.
self.mCounter = 0

```

...

Como siempre, en el constructor inicializamos todas las variables y constantes necesarias para el objeto, y usaremos la función `setState()` para poner al objeto en su estado inicial. Cada vez que cambiemos de estado, invocamos a la función `setState()` y le pasamos el

nuevo estado como parámetro. Esta función, establecerá los valores que haya que establecer para inicializar el estado. Por ejemplo, en el caso de pasar al estado `CPlayer.DYING`, la nave deja de moverse y se resetea el tiempo a contar antes de revivir. Este tipo de variables (denominados *timers*, *contadores*, etc.) siempre se inicializan cuando se pasa de un estado a otro.

```
# Establece el estado actual e inicializa
# las variables correspondientes al estado.
def setState(self, aState):

    self.mState = aState

    if self.mState == CPlayer.NORMAL:
        pass
    elif self.mState == CPlayer.DYING:
        self.mTimeDying = 0
        self.mCounter = 0
        self.stopMove()
```

Para detener completamente al objeto al morir, hemos hecho una función `stopMove()` en la clase `CGameObject` que lo que hace es poner la velocidad y la aceleración del objeto en cero, para que no se mueva más. Si no hacemos esto tendremos comportamientos no deseados, como por ejemplo, que el objeto si tenía una velocidad, al chocar con una bala, se seguirá moviendo mientras muere (porque en el estado en el que está muriendo no se controla con el teclado). La función `stopMove()` de `CGameObject` es la siguiente:

```
# Detiene el movimiento del objeto.
def stopMove(self):
    self.setVelX(0)
    self.setVelY(0)
    self.setAccelX(0)
    self.setAccelY(0)
```

Nota: Es fundamental inicializar bien los estados (inicializar correctamente las variables a ser usadas en el estado), para evitar tener comportamientos no deseados, producto de tener algunas variables no inicializadas, como el caso, por ejemplo, de una nave que explota pero se sigue moviendo.

Volviendo a la clase `CPlayer`, si vemos el listado mostrado anteriormente, veremos que todo el código que controla al jugador, ahora se encuentra en dentro de una función `move()`. Esto lo hemos hecho así, porque en algunos estados el jugador podrá controlarse con el teclado (por ejemplo en el estado `CPlayer.NORMAL`) y en otros estados no (cuando muere por ejemplo). Entonces, en los estados en los cuales se pueda mover la nave, se invoca a la

función `move()`. Como generalmente esto ocurre en varios estados, es poner una sola línea y no queda el código duplicado.

Entonces, la función `move()` es una función que controla la nave con el teclado y dispara si pulsamos el disparo. Es el mismo código que había antes, pero ahora se puso en una función, porque varios estados del jugador la van a utilizar (en resumen, se pone en una función para no repetir el código entre diferentes estados que lo usen).

Por último, la magia se encuentra en la función `update()`. Si miramos esta función, veremos que el código queda mucho más claro de entender qué es lo que hace, dado que es parecido a la tabla o diagrama que hicimos con la *máquina de estados*. En el código de `update()` se ven los estados y las condiciones en una estructura de `if`:

```
# Mover el objeto.
def update(self):

    # Invocar update() de CSprite.
    CSprite.update(self)

    # Actualizar el contador para parpadear.
    self.mCounter = self.mCounter + 1

    # Lógica del estado normal.
    if self.mState == CPlayer.NORMAL:
        if CEnemyManager.inst().collides(self):
            self.setState(CPlayer.DYING)
            return
        self.move()

    # Lógica del estado muriendo.
    elif self.mState == CPlayer.DYING:
        self.mTimeDying = self.mTimeDying + 1
        if (self.mTimeDying > CPlayer.TIME_DYING):
            self.setState(CPlayer.NORMAL)
            return
```

El contador (`self.mCounter`) es simplemente una variable que se incrementa de a uno y se utiliza en la función `render()` para hacer el parpadeo. Ya veremos eso en un momento.

La función `update()` se puede traducir al español de la siguiente manera:

- Si la nave está en estado normal:
 - Si la nave choca contra un enemigo, pasar al estado de morir.
 - Sino, mover la nave con el teclado.
- Si la nave está en estado muriendo:

- Incrementar la cuenta del tiempo.
- Si ya ha pasado más de un segundo, pasar al estado normal.

Como vemos, el código queda bastante más sencillo de entender y sobre todo, de modificar. El objeto (el jugador) hace una cosa u otra, y pasa a un estado u otro según determinadas condiciones. Es muy sencillo modificar este programa para que el jugador haga cosas más complejas. Más adelante agregaremos más estados en el jugador, lo que es lo mismo que decir, que le agregaremos más *comportamientos*.

Por último, como queremos que en el estado *muriendo* el jugador haga un parpadeo, en la función `render()` saltamos un cuadro (no dibujamos cuadro por medio, y eso lo hace parpadear). En el estado normal, la nave se dibuja siempre, y cuando está muriendo, se dibuja un cuadro sí y uno no:

```
# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # Si es estado normal, dibujar normalmente.
    if self.mState == CPlayer.NORMAL:
        CSprite.render(self, aScreen)
    # Si es estado muriendo, dibujar cuadro por medio.
    elif self.mState == CPlayer.DYING:
        if self.mCounter % 2 == 0:
            CSprite.render(self, aScreen)
```

Nota: Para evitar problemas durante el desarrollo del juego o para evitar *bugs*, es necesario asegurarse de inicializar bien las variables en cada estado y chequear bien las condiciones para las transiciones entre los estados.

Nota: Es buena práctica siempre usar `setState()` para cambiar de estado y las variables inicializarlas en esta función. Por otra parte asegurarse de no ejecutar código de lógica luego de invocar a `setState()`. Esto último se logra colocando una sentencia `return` luego de cada `setState()`.

Agregando más Estados al Jugador

Una vez que tenemos la máquina de estados implementada en el objeto, agregar estados es sencillo. Lo que tenemos que hacer es agregar las constantes correspondientes, inicializar esos estados en `setState()`, y programar las transiciones y los estados en la función `update()`.

Vamos a agregarle más estados al jugador, de forma tal de mejorar lo que hace al morir. Le

agregaremos un estado para cuando lo toca un enemigo y luego uno para explotar. Antes de programar, hacemos los estados completos del jugador en forma de tabla o diagrama. Mira la siguiente figura:

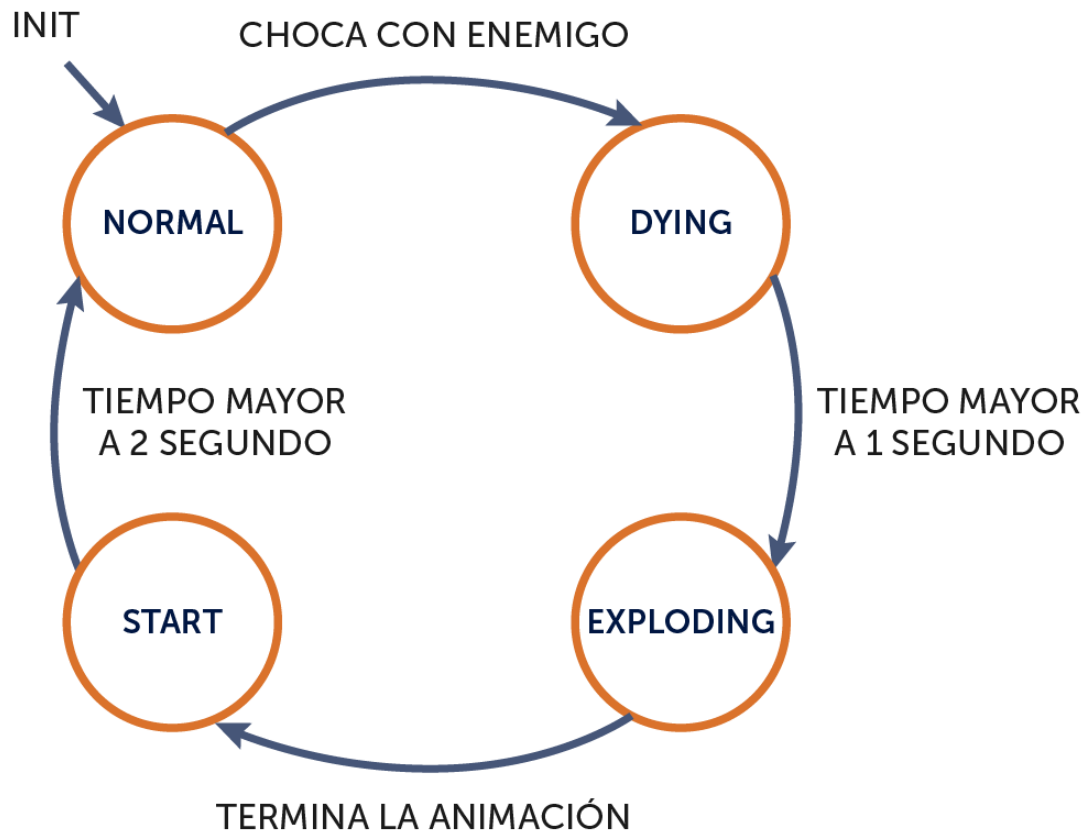


Figura 13-3: Máquina de estados completa del jugador.

El funcionamiento es el siguiente. La nave normal se puede controlar con el teclado. Cuando una bala o enemigo la toca, parpadea durante un segundo y luego explota. Luego que termina la explosión, la nave reaparece inmune por dos segundos. En este estado la nave se puede mover y disparar, pero no puede morir. Luego de dos segundos la nave vuelve a estar en estado normal.

Veamos el ejemplo que se encuentra en la carpeta `capitulo_13\002_maquina_de_estados_jugador_completa`.

En la clase `CPlayer`, hemos agregado constantes para los nuevos estados, y se ha puesto código para cada estado en las funciones `setState()`, `update()` y `render()`.

A continuación se muestra en **negrita** el código agregado o modificado:

...

```
# La clase CPlayer hereda de CSprite.
class CPlayer(CSprite):
```

```

# Tipos de jugador.
TYPE_PLAYER_1 = 0
TYPE_PLAYER_2 = 1

# Máquina de estados.
NORMAL = 0
DYING = 1
EXPLODING = 2
START = 3

# Tiempos que duran los estados.
TIME_DYING = 60
TIME_EXPLODING = 60
TIME_START = 120

# -----
# Constructor. Recibe el tipo de nave (TYPE_PLAYER_1 o
TYPE_PLAYER_2).
# -----
def __init__(self, aType):
    CSprite.__init__(self)

    # Segun el tipo de la nave, la imagen que se carga.
    self.mType = aType

    if self.mType == CPlayer.TYPE_PLAYER_1:
        imgFile = "assets/images/player0"
    elif self.mType == CPlayer.TYPE_PLAYER_2:
        imgFile = "assets/images/player1"

    self.mFrames = []
    i = 0
    while (i <= 2):
        tmpImg = pygame.image.load(imgFile + str(i) + ".png")
        tmpImg = tmpImg.convert_alpha()
        self.mFrames.append(tmpImg)
        i = i + 1

    self.setImage(self.mFrames[0])

    # Estado inicial.
    self.mState = CPlayer.NORMAL
    self.setState(CPlayer.NORMAL)

# Control del tiempo en el estado actual.
self.mTimeState = 0

```

```

    # Variable para hacer parpadear el sprite.
    self.mCounter = 0

# Mover el objeto.
def update(self):

    # Invocar update() de CSprite.
    CSprite.update(self)

    # Incrementar el tiempo del estado actual.
    self.mTimeState = self.mTimeState + 1

    # Actualizar el contador para parpadear.
    self.mCounter = self.mCounter + 1

    # Lógica del estado normal.
    if self.mState == CPlayer.NORMAL:
        if CEnemyManager.inst().collides(self):
            self.setState(CPlayer.DYING)
            return
        self.move()

    # Lógica del estado muriendo.
    elif self.mState == CPlayer.DYING:
        if (self.mTimeState > CPlayer.TIME_DYING):
            self.setState(CPlayer.EXPLODING)
            return

    # Lógica del estado explotando.
    elif self.mState == CPlayer.EXPLODING:
        if (self.mTimeState > CPlayer.TIME_EXPLODING):
            self.setState(CPlayer.START)
            return

    # Lógica del estado start.
    elif self.mState == CPlayer.START:
        if (self.mTimeState > CPlayer.TIME_START):
            self.setState(CPlayer.NORMAL)
            return
        self.move()

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # Si es estado normal, dibujar normalmente.
    if self.mState == CPlayer.NORMAL:

```

```

        CSprite.render(self, aScreen)
# Si es estado muriendo, dibujar cuadro por medio.
elif self.mState == CPlayer.DYING:
    if self.mCounter % 8 == 0:
        CSprite.render(self, aScreen)
# Si está explotando, por ahora no hacemos nada.
elif self.mState == CPlayer.EXPLODING:
    pass
# Si el estado es start, parpadear mas lento.
elif self.mState == CPlayer.START:
    if self.mCounter % 16 == 0:
        CSprite.render(self, aScreen)

# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar destroy() de CSprite.
    CSprite.destroy(self)

    i = len(self.mFrames)
    while i > 0:
        self.mFrames[i - 1] = None
        self.mFrames.pop(i - 1)
        i = i - 1
    mFrames = None

# Establece el estado actual e inicializa
# las variables correspondientes al estado.
def setState(self, aState):

    self.mState = aState

# Resetear el tiempo del estado actual.
self.mTimeState = 0

    if self.mState == CPlayer.NORMAL:
        pass
    elif self.mState == CPlayer.DYING:
        self.mCounter = 0
        self.stopMove()
elif self.mState == CPlayer.EXPLODING:
    pass
elif self.mState == CPlayer.START:
    self.mCounter = 0

...

```

Lo primero que hacemos es agregar una constante para cada estado.

Hemos puesto una variable `self.mTimeState` para no tener un contador diferente por estados y así, usamos esta misma variable para llevar el tiempo que se está en un estado determinado. Para esto, en `setState()` esta variable se pone en *cero* y en `update()` se incrementa. Entonces, de esta forma, en cualquier estado es posible saber cuánto tiempo ha pasado en el estado actual, y esto se usa para saber si hay que pasar a otro estado.

Para agregar estados, debemos incluirlos en los `if` de la función `update()` y programar su lógica. Para agregar estados a lo que ya hacía el jugador, debemos “enganchar” los estados según el *diagrama de estados*. En este caso, ahora la nave pasa de estado `CPlayer.DYING` a `CPlayer.EXPLODING` (antes desde `CPlayer.DYING` pasaba a estado `CPlayer.NORMAL` y ahora enganchamos a un nuevo estado: `CPlayer.EXPLODING`).

En el estado `CPlayer.EXPLODING`, mostraremos una animación de una explosión, cosa que no estamos haciendo ahora para no hacer dos cosas a la vez. A veces es mejor programar una cosa chica a la vez, asegurarse que funcione y luego pasar a la siguiente. En este caso, una vez que tenemos el estado listo, le agregaremos la explosión. Esto es lo que haremos a continuación, aunque primero debemos mejorar la clase `CAnimatedSprite` para soportar varios tipos de animaciones.

Mejorando la clase `AnimatedSprite`: `gotoAndPlay()` y `gotoAndStop()`

Si recordamos cómo estamos manejando la animación de la nave del jugador (sino observa el código de la clase `CPlayer`), vemos que si nos movemos a la izquierda (en la función `move()`), usamos la función `setImage()` para establecer la imagen de la nave inclinada a la izquierda. Lo mismo para la derecha y lo mismo cuando se sueltan las teclas. Siempre usamos la función `setImage()` para establecer la imagen de la nave según su movimiento.

Pues bien, ahora la clase `CPlayer` hereda de `CSprite`. Si queremos hacer que la nave explote, debemos hacer que `CPlayer` herede de `CAnimatedSprite`. Esto es, porque ahora pasará a ser un sprite animado, y tendrá varias animaciones diferentes.

El problema es, que si `CPlayer` es un sprite animado, no podemos establecer una imagen manualmente, sino una *animación*. Lo que tenemos que hacer entonces, es mejorar la clase `CAnimatedSprite` para que soporte tanto una secuencia de animación, como poder establecer un cuadro fijo (una imagen). Esto es lo que haremos a continuación.

Implementaremos varias funciones nuevas en la clase `CAnimatedSprite` que nos permita tener absoluto control de las animaciones de un sprite animado. Por ejemplo, tendremos una función `gotoAndStop()` que irá a un cuadro determinado de la animación y quedará fijo en ese cuadro (esto es igual a establecer una imagen), y la función `gotoAndPlay()` que irá a un cuadro y seguirá la animación a partir de ese cuadro. Luego tendremos otras funciones que ya veremos.

Nota: Los nombres `gotoAndStop()` y `gotoAndPlay()` vienen del lenguaje *Actionscript*, del software *Adobe Flash*, en donde dada una secuencia de frames de una animación, se tienen estas funciones para ir a un cuadro determinado y detenerse, o correr la animación desde un cuadro en particular, respectivamente. Este tipo de funciones hacen muy sencilla la programación de la lógica de los sprites animados.

Lo que haremos en la clase `CPlayer` es cargar una animación con los tres cuadros que tiene la nave (centro, izquierda, derecha) y usamos la función `gotoAndStop()` para indicarle que cuadro queremos mostrar. Más adelante agregaremos una explosión, y esta animación correrá desde el inicio, usando la función `gotoAndPlay()`.

Nota: Es bastante común que un jugador o un enemigo tenga un conjunto de animaciones y vayamos cambiando entre una y otra según lo que haga. Esto está naturalmente relacionado a la máquina de estados, dado que es muy probable que tengamos transiciones de un estado a otro cuando una animación termina.

Abre el ejemplo que se encuentra en la carpeta `capitulo_13\003_goto_and_stop_y_goto_and_play`.

En la clase `CAnimatedSprite`, hemos implementado varias funciones. En letra negrita podemos ver el código agregado:

...

```
class CAnimatedSprite(CSprite):

    def __init__(self):

        CSprite.__init__(self)

        # Array con los frames de la animación (las imágenes).
        self.mFrame = None

        # Frame actual de la animación.
        self.mCurrentFrame = 0

        # Control de cuando pasar de frame.
        self.mTimeFrame = 0
        # Cuantos frames deben pasar antes de pasar de imagen.
        self.mDelay = 0

        # Indica si la animación es cíclica (True) o no (False).
        self.mIsLoop = True

        # Indica si la animación no es cíclica y ha terminado.
        self.mEnded = False
```

```

    # Inicializa la animación del sprite. Establece el array de
imágenes
    # de la animación, el frame actual el delay de la animación y si
la
    # animación es cíclica (al terminar comienza desde el principio),
o no.
    def initAnimation(self, aFramesArray, aStartFrame, aDelay,
aIsLoop):
        self.mFrame = aFramesArray
        self.mCurrentFrame = aStartFrame
        self.mTimeFrame = 0
        self.mDelay = aDelay
        self.mIsLoop = aIsLoop
        self.mEnded = False
        self.setImage(self.mFrame[self.mCurrentFrame])

    def update(self):

        CSprite.update(self)

        # Ver si hay que cambiar de frame.
        self.mTimeFrame = self.mTimeFrame + 1
        if (self.mTimeFrame > self.mDelay):
            # Resetear el tiempo.
            self.mTimeFrame = 0

        # Si la animación no ha terminado, actualizar el cuadro de
animación.
        if not self.mEnded:
            # Si es loop, se comienza desde el inicio. Sino queda
            # en el último cuadro.
            self.mCurrentFrame = self.mCurrentFrame + 1
            if self.mCurrentFrame >= len(self.mFrame):
                # Si la animación es cíclica, comienza desde el
inicio.

                if self.mIsLoop:
                    self.mCurrentFrame = 0
                # Si la animación no es cíclica, queda parado en
el
                # último cuadro y se marca que la animación ha
terminado.

            else:
                self.mCurrentFrame = len(self.mFrame) - 1
                self.mEnded = True

        self.setImage(self.mFrame[self.mCurrentFrame])

    def render(self, aScreen):

```

```

        CSprite.render(self, aScreen)

def destroy(self):

    CSprite.destroy(self)

    i = len(self.mFrame)
    while i > 0:
        self.mFrame[i-1] = None
        self.mFrame.pop(i-1)
        i = i - 1

    # True si la animación no es cíclica y ya ha terminado. False en
    otro caso.
    def isEnded(self):
        return self.mEnded

    # Función para ir a un cuadro determinado de la animación
    # y quedarse en ese cuadro.
    def gotoAndStop(self, aFrame):
        if aFrame >= 0 and aFrame <= (len(self.mFrame) - 1):
            self.mCurrentFrame = aFrame
            self.setImage(self.mFrame[self.mCurrentFrame])
            self.mEnded = True

    # Función para ir a un cuadro determinado de la animación
    # y seguir con los cuadros siguientes.
    def gotoAndPlay(self, aFrame):
        if aFrame >= 0 and aFrame <= (len(self.mFrame) - 1):
            self.mCurrentFrame = aFrame
            self.setImage(self.mFrame[self.mCurrentFrame])
            self.mEnded = False
            self.mTimeFrame = 0

```

Lo primero que agregamos son dos variables de control: `self.mIsLoop` indicará si la animación es cíclica (si al terminar comienza desde el inicio nuevamente), o no, y `self.mIsEnded` indicará cuando la animación haya terminado, esto es, cuando haya alcanzado el último cuadro. Si la animación es cíclica, se entiende que la animación nunca termina.

Luego, la función `initAnimation()` ahora agrega un parámetro al final (`aIsLoop`), que indica si la animación es cíclica o no. Cada vez que se llama a la función `initAnimation()` se pone la variable `self.mIsEnded` en `False` para marcar que la animación no ha terminado.

La función `update()`, es quien controla la animación, como lo hacía antes, pero ahora

agregamos controles sobre la animación. Una vez que la animación ha terminado, ya no se cambia de cuadro (sentencia: `if not self.mIsEnded`).

Luego, al llegar al final de la animación (o sea, al llegar al último elemento del array de frames), si la animación es cíclica, se vuelve al inicio (al frame cero). Si la animación no es cíclica, entonces se ha terminado (porque ha llegado al final) y se establece la variable `self.mIsEnded` en `True`. Esto hace que ya no se anime más y quede fijo en ese cuadro.

```
# True si la animación no es cíclica y ya ha terminado. False en otro caso.
def isEnded(self):
    return self.mEnded
```

La función `isEnded()` nos dice si la animación ha terminado o no. Como vemos, solamente retorna el valor de la variable `self.mIsEnded`, dado que `update()` se encarga de establecer su valor como vimos antes. De esta forma, podemos correr una animación en un objeto del juego usando la función `initAnimation()` y luego cuando `isEnded()` sea `True`, podemos pasar a otro estado. Esto es lo que vamos a usar en el jugador cuando explota. Pasaremos a un estado *explotando* y cuando termine la animación pasaremos a otro estado.

Las funciones `gotoAndStop()` y `gotoAndPlay()` reciben el número de cuadro al que hay que ir en la animación, y se detiene en ese cuadro o continúa la animación, respectivamente.

```
# Función para ir a un cuadro determinado de la animación
# y quedarse en ese cuadro.
def gotoAndStop(self, aFrame):
    if aFrame >= 0 and aFrame <= (len(self.mFrame) - 1):
        self.mCurrentFrame = aFrame
        self.setImage(self.mFrame[self.mCurrentFrame])
        self.mEnded = True

# Función para ir a un cuadro determinado de la animación
# y seguir con los cuadros siguientes.
def gotoAndPlay(self, aFrame):
    if aFrame >= 0 and aFrame <= (len(self.mFrame) - 1):
        self.mCurrentFrame = aFrame
        self.setImage(self.mFrame[self.mCurrentFrame])
        self.mEnded = False
        self.mTimeFrame = 0
```

Ambas funciones al inicio controlan que no le pasemos un número de cuadro que esté fuera del array de frames (daría error si pasamos un número menor a cero o mayor a la cantidad de elementos del array menos uno).

En la función `gotoAndStop()` la variable `self.mIsEnded` se pone en `True` (como es un cuadro solo, se considera que la animación ha terminado), y en la función `gotoAndPlay()` se pone en `False` (porque la animación recién comienza).

Con esto, ya tenemos todo pronto para tener animaciones más evolucionadas. Ahora heredamos la clase `CPlayer` de `CAnimatedSprite`, inicializamos la animación y utilizamos `gotoAndStop()` para ir a un cuadro determinado cuando nos movemos (en la función `move()`). A continuación vemos en **negrita** los cambios realizados en la clase `CPlayer`:

```
...

# Importar Pygame.
import pygame

# Importar la clase CAnimatedSprite.
from api.CAnimatedSprite import *

...

# La clase CPlayer hereda de CAnimatedSprite.
class CPlayer(CAnimatedSprite):

    '''

    def __init__(self, aType):

        # Invocar al constructor de CAnimatedSprite.
        CAnimatedSprite.__init__(self)

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType

        if self.mType == CPlayer.TYPE_PLAYER_1:
            imgFile = "assets/images/player0"
        elif self.mType == CPlayer.TYPE_PLAYER_2:
            imgFile = "assets/images/player1"

        self.mFrames = []
        i = 0
        while (i <= 2):
            tmpImg = pygame.image.load(imgFile + str(i) + ".png")
            tmpImg = tmpImg.convert_alpha()
            self.mFrames.append(tmpImg)
            i = i + 1

        # Control del tiempo en el estado actual.
        self.mTimeState = 0
```

```

# Variable para hacer parpadear el sprite.
self.mCounter = 0

# Estado inicial.
self.mState = CPlayer.NORMAL
self.setState(CPlayer.NORMAL)

# Mover el objeto.
def update(self):

    # Invocar update() de CAnimatedSprite.
    CAnimatedSprite.update(self)

    ...

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # Si es estado normal, dibujar normalmente.
    if self.mState == CPlayer.NORMAL:
        CAnimatedSprite.render(self, aScreen)
    # Si es estado muriendo, dibujar cuadro por medio.
    elif self.mState == CPlayer.DYING:
        if self.mCounter % 8 == 0:
            CAnimatedSprite.render(self, aScreen)
    # Si está explotando, por ahora no hacemos nada.
    elif self.mState == CPlayer.EXPLODING:
        pass
    # Si el estado es start, parpadear mas lento.
    elif self.mState == CPlayer.START:
        if self.mCounter % 16 == 0:
            CAnimatedSprite.render(self, aScreen)

# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar destroy() de CAnimatedSprite.
    CAnimatedSprite.destroy(self)

    ...

# Establece el estado actual e inicializa
# las variables correspondientes al estado.
def setState(self, aState):

```

```

self.mState = aState

# Resetear el tiempo del estado actual.
self.mTimeState = 0

if self.mState == CPlayer.NORMAL:
    self.initAnimation(self.mFrames, 0, 0, False)
    self.gotoAndStop(0)

elif self.mState == CPlayer.DYING:
    self.mCounter = 0
    self.stopMove()
    self.initAnimation(self.mFrames, 0, 0, False)
    self.gotoAndStop(0)

elif self.mState == CPlayer.EXPLODING:
    pass

elif self.mState == CPlayer.START:
    self.mCounter = 0
    self.initAnimation(self.mFrames, 0, 0, False)
    self.gotoAndStop(0)

# Movimiento de la nave.
def move(self):
    # Obtener los controles.
    if self.mType == CPlayer.TYPE_PLAYER_1:
        left = CKeyboard.inst().leftPressed()
        right = CKeyboard.inst().rightPressed()
        fire = CKeyboard.inst().fire()
    else:
        left = CKeyboard.inst().APressed()
        right = CKeyboard.inst().DPressed()
        fire = CKeyboard.inst().fire2()

    # Mover la nave según las teclas.
    if (not left and not right):
        self.setVelX(0)
        self.gotoAndStop(0)
    else:
        if left:
            self.setVelX(-4)
            self.gotoAndStop(1)
        elif right:
            self.setVelX(4)
            self.gotoAndStop(2)

    ...

```

Hay un cambio conceptual importante en la función `setState()`. Como vemos en esta función, en cada estado ahora inicializamos la animación. En lugar de hacerlo en la función `init()` y luego asumir que ya está inicializada, es una mejor práctica de programación inicializar todas las variables cada vez que cambiamos de estado, reduciendo de esta forma la posibilidad de errores.

Al inicializar la animación le pasamos `False` como último parámetro, indicando que esta animación no es cíclica y usamos `gotoAndStop(0)` para ir al primer cuadro.

Nota: Si no llamamos a `initAnimation()` nos dará error en `CAnimatedSprite`, dado que esta clase utiliza un array de animaciones. Para evitar errores, es que en todos los estados asignamos el array invocando a `initAnimation()`.

En el estado `CPlayer.EXPLODING` no hacemos nada por ahora (ni inicializar la animación ni mostrarla en `render()`), pero ya tenemos todo pronto para agregar una animación cuando el jugador explota, eso es lo que haremos a continuación.

Por último, necesitamos también modificar la llamada a `initAnimation()` desde el constructor de `CNave`, dado que ahora le hemos agregado un parámetro para el loop. Como las naves enemigas tienen animación cíclica, le pasamos `True` como parámetro.

```
class CNave(CAnimatedSprite):

    ...

    def __init__(self, aType):
        CAnimatedSprite.__init__(self)

        ...

        # Inicializar la animación.
        # Se pasa el array de imágenes, el primer frame,
        # el delay de la animación y si es loop o no.
        self.initAnimation(self.mFrames, 0, 8, True)

    ...
```

Agregando la Animación de la Explosión al Morir el Jugador

Vamos ahora a implementar la explosión de la nave del jugador. Cuando el jugador muere, pasa al estado `CPlayer.EXPLODING`, en el cual se muestra la animación de la explosión.

Cuando la animación de la explosión termina, se pasará al estado `CPlayer.START`.

Colocamos en la carpeta de assets, las imágenes correspondientes a la explosión. Estas imágenes se nombran `explosion00.png`, `explosion01.png` hasta `explosion09.png`. La explosión contiene diez cuadros en total.



Figura 13-4: Cuadros de la animación de la explosión.

Mira el ejemplo que se encuentra en la carpeta `capitulo_13\004_explosion_jugador`. Ve a la carpeta `assets\images` y observa la secuencia de imágenes de la explosión. Como siempre, copia estos archivos a tu propio proyecto.

Si ejecutamos el juego, vemos que cuando una bala toca al jugador, éste explota. Para eso, hemos agregado en la clase `CPlayer` un array con la secuencia de imágenes que componen la animación de la explosión.

Veamos el código de la clase `CPlayer`, viendo solamente las partes del código agregadas que tienen que ver con el manejo de la explosión.

...

```
# La clase CPlayer hereda de CAnimatedSprite.
class CPlayer(CAnimatedSprite):

    ...

    def __init__(self, aType):

        # Invocar al constructor de CAnimatedSprite.
        CAnimatedSprite.__init__(self)

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType

        if self.mType == CPlayer.TYPE_PLAYER_1:
            imgFile = "assets/images/player0"
        elif self.mType == CPlayer.TYPE_PLAYER_2:
            imgFile = "assets/images/player1"

        self.mFrames = []
```

```

i = 0
while (i <= 2):
    tmpImg = pygame.image.load(imgFile + str(i) + ".png")
    tmpImg = tmpImg.convert_alpha()
    self.mFrames.append(tmpImg)
    i = i + 1

# Cargar la secuencia de imágenes de la explosión.
self.mFramesExplosion = []
i = 0
while (i <= 9):
    num = "0" + str(i)
    tmpImg = pygame.image.load("assets/images/explosion" + num
+ ".png")
    tmpImg = tmpImg.convert_alpha()
    self.mFramesExplosion.append(tmpImg)
    i = i + 1

# Control del tiempo en el estado actual.
self.mTimeState = 0

# Variable para hacer parpadear el sprite.
self.mCounter = 0

# Estado inicial.
self.mState = CPlayer.NORMAL
self.setState(CPlayer.NORMAL)

# Mover el objeto.
def update(self):

    ...

    # Lógica del estado explotando.
    elif self.mState == CPlayer.EXPLODING:
        if self.isEnded():
            self.setState(CPlayer.START)
            return

    ...

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    ...

```

```

        # Si está explotando, dibujar la explosión.
        elif self.mState == CPlayer.EXPLODING:
            CAnimatedSprite.render(self, aScreen)

        ...

# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar destroy() de CAnimatedSprite.
    CAnimatedSprite.destroy(self)

    i = len(self.mFrames)
    while i > 0:
        self.mFrames[i - 1] = None
        self.mFrames.pop(i - 1)
        i = i - 1
    mFrames = None

    i = len(self.mFramesExplosion)
    while i > 0:
        self.mFramesExplosion[i - 1] = None
        self.mFramesExplosion.pop(i - 1)
        i = i - 1
    self.mFramesExplosion = None

# Establece el estado actual e inicializa
# las variables correspondientes al estado.
def setState(self, aState):

    self.mState = aState

    # Resetear el tiempo del estado actual.
    self.mTimeState = 0

    if self.mState == CPlayer.NORMAL:
        self.initAnimation(self.mFrames, 0, 0, False)
        self.gotoAndStop(0)

    elif self.mState == CPlayer.DYING:
        self.mCounter = 0
        self.stopMove()
        self.initAnimation(self.mFrames, 0, 0, False)
        self.gotoAndStop(0)

    elif self.mState == CPlayer.EXPLODING:
        self.initAnimation(self.mFramesExplosion, 0, 2, False)

```

```

elif self.mState == CPlayer.START:
    self.mCounter = 0
    self.initAnimation(self.mFrames, 0, 0, False)
    self.gotoAndStop(0)

...

```

Hemos agregado en la función de inicialización la carga de las imágenes de las explosiones en un array llamado `self.mFramesExplosion`. Luego, en la función `setState()`, cuando se pasa al estado `CPlayer.EXPLODING`, se carga esa animación con la función `initAnimation()`. A esta función se le pasa el array de imágenes, *cero* para arrancar desde el primer cuadro, un delay de *dos* cuadros para que no se muestre tan rápido y `False` al final porque la animación no es cíclica.

En la función `render()` se agrega la línea en el estado `CPlayer.EXPLODING` para que muestre la animación y en la función `update()` se agrega la condición para terminar el estado, que es cuando termina la animación de la explosión.

Notar que para esto hemos usado la función `isEnded()` de `CAnimatedSprite`, que nos dice si la animación actual ha terminado o no. Cuando termina se pasa al estado `CPlayer.START` para que el jugador arranque nuevamente una vida.

Nota: En general, agregar un estado a un objeto del juego implica definir la constante para el estado, inicializar el estado en `setState()`, programar el comportamiento en `update()` y dibujar en `render()`.

Nota: No olvidar nunca al final destruir lo que se haya creado en memoria. En el código anterior, vemos que en la función `destroy()` eliminamos las imágenes y el array creado para la animación de la explosión.

Agregando Explosiones a los Enemigos

Lo que haremos ahora es similar a lo que hemos hecho con el jugador. Vamos a hacer que cuando una bala del jugador toque a la nave enemiga, la haga explotar. Para esto, debemos hacer una máquina de estados (nuestro enemigo aún no tiene máquina estados) con dos estados: `CNave.NORMAL` y `CNave.EXPLODING`. Luego de eso debemos cargar las imágenes de la animación de la explosión, y colocaremos como primer estado el estado `CNave.NORMAL`.

Como siempre, crear estados consiste en definir las constantes para cada estado, establecer un estado inicial invocando a la función `setState()`, inicializar el estado en `setState()`, y programar su comportamiento en la función `update()` y su dibujado en la función `render()`.

La máquina de estados que hemos definido para la nave enemiga tiene dos estados. Del

estado *normal* pasa al estado *explotando* cuando lo toca una bala. Luego que termina la animación de la explosión, el objeto se muere.

La máquina de estados de definida se muestra en la siguiente figura:



Figura 13-5: Máquina de estados de los enemigos.

Vamos el ejemplo que se encuentra en la carpeta `capitulo_13\005_explosion_enemigos`. Cuando acertamos un disparo, el enemigo muere con una explosión.

La explosión es la misma secuencia de imágenes que usamos para el jugador. Hemos usado los mismos assets (gráficos) temporalmente (*placeholders*). Más adelante los podemos cambiar.

Veamos el código de la clase `CNave` con los cambios que implementan los estados y la explosión.

...

```
# La clase CNave hereda de CAnimatedSprite.
```

```
class CNave(CAnimatedSprite):
```

```
    # Tipos de naves.
```

```
    TYPE_PLATINUM = 0
```

```
    TYPE_GOLD = 1
```

```
    TYPE_RED = 2
```

```
    TYPE_GREEN = 3
```

```

TYPE_CYAN = 4

# Máquina de estados.
NORMAL = 0
EXPLODING = 1

# -----
# Constructor. Recibe el tipo de nave (TYPE_PLATINUM o TYPE_GOLD).
# -----
def __init__(self, aType):
    CAnimatedSprite.__init__(self)

    ...

    # Cargar la secuencia de imágenes.
    self.mFramesNormal = []
    i = 0
    while (i <= 8):
        tmpImg = pygame.image.load(imgFile + str(i) + ".png")
        tmpImg = tmpImg.convert_alpha()
        self.mFramesNormal.append(tmpImg)
        i = i + 1

    # Cargar la secuencia de imágenes de la explosion.
    self.mFramesExplosion = []
    i = 0
    while (i <= 9):
        num = "0" + str(i)
        tmpImg = pygame.image.load("assets/images/explosion" + num
+ ".png")
        tmpImg = tmpImg.convert_alpha()
        self.mFramesExplosion.append(tmpImg)
        i = i + 1

    # Control del tiempo en el estado actual.
    self.mTimeState = 0

    # Estado inicial.
    self.mState = CNav.e.NORMAL
    self.setState(CNav.e.NORMAL)

# Mover el objeto.
def update(self):

    # Invocar update() de CAnimatedSprite.
    CAnimatedSprite.update(self)

    if self.mState == CNav.e.NORMAL:

```

```

        self.controlFire()

    elif self.mState == CNave.EXPLODING:
        if self.isEnded():
            self.die()
            return

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # Invocar render() de CAnimatedSprite.
    CAnimatedSprite.render(self, aScreen)

# Liberar lo que haya creado el objeto.
def destroy(self):

    # Invocar destroy() de CAnimatedSprite.
    CAnimatedSprite.destroy(self)

    # Eliminar todos los frames creados.
    i = len(self.mFramesNormal)
    while i > 0:
        self.mFramesNormal[i - 1] = None
        self.mFramesNormal.pop(i - 1)
        i = i - 1
    # Eliminar el array.
    self.mFramesNormal = None

    i = len(self.mFramesExplosion)
    while i > 0:
        self.mFramesExplosion[i - 1] = None
        self.mFramesExplosion.pop(i - 1)
        i = i - 1
    self.mFramesExplosion = None

def setState(self, aState):
    self.mState = aState
    self.mTimeState = 0

    if self.mState == CNave.NORMAL:
        self.initAnimation(self.mFramesNormal, 0, 0, True)
    elif self.mState == CNave.EXPLODING:
        self.initAnimation(self.mFramesExplosion, 0, 0, False)
        self.stopMove()

# Invocada desde Bullet cuando la Nave es alcanzada por una bala.

```

```

def hit(self):
    self.setState(CNave.EXPLODING)
    print("NAVE EXPLOTA")

def controlFire(self):
    # Ver si la nave dispara.
    if random.randrange(1, 500) == 1:
        b = CEnemyBullet()
        b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY() + self.getHeight())
        b.setVelX(0)
        b.setVelY(10)
        b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
        b.setBoundAction(CGameObject.DIE)
        CEnemyManager.inst().add(b)

```

Si examinamos el código de la clase `CNave`, vemos que el código para la máquina de estados no es muy diferente a lo que escribimos en la clase `CPlayer`. Observa bien el código para intentar seguir su lógica. Lo que se hace para agregar estados es lo de siempre: se definen las constantes de los estados, se cargan las animaciones en el constructor, se programa la inicialización de los estados en la función `setState()`, se programa el comportamiento en la función `update()`, se dibuja en `render()`, y finalmente se libera toda la memoria creada en la función `destroy()`.

Cuando la nave es alcanzada por una bala, se invoca a la función `hit()` en la nave. La función `hit()` simplemente pasa al estado `CNave.EXPLODING`. Si vemos en la función `update()`, al terminar la animación de la explosión, se mata el objeto (se destruye) invocando a `self.die()`. Recuerda que la función `die()` prende una variable indicando que está marcado para morir, lo que hace que el manager de enemigos lo destruya. Si no recuerdas esto, examina el código de `CManager`.

A continuación se muestra en la clase `CBullet` (la bala del jugador) cómo se invoca a la función `hit()` en el enemigo cuando éste es alcanzado. Antes se lo mataba con `enemy.die()` y ahora se cambió por `enemy.hit()` para hacer que el enemigo explote (que pase por el estado `CNave.EXPLODING`), antes de eliminarse del todo.

...

```

# La clase CPlayerBullet hereda de CSprite.
class CPlayerBullet(CSprite):

```

...

```

    # Mover el objeto.

```



```

def update(self):

    CSprite.update(self)

    # Detectar choque contra los enemigos.
    # Si la bala choca un enemigo, mueren ambos.
    enemy = CEnemyManager.inst().collides(self)
    if enemy != None:
        print("*****COLISION ENTRE BALA DEL JUGADOR Y ENEMIGO")
        enemy.hit()
        self.die()

...

```

Entonces, es la función `update()` de la clase `CPlayerBullet`, cuando la bala choca con un enemigo, ya no va a invocar a `die()` porque sino el enemigo se mata instantáneamente. Lo que haremos ahora es invocar a la función `hit()` para informarle que le pegó una bala. Esta función `hit()` entonces pasa la nave al estado `CNave.EXPLODING`, que mostrará la explosión. Luego que la explosión termine, se invoca a la función `die()`.

Nota: Como tanto las naves enemigas como las balas enemigas están en el mismo manager de enemigos, cuando una bala del jugador les pega, se les invoca la función `hit()`. Por lo tanto, debemos definir esta función en la clase `CEnemyBullet`, de lo contrario dará error si una bala del jugador choca contra una bala enemiga.

```

...

class CEnemyBullet(CSprite):

    ...

    # Invocada desde Bullet cuando la bala enemiga es alcanzada por
    # una bala del jugador.
    def hit(self):
        self.die()
        print("COLISION BALA CONTRA BALA")

```

Nota: En este ejemplo hemos inicializado la explosión de los enemigos con *cero* de delay (la velocidad de la animación), así explota muy rápido y queda más lindo. De lo contrario la explosión permanece mucho tiempo en el mismo lugar mientras las otras naves le pasan por encima y eso no queda del todo lindo. Detalles como estos abundan al desarrollar videojuegos, y llevan bastante tiempo (a esto se le llama *pulir* o *polishing*). ¡Hay que tenerlo en cuenta!

Ignorando las Colisiones de las Balas contra las Explosiones

En el ejemplo anterior, si disparamos varias balas seguidas y le acertamos a un enemigo, vemos que el enemigo explota, pero las balas siguen chocando con la explosión y la hace explotar nuevamente (prueba dispararle muy rápido al mismo enemigo). Esto es un error muy común al desarrollar juegos, y se debe a que cada vez que choca una bala con un enemigo, se le llama a la función `hit()` de este último. Pero si el enemigo ya está en estado *explotando*, también se le llama a la función `hit()`, que lo vuelve a poner en el estado *explotando* y por lo tanto comienza la explosión nuevamente.

Esto se arregla fácil, poniendo en la clase `CNave`, un control en la función `hit()`, haciendo que explote solo si está en estado *normal*. De esta forma, si ya estaba explotando no lo vuelve a hacer.

```
# Invocada desde Bullet cuando la Nave es alcanzada por una bala.
def hit(self):
    if self.mState == CNave.NORMAL:
        self.setState(CNave.EXPLODING)
        print("NAVE EXPLOTA")
```

Esto soluciona el hecho de que la nave vuelve a explotar, pero aún falta hacer que la bala del jugador no se muera al chocar contra una explosión. Entonces, un control similar hay que hacer desde la bala del jugador, para no invocar a `die()` que mata la bala, si la colisión es con una explosión (o sea, colisiona con el enemigo pero éste está explotando, por lo cual la bala debe seguir de largo, y no morir contra en la explosión).

En la función `update()` de la bala del jugador (clase `CPlayerBullet`), chequeamos que la colisión sea contra el enemigo en estado *normal* para que sea una colisión válida y matar la bala. Sino, se ignora la colisión y la bala sigue de largo.

...

```
class CPlayerBullet(CSprite):

    ...

    # Mover el objeto.
    def update(self):

        CSprite.update(self)

        # Detectar choque contra los enemigos.
        # Si la bala choca un enemigo, mueren ambos.
        enemy = CEnemyManager.inst().collides(self)
```

```

        if enemy != None:
            if enemy.isStateNormal():
                print("*****COLISION ENTRE BALA DEL JUGADOR Y
ENEMIGO")
                enemy.hit()
                self.die()

...

```

En la clase `CNave`, implementamos la función `isStateNormal()` que retorna `True` si la nave se encuentra en estado normal:

```

# Retorna True si la nave está en estado normal.
def isStateNormal(self):
    return self.mState == Nave.NORMAL

```

Nota: Dado que las balas enemigas y las naves se encuentran en el mismo manager de enemigos, también tenemos que implementar esta función en esa clase. En este caso la función retorna solamente `False` para que las balas del jugador y las balas enemigas no choquen entre sí. Se agrega entonces esta función también en la clase `CEnemyBullet`:

```

# La bala enemigas no chocan contra las balas del jugador.
def isStateNormal(self):
    return False

```

Ejecuta el ejemplo que se encuentra en la carpeta `capitulo_13\006_balas_ignoran_explosiones` y mira el código para ver implementados estos últimos cambios. Dispara muy seguido para ver que ahora las balas del jugador se comportan correctamente y no hacen reiniciar las explosiones ni se mueren las balas contra las explosiones.

Nota: Hemos hecho que las balas del jugador no choquen contra las balas enemigas, pero esto es una cuestión de gustos. No tiene porqué ser así. Tu puedes modificar lo que desees para cambiar el juego y hacerlo diferente, a tu gusto.

Reorganizando el Código de `CSprite` y `CGameObject` para Reutilizar

Para terminar este capítulo, haremos una breve *reestructura* (a esto se le denomina *refactor*) del código para evitar partes duplicadas. Como siempre, hacer esto siempre nos permite tener objetos más reusables y más fáciles de manejar.

Si vemos el código de `CPlayer` y `CNave`, tenemos cosas duplicadas (algunas variables y funciones). Por ejemplo, la variable `self.mState`, dado que ambas clases implementan una máquina de estado, es candidata a ser colocada en una clase base (la pondremos en `CGameObject`), para que no esté duplicada en cada clase. Recuerda que lo que se ponga en una clase base (variables y funciones), estará disponible para todas las clases que la heredan.

Otra cosa que pondremos en la clase `CGameObject` es el control del tiempo que el objeto está en el estado actual (variable `self.mTimeState`). Esto ahora lo tenemos en las dos clases (`CPlayer` y `CNave`) y lo usamos para pasar de un estado a otro cuando pasa cierto tiempo. Si este comportamiento lo quisiéramos en cualquier otra clase, deberíamos duplicarlo. Por este motivo es que la lógica de control de tiempo de los estados también la pondremos en `CGameObject`.

Recuerda que el código que lleva la cuenta del tiempo del estado actual funciona así: en la función `setState()` se inicializa la variable `self.mTimeState` en *cero*, y en la función `update()` se incrementa. Todo esto ahora lo colocaremos en `CGameObject`.

Abre el ejemplo que se encuentra en la carpeta `capitulo_13\007_reorganizando_el_codigo` y mira la clase `CGameObject` para ver estos cambios que se han explicado arriba. Ahora cualquier *game object* soporta tener una máquina de estados. Miremos los cambios en la clase `CGameObject`:

...

```
class CGameObject(object):

    ...

    def __init__(self):

        ...

        # Indica si el objeto está vivo o muerto (debe morir).
        self.mIsDead = False

        # Estado actual.
        self.mState = 0

        # Control del tiempo en el estado actual.
        self.mTimeState = 0

    ...

    # Update mueve el objeto según su velocidad.
```

```

def update(self):

    # Incrementar el tiempo del estado actual.
    self.mTimeState = self.mTimeState + 1

    # Modificar la velocidad según la aceleración.
    self.mVelX += self.mAccelX
    self.mVelY += self.mAccelY

    # Mover el objeto.
    self.mX += self.mVelX
    self.mY += self.mVelY

    # Comportamiento con el borde.
    self.checkBounds()

    ...

# Retorna el estado actual.
def getState(self):
    return self.mState

# Establece el estado actual.
def setState(self, aState):
    self.mState = aState
    # Resetear el tiempo del estado actual.
    self.mTimeState = 0

# Retorna el tiempo en el estado actual.
def getTimeState(self):
    return self.mTimeState

```

Como vemos en el código de `CGameObject`, hemos agregado las funciones `getState()` para retornar el estado actual, `setState()` que establece el nuevo estado actual y resetea la cuenta del tiempo y la función `getTimeState()` que retorna el tiempo (en *frames*) que el objeto está en el estado actual.

En la función `update()` se incrementa la variable de control del tiempo del estado actual y cada vez que se cambia de estado, se resetea la cuenta (*resetear* una variable significa ponerla en su valor inicial, generalmente *cero*).

En las clases `CPlayer` y `CNave`, se elimina la variable `self.mState` y se cambia por la función `self.getState()`. También, la función `setState()` de la clase `CPlayer` y `CNave`, ahora invocan a la función `setState()` de la clase base. Esto es lo mismo que ocurre con las funciones `render()` y `update()` que invocan a sus respectivas funciones de la clase base. Siempre que se tenga una función en la clase base y se usen en la clase superior, se deben invocar.

Veamos los cambios de CPlayer:

...

La clase CPlayer hereda de CAnimatedSprite.

class CPlayer(CAnimatedSprite):

...

Constructor. Recibe el tipo de nave (TYPE_PLAYER_1 o
TYPE_PLAYER_2).

def __init__(self, aType):

Invocar al constructor de CAnimatedSprite.
CAnimatedSprite.__init__(self)

...

Variable para hacer parpadear el sprite.
self.mCounter = 0

Estado inicial.

self.setState(CPlayer.NORMAL)

Mover el objeto.

def update(self):

Invocar update() de CAnimatedSprite.
CAnimatedSprite.update(self)

Actualizar el contador para parpadear.
self.mCounter = self.mCounter + 1

Lógica del estado normal.

if **self.getState()** == CPlayer.NORMAL:
 if CEnemyManager.inst().collides(self):
 self.setState(CPlayer.DYING)
 return
 self.move()

Lógica del estado muriendo.

elif **self.getState()** == CPlayer.DYING:
 if (**self.getTimeState()** > CPlayer.TIME_DYING):
 self.setState(CPlayer.EXPLODING)

```

        return

# Lógica del estado explotando.
elif self.getState() == CPlayer.EXPLODING:
    if self.isEnded():
        self.setState(CPlayer.START)
        return

# Lógica del estado start.
elif self.getState() == CPlayer.START:
    if (self.getTimeState() > CPlayer.TIME_START):
        self.setState(CPlayer.NORMAL)
        return
    self.move()

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # Si es estado normal, dibujar normalmente.
    if self.getState() == CPlayer.NORMAL:
        CAnimatedSprite.render(self, aScreen)
    # Si es estado muriendo, dibujar cuadro por medio.
    elif self.getState() == CPlayer.DYING:
        if self.mCounter % 8 == 0:
            CAnimatedSprite.render(self, aScreen)
    # Si está explotando, dibujar la explosión.
    elif self.getState() == CPlayer.EXPLODING:
        CAnimatedSprite.render(self, aScreen)
    # Si el estado es start, parpadear mas lento.
    elif self.getState() == CPlayer.START:
        if self.mCounter % 16 == 0:
            CAnimatedSprite.render(self, aScreen)

...

# Establece el estado actual e inicializa
# las variables correspondientes al estado.
def setState(self, aState):
    CAnimatedSprite.setState(self, aState)

    if self.getState() == CPlayer.NORMAL:
        self.initAnimation(self.mFrames, 0, 0, False)
        self.gotoAndStop(0)

    elif self.getState() == CPlayer.DYING:
        self.mCounter = 0

```

```

        self.stopMove()
        self.initAnimation(self.mFrames, 0, 0, False)
        self.gotoAndStop(0)

    elif self.getState() == CPlayer.EXPLODING:
        self.initAnimation(self.mFramesExplosion, 0, 2, False)

    elif self.getState() == CPlayer.START:
        self.mCounter = 0
        self.initAnimation(self.mFrames, 0, 0, False)
        self.gotoAndStop(0)

    ...

```

Los cambios en CNavé son similares:

```

...

# La clase CNavé hereda de CAnimatedSprite.
class CNavé(CAnimatedSprite):

    ...

    def __init__(self, aType):
        CAnimatedSprite.__init__(self)

    ...

    # Estado inicial.
    self.setState(CNavé.NORMAL)

    # Mover el objeto.
    def update(self):

        # Invocar update() de CAnimatedSprite.
        CAnimatedSprite.update(self)

        if self.getState() == CNavé.NORMAL:
            self.controlFire()

        elif self.getState() == CNavé.EXPLODING:
            if self.isEnded():
                self.die()
                return

    ...

```



```

def setState(self, aState):
    CAnimatedSprite.setState(self, aState)

    if self.getState() == CNavе.NORMAL:
        self.initAnimation(self.mFramesNormal, 0, 0, True)
    elif self.getState() == CNavе.EXPLODING:
        self.initAnimation(self.mFramesExplosion, 0, 0, False)
        self.stopMove()

# Invocada desde Bullet cuando la Nave es alcanzada por una bala.
def hit(self):
    if self.getState() == CNavе.NORMAL:
        self.setState(CNavе.EXPLODING)
        print("NAVE EXPLOTA")

def controlFire(self):
    # Ver si la nave dispara.
    if random.randrange(1, 500) == 1:
        b = CEnemyBullet()
        b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY() + self.getHeight())
        b.setVelX(0)
        b.setVelY(10)
        b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
        b.setBoundAction(CGameObject.DIE)
        CEnemyManager.inst().add(b)

# Retorna True si la nave está en estado normal.
def isStateNormal(self):
    return self.getState() == CNavе.NORMAL

```

A esta reorganización del código que hemos hecho, le agregaremos uno relacionado al parpadeo de los sprites, y con esto terminaremos este capítulo.

Haciendo Parpadear un Sprite (Visible / Invisible)

Cuando el jugador es alcanzado por una bala enemiga, parpadea rápido antes de explotar y luego al volver a la vida, parpadea más lento mientras es inmune.

Este parpadeo (en inglés se denomina *flicker*) es algo que puede ser muy común en los juegos. Sin embargo, ahora lo tenemos programado solamente en la clase `CPlayer`. Para hacer parpadear el sprite, lo que estamos haciendo es saltar su dibujado cada cierta cantidad de frames. Lo que haremos ahora, es mejorar cómo está hecho esto, de forma tal que pueda ser reusado por todos los objetos del juego.

Vamos a poner una función para controlar el parpadeo del sprite. La pondremos en la clase `CSprite` y no en `CGameObject` porque el parpadeo está asociado a mostrar o no una imagen (un *sprite*), y `CGameObject` no tiene imagen (es un objeto *abstracto*).

Haremos en la clase `CSprite` una función `setVisible()` que recibe `True` como parámetro si el sprite se muestra en pantalla y `False` si queremos que no se dibuje. Utilizaremos esto para hacer el parpadeo.

Abre el ejemplo que se encuentra en la carpeta `capitulo_13\008_parpadeo_de_sprite` y mira la clase `CSprite`, cuyo código se muestra a continuación:

```
...

class CSprite(CGameObject):

    # Constructor:
    def __init__(self):

        CGameObject.__init__(self)

        # Imagen (superficie) del sprite.
        # Usar setImage() en la clase superior para establecer una
imagen.
        self.mImg = None

        # Ancho y alto de la imagen. La función setImage() los
actualiza.
        self.mWidth = 0
        self.mHeight = 0

        # Indica si el sprite es visible o no.
self.mVisible = True

    ...

    # Dibuja el objeto en la pantalla.
    # Parámetros: La superficie de la pantalla donde dibujar la
imagen.
    def render(self, aScreen):

        if (self.mImg != None):
            if self.mVisible:
                aScreen.blit(self.mImg, (self.getX(), self.getY()))

    ...
```

```

# Establece si el sprite es visible o no.
# Parámetro: True para que se dibuje, False para que no se dibuje.
def setVisible(self, aVisible):
    self.mVisible = aVisible

# Retorna True si el sprite es visible y False si no lo es.
def isVisible(self):
    return self.mVisible

```

En la clase `CSprite`, hemos agregado una variable `self.mVisible` que indicará si el sprite está visible o no. Esta variable se usa en la función `render()` para determinar si se dibuja o no el sprite. Luego, tenemos dos funciones, `setVisible()` y `isVisible()` para establecer y obtener la visibilidad del sprite respectivamente.

En la clase `CPlayer`, quitamos la variable `self.mCounter` que usábamos para hacer parpadear el sprite, y ahora usamos la variable que lleva el tiempo en el estado actual, dado que solamente es un número que crece y es lo que necesitamos. No hay necesidad de tener varias variables para lo mismo.

Ahora, en la clase `CPlayer`, simplemente llamamos a `setVisible(True)` o `setVisible(False)` según el contador, para hacer parpadear al sprite en los estados `CPlayer.DYING` y `CPlayer.START`. Miremos la función `render()` de la clase `CPlayer`:

```

def render(self, aScreen):

    # Poner visible o invisible según el caso.
    if self.getState() == CPlayer.DYING:
        if self.getTimeState() % 8 == 0:
            self.setVisible(True)
        else:
            self.setVisible(False)
    elif self.getState() == CPlayer.START:
        if self.getTimeState() % 16 == 0:
            self.setVisible(True)
        else:
            self.setVisible(False)
    else:
        self.setVisible(True)

    CAnimatedSprite.render(self, aScreen)

```

Por último, en la clase `CPlayer`, en la función `setState()` que se invoca cuando se pasa de estado, se pone el sprite visible. Esto es para que si al cambiar de estado justo cuando el sprite está invisible, no quede invisible. De esta forma cada vez que se pasa de estado el

sprite vuelve a estar visible.

```
def setState(self, aState):  
    CAnimatedSprite.setState(self, aState)  
  
    # Por defecto poner el sprite visible.  
    self.setVisible(True)  
  
    ...
```

Nota: Es una buena práctica de programación el inicializar todas las variables cuando se pasa de estado. De esta forma se evitan potenciales problemas que se pueden dar al pasar entre estados dejando variables con valores pertenecientes a otros estados.

Con esto hemos terminado el capítulo de máquinas de estados. Como vemos, la máquina de estados es una técnica muy poderosa, y es una de las claves para hacer buenos videojuegos. Los estados se relacionan con las acciones de los objetos y con sus animaciones.

En el siguiente capítulo veremos el manejo de los sonidos y la música, otro de los aspectos fundamentales en un juego.

14. Audio: Música y Efectos

Nuestro juego está tomando forma. En este capítulo vamos a ver como agregar audio al juego. Vamos a ver como se pone una música (o sonido ambiente) de fondo y cómo agregarle efectos de sonido (denominados *fx*) al juego. Para eso, primero veremos los formatos de audio con los cuales trabaja *Pygame* y aprenderemos a cargar los archivos de audio y a ejecutarlos en el programa.

El Audio en los Juegos

El audio en un videojuego es tan importante como los gráficos. La música nos pone en el ambiente y ayuda a la inmersión del juego. Los efectos de sonido (llamados *sound fx*) también son importantísimos en el juego porque nos dan *feedback* (información) de lo que está pasando en el juego, por ejemplo cuando las cosas colisionan o agarramos una moneda en el juego, siempre hay un sonido para indicarlo y mejorar la experiencia del jugador.

Vamos a ver como cargar el audio para nuestros juegos y cómo manejamos el sonido en *Pygame*. Al igual que con los gráficos, el audio es realizado utilizando software especializado en audio y generando archivos, que son los que cargaremos en *Pygame* y reproduciremos cuando ocurran determinados eventos.

Formatos de Archivos de Audio

Los formatos de audio más usados son *ogg*, *mp3* y *wav*. El formato *wav* es un formato de audio muy usado, pero tiene la desventaja de que es un formato *sin compresión*, por lo cual los tamaños de archivo son muy grandes. El formato *mp3* es un formato que tiene *compresión* y es muy popular su uso, pero tiene la desventaja de que no es open source. El formato *ogg* es un formato de audio parecido al formato *mp3* y tiene la ventaja de que es un formato open source.

Pygame puede usar cualquiera de estos formatos de audio mencionados: *wav*, *mp3* y *ogg*. Es recomendable utilizar el formato *ogg* cuando se trata de un juego que se va a llevar a varias plataformas. De todas formas es sencillo convertir de un formato a otro utilizando *software de edición de audio*.

Inicializar el Sistema de Audio

Antes de ejecutar cualquier sonido, debemos inicializar el sistema de audio de *Pygame*, al cual se le llama *mixer*. Esta tarea se hará una sola vez, al inicio de nuestro programa y con esto ya podremos cargar y ejecutar sonidos.

Para inicializar el sistema de audio de *Pygame*, invocamos a la función `pygame.mixer.init()` inmediatamente después de haber inicializado *Pygame*. El mixer es el módulo de *Pygame* que maneja el audio. Entonces, lo primero que hacemos luego de inicializar *Pygame*, es inicializar el mixer.

Cargar y Ejecutar un Sonido

Una vez que tenemos correctamente inicializado el sistema de audio, cargaremos los sonidos que vamos a usar en el juego. Luego los ejecutaremos cuando ocurran determinados eventos.

Vamos a comenzar poniendo nuestro primer sonido. Haremos que la nave del jugador reproduzca un sonido cuando dispara. Luego de inicializar el sistema de audio, para colocar un nuevo sonido en el juego, se requieren tres pasos:

- 1) Colocar el archivo de sonido en la carpeta de `assets`. Los colocaremos en la carpeta `assets\audio`.
- 2) Al inicio del juego cargamos el sonido, creando un objeto de la clase `pygame.mixer.Sound`. Para esto, le pasamos la ruta donde se encuentra el archivo. Ya veremos esto a continuación.
- 3) Cuando ocurre determinado evento, como por ejemplo, al disparar, se utiliza la función `play()` en el sonido para ejecutarlo.

Lo que haremos ahora es colocar el primero sonido en el juego, que será el sonido cuando el jugador dispara.

Disparo del Jugador

Comenzaremos con el primer sonido que pondremos en el juego, que será el sonido que se reproduzca cuando el jugador dispare. Abre el ejemplo que se encuentra en la carpeta `capitulo_14\001_sonido_al_disparar_jugador` y ejecútalo. Verás que al disparar se reproduce un sonido de disparo.

Creemos una carpeta `audio` debajo de la carpeta `assets` que será la carpeta en donde vamos a poner los archivos de audio del juego. En la carpeta `assets\audio`, se encuentra el sonido que vamos a disparar cuando el jugador dispara. Este sonido se encuentra en el archivo `player_shoot.wav`. Crea la carpeta en tu propio proyecto y copia este archivo.

En el programa principal, `main.py`, agregamos la llamada a la función para inicializar el sistema de audio de *Pygame*:

```
# Función de inicialización.
```

```
def init():

    ...

    # Inicializar Pygame.
    pygame.init()

    # Inicializar el mixer de audio de Pygame.
    pygame.mixer.init()

    ...
```

Luego, en la clase `CPlayer`, cargaremos el archivo de sonido correspondiente al disparo del jugador en una variable (objeto) que necesitamos para usar el sonido. Para crear un sonido, utilizamos la clase `Sound` de *Pygame* y le pasamos como parámetro la ruta del archivo de sonido (en este caso será `assets/audio/player_shoot.wav`).

En el constructor de la clase `CPlayer` se pone una línea para cargar el sonido y guardarlo en una variable:

```
def __init__(self, aType):

    # Invocar al constructor de CAnimatedSprite.
    CAnimatedSprite.__init__(self)

    ...

    # Crear el sonido de disparo.
    self.mSoundShoot =
pygame.mixer.Sound("assets/audio/player_shoot.wav")

    # Estado inicial.
    self.setState(CPlayer.NORMAL)
```

Esto cargará el archivo de sonido en memoria, y creamos un objeto de clase `Sound` (la variable que hemos llamado `self.mSoundShoot`) que será la variable que usaremos para reproducir el sonido.

Para reproducir el sonido cuando el jugador dispara, debemos identificar en el código dónde es que ocurre el evento. En nuestro juego, éste ocurre en la función `move()` de la clase `CPlayer` (que es por supuesto la clase en la cual pusimos la variable del sonido).

Agregamos una línea cuando se crea la bala del jugador, que dispare el sonido que hemos agregado:

```

# Movimiento de la nave.
def move(self):

    ...

# Disparar.
if fire:
    print("FIRE")
    b = CPlayerBullet()
    b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() / 2,
self.getY())
    b.setVelX(0)
    b.setVelY(-10)
    b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
    b.setBoundAction(CGameObject.DIE)
    CBulletManager.inst().add(b)

# Sonido de disparo.
self.mSoundShoot.play()

```

De esta forma, una vez que los sonidos están cargados, se pueden disparar en cualquier momento que se desee. Para disparar el sonido en cualquier momento en el juego, se usa el método `play()` del objeto de la clase `Sound` que hemos creado (en este caso `self.mSoundShoot`). Hemos puesto esta variable en la clase `CPlayer` porque es en esta clase en donde se va a ejecutar el sonido.



Figura 14-1: Ahora el juego tiene sonido al disparar. Se siente mejor.

La mayoría de los sonidos ocurren cuando hay algún evento, como por ejemplo cuando una nave dispara, cuando le pegan a un objeto y explota, cuando se agarra un poder (*powerup*), etc.

Nota: Hay que tener muy prolijo y ordenado el código de forma tal de que sea fácil colocar la línea que ejecuta el sonido.

Agregando Todos los Sonidos al Juego

Cuando colocamos los sonidos en el juego, el proceso siempre es el mismo. Primero se inicializa el sistema de audio, luego se colocan los sonidos en la carpeta de assets, luego se inicializan las variables, cargando los sonidos y por último se ejecutan los sonidos con la función `play()`.

El problema es, que cuando el juego es muy grande, esta tarea de ubicar la línea exacta en donde se dispara el sonido, puede tomar tiempo. Es importantísimo ser muy prolijo a la hora de programar, y de no tener varias ramas que pueden llevar a ejecutar el mismo sonido.

Ahora vamos a agregar todos los efectos de sonido al juego. Cuando llegamos a esta tarea de poner los sonidos, lo que tenemos que hacer es una lista de los sonidos y cuándo es que se ejecutan. Entonces, la lista para este juego quedará así:

Archivo de Sonido:	Evento:
player_shoot.wav	Cuando el jugador dispara una bala.
player_hit.wav	Cuando al jugador lo toca una bala y parpadea.
player_explosion.wav	Cuando el jugador explota.
enemy_shoot	Cuando un enemigo dispara.
enemy_explosion.wav	Cuando el enemigo explota al tocarlo una bala.

Figura 14-1: La tabla con los sonidos a poner en el juego.

Para armar la lista de sonidos necesarios, debemos pensar en el juego y hacer la lista de los eventos que ocurren. Esto básicamente es una tarea mental, imaginándote estar jugando al juego en tu cabeza y anotar los sonidos.

Los sonidos casi siempre van asociados a las colisiones entre los objetos y a los eventos (disparar, saltar, agarrar objetos, etc).

Colocamos estos archivos de sonido en la carpeta `assets/audio` y cargamos los sonidos en las clases `CNave` y `CPlayer`. Luego, ponemos en el código las llamadas a `play()` en el lugar que corresponda al evento.

Veamos el ejemplo ubicado en la carpeta `capitulo_14\002_todos_los_sonidos`. En la

carpeta `assets\audio` hemos ubicado los archivos de sonido. Copia estos archivos a tu propio proyecto.

Al inicio de la clase cargamos los sonidos y luego ubicamos la sentencia `play()` donde corresponda para ejecutar el sonido. A continuación se muestra en negrita en el código de `CPlayer` las líneas agregadas. En la función `init()` se crean los sonidos:

```
def __init__(self, aType):

    # Invocar al constructor de CAnimatedSprite.
    CAnimatedSprite.__init__(self)

    ...

    # Crear el sonido de disparo.
    self.mSoundShoot =
pygame.mixer.Sound("assets/audio/player_shoot.wav")
    # Sonido de hit (cuando le pegan).
    self.mSoundHit = pygame.mixer.Sound("assets/audio/player_hit.wav")
    # Sonido de explosión.
    self.mSoundExplosion =
pygame.mixer.Sound("assets/audio/player_explosion.wav")

    # Estado inicial.
    self.setState(CPlayer.NORMAL)
```

En la función `setState()`, ubicamos las líneas que disparan los sonidos. Es muy probable que los sonidos ocurran cuando hay un cambio de estado, como es el caso cuando explota o cuando va a morir el jugador. Si armamos el código en forma prolija y se pasa de estado con la función `setState()`, es muy sencillo agregar sonidos:

```
def setState(self, aState):
    CAnimatedSprite.setState(self, aState)

    # Por defecto poner el sprite visible.
    self.setVisible(True)

    if self.getState() == CPlayer.NORMAL:
        self.initAnimation(self.mFrames, 0, 0, False)
        self.gotoAndStop(0)

    elif self.getState() == CPlayer.DYING:
        self.stopMove()
        self.initAnimation(self.mFrames, 0, 0, False)
        self.gotoAndStop(0)
```

```

        # Ejecutar sonido de morir.
        self.mSoundHit.play()

elif self.getState() == CPlayer.EXPLODING:
    self.initAnimation(self.mFramesExplosion, 0, 2, False)

    # Ejecutar el sonido de la explosión.
    self.mSoundExplosion.play()

elif self.getState() == CPlayer.START:
    self.initAnimation(self.mFrames, 0, 0, False)
    self.gotoAndStop(0)

```

De la misma manera, en la clase `CNave`, agregamos en la función `init()` la carga de los sonidos y luego ubicamos las líneas para reproducir los sonidos:

```

def __init__(self, aType):
    CAnimatedSprite.__init__(self)

    ...

    # Crear el sonido de disparo.
    self.mSoundShoot =
pygame.mixer.Sound("assets/audio/enemy_shoot.wav")
    # Sonido de explosión.
    self.mSoundExplosion =
pygame.mixer.Sound("assets/audio/enemy_explosion.wav")

    # Estado inicial.
    self.setState(CNave.NORMAL)

```

Luego de creados los sonidos, el sonido de disparo se ejecuta en la función `controlFire()`:

```

def controlFire(self):
    # Ver si la nave dispara.
    if random.randrange(1, 500) == 1:
        b = CEnemyBullet()
        b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() / 2,
self.getY() + self.getHeight())
        b.setVelX(0)
        b.setVelY(10)
        b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,

```

```

CGameConstants.SCREEN_HEIGHT)
    b.setBoundAction(CGameObject.DIE)
    CEnemyManager.inst().add(b)

    # Ejecutar el sonido de disparo.
    self.mSoundShoot.play()

```

Y por último, el sonido de la explosión se coloca al entrar en ese estado, en la función `setState()`:

```

def setState(self, aState):
    CAnimatedSprite.setState(self, aState)

    if self.getState() == CNav.e.NORMAL:
        self.initAnimation(self.mFramesNormal, 0, 0, True)
    elif self.getState() == CNav.e.EXPLODING:
        self.initAnimation(self.mFramesExplosion, 0, 0, False)
        self.stopMove()

    # Ejecutar el sonido de explosión.
    self.mSoundExplosion.play()

```

Con esto, terminamos de colocar los efectos de sonido en el juego. Luego de poner los sonidos hay que testear bien el juego porque puede pasar que bajo ciertas condiciones un sonido no se ejecute, o que se ejecute otro. Esto suele ocurrir cuando el código tiene varias ramas o lógica complicada, cosa que conviene minimizar y aprofundar.

Si dejamos el juego corriendo y escuchamos los efectos de sonido, veremos que dos por tres algún sonido no se ejecuta. Esto se debe a la forma en que estamos usando los canales de sonido, y lo corregiremos más adelante en este capítulo.

Agregando la Música

Vamos a agregar una música para el nivel del juego. La música es un archivo de sonido que se ejecuta en forma *cíclica* (denominado *loop*). Esto quiere decir que cuando se termina de reproducir la música, ésta comenzará nuevamente desde el inicio, y la banda de sonido está diseñada para que ese momento de *loopeo* no se note.

Veamos el ejemplo ubicado en la carpeta `capitulo_14\003_background_music`.

Si ejecutamos el juego, escucharemos una música cuando estamos jugando. La música del nivel se encuentra en el archivo `music_game.ogg`, y se encuentra ubicado en la carpeta `assets/audio`. Copia este archivo a tu propio proyecto.

Mira el programa `main.py`, al final de la función `init()` que es donde se inicializa la música y se reproduce. Para inicializar un archivo de música, utilizamos la clase `pygame.mixer.music`. El código para inicializar un archivo de música es el siguiente:

```
# Función de inicialización.
def init():

    ...

    # Inicializar Pygame.
    pygame.init()

    # Inicializar el mixer de audio de Pygame.
    pygame.mixer.init()

    ...

    # Ejecutar la música de background (loop) del juego.
    pygame.mixer.music.load("assets/audio/music_game.ogg")
    pygame.mixer.music.play(-1)
    # Poner el volumen de la música.
    pygame.mixer.music.set_volume(0.5)
```

La función `load()` inicializa la música, y recibe como parámetro la ruta donde se encuentra el archivo. Como siempre, ponemos los assets de sonido en la carpeta `assets\audio`.

La clase `pygame.mixer.music` es diferente a `pygame.mixer.sound` en cuanto a que la música es cargada a demanda (a esto se le llama *streamed*) y nunca se carga entera en memoria. El *mixer* en *Pygame* solamente soporta una sola música a la vez. Cuando cargamos sonidos, creamos una variable para cada sonido, pero para la música no usamos variables.

Luego, con la función `play()` se reproduce la música. A esta función se le pasa `-1` como parámetro para que se reproduzca indefinidamente (en *loop*).

Nota: Recuerda que siempre puedes consultar la documentación de *Pygame* en el sitio www.pygame.org para ver que otras funciones se pueden utilizar. Por ejemplo, en el caso del audio, es posible manejar el volumen, pausar la música, obtener la posición en la cual se encuentra el reproductor, encolar varias músicas para que se toquen una tras otra, etc. Cada clase de *Pygame* ofrece más funciones de las que se pueden explicar en este libro.

En el caso que quisiéramos detener la música (por ejemplo al ir a una pausa, o cuando termina el nivel), usamos la función `pygame.mixer.music.stop()`.

Nota: Algunos soportes de audio en *Pygame* se encuentran limitados en algunas plataformas, lo que podría hacer que se tranque el juego. Se recomienda el uso del formato *ogg* para los

audios.

El Manager de Audio

Como vimos antes, en algunos casos, algún efecto de sonido puede ser que no se reproduzca correctamente. Esto depende del sistema, pero se debe a que no estamos manejando los canales de sonido correctamente.

Cada sonido se reproduce en un determinado canal. Existen varios canales y debemos asegurarnos que cuando ejecutamos un sonido, éste no se dispare en un canal que está siendo usado, porque eso significa que o bien el sonido no se reproducirá, o bien corte el sonido actual para reproducir el nuevo.

Para eso, escribiremos una clase, que llamaremos `CAudioManager`, que maneje automáticamente los canales, asignando un canal no utilizado a cada sonido que vayamos a reproducir. Esta clase será una clase *singleton* (para poderla utilizar desde cualquier clase) y tendrá una función `play()` que recibirá un sonido (por ejemplo, los sonidos que cargamos en `CPlayer` o en `CNave`) y la función reproducirá este sonido en un canal libre.

Mira el ejemplo ubicado en la carpeta `capitulo_14\004_audio_manager`. En la carpeta `\api`, hemos puesto esta clase. A continuación se muestra el código de la clase `CAudioManager`:

```
# -*- coding: utf-8 -*-

#-----
# Clase CAudioManager.
# Clase para manejar el audio y los canales.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

class CAudioManager(object):

    mInstance = None
    mInitialized = False

    mChannels = 8

    def __new__(self, *args, **kwargs):
```

```

        if (CAudioManager.mInstance is None):
            CAudioManager.mInstance = object.__new__(self, *args,
**kargs)
            self.init(CAudioManager.mInstance)
        else:
            print("Cuidado: CAudioManager(): No se debería instanciar
más de una vez esta clase. Usar CAudioManager.inst().")
            return CAudioManager.mInstance

    @classmethod
    def inst(cls):
        if (not cls.mInstance):
            return cls()
        return cls.mInstance

    def init(self):
        if (CAudioManager.mInitialized):
            return
        CAudioManager.mInitialized = True

        CAudioManager.mChannels = pygame.mixer.get_num_channels()

        # Crear el sonido de disparo del jugador.
        CAudioManager.mSoundShootPlayer =
pygame.mixer.Sound("assets/audio/player_shoot.wav")
        # Sonido de hit (cuando le pegan al jugador).
        CAudioManager.mSoundHitPlayer =
pygame.mixer.Sound("assets/audio/player_hit.wav")
        # Sonido de explosión del jugador.
        CAudioManager.mSoundExplosionPlayer =
pygame.mixer.Sound("assets/audio/player_explosion.wav")
        # Crear el sonido de disparo del enemigo.
        CAudioManager.mSoundShootEnemy =
pygame.mixer.Sound("assets/audio/enemy_shoot.wav")
        # Sonido de explosión del enemigo.
        CAudioManager.mSoundExplosionEnemy =
pygame.mixer.Sound("assets/audio/enemy_explosion.wav")

    def play(self, aSound):
        CAudioManager.inst().getChannel().play(aSound)

    def getChannel(self):
        c = pygame.mixer.find_channel(True)
        while c is None:
            CAudioManager.mChannels += 1
            pygame.mixer.set_num_channels(CAudioManager.mChannels)
            c = pygame.mixer.find_channel()
        return c

```

```
def destroy(self):
    CAudioManager.mInstance = None

    CAudioManager.mSoundShootPlayer = None
    CAudioManager.mSoundHitPlayer = None
    CAudioManager.mSoundExplosionPlayer = None
    CAudioManager.mSoundShootEnemy = None
    CAudioManager.mSoundExplosionEnemy = None
```

En la función `init()` se cargan los sonidos del juego, tanto los que ejecuta el jugador como los que ejecuta los enemigos. Los sonidos ya no se cargan en cada clase, sino en esta función, en forma centralizada.

Nota: esta parte donde se cargan todos los sonidos del juego, luego se va a poner en algún estado de carga inicial del juego (*loading*). Pero por ahora trabajaremos de esta manera mientras aprendemos a hacer juegos.

Veamos la parte de ejecutar sonidos. Básicamente lo que hace la función `play()` es usar la función `getChannel()` de nuestra clase para obtener el primer canal libre. Para eso, se usa la función `find_channel()` de *Pygame*. Si no hubiera ningún canal libre (hay ocho disponibles al inicio), se crea uno nuevo usando la función `set_num_channels()`. La variable `CAudioManager.mChannels` lleva la cuenta de cuántos canales hay.

Nota: Es importante consultar la documentación de *Pygame* para ver cómo es que funciona cada una de las funciones de *Pygame* que estamos usando, que hace internamente, que retorna, qué parámetros espera, etc. Si hacemos esto, entenderemos bastante mejor su funcionamiento. Sin embargo, podemos utilizar el código de los ejemplos, que está bien escrito, aunque como todo, siempre es mejorable o adaptable a futuras necesidades.

Nota: En este libro estamos usando una nomenclatura propia. Las clases comienzan con “C”, las variables con “m”, los argumentos con “a” (parámetros) y las funciones con nomenclatura denominada “*camelCase*”, comienzan con minúscula y luego cada palabra va con mayúscula. De esta forma, podemos distinguir fácilmente nuestras funciones (por ejemplo “`getChanneel()`” de las de *Pygame* (por ejemplo “`find_channel()`”). Esto es fundamental para entender el código.

Una vez que tenemos implementada la clase `CAudioManager`, utilizamos su función `play()` para reproducir sonidos. Por ejemplo, en la clase `CPlayer`, para reproducir los sonidos, cambiamos la llamada por las siguientes líneas:

```
...
```

```
# Importar Pygame.
import pygame
```



```

...

# Importar la clase del manager de audio.
from api.CAudioManager import *

# La clase CPlayer hereda de CAnimatedSprite.
class CPlayer(CAnimatedSprite):

    ...

    # Establece el estado actual e inicializa
    # las variables correspondientes al estado.
    def setState(self, aState):
        CAnimatedSprite.setState(self, aState)

        # Por defecto poner el sprite visible.
        self.setVisible(True)

        if self.getState() == CPlayer.NORMAL:
            self.initAnimation(self.mFrames, 0, 0, False)
            self.gotoAndStop(0)

        elif self.getState() == CPlayer.DYING:
            self.stopMove()
            self.initAnimation(self.mFrames, 0, 0, False)
            self.gotoAndStop(0)

            # Ejecutar sonido de morir.
            CAudioManager.inst().play(CAudioManager.mSoundHitPlayer)

        elif self.getState() == CPlayer.EXPLODING:
            self.initAnimation(self.mFramesExplosion, 0, 2, False)

            # Ejecutar el sonido de la explosión.
            CAudioManager.inst().play(
                CAudioManager.mSoundExplosionPlayer)

        elif self.getState() == CPlayer.START:
            self.initAnimation(self.mFrames, 0, 0, False)
            self.gotoAndStop(0)

    # Movimiento de la nave.
    def move(self):

        ...

        # Disparar.

```

```

        if fire:
            print("FIRE")
            b = CPlayerBullet()
            b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY())
            b.setVelX(0)
            b.setVelY(-10)
            b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
            b.setBoundAction(CGameObject.DIE)
            CBulletManager.inst().add(b)

# Sonido de disparo.
CAudioManager.inst().play(CAudioManager.mSoundShootPlayer)

```

En la clase CNavé, también cambiamos las líneas para ejecutar sonidos, de la siguiente manera:

```

...

# Importar Pygame.
import pygame

...

# Importar la clase del manager de audio.
from api.CAudioManager import *

# La clase CNavé hereda de CAnimatedSprite.
class CNavé(CAnimatedSprite):

    ...

    def setState(self, aState):
        CAnimatedSprite.setState(self, aState)

        if self.getState() == CNavé.NORMAL:
            self.initAnimation(self.mFramesNormal, 0, 0, True)
        elif self.getState() == CNavé.EXPLODING:
            self.initAnimation(self.mFramesExplosion, 0, 0, False)
            self.stopMove()

# Ejecutar el sonido de explosión.
CAudioManager.inst().play(
CAudioManager.mSoundExplosionPlayer)

```

```

# Invocada desde Bullet cuando la Nave es alcanzada por una bala.
def hit(self):
    if self.getState() == CNave.NORMAL:
        self.setState(CNave.EXPLODING)
        print("NAVE EXPLOTA")

def controlFire(self):
    # Ver si la nave dispara.
    if random.randrange(1, 500) == 1:
        b = CEnemyBullet()
        b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY() + self.getHeight())
        b.setVelX(0)
        b.setVelY(10)
        b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
        b.setBoundAction(CGameObject.DIE)
        CEnemyManager.inst().add(b)

    # Ejecutar el sonido de disparo.
    CAudioManager.inst().play(CAudioManager.mSoundShootEnemy)

...

```

Con esto, tenemos terminado el soporte básico para los sonidos y la música del juego. En el siguiente capítulo continuaremos mejorando el juego, haciendo el marcador de puntaje y las vidas, viendo cómo mostrar textos.

15. Manejo de Textos y Fuentes

En este capítulo vamos a ver cómo dibujar texto en la pantalla. En *Pygame*, podemos dibujar textos usando alguna *fuentes* (*tipos de letra*). Vamos a comenzar mostrando en la pantalla del juego, un texto con las vidas y el puntaje (*score*) del jugador. En los siguientes capítulos haremos todas las pantallas del juego, como el menú principal, la pantalla de ayuda, la pantalla de créditos, etc. y en todas esas pantallas necesitaremos dibujar texto.

Dibujando un Texto

En *Pygame* los textos son dibujados en la pantalla como imágenes. Es decir, cuando mostramos texto en la pantalla, lo que estamos haciendo realmente es creando una imagen y dibujándola en la pantalla.

Para mostrar un texto, debemos crear primero un objeto de clase `pygame.font.Font`, al cual le pasamos como parámetro la fuente a usar (la ruta del archivo de la fuente) y el tamaño de la fuente. Luego usamos la función `render()` en esa fuente que hemos creado, lo que da como resultado una nueva imagen conteniendo el texto que le pasamos como parámetro dibujado en la imagen. Al final, mostramos esa imagen en la pantalla, normalmente, como hacemos con cualquier imagen.

Vamos a hacer una función para dibujar un texto en la pantalla o en una imagen. A esta función le pasamos varios parámetros. Veamos la función a continuación:

```
# Función para dibujar texto.
# Parámetros: La pantalla donde dibujar, coordenadas (x,y),
# texto ,tamaño de fuente y color del texto.
def drawText(aScreen, aX, aY, aMsg, aSize, aColor=(0,0,0)):
    font = pygame.font.Font("assets/fonts/days.ot", aSize)
    imgTxt = font.render(aMsg, True, aColor)
    aScreen.blit(imgTxt, (aX, aY))
```

Esta función recibe como parámetros lo siguiente:

`aScreen`: Superficie donde va a dibujar el texto. Puede ser la pantalla o cualquier imagen.

`aX, aY`: Coordenadas en donde dibujar el texto.

`aMsg`: String (texto) que se quiere mostrar.

`aSize`: Tamaño de la fuente que se va a usar.

`aColor`: Color del texto.

En primera línea de la función, se crea un objeto `pygame.font.Font`, al cual se le pasa como parámetro la fuente a usar y el tamaño de la fuente (el tamaño es similar a cuando

elegimos una fuente y tamaño en un editor de texto). Como los archivos de fuentes son *assets*, hemos hecho una carpeta para las fuentes en la carpeta de *assets* en donde colocamos las fuentes usadas en el juego (*assets/fonts*).

La segunda línea usa la función `render()` de la fuente para crear una imagen a partir del texto que queremos mostrar, utilizando la información de la fuente para generar la imagen. A la función `render()` se le pasa como segundo parámetro un valor booleano (`True` o `False`) que indica si se quiere el texto con *antialias* (suavizado de los píxeles) (`True`) o sin *antialias* (`False`). El tercer parámetro es el color del texto. Recordar que el color siempre es una tupla con los valores *RGB* (*red, green, blue*).

Nota: La función `font.render()` podría recibir un cuarto parámetro que es el color de fondo. Este parámetro no lo usamos porque sino quedaría un rectángulo de fondo con el color. Usaremos siempre el texto con un fondo transparente.

Nota: El uso de *antialias* hace que los textos se vean mejor, con más suavizado en los bordes. Si no se usa *antialias* se ven los píxeles bien marcados en los bordes. En la función usaremos *antialias*, pero hay que tener en cuenta que es una operación más lenta que dibujar sin *antialias*. Si son muchos los textos del juego, es algo a tener en cuenta.

Por último, usamos la ya conocida función `blit()` en la imagen para dibujar la imagen generada con el texto, en las coordenadas indicadas.

Abre y ejecuta el ejemplo que se encuentra en la carpeta `capitulo_15\001_mostrar_texto`. En el programa `main.py`, se encuentra implementada la función `drawText()`, explicada antes, casi al final del programa. Esta función usa la fuente `days.otf` que se encuentra en la carpeta `assets/fonts`. Copia esta carpeta a tu propio proyecto.

En la función `render()` en `main.py`, mostramos dos textos, el primero para mostrar el puntaje del jugador en la parte superior izquierda de la pantalla, y el segundo para mostrar las vidas, en la parte inferior de la pantalla.

```
# Dibujar el frame y actualizar la pantalla.
def render():
    # Dibujar el fondo.
    screen.blit(imgBackground, (0, 0))

    # Dibujar los enemigos.
    CEnemyManager.inst().render(screen)

    # Dibujar los jugadores.
    player1.render(screen)
    player2.render(screen)

    # Dibujar las balas.
```

```

CBulletManager.inst().render(screen)

# Dibujar el texto del score y las vidas.
drawText(screen, 5, 5, "SCORE: 00000", 20, (255, 255, 255))
drawText(screen, 5, SCREEN_HEIGHT - 20 - 5, "VIDAS: 3", 20, (255,
255, 255))

# Actualizar la pantalla.
pygame.display.flip()

```

Dibujamos el score en la parte de arriba de la pantalla y las vidas en la parte inferior. Recordemos que el segundo y tercer parámetro corresponde a la coordenada x e y donde se va a dibujar el texto. Como al texto le pasamos un alto de 20 puntos, las vidas, el segundo texto, lo dibujamos en la coordenada correspondiente al alto de la pantalla menos el alto del texto. Usamos 5 de margen, tanto arriba como abajo, para que los textos no queden pegados a los bordes de la pantalla.

Los marcadores que se muestran en el juego (puntaje, vidas, nivel, etc.) se denomina *HUD* (por las siglas de *head-up display*). Por ahora solamente mostramos un texto fijo, porque recién estamos viendo cómo mostrar textos. Los marcadores del *HUD* por ahora no estarán funcionales.

Más adelante si perdemos una vida, actualizaremos el contador de vidas en el *HUD*. Lo mismo cuando matamos a un enemigo, se incrementará el marcador del puntaje.

Al colocar estos textos, vemos que las naves de los jugadores lo tocan. Debemos subir un poco la ubicación de las naves para que no toquen el texto para que queden arriba del texto de las vidas y no lo toque. Como el texto tiene 20 puntos, es la distancia que subimos las naves de los jugadores. En la función `init()` de `main.py` restamos la coordenada vertical de las naves y también subimos 20 píxeles las naves enemigas:

```

# Función de inicialización.
def init():

    ...

    # Crear la formación inicial.
    f = 0
    while f <= 4:
        c = 0
        while c <= 4:
            n = CNave(f)
            n.setXY(100 + (70 * c), 30 + (35 * f))
            n.setVelX(4)
            n.setVelY(0)
            n.setBounds(0, 0, SCREEN_WIDTH - n.getWidth(),

```

```

SCREEN_HEIGHT - n.getHeight())
    n.setBoundAction(CGameObject.BOUNCE)
    CEnemyManager.inst().add(n)
    c = c + 1
    f = f + 1

# Crear el jugador 1.
player1 = CPlayer(CPlayer.TYPE_PLAYER_1)
player1.setXY(SCREEN_WIDTH / 4 - player1.getWidth() / 2,
SCREEN_HEIGHT - player1.getHeight() - 20)
player1.setBounds(0, 0, SCREEN_WIDTH - player1.getWidth(),
SCREEN_HEIGHT)

# Crear el jugador 2.
player2 = CPlayer(CPlayer.TYPE_PLAYER_2)
player2.setXY(SCREEN_WIDTH / 4 * 3 - player2.getWidth() / 2,
SCREEN_HEIGHT - player2.getHeight() - 20)
player2.setBounds(0, 0, SCREEN_WIDTH - player2.getWidth(),
SCREEN_HEIGHT)

...

```

Al ejecutar el juego, vemos que se muestran los textos del puntaje y de las vidas, y que ni las naves de los jugadores ni las naves enemigas tocan los textos.



Figura 15-1: El juego muestra textos ahora.

Nota: Al crear la fuente en la función `drawText()`, podríamos pasar como parámetro el nombre de la fuente en lugar del archivo de la fuente, pero como los diferentes sistemas

operativos (*Windows*, *Linux*, *Mac*, etc.) tienen diferentes fuentes en el sistema, la fuente usada podría no existir. Por eso, es mejor poner el archivo de fuente en una carpeta de fuentes y distribuirla con el juego (esto es lo que estamos haciendo). Casi siempre vamos a usar un archivo de fuente propio del juego, porque en un juego las fuentes deben de ser lindas o de cierto tema, y las fuentes del sistema generalmente no lo son.

Usando una Fuente del Sistema (System Font)

Pygame nos brinda la función `pygame.font.SysFont` para crear una fuente del sistema. Esto es, una de las fuentes que trae ya instalado el sistema operativo. Para crear una fuente del sistema, le pasamos como parámetros a la función, el nombre de la fuente y el tamaño. Por ejemplo:

```
font = pygame.font.SysFont("Comic Sans MS", aSize)
```

La ventaja de usar una fuente del sistema es que no tenemos que incorporar una fuente con el juego, pero la desventaja es que la fuente podría no existir en el sistema (recuerda que al juego, si tiene éxito, se va a jugar en diferentes plataformas y es importante tener en cuenta este aspecto al desarrollar juegos).

No hay ninguna garantía de que determinada fuente se encuentre en el sistema. Hay pocas fuentes que son compartidas entre diferentes sistemas operativos como *Linux*, *Mac* y *Windows*.

Usando `pygame.font.SysFont()`, si la fuente no se encuentra en el sistema, *Pygame* la reemplaza por otra y el programa sigue funcionando (no da error). Si usamos `pygame.font.Font()` y nos olvidamos de incluir la fuente en el juego, el programa dará error.

Nota: En un juego nuestro, casi siempre utilizaremos una fuente propia, dado que la fuente tiene que ir con la estética del juego. Estas fuentes las colocamos en la carpeta de assets (`assets/fonts`).

Usando una Fuente Propia del Juego (Custom Font)

En nuestra función `drawText()`, hemos creado una fuente propia (denominada *custom font*) usando la función `pygame.font.Font` y pasándole como parámetro el nombre del archivo de fuente (la ruta del archivo) y el tamaño. Por ejemplo:

```
font = pygame.font.Font("assets/fonts/days.otf", aSize)
```

Estos archivos de fuentes se consiguen en sitios especializados en fuentes, o bien se puede crear una fuente nueva si se usa algún editor (software) para hacer una nueva fuente (aunque esto lleva tiempo y no es tan sencillo).

Creando el HUD: Agregando los Puntajes y las Vidas

En este momento ya tenemos el juego funcionando bien y sabemos mostrar texto. Entonces, vamos a agregar el *HUD* al juego, que muestre el puntaje y las vidas que tienen los jugadores. Recuerda que *HUD* significa *head-up display* y se refiere a los marcadores que aparecen sobreimpresos en el juego (vidas, puntos, energía, etc).

Los marcadores le darán al jugador una idea de cómo lo está haciendo en el juego (si va bien o mal). Cuando un juego muestra información de alguna manera (tanto con audio como con gráficos o con texto), se le denomina dar *feedback* (dar información).

Los jugadores comenzará con 3 vidas cada uno y cada vez que choquemos contra un enemigo (una bala por ejemplo), vamos a perder una vida. Si perdemos todas las vidas el juego termina (más adelante haremos que salga del juego al terminar).

Abre y ejecuta el ejemplo que se encuentra en la carpeta `capitulo_15\002_hud_score_y_vidas`. Al ejecutar el juego vemos que ahora tenemos en pantalla el puntaje y las vidas de los dos jugadores.

El juego ahora tiene cuatro datos que llevar: los puntajes y las vidas de los dos jugadores. En lugar de utilizar variables en el programa `main.py`, que haría complicado su seguimiento por todo el código que hay, centralizamos los datos del juego en una clase llamada `CGameData`, que colocaremos en la carpeta `game` (porque es algo relacionado al juego).

La clase `CGameData` será una clase *singleton* (porque queremos acceder a sus funciones desde cualquier parte del código sin necesitar una instancia). Esta clase tendrá cuatro variables que son las que necesitamos para el juego: `mScore1` y `mScore2` son el puntaje del primer y segundo jugador respectivamente y `mLives1` y `mLives2` son las vidas del primer y segundo jugador respectivamente.

La clase `CGameData` se compone de las variables, más un conjunto de funciones denominadas *setters* y *getters* que son funciones para establecer y obtener los valores de cada una de las variables. El código es bastante fácil de entender. Una parte corresponde a implementar el comportamiento del patrón *singleton* y la otra parte a las funciones para el control de las variables que almacena. Veamos a continuación el código de la clase `CGameData`:

```
# -*- coding: utf-8 -*-

#-----
# Clase GameData.
# Clase para manejar los datos del juego.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
```

```

#-----

# Importar Pygame.
import pygame

class CGameData(object):

    mInstance = None
    mInitialized = False

    mScore1 = 0
    mLives1 = 0
    mScore2 = 0
    mLives2 = 0

    def __new__(self, *args, **kargs):
        if (CGameData.mInstance is None):
            CGameData.mInstance = object.__new__(self, *args, **kargs)
            self.init(CGameData.mInstance)
        else:
            print("Cuidado: CGameData(): No se debería instanciar más
de una vez esta clase. Usar CGameData.inst().")
            return CGameData.mInstance

    @classmethod
    def inst(cls):
        if (not cls.mInstance):
            return cls()
        return cls.mInstance

    def init(self):
        if (CGameData.mInitialized):
            return
        CGameData.mInitialized = True

        CGameData.mScore1 = 0
        CGameData.mLives1 = 0
        CGameData.mScore2 = 0
        CGameData.mLives2 = 0

    def setScore1(self, aScore):
        CGameData.mScore1 = aScore
        self.controlScores()

    def setScore2(self, aScore):
        CGameData.mScore2 = aScore
        self.controlScores()

```

```

def addScore1(self, aScore):
    CGameData.mScore1 += aScore
    self.controlScores()

def addScore2(self, aScore):
    CGameData.mScore2 += aScore
    self.controlScores()

def controlScores(self):
    # Controlar que los scores no sean negativos o muy grandes.
    if (CGameData.mScore1 < 0):
        CGameData.mScore1 = 0
    if (CGameData.mScore1 > 999999):
        CGameData.mScore1 = 999999
    if (CGameData.mScore2 < 0):
        CGameData.mScore2 = 0
    if (CGameData.mScore2 > 999999):
        CGameData.mScore2 = 999999

def getScore1(self):
    return CGameData.mScore1

def getScore2(self):
    return CGameData.mScore2

def setLives1(self, aLives):
    CGameData.mLives1 = aLives
    self.controlLives()

def setLives2(self, aLives):
    CGameData.mLives2 = aLives
    self.controlLives()

def addLives1(self, aLives):
    CGameData.mLives1 += aLives
    self.controlLives()

def addLives2(self, aLives):
    CGameData.mLives2 += aLives
    self.controlLives()

def controlLives(self):
    # Controlar que las vidas no sean negativas o muy grandes.
    if (CGameData.mLives1 < 0):
        CGameData.mLives1 = 0
    if (CGameData.mLives1 > 9):
        CGameData.mLives1 = 9
    if (CGameData.mLives2 < 0):

```

```

        CGameData.mLives2 = 0
    if (CGameData.mLives2 > 9):
        CGameData.mLives2 = 9

    def getLives1(self):
        return CGameData.mLives1

    def getLives2(self):
        return CGameData.mLives2

    def destroy(self):
        CGameData.mInstance = None

```

Como vemos en el código, la clase `CCGameData` contiene funciones para establecer los valores de cada una de las variables (*setters*) y funciones para obtener sus valores (*getters*). El concepto importante aquí es que las variables (los datos del juego) solamente se modifican mediante estas funciones. Cuando se modifica el puntaje o las vidas, se llama internamente a una función que controla que las vidas estén entre 0 y 9 (esto es arbitrario y se puede cambiar) y que el puntaje esté entre 0 y 999999.

Hacer este control cada vez que modificamos un dato se denomina *chequeo de integridad*, y con esto nos aseguramos de no cometer errores que son comunes en los juegos amateurs, como tener vidas negativas cuando perdamos muchas vidas, o tener un puntaje negativo, o muy alto que de la vuelta hasta el cero. Cada vez que se modifica el valor de un dato, se chequea su integridad. De esta forma no hay errores.

Una vez que tenemos los datos del juego en la clase `CGameData`, en el programa `main.py` los utilizamos. Para eso, importamos la clase, inicializamos los datos usando las funciones *setters*, y los dibujamos en el *HUD* utilizando las funciones *getters*. A continuación se muestra resaltado en negrita los lugares de `main.py` relacionado con los datos del juego:

```

...

# Importar Pygame.
import pygame

...

# Importar la clase de los datos del juego.
from game.CGameData import *

...

# Función de inicialización.
def init():

```

```

...

# Inicializar los datos del juego.
CGameData.inst().setScore1(0)
CGameData.inst().setLives1(3)
CGameData.inst().setScore2(0)
CGameData.inst().setLives2(3)

...

# Dibujar el frame y actualizar la pantalla.
def render():
    # Dibujar el fondo.
    screen.blit(imgBackground, (0, 0))

    # Dibujar los enemigos.
    CEnemyManager.inst().render(screen)

    # Dibujar los jugadores.
    player1.render(screen)
    player2.render(screen)

    # Dibujar las balas.
    CBulletManager.inst().render(screen)

    # Dibujar el texto del score y las vidas.
    drawText(screen, 5, 5, "SCORE: " +
str(CGameData.inst().getScore1()), 20, (255, 255, 255))
    drawText(screen, 5, SCREEN_HEIGHT - 20 - 5, "VIDAS: " +
str(CGameData.inst().getLives1()), 20, (255, 255, 255))
    drawText(screen, 530, 5, "SCORE: " +
str(CGameData.inst().getScore2()), 20, (255, 255, 255))
    drawText(screen, 540, SCREEN_HEIGHT - 20 - 5, "VIDAS: " +
str(CGameData.inst().getLives2()), 20, (255, 255, 255))

    # Actualizar la pantalla.
    pygame.display.flip()

...

```

Como vemos, simplemente usamos `CGameData.inst().setScore1()` si queremos establecer el puntaje del primer jugador, y usamos `CGameData.inst().getScore1()` cuando necesitamos obtener su valor para mostrarlo en el *HUD*. Lo mismo para cada uno de los datos de esa clase.

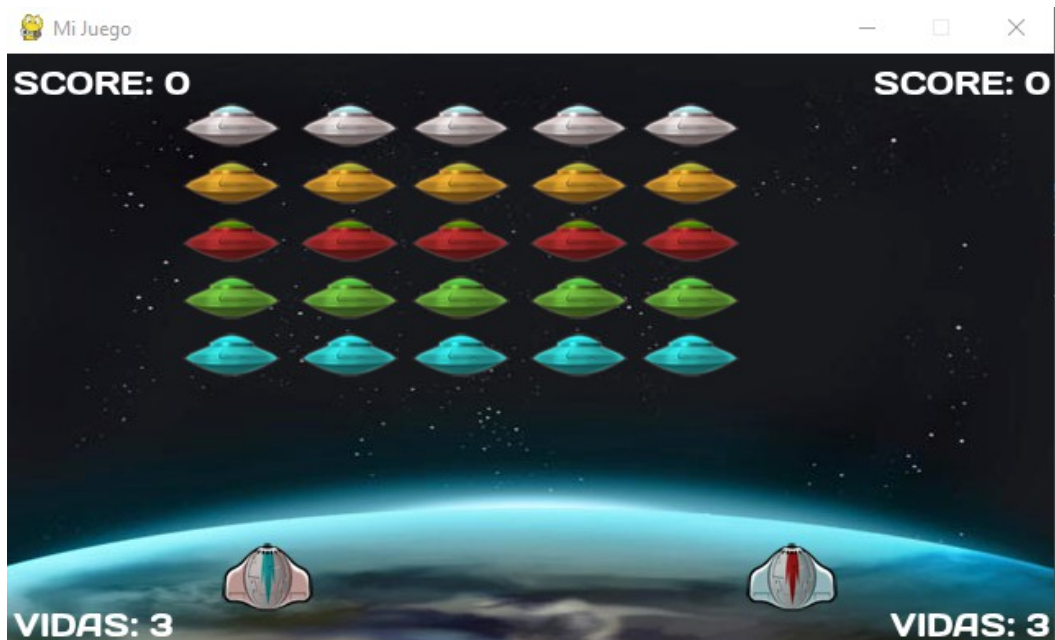


Figura 15-2: El HUD ahora está mostrándose en pantalla.

Nota: En la clase `CGameData` podríamos haber implementado una solución mejor para manejar los datos de cada jugador, por ejemplo utilizando un *array*. Lo hemos hecho con variables por separado para que quede más claro, dado que son pocas variables y funciona bien para nuestro propósito.

Controlando el Puntaje de los Jugadores

En esta sección, haremos que cuando una bala toque a un enemigo, incrementar el puntaje, sumándole el que corresponda a cada enemigo.

Cada vez que matamos a un enemigo, el score aumenta. Haremos que los enemigos de la primera línea valgan 5 puntos, los de la segunda línea valdrán 10 puntos, y así sucesivamente.

Veamos el ejemplo que se encuentra en la carpeta `capitulo_15\003_score_funcional`. Cuando cada jugador mata a un enemigo, vemos que su score se incrementa.

Para lograr esto, necesitamos saber cuantos puntos nos da cada enemigo. Para esto, pondremos una variable en la clase `CSprite`, de modo que todos los sprites del juego tengan un score. Es discutible si poner el puntaje en la clase `CSprite`, en `CGameObject` o en donde sea (es cuestión de gustos), pero en algún lado lo debemos poner. Lo vamos a hacer en la clase `CSprite`.

Tendremos una variable `mScore` que indica cuantos puntos da el sprite al matarlo, y como siempre, tendremos funciones para establecer y obtener su valor.. Veamos el código agregado en la clase `CSprite`:

```

...

class CSprite(CGameObject):

    # Constructor:
    def __init__(self):

        CGameObject.__init__(self)

        ...

        # Score del sprite.
        self.mScore = 0

...

    # Establecer el score del sprite.
    def setScore(self, aScore):
        self.mScore = aScore

    # Obtener el score del sprite.
    def getScore(self):
        return self.mScore

```

Ahora, cualquier sprite del juego puede tener un score si lo asignamos con la función `setScore()`, de lo contrario el score será *cero*.

Como cada nave ahora tiene su score, en la clase `CNave` debemos establecerlo. En el constructor de la clase asignamos el score según el tipo de nave:

```

...

class CNave(CAnimatedSprite):

    ...

    def __init__(self, aType):
        CAnimatedSprite.__init__(self)

        # Segun el tipo de la nave, la imagen que se carga.
        self.mType = aType

        if self.mType == CNave.TYPE_PLATINUM:
            imgFile = "assets/images/grey_ufo_0"
            self.setScore(25)
        elif self.mType == CNave.TYPE_GOLD:

```

```

        imgFile = "assets/images/yellow_ufo_0"
        self.setScore(20)
    elif self.mType == CNave.TYPE_RED:
        imgFile = "assets/images/red_ufo_0"
        self.setScore(15)
    elif self.mType == CNave.TYPE_CYAN:
        imgFile = "assets/images/cyan_ufo_0"
        self.setScore(10)
    elif self.mType == CNave.TYPE_GREEN:
        imgFile = "assets/images/green_ufo_0"
        self.setScore(5)

    ...

```

En la clase `CNave`, cambiamos las constantes de las naves para que la nave verde sea la de más abajo de la formación (esto es porque es la que luce más fea).

```

...

class CNave(CAnimatedSprite):

    # Tipos de naves.
    TYPE_PLATINUM = 0
    TYPE_GOLD = 1
    TYPE_RED = 2
    TYPE_CYAN = 3
    TYPE_GREEN = 4

    ...

```

Con esto, cada nave tendrá almacenado su score. Ahora, tenemos que hacer que cuando una bala del jugador mata a un enemigo, se le sume al puntaje el valor del *score* de la nave que matamos. Esto se encuentra en la clase `CPlayerBullet`, cuando chequeamos colisiones contra los enemigos del manager de enemigos.

Pero antes tenemos que solucionar algo. Cuando una bala le pega a un enemigo, debemos incrementar el score del jugador que disparó la bala. Para esto, en la clase `CPlayer`, al disparar la bala, le pasaremos como parámetro la referencia al jugador que la dispara. Veamos esta línea en la función `move()`, al disparar:

```

# Movimiento de la nave.
def move(self):

    ...

```



```

# Disparar.
if fire:
    print("FIRE")
    b = CPlayerBullet(self)
    b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() / 2,
self.getY())
    b.setVelX(0)
    b.setVelY(-10)
    b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
    b.setBoundAction(CGameObject.DIE)
    CBulletManager.inst().add(b)

# Sonido de disparo.
CAudioManager.inst().play(CAudioManager.mSoundShootPlayer)

```

Esto es clave en los juegos y es un concepto muy poderoso. Le pasamos a la bala la referencia `self` (una referencia a mí mismo). Como estamos en la clase `CPlayer`, esto es una referencia al jugador que disparó la bala. Entonces, en la clase `CPlayerBullet` guardamos esa referencia, y con ella podremos acceder a cualquiera de sus funciones. Esto lo usaremos para saber si es el primer jugador o el segundo.

En la clase `CPlayer`, agregamos una función para preguntar si es el primer jugador o no:

```

# Retorna True si es el primer jugador. False si es el segundo.
def isPlayerOne(self):
    return self.mType == CPlayer.TYPE_PLAYER_1

```

Y en la clase `CPlayerBullet`, cuando la bala del jugador le pega a una nave enemiga, se incrementa el score del jugador que corresponda. Veamos los cambios en la clase `CPlayerBullet`:

```

# -*- coding: utf-8 -*-

#-----
# Clase CPlayerBullet.
# Balas del jugador.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

```

```

# Importar Pygame.
import pygame

# Importar la clase CSprite.
from api.CSprite import *

# Importar el manager de enemigos para detectar colisiones.
from api.CEnemyManager import *

# Importar la clase para guardar datos.
from game.CGameData import *

# La clase CPlayerBullet hereda de CSprite.
class CPlayerBullet(CSprite):

    # Constructor.
    # Recibe como parámetro quién disparó la bala.
    def __init__(self, aPlayer):
        CSprite.__init__(self)

        img = pygame.image.load("assets/images/player_bullet.png")
        img = img.convert_alpha()
        self.setImage(img)

        # Guardar la referencia a quién disparó la bala.
        self.mPlayer = aPlayer

    # Mover el objeto.
    def update(self):

        CSprite.update(self)

        # Detectar choque contra los enemigos.
        # Si la bala choca un enemigo, mueren ambos.
        enemy = CEnemyManager.inst().collides(self)
        if enemy != None:
            if enemy.isStateNormal():
                print("*****COLISION ENTRE BALA DEL JUGADOR Y
ENEMIGO")
                enemy.hit()
                self.die()

            if self.mPlayer.isPlayerOne():
                CGameData.inst().addScore1(enemy.getScore())
            else:
                CGameData.inst().addScore2(enemy.getScore())

    # Dibuja el objeto en la pantalla.

```

```

# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    CSprite.render(self, aScreen)

# Liberar lo que haya creado el objeto.
def destroy(self):

    CSprite.destroy(self)

```

En el constructor se almacena la referencia al jugador que dispara la bala, y en la función `update()`, si la bala choca con un enemigo, se usa la referencia al jugador que la disparó, para preguntar si es el primer jugador (si no es el primero, es el segundo) y decidir a quién le asignamos el score.

Nota: Recuerda que como en la lista de enemigos (`CEnemyManager`) tenemos tanto naves enemigas como balas enemigas, es posible que una bala del jugador choque contra una bala enemiga. Cuando invocamos a la función `addScore()`, usamos `enemy.getScore()` para obtener su puntaje, por lo cual puede ser llamado tanto para naves como para balas enemigas (en este caso el score será cero). Debemos asegurarnos que esta función esté en ambos objetos. Por esta razón, hemos puesto el manejo del score en la clase `CSprite`, así todos los sprites del juego tienen score. Si no hacemos esto, y hubiéramos puesto el score en la clase `CNave`, cuando la bala choca con una bala enemiga, intenta llamar a esta función y si no existe da error. Hay que tener cuidado con estas cosas. Por eso es muy importante diseñar bien las clases bases del motor (las clases en la carpeta `api`).

Controlando las Vidas de los Jugadores

Lo que haremos ahora es tener funcional el contador de vidas. Cada vez que se pierda una vida, el contador de vidas debe decrementarse.

Abre el ejemplo que se encuentra en la carpeta `capitulo_15\004_vidas_funcional`. Si ejecutas el juego, verás que cuando el jugador pierde una vida, se decrementa el contador de vidas y que cuando no le quedan más vidas no vuelve a aparecer la nave. ¿Cómo hacemos esto?

Lo primero, como siempre, es pensar y analizar cuándo es que el contador de vidas debe bajar. Cuando al jugador lo alcanza una bala, parpadea, luego explota y luego vuelve a aparecer (mira la máquina de estados del jugador si no lo recuerdas). Entonces, en el momento en el cual se pasa del estado `CPlayer.EXPLODING` a `CPlayer.START` es cuando decrementamos la cantidad de vidas. En este mismo momento, si las vidas ya son cero, no pasamos al estado `CPlayer.START`, sino que lo pasaremos a un nuevo estado que llamaremos `CPlayer.GAME_OVER`. Este es un estado nuevo que agregamos, en el cual el

jugador no hace nada (en la función `render()` no se dibuja y en `update()` no hace nada).

Veamos el código de la función de la clase `CPlayer`, mostrando en negrita los agregados. Mira en la función `update()`, en el momento en que termina la explosión y decrementamos el contador de vidas, chequeando si la nave vuelve a comenzar o no:

```
...

# La clase CPlayer hereda de CAnimatedSprite.
class CPlayer(CAnimatedSprite):

    ...

    # Máquina de estados.
    NORMAL = 0
    DYING = 1
    EXPLODING = 2
    START = 3
    GAME_OVER = 4

    ...

    # Mover el objeto.
    def update(self):

        # Invocar update() de CAnimatedSprite.
        CAnimatedSprite.update(self)

        # Lógica del estado normal.
        if self.getState() == CPlayer.NORMAL:
            if CEnemyManager.inst().collides(self):
                self.setState(CPlayer.DYING)
                return
            self.move()

        # Lógica del estado muriendo.
        elif self.getState() == CPlayer.DYING:
            if (self.getTimeState() > CPlayer.TIME_DYING):
                self.setState(CPlayer.EXPLODING)
                return

        # Lógica del estado explotando.
        elif self.getState() == CPlayer.EXPLODING:
            if self.isEnded():
                # En este punto comienza una nueva vida.
                if self.isPlayerOne():
                    if (CGameData.inst().getLives1() == 0):
```

```

        print("LAYER 1 NO JUEGA MAS")
        self.setState(CPlayer.GAME_OVER)
    else:
        CGameData.inst().addLives1(-1)
        self.setState(CPlayer.START)
    else:
        if (CGameData.inst().getLives2() == 0):
            print("PLAYER 2 NO JUEGA MAS")
            self.setState(CPlayer.GAME_OVER)
        else:
            CGameData.inst().addLives2(-1)
            self.setState(CPlayer.START)
    return

# Lógica del estado start.
elif self.getState() == CPlayer.START:
    if (self.getTimeState() > CPlayer.TIME_START):
        self.setState(CPlayer.NORMAL)
        return
    self.move()

# En el estado game over no hace nada.
elif self.getState() == CPlayer.GAME_OVER:
    return

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # En el estado game over la nave no se dibuja.
    if self.getState() == CPlayer.GAME_OVER:
        return

    ...

...

# Establece el estado actual e inicializa
# las variables correspondientes al estado.
def setState(self, aState):

    ...

    elif self.getState() == CPlayer.GAME_OVER:
        self.setVisible(False)

...

```

El código es bastante simple. Es sencillo de seguir. Si es el primer jugador se tiene en cuenta el score del primer jugador. Esto se podría haber hecho con una variable que lleve las vidas en la propia clase `CPlayer` y pasarle el valor a `CGameData`. Lo hemos hecho de esta manera por claridad.

En la función `update()` cuando el estado es `CPlayer.GAME_OVER`, ya no se hace mas nada (por eso solo ejecuta la sentencia `return`). Se retorna de la función sin hacer nada.

En la función `render()` el jugador no se muestra si el jugador está en estado `CPlayer.GAME_OVER`, y en la función `setState()` se pone el jugador invisible cuando se pasa a este estado:

Al ejecutar el juego, si perdemos todas las vidas, no podemos seguir jugando y las nave no aparece más. Es hora de trabajar en qué hacemos si ganamos (cuando matamos a todos los enemigos) o perdemos (cuando perdemos todas las vidas).

Condición de Ganar y Condición de Perder

En este punto, estamos listos para agregar al juego el control de la condición de ganar (denominada *win condition*) y de la condición de perder (denominada *lose condition*). Esto es, chequear cuando se da la condición para ganar el juego (o el nivel) y cuando se da la condición para perder el juego.

En todos los juegos, la condición para ganar y perder son diferentes, dado que depende del juego que estemos haciendo. En este caso, la condición para ganar el juego es que no queden más enemigos en pantalla y la condición para perder el juego es que ambos jugadores hayan muerto completamente.

Pensando en términos de programación, esto se traduce en:

Ganar: La cantidad de enemigos en la clase `CEnemyManager` es **cero**.

Perder: Ambos jugadores están en estado `CPlayer.GAME_OVER`.

Lo que ocurra primero hará que pasemos a la pantalla de ganar o perder el juego. Por ahora solamente chequeamos la condición de ganar y de perder y mostraremos un mensaje en la consola. Más adelante cambiaremos de pantalla cuando ocurra una de estas condiciones.

Veamos el ejemplo ubicado en la carpeta `capitulo_15\005_condicion_ganar_perder`. En el programa `main.py`, en la función `update()`, chequeamos ambas condiciones:

```
# Correr la lógica del juego.
def update():
    ...
```

```

# Actualizar los enemigos.
CEnemyManager.inst().update()

# Lógica de los jugadores.
player1.update()
player2.update()

# Detectar la colision entre los dos jugadores.
if player1.collides(player2):
    print("COLISION ENTRE LOS JUGADORES")
else:
    print("...")

if player1.isGameOver() and player2.isGameOver():
    print("AMBOS JUGADORES MUEREN - LOSE CONDITION")
if CEnemyManager.inst().getLength() == 0:
    print("TODOS LOS ENEMIGOS MUEREN - WIN CONDITION")

...

```

Como vemos, el chequeo es simple y estamos usando funciones *descriptivas* (que tienen nombres que lo hagan fácil de leer). En la clase `CPlayer`, agregamos una función llamada `isGameOver()` que nos dice si el jugador está muerto o no. Esta función retorna `True` si el estado del jugador es `CPlayer.GAME_OVER`. Esto es, cuando ha terminado de jugar todas sus vidas.

```

# Retorna True si el jugador está en estado game over (si ya no tiene
vidas).
def isGameOver(self):
    return self.mState == CPlayer.GAME_OVER

```

En la clase `CManager` en la carpeta `api` (recuerda que es la clase base del managers de enemigos) agregamos una función `getLenght()`, que simplemente retorna la cantidad de elementos del array. Cuando se trata del manager de enemigos, esto significa la cantidad de enemigos en pantalla.

```

# Retorna la cantidad de objetos del manager.
def getLength(self):
    return len(self.mArray)

```

De esta forma, básicamente hemos terminado lo que es el juego en sí. Ahora pasaremos en los siguientes capítulos al manejo de las diferentes pantallas que tiene el juego (el menú, los créditos, la pantalla de ayuda, etc), luego al manejo del mouse, y luego de eso, vamos a pulir

el juego y a finalizarlo.

16. Leyendo el Mouse

En este capítulo veremos cómo leer el movimiento del *mouse*, lo que hace el jugador con él y qué botones toca. Utilizaremos el mouse para determinados tipos de juegos (*puzzles* por ejemplo), pero también lo utilizaremos para pulsar los botones de las pantallas que haremos para nuestros juegos (por ejemplo, la pantalla del menú principal).

Eventos del Mouse

¿Recuerdas cómo leemos el teclado, capturando los eventos del teclado en el loop principal?, ahora capturaremos los eventos relacionados al *mouse* de forma similar.

A forma de recordatorio, cada vez que ocurre un evento en el sistema operativo, el evento es enviado a la ventana del juego. Estos eventos ocurren cuando se pulsa una tecla, o se cierra la ventana, o como veremos a continuación, cuando se pulsa un botón del mouse, se mueve el mouse, etc.

Abre el proyecto ubicado en la carpeta `capitulo_16\001_eventos_mouse`. Este proyecto contiene solamente un archivo `main.py`, dado que es simplemente un programa para mostrar en la consola mensajes de texto sobre los eventos que capturamos en el loop principal, al igual que el que hicimos cuando vimos los eventos del teclado,.

Veamos el código de `main.py` para ver cómo capturamos los eventos en el loop principal:

```
while not salir:

    clock.tick(30)

    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():

        # Evento cuando se pulsa el botón del mouse.
        if event.type == pygame.MOUSEBUTTONDOWN:
            print("Se pulsa el botón del mouse: ",
                  pygame.mouse.get_pos())
        # Evento cuando se libera el botón del mouse.
        if event.type == pygame.MOUSEBUTTONUP:
            print("Se libera el botón del mouse: ",
                  pygame.mouse.get_pos())
        # Evento cuando se mueve el mouse.
        if event.type == pygame.MOUSEMOTION:
            (mouseX, mouseY) = pygame.mouse.get_pos()
            print("Se mueve el mouse: (%d, %d)" % (mouseX, mouseY))
```

```

...

# Saber en el momento que botones del mouse están pulsados.
# mouse_buttons = pygame.mouse.get_pressed()
# print(mouse_buttons)

```

Al igual que con los eventos del teclado, cuando el jugador hace algo con el *mouse*, se envía un evento a la ventana del juego. Cuando se pulsa el botón del mouse, `event.type` equivale a `pygame.MOUSEBUTTONDOWN` y cuando se libera el botón, `event.type` equivale a `pygame.MOUSEBUTTONUP`. Cuando se mueve el mouse, se recibe un evento donde `event.type` equivale a `pygame.MOUSEMOTION`.

Usando la función `pygame.mouse.get_pos()` obtenemos una *tupla* con la coordenada *x* e *y* del *mouse*. Para cargar las coordenadas del mouse en nuestras variables `mouseX` y `mouseY`, hacemos:

```
(mouseX, mouseY) = pygame.mouse.get_pos()
```

Esta línea carga la coordenada *x* del mouse en `mouseX` y la coordenada *y* del mouse en `mouseY`. Si estuviéramos haciendo un juego en donde usamos el mouse para seleccionar y mover objetos, necesitamos estas coordenadas para saber qué objeto está debajo del mouse.

Detectando los Botones del Mouse

Es posible detectar en cualquier momento (sin necesidad de capturar el evento) qué botones están siendo presionados o no en el *mouse*, usando la función `pygame.mouse.get_pressed()`. En el ejemplo anterior, se encuentran comentadas estas líneas. Elimina los comentarios de esas líneas y corre el ejemplo para ver en la consola estos valores.

```

# Saber en el momento que botones del mouse están pulsados.
# mouse_buttons = pygame.mouse.get_pressed()
# print mouse_buttons

```

Esta función retorna una tupla de tres valores, uno para cada botón del mouse. Mira la siguiente figura:

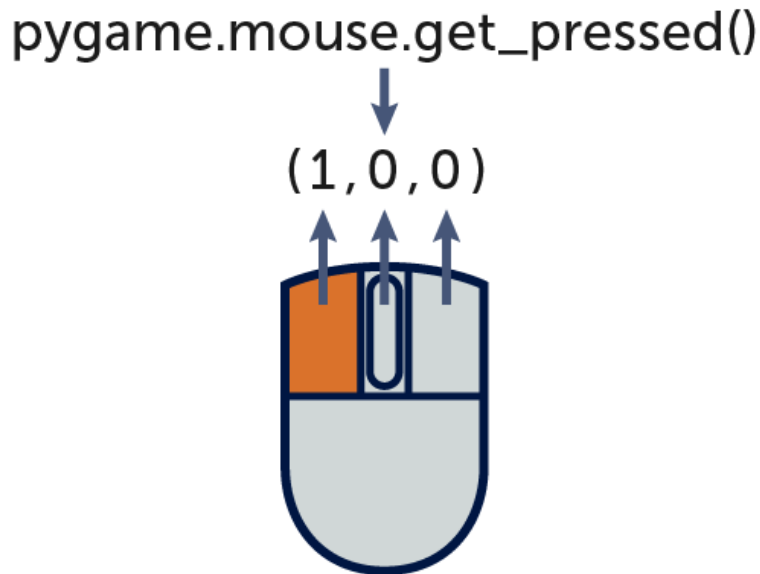


Figura 16-1: La función `pygame.mouse.get_pressed()` para leer los botones.

El valor de cada elemento de la tupla es *uno* si el botón está presionado y *cero* si no lo está. De la misma forma que para el teclado podemos utilizar `pygame.key.get_pressed()` para saber el estado de cada tecla, para el mouse usamos `pygame.mouse.get_pressed()`.

Nota: Podemos usar estas funciones, que hacen innecesario capturar los eventos, pero esto es algo que nos brinda *Pygame*. Si queremos en un futuro portar nuestro juego a otro lenguaje como *Java* o *C++*, debemos saber y conocer cómo funciona el sistema de eventos, y en general el funcionamiento del sistema.

La Clase `CMouse`

Lo que haremos, al igual que como hicimos con la clase `CKeyboard`, haremos una clase `CMouse` que maneje el puntero del mouse y la información relacionada al mismo, y tendrá funciones como `click()`, que servirá para saber cuando se hace un *click* con el mouse (un *click* es pulsar el botón y luego liberarlo).

Nota: Siempre que tratamos un tema nuevo, como por ejemplo el *mouse*, tenemos funciones de *Pygame* que nos permiten hacer ciertas cosas, pero no todo. El caso de la función `click()` es un claro ejemplo. *Pygame* no nos da nada para saber cuando el usuario hace *click* (cuando se pulsa el botón por primera vez). Siempre debemos escribir una clase para abstraer y para extender la funcionalidad básica y que sea aplicable a nuestros juegos. A esto se le denomina implementar el *motor de juegos (engine)*. Nosotros estamos poniendo las clases del motor en la carpeta `api`.

Veamos el ejemplo ubicado en la carpeta `capitulo_16\002_clase_mouse_y_puntero`. Por ahora veamos solamente la clase `CMouse`, que es similar a la clase que hicimos para el

teclado:

```
# -*- coding: utf-8 -*-

#-----
# Clase CMouse.
# Clase para manejar el estado de los botones del mouse y la
# posición.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

class CMouse(object):

    mInstance = None
    mInitialized = False

    mLeftPressed = False
    mRightPressed = False
    mCenterPressed = False

    mLeftPressedPreviousFrame = False

    def __new__(self, *args, **kwargs):
        if (CMouse.mInstance is None):
            CMouse.mInstance = object.__new__(self, *args, **kwargs)
            self.init(CMouse.mInstance)
        else:
            print("Cuidado: Mouse(): No se debería instanciar más de
una vez esta clase. Usar Mouse.inst().")
            return CMouse.mInstance

    @classmethod
    def inst(cls):
        if (not cls.mInstance):
            return cls()
        return cls.mInstance

    def init(self):
        if (CMouse.mInitialized):
            return
        CMouse.mInitialized = True
```

```

CMouse.mLeftPressed = False
CMouse.mRightPressed = False
CMouse.mCenterPressed = False
CMouse.mLeftPressedPreviousFrame = False

def update(self):
    CMouse.mLeftPressedPreviousFrame = CMouse.mLeftPressed
    CMouse.mLeftPressed = pygame.mouse.get_pressed()[0]
    CMouse.mRightPressed = pygame.mouse.get_pressed()[2]
    CMouse.mCenterPressed = pygame.mouse.get_pressed()[1]

def leftPressed(self):
    return CMouse.mLeftPressed

def rightPressed(self):
    return CMouse.mRightPressed

def centerPressed(self):
    return CMouse.mCenterPressed

def click(self):
    return CMouse.mLeftPressed == False and
CMouse.mLeftPressedPreviousFrame == True

def getPos(self):
    return pygame.mouse.get_pos()

def getX(self):
    return pygame.mouse.get_pos()[0]

def getY(self):
    return pygame.mouse.get_pos()[1]

def destroy(self):
    CMouse.mInstance = None

```

Esta clase es parecida a la clase del teclado. Es una clase *singleton* porque la utilizaremos sin necesitar una instancia de esa clase. Recuerda que las clases *singleton* las podemos acceder desde cualquier parte del código sin necesitar crearla cada vez que la usemos (se crea una vez sola).

En la función `update()` se guardan en variables los estados de cada uno de los botones del *mouse*. Luego tenemos varias funciones que son sencillas, para obtener el estado de cada uno de los botones, la posición del mouse (`getX()` y `getY()`) y la más importante es la función `click()`, que retornará `True` solamente en el frame en el cual se presiona el botón del mouse y antes no estaba presionando, o sea, en el frame en el cual se pulsa el botón del

mouse. Luego esta función no vuelve a retornar `True` hasta que se suelte el botón y se vuelva a presionar.

En la clase `main.py`, debemos importar la clase, y en `update()` invocar a la función `update()` de la clase `Mouse`. Esto lo hacemos una vez, y ya nos queda para todos los juegos, porque es parte del motor. Es necesario hacer esta llamada porque esta función actualiza la posición del mouse y el estado de sus botones en cada cuadro:

```
...

# Importar la clase del mouse.
from api.CMouse import *

...

# Correr la lógica del juego.
def update():
    ...

    # Llamar a update() de CKeyboard.
    CKeyboard.inst().update()

    # Llamar a update() de CMouse.
    CMouse.inst().update()

...

...
```

Con esto ya tenemos el mouse funcional. Ahora lo que nos queda es crear un sprite para el puntero del mouse, que es lo que haremos a continuación.

El Puntero del Mouse

Como seguramente has visto si has jugado juegos en *PC*, los juegos no utilizan el puntero del *mouse* del sistema (la flechita blanca), sino que implementan su propio puntero para que vaya con el estilo gráfico del juego, ya que influye en la inmersión. Vamos a colocar nuestro propio sprite como puntero del mouse.

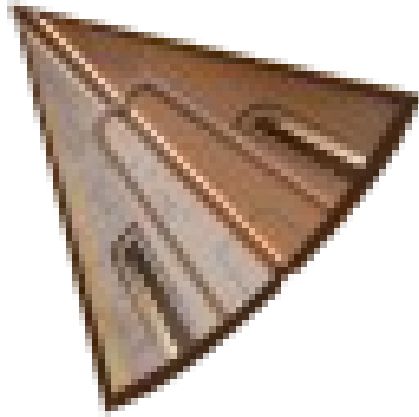


Figura 16-2: El sprite del puntero del mouse.

La imagen del cursor la ponemos como siempre en la carpeta `assets/images`. Copia esta imagen (`cursor.png`) a tu propio proyecto. Vamos a crear una clase para manejar este sprite, que llamaremos `CMousePointer`.

Lo que hacemos para implementar el cursor propio para el mouse es crear un sprite como siempre (como es para el mouse, lo hacemos en el programa `main.py`) y en cada frame (en cada `update()`) lo ponemos en la posición donde se encuentra el mouse.

Veamos la clase `CMousePointer` que es un sprite como cualquier otro en el juego (hereda de `CSprite`):

```
# -*- coding: utf-8 -*-

#-----
# Clase MousePointer.
# Sprite del puntero del mouse.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# Importar la clase CSprite.
from api.CSprite import *

# Importar la clase CMouse.
from api.CMouse import *

# La clase hereda de CSprite.
class CMousePointer(CSprite):
```

```

def __init__(self):
    CSprite.__init__(self)

    img = pygame.image.load("assets/images/cursor.png")
    img = img.convert_alpha()
    self.setImage(img)

def update(self):
    CSprite.update(self)

    # Colocar el sprite en donde está el mouse.
    self.setXY(CMouse.inst().getX(), CMouse.inst().getY())

def render(self, aScreen):
    CSprite.render(self, aScreen)

def destroy(self):
    CSprite.destroy(self)

```

Como vemos, la única particularidad de este sprite es que en la función `update()` se pone el sprite en la posición en donde está el cursor del sistema. El resto es igual a como se maneja cualquier otro sprite del juego.

En el programa `main.py`, es donde manejamos el cursor. Lo hemos hecho en el programa principal, porque el cursor lo necesitaremos en todas las pantallas del juego. Para tener un puntero, lo que hacemos es básicamente lo de siempre: crear el sprite en `init()`, actualizarlo en `update()`, dibujarlo en `render()` y eliminarlo en `destroy()`. Veamos los lugares en `main.py` donde se maneja el sprite del puntero del mouse:

```

...

# Importar la clase del puntero del mouse.
from game.CMousePointer import *

...

# Sprite del puntero del mouse.
mousePointer = None

# Función de inicialización.
def init():

    global screen
    global imgBackground
    global imgSpace

```



```

global clock
global player1
global player2
global mousePointer

# Inicializar Pygame.
pygame.init()

...

# Inicializar los datos del juego.
CGameData.inst().setScore1(0)
CGameData.inst().setLives1(3)
CGameData.inst().setScore2(0)
CGameData.inst().setLives2(3)

# Ocultar el mouse del sistema.
pygame.mouse.set_visible(False)
# Crear el sprite puntero del mouse.
mousePointer = CMousePointer()

# Correr la lógica del juego.
def update():
    global salir
    global screen
    global isFullscreen

    # Timer que controla el frame rate.
    clock.tick(60)

    # Llamar a update() de CKeyboard.
    CKeyboard.inst().update()

    # Llamar a update() de CMouse.
    CMouse.inst().update()

    # Actualizar el sprite del puntero del mouse.
    mousePointer.update()

    ...

# Dibujar el frame y actualizar la pantalla.
def render():
    # Dibujar el fondo.
    screen.blit(imgBackground, (0, 0))

    ...

```

```

# Dibujar el puntero del mouse.
mousePointer.render(screen)

# Actualizar la pantalla.
pygame.display.flip()

# Función de destrucción.
def destroy():
    global player1
    global player2
    global mousePointer

    # Destruir la nave de los jugadores.
    player1.destroy()
    player1 = None
    player2.destroy()
    player2 = None

    # Destruir las balas.
    CBulletManager.inst().destroy()

    # Destruir los enemigos.
    CEnemyManager.inst().destroy()

    # Destruir el puntero del mouse.
    mousePointer.destroy()
    mousePointer = None
    pygame.mouse.set_visible(True)

    # Cerrar Pygame y liberar los recursos que pidió el programa.
    pygame.quit()

...

```

Como vemos en el código, lo que hacemos es básicamente lo que hacemos con cualquier sprite. En la función `render()`, dibujamos el sprite del mouse a lo último, de modo que aparezca por encima de todos los gráficos del juego. En la función `update()` actualiza su posición según la posición del mouse.

Ocultar o Hacer Visible el Mouse del Sistema

Cuando mostramos un sprite propio como el puntero del mouse, debemos ocultar el puntero del mouse del sistema. De esta forma, el puntero del sistema (la flecha blanca) será invisible, y en su lugar se mostrará nuestro sprite.

Para ocultar el puntero del mouse usamos la siguiente sentencia:

```
pygame.mouse.set_visible(False)
```

Esto hace que en la posición del mouse no se muestre la flecha blanca, aunque el mouse sigue estando ahí (imagínate una flecha invisible). Como nuestro sprite lo colocamos en la posición del mouse en cada `update()`, solamente se muestra nuestro sprite para el puntero del mouse.

Para mostrar el puntero del sistema hacemos lo mismo pero le pasamos `True` como parámetro a la función:

```
pygame.mouse.set_visible(True)
```

Por último, en la función `destroy()`, eliminamos lo que hayamos creado (el sprite del puntero por ejemplo) y ponemos visible nuevamente el puntero del sistema.

Nota: Recuerda que en la función `destroy()` debemos siempre liberar toda la memoria creada, liberando las imágenes, destruyendo los objetos creados y generalmente liberando lo creado en la función `init()`.

Con esto hemos terminado el manejo del mouse. Tenemos nuestro propio puntero funcionando y sabemos detectar clicks del mouse. En el capítulo siguiente implementaremos las diferentes pantallas del juego.

17. Las Pantallas del Juego

En este capítulo implementaremos las diferentes pantallas que va a tener el juego: el menú principal, la pantalla de créditos, la pantalla de ayuda, etc.

Para esto, haremos una *máquina de estados* del juego. Cada estado corresponderá a cada una de las pantallas del juego, y cuando pasemos de un estado a otro, lo que haremos será pasar de una pantalla a otra. Si recordamos cuando hicimos la máquina de estados del jugador y de los enemigos, una máquina de estados se dibuja como un diagrama en donde se muestran los *estados* y cómo se pasa de un estado a otro (las *transiciones* entre estados).

Pues bien, esto nos sirve perfectamente para las pantallas del juego. Si dibujamos las pantallas del juego y cómo se pasa de una a otra, nos quedará el diagrama de la máquina de estados de las pantallas del juego. Comenzaremos con el menú y el juego.

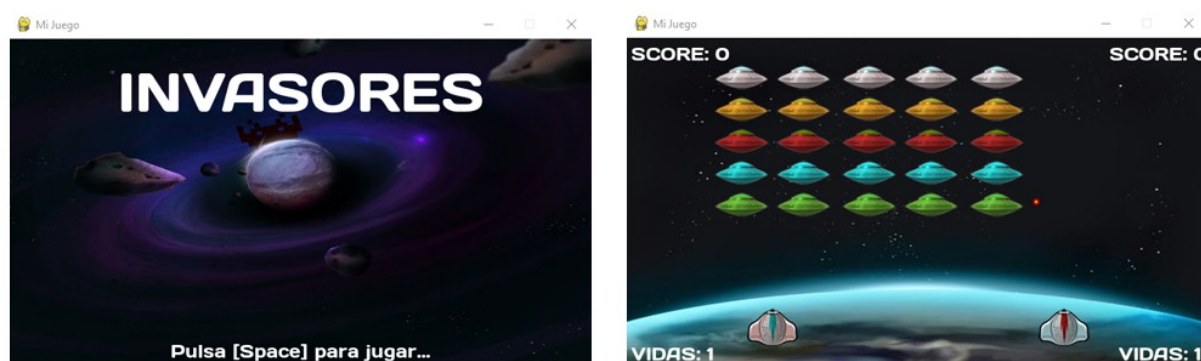


Figura 17-1: Las pantallas del juego.

En la mayoría de las pantallas, pasamos de una a otra pantalla seleccionando los botones correspondientes o pulsando alguna tecla. Cuando estamos en el juego y termina el mismo, volvemos al menú principal. Hay diferentes condiciones para las transiciones entre pantallas, pero la máquina de estados funcionará de forma similar a las que hemos implementado antes. A continuación comenzaremos a programar la máquina de estados del juego.

La clase CGame

Para poder tener varias pantallas en el juego, y poder pasar entre una y otra, debemos tener una clase que controle eso. Esa clase tendrá el estado actual del juego y una función llamada `setState()` para pasar de un estado a otro. Esta clase será una clase *singleton* y la denominaremos `CGame`. La pondremos en la carpeta `api`, porque será una clase que pertenece al motor del juego. De hecho, la clase `CGame` va a sustituir a `main.py` como clase principal, como ya veremos.

La clase `CGame` se encargará de controlar el estado actual del juego, y tendrá una función `setState()` para cambiar entre estados del juego (cambiar entre pantallas). Al cambiar de pantalla, se va a invocar a la función `destroy()` del estado actual (para eliminar lo creado al momento) y se invoca a `init()` en el nuevo estado (para crear lo necesario para la nueva pantalla). Ya veremos el código en un momento.

La clase `CGame` también llevará el control del puntero del mouse, y en general de todo lo que afecte al juego como un todo (o sea, a las cosas a nivel de la *aplicación*).

La clase `CGameState`

La clase `CGameState` será la clase base de cualquier *estado del juego*. Cuando decimos “estado del juego” nos estamos refiriendo a una de las pantallas del juego. Esta clase, tendrá cuatro funciones: `init()`, `update()`, `render()` y `destroy()`.

La clase `CGame` se encargará de que cada estado del juego llame a `init()` cuando se crea, luego entra en el ciclo `update()` / `render()` y al final, al salir del estado se llama a `destroy()` para eliminar lo creado en la pantalla.

Cualquier clase que implemente una de las pantallas del juego, será un estado del juego y esa clase heredarán de `CGameState`. Esta clase tendrá las funciones vacías, porque implementar estas funciones es responsabilidad de las clases de estados que hagamos.

La clase `CLevelState`

Hasta ahora teníamos todo el código en el programa `main.py`, pero esto no es conveniente si vamos a tener muchas pantallas en el juego. Cuando ejecutamos el programa, ahora ya arranca en el juego, pero esto no va a ser así. El juego va a empezar en una pantalla con el menú principal. Luego si elegimos la opción de jugar, pasaremos al juego.

Quedaría muy complicado y largo el código en la clase `main.py` si programáramos todas las pantallas en ese programa sin usar estados y/o clases. Entonces, lo que haremos es separar cada pantalla (estado) en clases diferentes y de esta forma el código quedará más simple y fácil de modificar y en el futuro será más fácil agregar pantallas.

Entonces, lo que haremos será poner el código relacionado al juego en sí (el nivel) en la clase `CLevelState`. Esta clase se encarga del juego cuando el jugador está jugando, y corresponde a la clase del juego o el nivel en sí, por eso su nombre. Esto es lo que haremos a continuación.

La Máquina de Estados del Juego

Para que podamos pasar entre pantallas y tengamos diferentes estados en el juego, debemos programar el motor para que esto sea posible. Debemos implementar las clases `CGame`, `CGameState` y `CLevelState`. Luego tendremos un programa `main.py` con un código muy chico que lo que hace es inicializar la máquina de estados del juego y poner el juego en el estado inicial.

Nota: La reestructura que haremos en el siguiente ejemplo no es tan sencilla de entender sin examinar bien el código, pero nos permitirá agregar muy fácil el resto de las pantallas del juego. Como los cambios más importantes están en el motor, es posible usarlos sin entender del todo cómo funciona en detalle. Puedes construir un juego partiendo de los ejemplos, y dejar para más adelante el entendimiento completo del funcionamiento del motor.

Abre y ejecuta el ejemplo ubicado en la carpeta `capitulo_17\001_estados_del_juego_level_state`.

Como puedes ver, el juego es exactamente el mismo del capítulo anterior, pero se han implementado las clases que han sido descritas arriba.

Nota: Parece que hacemos un montón de cosas complicadas y el juego sigue haciendo lo mismo que antes, pero no es así. De esta forma en que componemos el código nos será muy fácil agregar nuevas pantallas. De todas formas, recuerda que mientras aprendemos cosas nuevas estamos escribiendo código de base y reestructurando las clases a medida que avanzamos, pero cuando hagamos un juego nuevo, será mucho más fácil, porque ya tendremos todas las cosas de base funcionando en un motor.

Ahora veamos el código de cada una de las nuevas clases.

La clase `CGameState`, es simplemente un *template* (una clase base) de la cual van a heredar los estados (las pantallas del juego). Esta clase va en la carpeta `api`, dado que es parte del motor. Tiene las funciones `init()`, `render()`, `update()` y `destroy()` vacías porque la clase que la hereda debe implementarlas. Veamos el código:

```
# -*- coding: utf-8 -*-

#-----
# Clase CGameState.
# Clase base de todos los estados del juego. Los estados del juego
heredan de
# esta clase porque se les invoca los métodos init(), update(),
render() y destroy()
# desde la clase CGame que es quien maneja los estados.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

class CGameState(object):
```

```

def __init__(self):
    print("GameState constructor")

def init(self):
    print("GameState init()")

def update(self):
    print("GameState update()")

def render(self):
    print("GameState render()")

def destroy(self):
    print("GameState destroy()")

```

Ahora hemos puesto sentencias `print` para probar que el programa funciona correctamente e invoca a estas funciones. Luego pondremos sentencias `pass` en lugar de `print`, si la función no hace nada.

La clase `CGame`, contiene la lógica de la aplicación. Esta clase ahora tiene el código que antes tenía `main.py`, sacando todo lo relacionado al juego en sí, que eso ya no irá en el programa principal. Se mueve todo el código de `main.py` a la clase `CGame`.

La clase `CGame` también es parte del motor, por lo que la colocamos en la carpeta `api`. Veamos el código:

```

# -*- coding: utf-8 -*-

#-----
# Clase CGame.
# Clase que maneja los estados del juego a nivel de aplicación.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

# Importar Pygame.
import pygame

# Importar la clase CKeyboard
from api.CKeyboard import *

# Importar la clase del mouse.
from api.CMouse import *

# Importar la clase del puntero del mouse.

```

```

from game.CMousePointer import *

# Importar el garbage collector.
import gc

# Importar la clase de constantes.
from api.CGameConstants import *

class CGame(object):

    mInstance = None
    mInitialized = False

    mScreen = None
    mClock = None
    mSalir = False
    mMousePointer = None

    # Pantalla (estado) actual del juego.
    mState = None

    SCREEN_WIDTH = 0
    SCREEN_HEIGHT = 0
    RESOLUTION = 0

    # Control del modo ventana o fullscreen.
    mIsFullscreen = False

    def __new__(self, *args, **kwargs):
        if (CGame.mInstance is None):
            CGame.mInstance = object.__new__(self, *args, **kwargs)
            self.init(CGame.mInstance)
        else:
            print("Cuidado: Game(): No se debería instanciar más de
una vez esta clase. Usar Game.inst().")
            return self.mInstance

    @classmethod
    def inst(cls):
        if (not cls.mInstance):
            return cls()
        return cls.mInstance

    # Función de inicialización.
    def init(self):
        if (CGame.mInitialized):
            return
        CGame.mInitialized = True

```



```

# Definir ancho y alto de la pantalla.
CGame.SCREEN_WIDTH = CGameConstants.SCREEN_WIDTH
CGame.SCREEN_HEIGHT = CGameConstants.SCREEN_HEIGHT
CGame.RESOLUTION = (CGame.SCREEN_WIDTH, CGame.SCREEN_HEIGHT)

# Inicializar Pygame.
pygame.init()

# Inicializar el mixer de audio de Pygame.
pygame.mixer.init()

# Poner el modo de video en ventana e indicar la resolución.
CGame.mScreen = pygame.display.set_mode(CGame.RESOLUTION)
# Poner el título de la ventana.
pygame.display.set_caption("Mi Juego")

# Crear la superficie del fondo o background.
CGame.mBackground = pygame.Surface(self.mScreen.get_size())
CGame.mBackground = self.mBackground.convert()

# Inicializar el reloj.
CGame.mClock = pygame.time.Clock()

# Inicializar la variable de control del game loop.
CGame.mSalir = False

# Sprite del puntero del mouse.
CGame.mMousePointer = CMousePointer()
# Ocultar el mouse del sistema.
pygame.mouse.set_visible(False)

CGame.mState = None

# Función para cambiar entre pantallas (estados) del juego.
def setState(self, aState):
    if (CGame.mState != None):
        CGame.mState.destroy()
        CGame.mState = None
        # Liberar memoria.
        print(gc.collect(), " objetos borrados.")

    CGame.mState = aState
    CGame.mState.init()

# Game loop del juego.
def gameLoop(self):

```

```

        while not self.mSalir:
            self.update()
            self.render()

# Correr la lógica del juego.
def update(self):

    # Timer que controla el frame rate.
    CGame.mClock.tick(60)

    # Llamar a update() de CKeyboard.
    CKeyboard.inst().update()

    # Llamar a update() de CMouse.
    CMouse.inst().update()

    # Actualizar el sprite del puntero del mouse.
    CGame.mMousePointer.update()

    # Procesar los eventos que llegan a la aplicación.
    for event in pygame.event.get():

        # Si se cierra la ventana se sale del programa.
        if event.type == pygame.QUIT:
            CGame.mSalir = True

        # Si se pulsa la tecla [Esc] se sale del programa.
        if event.type == pygame.KEYUP:
            if (event.key == pygame.K_ESCAPE):
                CGame.mSalir = True

        # Si se pulsa la tecla [F], se cambia entre ventana y
fullscreen.
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_f:
                CGame.mIsFullscreen = not self.mIsFullscreen
                if self.mIsFullscreen:
                    CGame.mScreen =
pygame.display.set_mode(CGame.RESOLUTION, pygame.FULLSCREEN)
                else:
                    CGame.mScreen =
pygame.display.set_mode(CGame.RESOLUTION)

        # Registrar cuando se apreta o suelta una tecla.
        if event.type == pygame.KEYDOWN:
            CKeyboard.inst().keyDown(event.key)
        if event.type == pygame.KEYUP:
            CKeyboard.inst().keyUp(event.key)

```

```

        # Actualizar el estado del juego.
        CGame.mState.update()

# Dibujar el frame y actualizar la pantalla.
def render(self):

    # Dibujar el fondo.
    CGame.mScreen.blit(self.mBackground, (0, 0))

    # Dibujar el estado del juego.
    CGame.mState.render()

    # Dibujar el puntero del mouse.
    CGame.mMousePointer.render(self.mScreen)

    # Actualizar la pantalla.
    pygame.display.flip()

def setBackground(self, aBackgroundImage):
    CGame.mBackground = None
    CGame.mBackground = aBackgroundImage
    self.blitBackground(CGame.mBackground)

def blitBackground(self, aBackgroundImage):
    CGame.mScreen.blit(aBackgroundImage, (0, 0))

# Obtener la referencia a la pantalla (usada para dibujar).
def getScreen(self):
    return CGame.mScreen

def destroy(self):
    if (CGame.mState != None):
        CGame.mState.destroy()
        CGame.mState = None

    CKeyboard.inst().destroy()
    CMouse.inst().destroy()

    # Destruir el puntero del mouse.
    CGame.mMousePointer.destroy()
    CGame.mMousePointer = None
    pygame.mouse.set_visible(True)

    CGame.mInstance = None

# Cerrar Pygame y liberar los recursos que pidió el programa.
pygame.quit()

```

Dando un pantallazo de la clase `CGame`, vemos que es bastante del código que estaba en `main.py`, que seguirá existiendo porque es el punto de arranque del programa, pero podemos decir que la clase `CGame` es la clase principal del juego.

Hemos cambiado algunos nombres de variables, sobre todo agregando el prefijo “m” (por *variable miembro*) que se le pone a cada variable de una clase para indicar que es una variable *miembro* de esa clase. Todo lo relacionado al manejo de la *aplicación* (o *programa*, como se le quiera decir) estará en la clase `CGame`.

La parte que conviene notar es que tenemos una variable `mState` que será una referencia a una clase con el estado del juego que crearemos para cada pantalla. Usaremos la función `setState()` para cambiar de estado. Esta función se encarga de invocar a la función `destroy()` en el estado actual e invocar a la función `init()` en el estado siguiente. También invoca a `gc.collect()` para liberar la memoria usada. Más adelante hablaremos sobre esto.

La clase `CGame` contiene el *loop principal* del juego, en el cual se procesan los eventos y se invoca a las funciones `update()` y `render()` del estado actual (la pantalla actual del juego). En breve estaremos creando esta clase para el estado del juego.

Por último, tenemos una función `setBackground()` que establece una imagen de fondo, y en la función `render()`, cuando se dibuja el juego, primero se dibuja el fondo, luego se dibujan los objetos en pantalla (invocando a `render()` en el estado (pantalla) del juego actual), y por último se dibuja el puntero sprite del puntero del mouse para que aparezca por encima de todo.

La función `setBackground()` sirve para poner una imagen de fondo desde un estado. Cada estado del juego (pantalla) tendrá su propia imagen de fondo como veremos más adelante.

Lo importante de tener una clase `CGame` que se encarga del *game loop* y de la *transición* entre estados, es que es fácil implementar varias pantallas en el juego. Lo que haremos ahora será ver la clase del estado del juego en sí. Esta clase se llama `CLevelState`.

Nota: Las clases relacionadas a las pantallas del juego las hemos puesto en la carpeta `game\states`. Recuerda que un *package* es un conjunto de clases en una carpeta, y para que *Python* reconozca la carpeta como *package* (para que reconozca a las clases en esa carpeta) debe contener el archivo `__init__.py`, que es un archivo vacío.

Veamos el código de la clase `CLevelState`:

```
# -*- coding: utf-8 -*-
```

```
#-----
```

```

# Clase CLevelState.
# Nivel del juego. Es el juego en sí.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

import pygame
from api.CGameState import *
from api.CGame import *
from game.CNave import *
from game.CPlayer import *
from api.CGameObject import *
from game.CGameData import *
from api.CGameConstants import *

class CLevelState(CGameState):

    mImgSpace = None
    mPlayer1 = None
    mPlayer2 = None

    def __init__(self):
        CGameState.__init__(self)
        print("LevelState constructor")

        self.mImgSpace = None
        self.mPlayer1 = None
        self.mPlayer2 = None

    # Función donde se inicializan los elementos necesarios del nivel.
    def init(self):
        CGameState.init(self)
        print("LevelState init")

        # Cargar la imagen del fondo. La imagen es de 640 x 360.
        self.mImgSpace =
pygame.image.load("assets/images/space_640x360.jpg")
        self.mImgSpace = self.mImgSpace.convert()

        # Dibujar la imagen cargada en la imagen de background.
        CGame.inst().setBackground(self.mImgSpace)

        # Crear la formación inicial.
        f = 0
        while f <= 4:
            c = 0

```

```

        while c <= 4:
            n = CNave(f)
            n.setXY(100 + (70 * c), 30 + (35 * f))
            n.setVelX(4)
            n.setVelY(0)
            n.setBounds(0, 0, CGame.SCREEN_WIDTH - n.getWidth(),
CGame.SCREEN_HEIGHT - n.getHeight())
            n.setBoundAction(CGameObject.BOUNCE)
            CEnemyManager.inst().add(n)
            c = c + 1
            f = f + 1

    # Crear el jugador 1.
    self.mPlayer1 = CPlayer(CPlayer.TYPE_PLAYER_1)
    self.mPlayer1.setXY(CGame.SCREEN_WIDTH / 4 -
self.mPlayer1.getWidth() / 2, CGame.SCREEN_HEIGHT -
self.mPlayer1.getHeight() - 20)
    self.mPlayer1.setBounds(0, 0, CGame.SCREEN_WIDTH -
self.mPlayer1.getWidth(), CGame.SCREEN_HEIGHT)

    # Crear el jugador 2.
    self.mPlayer2 = CPlayer(CPlayer.TYPE_PLAYER_2)
    self.mPlayer2.setXY(CGame.SCREEN_WIDTH / 4 * 3 -
self.mPlayer2.getWidth() / 2, CGame.SCREEN_HEIGHT -
self.mPlayer2.getHeight() - 20)
    self.mPlayer2.setBounds(0, 0, CGame.SCREEN_WIDTH -
self.mPlayer2.getWidth(), CGame.SCREEN_HEIGHT)

    # Ejecutar la música de background (loop) del juego.
    pygame.mixer.music.load("assets/audio/music_game.ogg")
    pygame.mixer.music.play(-1)
    # Poner el volumen de la música.
    pygame.mixer.music.set_volume(0.5)

    # Inicializar los datos del juego.
    CGameData.inst().setScore1(0)
    CGameData.inst().setLives1(3)
    CGameData.inst().setScore2(0)
    CGameData.inst().setLives2(3)

    # Actualizar los objetos del nivel.
    def update(self):
        CGameState.update(self)
        print("LevelState update()")

    # Actualizar los enemigos.
    CEnemyManager.inst().update()

```

```

# Mover las balas.
CBulletManager.inst().update()

# Lógica de los jugadores.
self.mPlayer1.update()
self.mPlayer2.update()

# Detectar la colision entre los dos jugadores.
if self.mPlayer1.collides(self.mPlayer2):
    print("COLISION ENTRE LOS JUGADORES")
else:
    print("...")

if self.mPlayer1.isGameOver() and self.mPlayer2.isGameOver():
    print("AMBOS JUGADORES MUEREN - LOSE CONDITION")
if CEnemyManager.inst().getLength() == 0:
    print("TODOS LOS ENEMIGOS MUEREN - WIN CONDITION")

# Dibujar el frame del nivel.
def render(self):
    CGameState.render(self)
    print("LevelState render()")

# Obtener la referencia a la pantalla.
screen = CGame.inst().getScreen()

# Dibujar los enemigos.
CEnemyManager.inst().render(screen)

# Dibujar las balas.
CBulletManager.inst().render(screen)

# Dibujar los jugadores.
self.mPlayer1.render(screen)
self.mPlayer2.render(screen)

# Dibujar el texto del score y las vidas.
self.drawText(screen, 5, 5, "SCORE: " +
str(CGameData.inst().getScore1()), 20, (255, 255, 255))
self.drawText(screen, 5, CGame.SCREEN_HEIGHT - 20 - 5, "VIDAS:
" + str(CGameData.inst().getLives1()), 20, (255, 255, 255))
self.drawText(screen, 530, 5, "SCORE: " +
str(CGameData.inst().getScore2()), 20, (255, 255, 255))
self.drawText(screen, 540, CGame.SCREEN_HEIGHT - 20 - 5,
"VIDAS: " + str(CGameData.inst().getLives2()), 20, (255, 255, 255))

# Destruir los objetos creados en el nivel.
def destroy(self):

```

```

CGameState.destroy(self)
print("LevelState destroy()")

# Destruir la nave de los jugadores.
self.mPlayer1.destroy()
self.mPlayer1 = None
self.mPlayer2.destroy()
self.mPlayer2 = None

self.mImgSpace = None

# Destruir las balas.
CBulletManager.inst().destroy()

# Destruir los enemigos.
CEnemyManager.inst().destroy()

CGameData.inst().destroy()

# Función para dibujar texto.
# Parámetros: La pantalla donde dibujar, coordenadas (x,y),
# texto ,tamaño de fuente y color del texto.
def drawText(self, aScreen, aX, aY, aMsg, aSize, aColor=(0,0,0)):
    font = pygame.font.Font("assets/fonts/days.otf", aSize)
    imgTxt = font.render(aMsg, True, aColor)
    aScreen.blit(imgTxt, (aX, aY))

```

La clase `CLevelState` corresponde a la pantalla (estado del juego) del juego en sí, cuando se está jugando un nivel del juego. Como vemos en el código, es el código relacionado al juego que estaba en `main.py`. De esta forma, hemos pasado de tener todo junto en `main.py`, a mover la parte que corresponde al motor a la clase `CGame` y el código que corresponde al nivel del juego a la clase `CLevelState`.

Por último, el programa `main.py` ahora queda muy simple. Veamos el código:

```

# -*- coding: utf-8 -*-

#-----
# Máquina de estados del juego. Clases CGame, CGameState y
CLevelState.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

```



```

from api.CGame import *
from game.states.CLevelState import *

# ===== Punto de entrada del programa. =====

# Inicializar los elementos necesarios del juego.
g = CGame()

# Crear el estado para el nivel del juego.
initState = CLevelState()
g.setState(initState)

# Loop principal del juego.
g.gameLoop()

# Liberar los recursos al final.
g.destroy()

```

Lo único que se hace ahora el programa `main.py` es crear el objeto de clase `CGame`, que será la clase principal, luego se crea el estado inicial del juego que corresponderá al juego en sí (el nivel), se pone al juego en ese estado y se invoca al *game loop*. Cuando se sale del juego se invoca a la función `destroy()` y se termina la ejecución.

En la siguiente sección haremos el menú principal del juego, y éste será el estado inicial, dado que el juego comienza en la pantalla del menú.

Agregando la Pantalla Principal (Menú)

Vamos a hacer la pantalla del menú principal. Esta será la primera pantalla que vemos en el juego. El menú tiene el nombre del juego, una ilustración (la carátula o portada del juego), y las opciones (jugar, ver la ayuda, ver los créditos, etc). Por ahora pondremos solamente un texto indicando que con la tecla *[Space]* arrancamos el juego. La siguiente figura muestra el menú principal.



Figura 17-2: La pantalla del menú principal.

Con la tecla `[Space]` pasaremos del menú al juego, y una vez jugando, al pulsar `[Esc]` o cuando termine el juego (al perder todas las vidas) volveremos al menú. En principio tendremos solamente dos estados (pantallas) del juego, la pantalla principal y el juego mismo (este estado es `CLevelState`, la clase que hicimos en el ejemplo anterior).

Por ahora el menú tendrá el título del juego, una imagen y un texto para comenzar a jugar. Más adelante podremos mejorar el menú principal, una vez que esté implementado.

Luego implementaremos el resto de las pantallas. Como siempre, vamos haciendo el juego de a poco, agregando una funcionalidad a la vez. Una vez que tengamos dos pantallas funcionando será más fácil agregar el resto de las pantallas. Siempre conviene hacer una cosa a la vez, paso por paso, en lugar de programar muchas cosas juntas que puede ser más engorroso.

Nota: A una funcionalidad en el juego se le denomina *feature* (en inglés, significa una característica o algo que hace el juego).

Abre y ejecuta el proyecto de ejemplo ubicado en la carpeta `capitulo_17\002_estados_del_juego_menu`. Este ejemplo muestra el menú al iniciar el juego.

Lo primero que hay que decir es que si agregamos una pantalla, debemos agregar una clase para manejar ese estado. En este caso, a la clase del menú la llamaremos `CMenuState`, y la colocamos en la carpeta de estados del juego `game\states`.

En este momento, los estados del juego son `CMenuState` que maneja el menú principal y `CLevelState` que maneja el nivel del juego. Al igual que `CLevelState`, la clase `CMenuState` hereda de `CGameState` y tiene que implementar las funciones del ciclo de vida: `init()`, `update()`, `render()` y `destroy()`.

La clase `CMenuState`, carga su imagen de fondo en `init()` e inicializa los sprites de texto para mostrar en pantalla (más adelante veremos cómo funcionan), luego en `render()` muestra el fondo y los textos necesarios, en `update()` chequea si se pulsa la tecla `[Space]` para pasar al juego, y por último en `destroy()` se destruyen los objetos creados. Veamos el código de la clase `CMenuState`:

```
# -*- coding: utf-8 -*-

#-----
# Clase CMenuState.
# Menú principal.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

import pygame
from api.CKeyboard import *
from api.CGame import *
from api.CGameState import *
from game.states.CLevelState import *
from api.CTextSprite import *

class CMenuState(CGameState):

    mImgSpace = None

    mTextTitle = None
    mTextPressFire = None

    def __init__(self):
        CGameState.__init__(self)

        self.mImgSpace = None
        self.mTextTitle = None
        self.mTextPressFire = None

    def init(self):
        CGameState.init(self)

        # Cargar la imagen del fondo. La imagen es de 640 x 360 al
        # igual que la pantalla.
        self.mImgSpace =
pygame.image.load("assets/images/menu_640x360.jpg")
        self.mImgSpace = self.mImgSpace.convert()
```

```

        # Dibujar la imagen cargada en la imagen de fondo del juego.
        CGame.inst().setBackground(self.mImgSpace)

        self.mTextTitle = CTextSprite("INVASORES", 60,
"assets/fonts/days.otf", (0xFF, 0xFF, 0xFF))
        self.mTextTitle.setXY((CGame.SCREEN_WIDTH -
self.mTextTitle.getWidth())/2, 20)
        self.mTextPressFire = CTextSprite("Pulsa [Space] para
jugar...", 20, "assets/fonts/days.otf", (0xFF, 0xFF, 0xFF))
        self.mTextPressFire.setXY((CGame.SCREEN_WIDTH -
self.mTextPressFire.getWidth())/2, 330)

    def update(self):
        CGameState.update(self)

        if CKeyboard.inst().fire():
            nextState = CLevelState()
            CGame.inst().setState(nextState)
            return

        self.mTextTitle.update()
        self.mTextPressFire.update()

    def render(self):
        CGameState.render(self)

        self.mTextTitle.render(CGame.inst().getScreen())
        self.mTextPressFire.render(CGame.inst().getScreen())

    def destroy(self):
        CGameState.destroy(self)

        self.mImgSpace = None

        self.mTextTitle.destroy()
        self.mTextTitle = None
        self.mTextPressFire.destroy()
        self.mTextPressFire = None

```

La clase `CMenuState` hereda de `CGameState`. Entonces, las funciones `init()`, `update()`, `render()` y `destroy()` llaman a las funciones homónimas de la clase base. Es recomendable hacer esto por si algún día colocamos funcionalidad en la clase base que sirva para todas las clases que la heredan. Debemos invocar a estas funciones, dado que en *Python* no se invocan automáticamente.

En la función `init()`, cargamos la imagen del fondo del menú (que previamente hemos

puesto en la carpeta `assets\images`) y creamos los sprites de texto que se mostrarán en pantalla. Se usa la función `CGame.inst().setBackground()` para establecer la imagen como fondo del juego. Esto es porque la imagen ahora la dibuja la clase `CGame`.

Observa cómo podemos centrar los textos fácilmente, dado que son sprites, podemos usar su función `setXY()` para posicionar el texto y `getWidth()` para saber el ancho del sprite para centrar el texto horizontalmente. Piensa cómo funciona esto.

```
self.mTextPressFire = CTextSprite("Pulsa [Space] para jugar...", 20,
                                   "assets/fonts/days.otf", (0xFF, 0xFF, 0xFF))
self.mTextPressFire.setXY((CGame.SCREEN_WIDTH -
                           self.mTextPressFire.getWidth())/2, 330)
```

En las funciones `update()` y `render()` se invocan a las funciones `update()` y `render()` de los sprites como siempre. Lo que vale la pena notar es que en la función `update()` se está chequeando si se pulsa la tecla `[Space]`. Si esto ocurre, se cambia de estado al igual que se hace en `main.py` para llamar al menú principal. Este código es el que usaremos en cualquier estado para pasar a otro estado del juego (pasar de pantalla):

```
if CKeyboard.inst().fire():
    nextState = CLevelState()
    CGame.inst().setState(nextState)
    return
```

Lo que hace este código es crear una instancia de la clase correspondiente al estado al que vamos a pasar, y se invoca a la función `setState()` de la clase `CGame` para establecer ese estado como el estado actual. Recuerda que la clase `CGame` se encarga de invocar a la función `destroy()` en el estado anterior y a la función `init()` en el estado al que vamos. Esto hace que sea sencillo cambiar de pantalla y sabemos que el motor se encarga de los detalles para que el sistema funcione correctamente.

Por último, en la función `destroy()` se eliminan los objetos creados por la clase. Siempre es necesario eliminar los objetos creados. Si no lo hacemos, aparecerán problemas de memoria (generalmente aparecen cuando jugamos mucho tiempo, el juego se puede poner lento).

Sentencia `return` Luego de Cambiar de Estado

Es muy importante mencionar que cada vez que cambiamos de estado debemos retornar inmediatamente del código que estamos ejecutando, sino, pueden haber errores. Miremos este caso: En `CMenuState`, en la función `update()`, cuando pulsamos la tecla `[Space]`, pasamos de pantalla. Miremos el código de la función `update()` y veamos que hay una

sentencia `return` (para salir de la función) luego de pasar de estado:

```
def update(self):
    CGameState.update(self)

    if CKeyboard.inst().fire():
        nextState = CLevelState()
        CGame.inst().setState(nextState)
        return

    self.mTextTitle.update()          (*)
    self.mTextPressFire.update()      (*)
```

Como se explicó antes, recordemos que al cambiar de estado se invoca a la función `destroy()` del estado actual (esta clase). La función `destroy()` elimina los sprites de textos creados. Entonces, si no retornamos de la función y el código sigue ejecutando, dará error al acceder a los sprites de texto (en la parte marcada con `(*)`). No hay que ejecutar más nada luego de cambiar de estado, dado que es como que este estado ya no existe en realidad (las variables de la clase contienen referencias a `None`).

Nota: Siempre al pasar de estado, hay que colocar una sentencia `return`, para evitar que se pueda ejecutar código usando objetos que ya han sido eliminados en la función `destroy()` que fue invocada luego de cambiar de estado. Esto es tanto para los estados del juego como para los estados del jugador, de los enemigos, etc.

Estado Inicial del Juego

En el programa `main.py`, ahora se comienza en el menú como primera pantalla. Siempre en `main.py` se coloca el estado inicial del juego, creando un objeto de la clase correspondiente al estado inicial y estableciendo esa clase como estado actual. El código de `main.py` queda de la siguiente manera:

```
...

from api.CGame import *
from game.states.CMenuState import *

# ===== Punto de entrada del programa. =====

# Inicializar los elementos necesarios del juego.
g = CGame()

# Crear el estado para el menu del juego.
```

```

initState = CMenuState()
g.setState(initState)

# Loop principal del juego.
g.gameLoop()

# Liberar los recursos al final.
g.destroy()

```

Nos queda ver qué son los sprites de texto que hemos usado en este ejemplo. Esto lo haremos a continuación.

Sprites de Texto

Hasta ahora, cuando mostramos un texto, simplemente usamos una función `drawText()` que hicimos, que lo que hace es dibujar un texto en la pantalla, usando determinada fuente, tamaño de fuente y color. Existen dos problemas a solucionar en nuestro motor relacionados al manejo de texto.

El primer problema ahora es que tenemos varias pantallas, entonces, ¿dónde colocamos la función para dibujar texto para que todas las clases la usen?

El segundo problema está relacionado a la optimización. Recordemos que al mostrar texto en *Pygame*, se genera una imagen y *Pygame* tiene que armar esa imagen cuando dibujamos un texto. Si el texto no ha cambiado, no hay porqué generar la imagen nuevamente. Por supuesto siempre la debemos mostrar en pantalla en cada cuadro, pero no es necesario recrear la imagen en cada frame si el texto no ha cambiado.

Entonces, lo que haremos es crear una clase `CTextSprite` que será un sprite (heredará de `CSprite`) y se le podrá asignar un texto en lugar de una imagen. Internamente esta clase generará la imagen según el texto y la usará como la imagen del sprite.

La clase `CTextSprite` va en la carpeta `\api` porque se trata de una clase del motor, que usaremos en todos nuestros juegos. Una gran ventaja de tener un sprite de texto, es que podremos mover los textos por la pantalla como lo hacemos con cualquier sprite. Al heredar de `CSprite` tenemos a disposición todas las funciones que tiene un sprite, incluidas las funciones de posición, velocidad y aceleración, o las funciones `getWidth()` y `getHeight()` que serán muy útiles al colocar los textos centrados en la pantalla.

Veamos el código de la clase `CTextSprite`:

```

# -*- coding: utf-8 -*-

#-----

```

```

# Clase TextSprite.
# Sprite de texto. Hereda de CSprite.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----

import pygame
from api.CSprite import *

class CTextSprite(CSprite):

    def __init__(self, aText = "", aFontSize = 10, aFontName = "",
aColor = (0xFF, 0xFF, 0xFF)):

        CSprite.__init__(self)

        self.mText = aText
        self.mFontSize = aFontSize
        self.mFontName = aFontName
        self.mColor = aColor
        self.updateImage()

    # Función genérica para dibujar un texto en una superficie.
    # Parámetros: La pantalla donde dibujar, coordenadas (x,y),
    # texto , tamaño de fuente y color del texto.
    @classmethod
    def drawText(self, aScreen, aX, aY, aMsg, aFontName, aFontSize,
aColor=(0,0,0)):
        font = pygame.font.Font(aFontName, aFontSize)
        imgTxt = font.render(aMsg, True, aColor)
        aScreen.blit(imgTxt, (aX, aY))

    def setText(self, aText):
        if self.mText != aText:
            self.mText = aText
            self.updateImage()

    def setFontName(self, aFontName):
        if self.mFontName != aFontName:
            self.mFontName = aFontName
            self.updateImage()

    def setSize(self, aFontSize):
        if self.mFontSize != aFontSize:
            self.mFontSize = aFontSize
            self.updateImage()

```



```

def setColor(self, aColor):
    if self.mColor != aColor:
        self.mColor = aColor
        self.updateImage()

def updateImage(self):
    if (self.mFontName == ""):
        font = pygame.font.SysFont("Comic Sans MS",
self.mFontSize)
    else:
        font = pygame.font.Font(self.mFontName, self.mFontSize)
    imgTxt = font.render(self.mText, True, self.mColor)
    self.setImage(imgTxt)

def update(self):
    CSprite.update(self)

def render(self, aScreen):
    CSprite.render(self, aScreen)

def destroy(self):
    CSprite.destroy(self)

```

La clase `CTextSprite` es bastante simple en su funcionamiento. En el constructor se pasan como parámetros el texto, el tamaño de la fuente, el nombre de la fuente y el color del texto. Con esos datos se arma la imagen que es la que se establece como imagen del sprite. Esto se hace en la función `updateImage()`.

La función `updateImage()` es invocada cada vez que hay que armar la imagen del texto. Esto es, en el constructor cuando se crea la imagen, y luego cada vez que se cambia alguna de las propiedades del sprite de texto que altere la imagen. Cuando cambia el texto, la fuente, el tamaño o el color, hay que rehacer la imagen. Si nos fijamos en cada función para establecer una propiedad, se llama a la función `updateImage()` para recrear la imagen. Pero si no cambia nada no hay necesidad de andar recreando la imagen. Cada función tiene una optimización, que pregunta si no ha cambiado el dato, en cuyo caso no será necesario recrear la imagen del texto.

Por último, esta clase tiene una función estática `drawText()` que es la misma que antes teníamos para mostrar texto, en caso de que necesitemos dibujar un texto directamente y no queramos hacer un sprite de texto.

Mira el código de `CLevelState` en este ejemplo que también está usando sprites de texto para dibujar los textos del *HUD*, en lugar de dibujarlo directamente como lo hacía antes. En la función `update()`, se cambia el texto en cada frame (esto se podría optimizar, controlando que no cambie el texto si no cambió el valor del puntaje o de las vidas).

Similar a lo que se hace en la clase `CMenuState`, en `CLevelState` se inicializan las variables de los sprites de texto en `init()`, luego se invoca a las funciones `update()` y `render()` para establecer el texto y dibujarlo, y al final se invoca a la función `destroy()` para eliminar los objetos creados. Veamos marcados en negrita los lugares donde se manejan los nuevos sprites de texto en `CLevelState`:

```
...
from api.CTextSprite import *

class CLevelState(CGameState):

    mImgSpace = None
    mPlayer1 = None
    mPlayer2 = None

    mTextLives1 = None
    mTextLives2 = None
    mTextScore1 = None
    mTextScore2 = None

    def __init__(self):
        CGameState.__init__(self)
        print("LevelState constructor")

        self.mImgSpace = None
        self.mPlayer1 = None
        self.mPlayer2 = None

        self.mTextScore1 = None
        self.mTextScore2 = None
        self.mTextLives1 = None
        self.mTextLives2 = None

    # Función donde se inicializan los elementos necesarios del nivel.
    def init(self):
        ...

        # Inicializar los datos del juego.
        CGameData.inst().setScore1(0)
        CGameData.inst().setLives1(3)
        CGameData.inst().setScore2(0)
        CGameData.inst().setLives2(3)

        self.mTextScore1 = CTextSprite("SCORE: " +
str(CGameData.inst().getScore1()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
```

```

        self.mTextScore1.setXY(5, 5)
        self.mTextLives1 = CTextSprite("VIDAS: " +
str(CGameData.inst().getLives1()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
        self.mTextLives1.setXY(5, CGame.SCREEN_HEIGHT - 20 - 5)

        self.mTextScore2 = CTextSprite("SCORE: " +
str(CGameData.inst().getScore2()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
        self.mTextScore2.setXY(530, 5)
        self.mTextLives2 = CTextSprite("VIDAS: " +
str(CGameData.inst().getLives2()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
        self.mTextLives2.setXY(540, CGame.SCREEN_HEIGHT - 20 - 5)

# Actualizar los objetos del nivel.
def update(self):
    ...

    if self.mPlayer1.isGameOver() and self.mPlayer2.isGameOver():
        print("AMBOS JUGADORES MUEREN - LOSE CONDITION")
    if CEnemyManager.inst().getLength() == 0:
        print("TODOS LOS ENEMIGOS MUEREN - WIN CONDITION")

    self.mTextScore1.update()
    self.mTextScore2.update()
    self.mTextLives1.update()
    self.mTextLives2.update()

# Dibujar el frame del nivel.
def render(self):
    ...

    # Dibujar los jugadores.
    self.mPlayer1.render(screen)
    self.mPlayer2.render(screen)

    # Dibujar el texto del score y las vidas.
    self.mTextScore1.setText("SCORE: " +
str(CGameData.inst().getScore1()))
    self.mTextLives1.setText("VIDAS: " +
str(CGameData.inst().getLives1()))
    self.mTextScore2.setText("SCORE: " +
str(CGameData.inst().getScore2()))
    self.mTextLives2.setText("VIDAS: " +
str(CGameData.inst().getLives2()))

    self.mTextScore1.render(screen)

```

```

        self.mTextLives1.render(screen)
        self.mTextScore2.render(screen)
        self.mTextLives2.render(screen)

# Destruir los objetos creados en el nivel.
def destroy(self):
    ...

    # Destruir las balas.
    CBulletManager.inst().destroy()

    # Destruir los enemigos.
    CEnemyManager.inst().destroy()

    self.mTextScore1.destroy()
    self.mTextScore1 = None
    self.mTextLives1.destroy()
    self.mTextLives1 = None
    self.mTextScore2.destroy()
    self.mTextScore2 = None
    self.mTextLives2.destroy()
    self.mTextLives2 = None

    CGameData.inst().destroy()

```

Esto es una muestra más de que agregar un objeto en el juego es crearlo, actualizarlo y borrarlo al final. O dicho de otra manera, implementar si ciclo de vida, invocando a las funciones `init()`, `update()` y `render()` y al final `destroy()`.

Garbage Collector

En la clase `CGame` al cambiar de estado en la función `setState()`, se llama a la función `gc.collect()` para liberar la memoria que no está utilizada. *Python* tiene lo que se conoce como *garbage collector*, que es un módulo de *Python* que se encarga de eliminar los objetos que no están siendo más referenciados. La función `gc.collect()` retorna el número de objetos que ha eliminado.

```

# Función para cambiar entre pantallas (estados) del juego.
def setState(self, aState):
    if (CGame.mState != None):
        CGame.mState.destroy()
        CGame.mState = None
    # Liberar memoria.

```

```
print(gc.collect(), " objetos borrados.")

CGame.mState = aState
CGame.mState.init()
```

Cuando creamos un objeto, se asigna espacio en la memoria de la computadora para almacenar el objeto. Cuando no usamos más el objeto lo que hacemos es eliminar su referencia, esto es, asignando a la variable que lo referencia, el valor `None` para que no le apunte más. De esta forma, cuando un objeto ya no es referenciado, es eliminado de la memoria por el *garbage collector*.

No es bueno que el *garbage collector* corra en cualquier momento, porque es una operación que puede tomar un tiempo (ínfimo, pero suficiente como para no querer que eso pase en el medio del juego, dado que se puede notar una trancada). Entonces, un buen lugar para invocar al *garbage collector* es justamente al pasar de estado del juego, momento en el que acabamos de destruir objetos y creando otros, siendo el punto más crítico del motor, en cuanto a manejo de memoria. Además, al momento de cambiar de estado, no estamos dentro del juego, por lo cual es un buen momento para que se limpie la memoria utilizada.

Como siempre, este tipo de cosas a nivel del sistema lo tenemos en cuenta cuando desarrollamos el *motor* o *engine del juego*. Luego, nos queda disponible para cualquier juego que hagamos.

La Pausa del Juego

Para terminar con la programación de los estados del juego, implementaremos la pausa del juego. Ningún juego de acción puede ver la luz del día sin tener una pausa. La pausa es algo muy necesario en un juego, dado que el jugador puede tener interrupciones mientras está jugando.

Haremos que el juego se ponga en pausa al pulsar la tecla `[P]` o `[Enter]`. Cuando el juego está en pausa, mostraremos un mensaje indicando que el juego está en pausa. Presionando de nuevo la tecla `[P]` o `[Enter]` el juego continúa. Mira la siguiente figura.



Figura 17-3: El juego cuando está pausado.

Abre y ejecuta el ejemplo ubicado en la carpeta `capitulo_17\003_pausa`. En este ejemplo vemos que el juego se detiene si pulsamos la tecla `[P]` o `[Enter]`. En la clase `CKeyboard` hemos agregado las funciones necesarias para detectar cuando se pulsan estas dos nuevas teclas. Observa esta clase para ver las nuevas teclas agregadas.

Hacer la pausa es algo bastante sencillo, si sabemos qué es lo que hacer. Para que el juego se detenga, simplemente tenemos que dejar de ejecutar la función `update()` del estado actual. Esto lo hacemos en la clase `CGame`, que es la clase que se encarga de esto.

Pondremos una variable *booleana* (una *bandera*) que indicará si el juego está en pausa o no. Cuando se pulsa la tecla de pausa, esta variable cambia su valor. También creamos un sprite de texto para mostrar el mensaje de que el juego está en pausa.

Entonces, si el juego está en pausa no se ejecuta la función `update()` del estado del juego y se muestra el texto. Cuando el juego no está en pausa, se ejecuta la función `update()` normalmente y no se muestra el texto. Veamos el código de `CGame` con el código de la pausa:

...

```
# Importar la clase de texto.
from api.CTextSprite import *
```

```
class CGame(object):
```

```
...
```

```
# Pausa.
mIsPaused = False
mTextPause = None
```

```

...

# Función de inicialización.
def init(self):

    ...

    CGame.mState = None

    # Variables para la pausa.
    CGame.mIsPaused = False
    CGame.mTextPause = CTextSprite("JUEGO EN PAUSA", 40,
"assets/fonts/days.otf", (0xFF, 0xFF, 0xFF))
    CGame.mTextPause.setXY((CGame.SCREEN_WIDTH -
CGame.mTextPause.getWidth()) / 2, (CGame.SCREEN_HEIGHT -
CGame.mTextPause.getHeight()) / 2)

    ...

# Game loop del juego.
def gameLoop(self):

    while not self.mSalir:
        self.update()
        self.render()

# Correr la lógica del juego.
def update(self):

    ...

    if CKeyboard.inst().pauseKey():
        self.togglePause()

    # Cuando el juego está en pausa no se corre update().
    if not CGame.mIsPaused:
        # Actualizar el estado del juego.
        CGame.mState.update()

# Dibujar el frame y actualizar la pantalla.
def render(self):

    # Dibujar el fondo.
    CGame.mScreen.blit(self.mBackground, (0, 0))

    # Dibujar el estado del juego.
    CGame.mState.render()

```

```

    # Si el juego está en pausa se muestra el mensaje.
    if CGame.mIsPaused:
        CGame.mTextPause.render(self.mScreen)

    # Dibujar el puntero del mouse.
    CGame.mMousePointer.render(self.mScreen)

    # Actualizar la pantalla.
    pygame.display.flip()

    ...

# Pausa o continua el juego.
def togglePause(self):
    CGame.mIsPaused = not CGame.mIsPaused

def destroy(self):
    ...

    CGame.mTextPause.destroy()
    CGame.mTextPause = None

    ...

```

Como vemos en el código, tenemos una variable *booleana* que indica si el juego está o no en pausa y según su estado se muestra o no el mensaje de pausa, o si se ejecuta o no la función `update()` del estado actual, como se explicó antes.

Nota: Para agregar una nueva funcionalidad al juego, generalmente no se necesita programar tanto, sino más bien saber en dónde hay que poner el código. Por este motivo, es que conviene no programar nada sin antes sentarse un momento a pensar bien qué es lo que se va a hacer y cómo.

En el caso de la pausa, la solución no es complicada. Es simplemente poner una bandera y correr o no la lógica (invocar o no a la función `update()`). Pero hacerlo sin pensarlo, puede llevar a errores. Sobre todo, porque se trata de la clase `CGame`, que es la clase de base para toda la aplicación. Debemos tener tiempo para diseñar y pensar la solución que vamos a realizar.

Imports y las Referencias Circulares en *Python*

Para finalizar este capítulo, vamos a hacer que al ganar el nivel (cuando se matan a todos los enemigos) o cuando se pierde el juego (cuando ambos jugadores se quedan sin vidas), que se vuelva al menú principal. Como vimos en la clase `CMenuState`, para pasar al nivel del

juego hacemos esto:

```
from game.states.CLevelState import CLevelState

...

nextState = CLevelState()
CGame.inst().setState(nextState)
```

Pues bien, en forma similar, cuando estamos en el nivel y ocurra la condición de ganar o de perder, el código para volver al menú sería el siguiente:

```
from game.states.CMenuState import CMenuState

...

if self.mPlayer1.isGameOver() and self.mPlayer2.isGameOver():
    print("AMBOS JUGADORES MUEREN - LOSE CONDITION")
    nextState = CMenuState()
    CGame.inst().setState(nextState)

if CEnemyManager.inst().getLength() == 0:
    print("TODOS LOS ENEMIGOS MUEREN - WIN CONDITION")
    nextState = CMenuState()
    CGame.inst().setState(nextState)
```

Pues bien, si probamos esto veremos que nos da un error. El error no es claro porque dice que no encuentra (no está definida) la clase `CMenuState`, cuando claramente está importado arriba. Entonces, ¿qué es lo que sucede?

Lo que pasa aquí, es un problema que se denomina *referencia circular*. El módulo `CMenuState` necesita importar `CLevelState` y el módulo `CLevelState` necesita importar `CMenuState`. El problema es que un módulo (*clase*) no puede compilarse porque necesita al segundo, el cual necesita del primero. Si las sentencias `import` se encuentran al inicio de los módulos, no funcionará.

Para evitar esto, lo que hacemos es importar el nombre de clase que necesitamos antes de instanciar la clase, en la misma función `update()` donde cambiamos de estado. Entonces, el código de la función `update()` de `CMenuState` queda de la siguiente manera:

```
def update(self):
    CGameState.update(self)
```

```

if CKeyboard.inst().fire():
    from game.states.CLevelState import CLevelState
    nextState = CLevelState()
    CGame.inst().setState(nextState)
    return

self.mTextTitle.update()
self.mTextPressFire.update()

```

Y la función `update()` de la clase `CLevelState`, ahora chequea por la condición de ganar o perder y cambia a la pantalla del menú cuando se gana o se pierde:

```

# Actualizar los objetos del nivel.
def update(self):

    ...

    if self.mPlayer1.isGameOver() and self.mPlayer2.isGameOver():
        print("AMBOS JUGADORES MUEREN - LOSE CONDITION")
        from game.states.CMenuState import CMenuState
        nextState = CMenuState()
        CGame.inst().setState(nextState)
        return

    if CEnemyManager.inst().getLength() == 0:
        print("TODOS LOS ENEMIGOS MUEREN - WIN CONDITION")
        from game.states.CMenuState import CMenuState
        nextState = CMenuState()
        CGame.inst().setState(nextState)
        return

    ...

```

Como siempre, cada vez que cambiamos de estado, lo sigue una función `return` para no continuar ejecutando código perteneciente a un estado que ya es obsoleto.

Si vemos el código del último ejemplo, veremos esto implementado. Al ejecutar el juego, si matamos a todos los enemigos o al perder todas las vidas, se vuelve al menú principal.

Ya tenemos el juego casi completo. De esta forma terminamos este capítulo. En el capítulo siguiente nos dedicaremos a terminar el juego, implementando los detalles finales, lo que se conoce como el pulido del juego.

18. Terminando el Nivel y Puliendo

Llegamos a la parte más linda o divertida de hacer un videojuego, el pulido (*polishing* en inglés). Esto es, trabajar en los detalles para que el juego sea muy divertido, se vea lindo y se sienta realmente bien.

Durante todo el desarrollo del juego estaremos realizando testing con personas, para ver si les gusta el juego y detectar posibles problemas que iremos solucionando. Atenderemos a las sugerencias que nos hagan nuestros amigos, y eso lleva a realizar un mejor juego.

En esta parte, ya tenemos el juego casi completo, y lo que haremos ahora será terminarlo, atacando los detalles que quedan. Naturalmente que en esta etapa podemos estar mucho tiempo, y podría no acabar nunca. Como se trata de un ejemplo para un libro, haremos pocas tareas. Tu eres libre de seguir este juego o crear el tuyo propio con todas las características que te sientas capaz de realizar.

A continuación arreglaremos varios detalles que tiene el juego, antes de terminarlo.

Matar la Bala de los Enemigos al Pegarle al Jugador

Si corremos el último ejemplo, vemos que cuando una bala enemiga le pega al jugador, lo mata, pero la bala continúa su camino. Lo que haremos es hacer que cualquier bala que le pegue al jugador, se muera. No importa en qué estado esté el jugador, la bala se morirá. El jugador, pasa al estado *explotando* solamente si lo toca una bala y se encuentra en el estado *normal*.

Veamos el ejemplo ubicado en la carpeta `capitulo_18\001_matar_bala_enemiga_al_chocar` para ver esto implementado y funcionando.

En función `update()` de la clase `CPlayer`, hacemos el siguiente cambio:

```
# Mover el objeto.
def update(self):

    # Invocar update() de CAnimatedSprite.
    CAnimatedSprite.update(self)

    # Obtener el enemigo con el cual chocamos.
    enemy = CEnemyManager.inst().collides(self)

    # Si chocamos con un enemigo, el enemigo se muere.
    if enemy != None:
```

```

        enemy.hit()

# Lógica del estado normal.
if self.getState() == CPlayer.NORMAL:
    if enemy != None:
        self.setState(CPlayer.DYING)
        return
    self.move()

...

```

Como vemos en el código, chequeamos colisiones contra todas las balas fuera de las estructuras `if` de los estados. De esta forma, la bala se muere, independientemente del estado del jugador. Luego, dentro del estado `CPlayer.NORMAL`, pasamos al estado `CPlayer.DYING` si se ha chocado con una bala (esto ocurre cuando la variable `enemy` no es `None`, o sea, contiene la referencia al enemigo con el que ha chocado). Repasa la clase `CManager` si no recuerdas cómo funcionaba esto.

Limitando la Cantidad de Disparos

Al jugar al juego como está ahora, nos damos cuenta que si disparamos mucho, matamos muy fácil a los enemigos. Lo que haremos es hacer que el jugador no pueda disparar más de una bala a la vez (se puede cambiar este valor). Cuando quiera disparar la segunda bala, sonará un sonido indicando que no puede disparar. En el momento que la bala llegue a destino (cuando alcance un enemigo o salga de la pantalla), se podrá disparar otra bala.

Observa el ejemplo ubicado en la carpeta `capitulo_18\002_maxima_cantidad_de_balas`. Veamos el código de la clase `CPlayer`, que es donde agregamos código para controlar los disparos:

```

...

# La clase CPlayer hereda de CAnimatedSprite.
class CPlayer(CAnimatedSprite):

    ...

    # Maxima cantidad de disparos a la vez.
    MAX_BULLETS = 1

    ...

    def __init__(self, aType):

        ...

```

```

    # Cantidad de balas en el momento.
    self.mBulletCount = 0

    # Estado inicial.
    self.setState(CPlayer.NORMAL)

# Movimiento de la nave.
def move(self):
    ...

    # Disparar.
    if fire:
        if self.mBulletCount < CPlayer.MAX_BULLETS:
            # Incrementar la cantidad de balas vivas.
            self.mBulletCount = self.mBulletCount + 1

            print("FIRE")
            b = CPlayerBullet(self)
            b.setXY(self.getX() + self.getWidth() / 2 - b.getWidth() /
2, self.getY())
            b.setVelX(0)
            b.setVelY(-10)
            b.setBounds(0, 0, CGameConstants.SCREEN_WIDTH,
CGameConstants.SCREEN_HEIGHT)
            b.setBoundAction(CGameObject.DIE)
            CBulletManager.inst().add(b)

            # Sonido de disparo.
            CAudioManager.inst().play(CAudioManager.mSoundShootPlayer)
        else:
            # Sonido de que no puede disparar.
            CAudioManager.inst().play(CAudioManager.mSoundCannotShoot)

    ...

# Invocada cuando una bala del jugador se destruye,
def bulletDestroyed(self):
    # Decrementar la cantidad de balas vivas.
    self.mBulletCount = self.mBulletCount - 1

```

Definimos una constante MAX_BULLETS con la cantidad máxima de balas que puede disparar el jugador a la vez. Este valor se pone en 1, y se puede cambiar. La variable self.mBulletCount lleva la cuenta de cuantas balas del jugador hay en la pantalla. Cada vez que se dispara se incrementa esta variable, y no se puede disparar si la cuenta es mayor que MAX_BULLETS. Si se puede disparar, se dispara normalmente y se incrementa la variable. Si no se puede disparar porque se ha alcanzado el máximo de balas, se dispara el sonido

indicando que no es posible disparar (por supuesto, habiendo colocado el archivo de sonido en la carpeta de assets). Todo esto se hace en la función `move()`, en la parte de disparar.

El sonido que agregamos se llama `player_cannot_shoot.wav` y en la clase `CAudioManager` lo agregamos (observa el proyecto de ejemplo para copiar el archivo e integrar este sonido).

Luego, cuando la bala se muere, debemos decrementar la cantidad de balas. Si recuerdas (mira el código de `move()`), cuando se crea la bala se le pasa como parámetro la referencia al jugador que la dispara. Esto era, para saber a quién asignarle el puntaje cuando la bala mataba un enemigo. Pues bien, ahora lo necesitaremos para saber a qué jugador decrementarle la cantidad de balas cuando la bala se destruye.

En la clase `CPlayerBullet`, cuando la bala es destruida, invoca a la función `bulletDestroyed()` de la clase `CPlayer`, y en esta función se decrementa la cuenta de balas, como se ve arriba. A continuación vemos en la clase `CPlayerBullet` donde se invoca la función al destruirse la bala:

```
# Liberar lo que haya creado el objeto.
def destroy(self):
    # Invocar esta función para decrementar la cuenta de balas vivas.
    self.mPlayer.bulletDestroyed()

    CSprite.destroy(self)
```

Esta forma de comunicarse entre objetos, es clave a la hora de programar videojuegos. Los objetos en un juego constantemente se están mandando mensajes entre sí. En este caso, es como que la bala dice: “he muerto, quien me haya disparado debe saberlo”.

Nota: El envío de mensajes entre objetos del juego puede ser en forma directa como en este caso (invocando a funciones) o a través de un manager. Por ejemplo, que la bala le avise al manager del juego y luego éste le avisa al jugador. Como sea, el resultado es el mismo. Muchas soluciones diferentes implican dónde se coloca el código, pero el funcionamiento en general es el mismo.

La Formación de Enemigos Baja

Lo que haremos a continuación será que la formación de enemigos baje cuando uno de los enemigos toque el borde. Cuando la formación baja, comienza a moverse para el lado contrario. El objetivo en este juego es evitar que las naves enemigas se acerquen al fondo de la pantalla.

Abre y ejecuta el ejemplo ubicado en la carpeta `capitulo_18\003_la_formacion_baja`. Mira como cuando un enemigo toca el borde, la formación entera baja y comienza a ir hacia el

otro lado.

Como la formación contiene a todos los enemigos, y ahora debemos escribir código de comportamiento que es específico para el conjunto de enemigos, colocaremos este código en la clase `CEnemyManager`, que es la clase que se encarga de los enemigos.

Lo primero que haremos será colocar esta clase en la carpeta `game`, dado que ya no puede estar en la carpeta `api` porque tendrá código específico para este juego. Cuando se hace esto hay que cambiar (o agregar) las sentencias `import` correspondientes, en las otras clases que la usen.

Para controlar la formación, tendremos una variable denominada `mDirection` que indicará si la formación de enemigos se mueve hacia la derecha o hacia la izquierda (usaremos constantes denominadas `RIGHT` y `LEFT`).

En la función `update()` del manager de enemigos, es donde controlamos que si la formación va hacia la derecha y uno de los enemigos toca el borde derecho, la formación baja y los enemigos comienzan a caminar para el lado contrario. De la misma manera, si la formación va hacia la izquierda, se controla si alguno de los enemigos toca el borde izquierdo. Si esto ocurre, la formación baja y los enemigos comienzan a moverse para el lado contrario.

Veamos el código de `CEnemyManager` para ver la programación relacionada a la formación de enemigos:

```
...

# Importar Pygame.
import pygame

from api.CManager import *
from api.CGameConstants import *
from game.CEnemyBullet import *

class CEnemyManager(CManager):

    mInstance = None
    mInitialized = False

    # Constantes para la dirección de la formación de enemigos.
    RIGHT = 0
    LEFT = 1

    # Dirección de la formación.
    mDirection = RIGHT

    ...
```

```

# Procesar los objetos.
def update(self):
    CManager.update(self)

    # Ver si alguno de los enemigos tocó el borde de la pantalla.
    # Si esto es así, bajar a los enemigos y luego invertir la
    # velocidad de todos los enemigos.
    touched = False
    i = 0
    while i < len(self.mArray):
        if not isinstance(self.mArray[i], CEnemyBullet):

            # Ver si van a la derecha y tocan el borde derecho.
            if CEnemyManager.mDirection == CEnemyManager.RIGHT:
                if self.mArray[i].getX() +
self.mArray[i].getWidth() > CGameConstants.SCREEN_WIDTH:
                    touched = True
                    break
            # Ver si van hacia la izquierda y tocan el borde izq.
            else:
                if self.mArray[i].getX() < 0:
                    touched = True
                    break

            i = i + 1
        if touched:
            self.invertFormation()

# Dibujar los objetos.
def render(self, aScreen):
    CManager.render(self, aScreen)

# Agregar un objeto a la lista.
def add(self, aEnemy):
    CManager.add(self, aEnemy)

# Destruir todos los objetos y removerlos de la lista.
# Destruir la lista.
def destroy(self):
    CManager.destroy(self)

    CEnemyManager.mInstance = None

    # Bajar los enemigos e invertir las velocidades de todos los
    enemigos.
    def invertFormation(self):
        if CEnemyManager.mDirection == CEnemyManager.RIGHT:

```



```

        CEnemyManager.mDirection = CEnemyManager.LEFT
    else:
        CEnemyManager.mDirection = CEnemyManager.RIGHT

    i = 0
    while i < len(self.mArray):
        if not isinstance(self.mArray[i], CEnemyBullet):
            self.mArray[i].setY(self.mArray[i].getY() +
self.mArray[i].getHeight() / 2)
            self.mArray[i].setVelX(self.mArray[i].getVelX() * -1)

        i = i + 1

```

Cuando un enemigo toca el borde en la dirección que lleva la formación, se invoca a la función `invertFormation()`. Esta función cambia la variable de dirección según corresponda, y se recorre la lista de enemigos, haciéndolos bajar con la función `setY()` e invirtiendo su velocidad para que comiencen a moverse para el otro lado, utilizando la función `setVelX()` y multiplicando la velocidad que tienen por `-1`. El efecto de multiplicar por `-1` es invertir el signo de la velocidad, lo que hace que el enemigo camine para el lado contrario al que se movía.

Nota: Observa en la clase `CGameObject` en el ejemplo, para ver las funciones `setX()` y `setY()` que agregamos a `CGameObject`. Estas funciones establecen las coordenadas por separado (hasta ahora usábamos la función `setXY()`). También hemos agregado en esta clase las funciones `getVelX()` y `getVelY()`. A medida que necesitemos funciones las iremos agregando en las clases de `api`.

Como nosotros ahora estamos controlando el comportamiento de los enemigos contra el borde de la pantalla, tenemos que hacer que al crearse los enemigos, no se establezca ningún comportamiento de borde (para esto, al crear los enemigos en la clase `CLevelState`, se utiliza `CGameObject.NONE` como comportamiento de borde).

Al recorrer la lista del manager de enemigos, debemos recordar que en la lista de enemigos se encuentran tanto las naves enemigas como las balas enemigas. Por esta razón, es que usamos la función `isinstance()` para saltar a las balas enemigas. Si el enemigo es una instancia de `CNave` (o sea, es una nave enemiga, de clase `CNave`), es tenido en cuenta. Si no, es saltada.

Nota: En la clase `CEnemyManager` hemos eliminado los comentarios de las sentencias `import`, porque son obvias. Está en cada programador el documentar más o menos. En mi caso, me gusta documentar cada línea, pero si las cosas son muy obvias, no. Elige tu forma de documentar como más te guste.

Nota: Como habrás notado, de a poco vamos cambiando los nombres de las variables y funciones a idioma inglés. Esto es necesario para irnos acostumbrando, ya que se espera en la industria de videojuegos que manejemos el idioma inglés como estándar, para que se

pueda trabajar con personas de otras nacionalidades. Obviamente se puede escribir en español, pero a la larga lo tendremos que hacer en inglés porque es una práctica profesional. Es algo para hacerlo con tiempo.

El juego, en este momento queda muy difícil porque la formación de enemigos se mueve rápida y al tocar los bordes baja. Esto lo mejoraremos a continuación.

Velocidad Incremental de los Enemigos

La velocidad de los enemigos comenzará lenta al inicio e irá acelerando a medida que queden menos enemigos en la pantalla. Para esto, tendremos una variable para tener la velocidad mínima (al arrancar el nivel) y la velocidad máxima (cuando queda un solo enemigo). En el medio, se usan valores entre el mínimo y el máximo según la cantidad de enemigos. A esta operación se le llama interpolación, y consiste en establecer un valor entre el mínimo y el máximo, gradualmente, según cierto criterio (en este caso, la cantidad de enemigos que quedan).

Veamos el código del ejemplo ubicado en la carpeta `capitulo_18\004_la_formacion_acelera`. En la clase `CEnemyManager` es donde se encuentran los cambios relacionados a la velocidad incremental de la formación de enemigos. Veamos el código.

...

```
class CEnemyManager(CManager):
```

```
    mInstance = None
    mInitialized = False
```

```
    # Constantes para la dirección de la formación de enemigos.
    RIGHT = 0
    LEFT = 1
```

```
    # Dirección de la formación.
    mDirection = RIGHT
```

```
    # Variables para la velocidad de la formación de enemigos.
    mMinVelX = 0
    mMaxVelX = 0
    mVelX = 0
    mMaxShips = 0
```

...

```
    # Procesar los objetos.
```

```

def update(self):
    CManager.update(self)

    # Establecer la velocidad de la formación según la cantidad
de enemigos.
    if self.countShips() != 0:
        # Vale 1 cuando hay un solo enemigo y 0 cuando están
        # todos los enemigos. En el medio se interpola.
        percent = 1 - (float(self.countShips()) /
float(self.mMaxShips))
    else:
        percent = 0

    vel = self.mMinVelX + ((self.mMaxVelX - self.mMinVelX) *
percent)

    if CEnemyManager.mDirection == CEnemyManager.RIGHT:
        self.setVelX(vel)
    else:
        self.setVelX(-vel)

    # Ver si alguno de los enemigos tocó el borde de la pantalla.
    # Si esto es así, bajar a los enemigos y luego invertir la
    # velocidad de todos los enemigos.
    touched = False
    i = 0
    while i < len(self.mArray):
        if not isinstance(self.mArray[i], CEnemyBullet):

            # Ver si van a la derecha y tocan el borde derecho.
            if CEnemyManager.mDirection == CEnemyManager.RIGHT:
                if self.mArray[i].getX() +
self.mArray[i].getWidth() > CGameConstants.SCREEN_WIDTH:
                    touched = True
                    break

            # Ver si van hacia la izquierda y tocan el borde izq.
            else:
                if self.mArray[i].getX() < 0:
                    touched = True
                    break

            i = i + 1
        if touched:
            self.invertFormation()

    ...

    # Establecer la velocidad mínima y máxima de la formación.

```

```

def setMinMaxVelX(self, aMinVelX, aMaxVelX):
    self.mMinVelX = aMinVelX
    self.mMaxVelX = aMaxVelX
    self.setVelX(aMinVelX)
    self.mMaxShips = self.countShips()

# Establecer la velocidad actual de la formación.
def setVelX(self, aMinVelX):
    self.mVelX = aMinVelX

    i = 0
    while i < len(self.mArray):
        if not isinstance(self.mArray[i], CEnemyBullet):
            self.mArray[i].setVelX(self.mVelX)
        i = i + 1

# Retorna la cantidad de naves en la formación.
def countShips(self):
    c = 0
    i = 0
    while i < len(self.mArray):
        if not isinstance(self.mArray[i], CEnemyBullet):
            c = c + 1
        i = i + 1
    return c

```

Cuando se crean los enemigos, se establece la velocidad mínima y máxima. Esto se hace en la función `setMinMaxVelX()` que se invoca desde `CLevelState` al crear los enemigos.

```

# Función donde se inicializan los elementos necesarios del nivel.
def init(self):
    ...

# Establecer la velocidad mínima y máxima de la formación.
CEnemyManager.inst().setMinMaxVelX(1, 8)

```

Luego, en esta función, se establece la velocidad mínima para todos los enemigos. Para esto, hacemos una función `setVelX()` en el manager de enemigos, que recibe como parámetro la velocidad, y establece esa velocidad para todos los enemigos (como siempre, usamos la función `isinstance()` para aplicarlo solo a los objetos de clase `CNave` y no a las balas enemigas).

En la función `update()`, calculamos (interpolamos o proyectamos) un número entre cero y uno, según el rango:

1.0 o 100%: Cuando hay un solo enemigo.
0.0 o 0%: Cuando están todos los enemigos.

Y en el medio, todos los valores según cuantos enemigos queden en la pantalla. La cuenta que hacemos es sumarle a la velocidad mínima, la diferencia entre la velocidad máxima y la velocidad mínima, interpolada por la cantidad de enemigos. Esta operación da la velocidad mínima cuando están todos los enemigos y la velocidad máxima cuando queda un solo enemigo. Para hacer esto, como se ve en el código, se suma a la velocidad mínima, la diferencia entre la velocidad máxima y la mínima, multiplicado por el porcentaje.

Nota: En la operación de porcentaje convertimos los números enteros a flotantes, porque sino la división entre enteros se toma como división entera (lo cual da solamente cero o uno en este caso). Como necesitamos un número entre 0.0 y 1.0, utilizamos números flotantes.

Por último, como ahora estamos estableciendo la velocidad de los enemigos en cada frame, en el estado `CNave.EXPLODING` de la nave, en la función `update()` de la clase `CNave`, le ponemos la velocidad en cero para que no se mueva.

```
# Mover el objeto.
def update(self):

    # Invocar update() de CAnimatedSprite.
    CAnimatedSprite.update(self)

    if self.getState() == CNave.NORMAL:
        self.controlFire()

    elif self.getState() == CNave.EXPLODING:

        # Como en update() del manager se establece velocidad,
        # detenerlo aquí.
        self.stopMove()

    if self.isEnded():
        self.die()
        return
```

También en este ejemplo le restamos un poco a lo que baja la nave para que no quede muy difícil el juego.

Nota: Mientras hacemos el juego, iremos modificando los valores (llamados *parámetros*) de jugabilidad. Esto hará que el juego resulte más difícil o más fácil. A esta tarea se le llama *gameplay balance* y es una tarea que hace el diseñador del juego (con ayuda del programador para indicarle dónde se encuentran los valores a cambiar)..

Ocultar o Mostrar el Mouse

En esta sección, incluiremos un par de mejoras simples al juego. La primera mejora es hacer que los jugadores no se vayan de la pantalla. Para eso, en la función `init()` de `CLevelState`, al crear los jugadores, se le pasa el comportamiento de borde `CGameObject.STOP`. Si recuerdas en código en `CGameObject`, este valor hacía que el jugador no se pase de los bordes que definimos con la función `setBounds()`. Si no recuerdas su uso, mira la clase `CLevelState` en el ejemplo que se encuentra en la carpeta `capitulo_18\005_ocultar_o_mostrar_mouse`.

La segunda mejora consiste en tener una función para no mostrar el sprite del puntero del mouse, que molesta durante el juego. Como el puntero del mouse se maneja en la clase `CGame`, ahí es donde colocaremos el código. Mira los cambios realizados en esta clase, relacionados a mostrar u ocultar el sprite del puntero del mouse.

...

```
class CGame(object):
```

```
    mInstance = None
    mInitialized = False
```

```
    mScreen = None
    mClock = None
    mSalir = False
    mMousePointer = None
```

```
    # Variable para mostrar o no el puntero del mouse.
    mShowGamePointer = False
```

...

```
# Función de inicialización.
def init(self):
```

```
    ...
```

```
    self.mShowGamePointer = False
    self.showGamePointer(False)
```

```
    ...
```

...

```
# Correr la lógica del juego.
def update(self):
```

```

...

# Actualizar el sprite del puntero del mouse.
if (CGame.mMousePointer != None):
    CGame.mMousePointer.update()

...

# Dibujar el frame y actualizar la pantalla.
def render(self):

    ...

    # Dibujar el puntero del mouse.
    if (CGame.mMousePointer != None):
        CGame.mMousePointer.render(self.mScreen)

    ...

...

def destroy(self):
    ...

    # Destruir el puntero del mouse.
    if (CGame.mMousePointer != None):
        CGame.mMousePointer.destroy()
        CGame.mMousePointer = None
    pygame.mouse.set_visible(True)

    ...

def showGamePointer(self, aShowGamePointer):
    self.mShowGamePointer = aShowGamePointer

    if (aShowGamePointer):
        if (CGame.mMousePointer == None):
            # Crear el sprite del puntero del mouse.
            CGame.mMousePointer = CMousePointer()
            # Ocultar el puntero del sistema.
            pygame.mouse.set_visible(False)
        else:
            # Eliminar el sprite del puntero del mouse.
            if (CGame.mMousePointer != None):
                CGame.mMousePointer.destroy()
                CGame.mMousePointer = None
            # Mostrar el puntero del sistema.

```

```
pygame.mouse.set_visible(True)
```

La variable `self.mShowGamePointer` indica si se está usando un cursor propio (un sprite) o se está usando el puntero del sistema. Para indicar si se usa un sprite o el cursor del sistema, se utiliza la función `showGamePointer()`, que recibe como parámetro `True` si se usa un sprite y `False` si se usa el cursor del sistema.

Esta función según el parámetro, pone activo o no el cursor del sistema, y en caso que se muestre el sprite del puntero del mouse, lo crea. Si ya existiera y se pasa al puntero del sistema, lo destruye.

Luego, en el loop principal, se agrega un chequeo de que la referencia del sprite del puntero del mouse no sea `None`, al dibujarlo y al actualizarlo (en `render()` y en `update()`). Recuerda que si el sprite del puntero del mouse no existe, invocar a una función da error si la referencia es `None` (no existe). Por este motivo es que se chequea que el sprite exista.

En el menú principal, en la función `init()` de la clase `CMenuState`, se invoca a `CGame.inst().showGamePointer(True)` para mostrar el sprite del puntero del mouse, y en la clase del nivel, `CLevelState`, en la función `init()`, se invoca con `False` para ocultarlo.

Máquina de Estados del Nivel

Cuando arrancamos el juego, el mismo comienza en forma inmediata, lo cual no da tiempo a prepararse y podemos perder una vida. De la misma manera, cuando perdemos todas las vidas, el juego termina abruptamente y no sabemos qué ha pasado. Vamos a corregir estos detalles.

Lo que haremos para que el juego quede más lindo, es agregar un mensaje que diga “Nivel 1” al inicio del juego, que durará unos segundos. Mientras esté el mensaje en pantalla, los enemigos no van a disparar.

De la misma manera, cuando perdemos todas las vidas, aparecerá por unos segundos un texto que dice “Game Over” en el centro de la pantalla y luego pasará al menú principal. Por último, cuando matamos a todos los enemigos, esperaremos unos segundos antes de comenzar el siguiente nivel.



Figura 18-1: Los mensajes de inicio del nivel y de fin del juego.

Cada uno de estos agregados corresponde a estados en el juego. Haremos una máquina de estados a nivel del juego (en la clase `CLevelState`), al igual que como la hicimos con el jugador, o los enemigos. Miremos el código de la clase `CLevelState` en el ejemplo que se encuentra ubicado en la carpeta `capitulo_18\006_maquina_de_estados_del_nivel`. Ejecuta el juego para ver cómo funciona el nivel con estos estados.

Como es la primera vez que ponemos una máquina de estados en una clase de una pantalla, examinemos todo el código, porque debemos de hacer lo mismo que hicimos con los objetos (manejar los estados y colocar las funciones `setState()`, `getState()` y `getTimeState()` tanto en la clase como en la clase base). Al igual que hicimos, por ejemplo, con `CPlayer` y `CGameobject`, para que todos los objetos del juego tengan máquina de estados, ahora haremos una máquina de estados en `CLevelState` y en `CGameState` para que todas las pantallas lo tengan disponible.

```
# -*- coding: utf-8 -*-
```

```
#-----
# Clase CLevelState.
# Nivel del juego. Es el juego en sí.
#
# Autor: Fernando Sansberro - Batovi Games Studio
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
#-----
```

```
import pygame
from api.CGameState import *
from api.CGame import *
from game.CNave import *
from game.CPlayer import *
from api.CGameobject import *
from game.CGameData import *
from api.CGameConstants import *
from api.CTextSprite import *
```

```

from game.CEnemyManager import *

class CLevelState(CGameState):

    mImgSpace = None
    mPlayer1 = None
    mPlayer2 = None

    mTextLives1 = None
    mTextLives2 = None
    mTextScore1 = None
    mTextScore2 = None

    # Máquina de estados.
    PLAYING = 0
    INIT_LEVEL = 1
    TRANSITION = 2
    GAME_OVER = 3

    # Variables para el control de los mensajes y estados.
    mLevel = 1
    TIME_SHOWING_LEVEL_TEXT = CGame.FPS * 2
    TIME_TRANSITION = CGame.FPS * 1
    TIME_SHOWING_GAME_OVER = CGame.FPS * 4
    mText = None

    def __init__(self):
        CGameState.__init__(self)
        print("LevelState constructor")

        self.mImgSpace = None
        self.mPlayer1 = None
        self.mPlayer2 = None

        self.mTextScore1 = None
        self.mTextScore2 = None
        self.mTextLives1 = None
        self.mTextLives2 = None

        # Comienzo del juego.
        self.mLevel = 1
        self.mText = None

    # Función donde se inicializan los elementos necesarios del nivel.
    def init(self):
        CGameState.init(self)
        print("LevelState init")

```

```

    # Cargar la imagen del fondo. La imagen es de 640 x 360.
    self.mImgSpace =
pygame.image.load("assets/images/space_640x360.jpg")
    self.mImgSpace = self.mImgSpace.convert()

    # Dibujar la imagen cargada en la imagen de background.
    CGame.inst().setBackground(self.mImgSpace)

# Crear la formación inicial.
self.initEnemies()

    # Crear el jugador 1.
    self.mPlayer1 = CPlayer(CPlayer.TYPE_PLAYER_1)
    self.mPlayer1.setXY(CGame.SCREEN_WIDTH / 4 -
self.mPlayer1.getWidth() / 2, CGame.SCREEN_HEIGHT -
self.mPlayer1.getHeight() - 20)
    self.mPlayer1.setBounds(0, 0, CGame.SCREEN_WIDTH -
self.mPlayer1.getWidth(), CGame.SCREEN_HEIGHT)
    self.mPlayer1.setBoundAction(CGameObject.STOP)

    # Crear el jugador 2.
    self.mPlayer2 = CPlayer(CPlayer.TYPE_PLAYER_2)
    self.mPlayer2.setXY(CGame.SCREEN_WIDTH / 4 * 3 -
self.mPlayer2.getWidth() / 2, CGame.SCREEN_HEIGHT -
self.mPlayer2.getHeight() - 20)
    self.mPlayer2.setBounds(0, 0, CGame.SCREEN_WIDTH -
self.mPlayer2.getWidth(), CGame.SCREEN_HEIGHT)
    self.mPlayer2.setBoundAction(CGameObject.STOP)

    # Ejecutar la música de background (loop) del juego.
    pygame.mixer.music.load("assets/audio/music_game.ogg")
    pygame.mixer.music.play(-1)
    # Poner el volumen de la música.
    pygame.mixer.music.set_volume(0.5)

    # Inicializar los datos del juego.
    CGameData.inst().setScore1(0)
    CGameData.inst().setLives1(3)
    CGameData.inst().setScore2(0)
    CGameData.inst().setLives2(3)

    self.mTextScore1 = CTextSprite("SCORE: " +
str(CGameData.inst().getScore1()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
    self.mTextScore1.setXY(5, 5)
    self.mTextLives1 = CTextSprite("VIDAS: " +
str(CGameData.inst().getLives1()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))

```

```

        self.mTextLives1.setXY(5, CGame.SCREEN_HEIGHT - 20 - 5)

        self.mTextScore2 = CTextSprite("SCORE: " +
str(CGameData.inst().getScore2()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
        self.mTextScore2.setXY(530, 5)
        self.mTextLives2 = CTextSprite("VIDAS: " +
str(CGameData.inst().getLives2()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
        self.mTextLives2.setXY(540, CGame.SCREEN_HEIGHT - 20 - 5)

        # Ocultar el sprite del puntero del mouse.
        CGame.inst().showGamePointer(False)

# Establecer el estado inicial.
self.setState(CLevelState.INIT_LEVEL)

# Actualizar los objetos del nivel.
def update(self):
    CGameState.update(self)
    print("LevelState update()")

    # Actualizar los enemigos.
    CEnemyManager.inst().update()

    # Mover las balas.
    CBulletManager.inst().update()

    # Lógica de los jugadores.
    self.mPlayer1.update()
    self.mPlayer2.update()

    if self.getState() == CLevelState.INIT_LEVEL:
        if self.getTimeState() >
CLevelState.TIME_SHOWING_LEVEL_TEXT:
            self.setState(CLevelState.PLAYING)
            return

    elif self.getState() == CLevelState.PLAYING:
        if self.mPlayer1.isGameOver() and
self.mPlayer2.isGameOver():
            print("AMBOS JUGADORES MUEREN - LOSE CONDITION")
            self.setState(CLevelState.GAME_OVER)
            return

    if CEnemyManager.inst().getLength() == 0:
        print("TODOS LOS ENEMIGOS MUEREN - WIN CONDITION")
        self.setState(CLevelState.TRANSITION)

```

```

        return

    elif self.getState() == CLevelState.GAME_OVER:
        if self.getTimeState() >
CLevelState.TIME_SHOWING_GAME_OVER:
            from game.states.CMenuState import CMenuState
            nextState = CMenuState()
            CGame.inst().setState(nextState)
            return

    elif self.getState() == CLevelState.TRANSITION:
        if self.getTimeState() > CLevelState.TIME_TRANSITION:
            self.nextLevel()
            return

    self.mTextScore1.update()
    self.mTextScore2.update()
    self.mTextLives1.update()
    self.mTextLives2.update()

    if self.mText != None:
        self.mText.update()

# Dibujar el frame del nivel.
def render(self):
    CGameState.render(self)
    print("LevelState render()")

    # Obtener la referencia a la pantalla.
    screen = CGame.inst().getScreen()

    # Dibujar los enemigos.
    CEnemyManager.inst().render(screen)

    # Dibujar las balas.
    CBulletManager.inst().render(screen)

    # Dibujar los jugadores.
    self.mPlayer1.render(screen)
    self.mPlayer2.render(screen)

    # Dibujar el texto del score y las vidas.
    self.mTextScore1.setText("SCORE: " +
str(CGameData.inst().getScore1()))
    self.mTextLives1.setText("VIDAS: " +
str(CGameData.inst().getLives1()))
    self.mTextScore2.setText("SCORE: " +
str(CGameData.inst().getScore2()))

```

```

        self.mTextLives2.setText("VIDAS: " +
str(CGameData.inst().getLives2()))

        self.mTextScore1.render(screen)
        self.mTextLives1.render(screen)
        self.mTextScore2.render(screen)
        self.mTextLives2.render(screen)

if self.mText != None:
    self.mText.render(screen)

# Destruir los objetos creados en el nivel.
def destroy(self):
    CGameState.destroy(self)
    print("LevelState destroy()")

    # Destruir la nave de los jugadores.
    self.mPlayer1.destroy()
    self.mPlayer1 = None
    self.mPlayer2.destroy()
    self.mPlayer2 = None

    self.mImgSpace = None

    # Destruir las balas.
    CBulletManager.inst().destroy()

    # Destruir los enemigos.
    CEnemyManager.inst().destroy()

    self.mTextScore1.destroy()
    self.mTextScore1 = None
    self.mTextLives1.destroy()
    self.mTextLives1 = None
    self.mTextScore2.destroy()
    self.mTextScore2 = None
    self.mTextLives2.destroy()
    self.mTextLives2 = None

if self.mText != None:
    self.mText.destroy()
    self.mText = None

    CGameData.inst().destroy()

# Establece el estado actual e inicializa
# las variables correspondientes al estado.
def setState(self, aState):

```

```

CGameState.setState(self, aState)

if self.getState() == CLevelState.INIT_LEVEL:
    CEnemyManager.inst().setCanShoot(False)
    self.mText = CTextSprite("NIVEL " + str(self.mLevel), 60,
"assets/fonts/days.otf", (0xFF, 0xFF, 0xFF))
    self.mText.setXY((CGame.SCREEN_WIDTH -
self.mText.getWidth()) / 2, (CGame.SCREEN_HEIGHT -
self.mText.getHeight()) / 2)
    elif self.getState() == CLevelState.TRANSITION:
        pass
    elif self.getState() == CLevelState.PLAYING:
        CEnemyManager.inst().setCanShoot(True)
    elif self.getState() == CLevelState.GAME_OVER:
        self.mText = CTextSprite("GAME OVER", 80,
"assets/fonts/days.otf", (0xFF, 0x00, 0x00))
        self.mText.setXY((CGame.SCREEN_WIDTH -
self.mText.getWidth()) / 2, 250)

# Crear la formación inicial.
def initEnemies(self):
    f = 0
    while f <= 4:
        c = 0
        while c <= 4:
            n = CNave(f)
            n.setXY(100 + (70 * c), 30 + (35 * f))
            n.setVelX(4)
            n.setVelY(0)
            n.setBounds(0, 0, CGame.SCREEN_WIDTH - n.getWidth(),
CGame.SCREEN_HEIGHT - n.getHeight())
            n.setBoundAction(CGameObject.NONE)
            n.setCanShoot(False)
            CEnemyManager.inst().add(n)
            c = c + 1
        f = f + 1

# Establecer la velocidad mínima y máxima de la formación.
CEnemyManager.inst().setMinMaxVelX(1, 8)

# Función para pasar de nivel.
def nextLevel(self):
    self.mLevel = self.mLevel + 1
    self.initEnemies()
    self.setState(CLevelState.INIT_LEVEL)

```

Como siempre hacemos cuando creamos una máquina de estados, definimos las constantes

para cada estado, implementamos la función `setState()` que inicializa cada estado, y se colocan las sentencias `if` correspondientes a los estados en las funciones `update()`, `render()`, etc. Mira el código para ver los cambios relacionados a la máquina de estados.

Luego definimos las constantes de tiempo para cuando se pasa de un estado a otro (`mTimeState`) y las funciones `setState()`, `getState()` y `getTimeState()` en la clase base (`CGameState`), de la misma forma que hicimos en `CGameobject` para los objetos para que todos hereden la máquina de estados, en este caso es para las pantallas.

Veamos ahora como queda la clase `CGameState` con las funciones de la máquina de estados que se han agregado:

...

```
class CGameState(object):

    def __init__(self):

        # Estado actual.
        self.mState = 0

        # Control del tiempo en el estado actual.
        self.mTimeState = 0

    def init(self):
        pass

    def update(self):
        # Incrementar el tiempo del estado actual.
        self.mTimeState = self.mTimeState + 1

    def render(self):
        pass

    def destroy(self):
        pass

    # Retorna el estado actual.
    def getState(self):
        return self.mState

    # Establece el estado actual.
    def setState(self, aState):
        self.mState = aState
        # Resetear el tiempo del estado actual.
        self.mTimeState = 0
```



```
# Retorna el tiempo en el estado actual.
def getTimeState(self):
    return self.mTimeState
```

Volvamos a ver el código de `CLevelState` y veamos los otros cambios realizados. .

En la función `init()`, se inicializan los enemigos invocando a la función `initEnemies()` y luego se establece el estado inicial invocando a `setState()`. El código de `InitEnemies()` es el mismo que había antes, pero se ha separado en una función por si se quiere usar el nivel como parámetro de velocidad, para hacer el juego más difícil en cada nivel.

Hemos puesto toda la inicialización de los enemigos en una función, llamada `initEnemies()`, porque cada vez que inicia el nivel hay que crear todos los enemigos de nuevo. Antes lo hacíamos solo al inicio, pero el juego no estaba completo, dado que si los matábamos a todos quedaba el juego sin poder continuar. Ahora, hay una función denominada `nextLevel()` que incrementa el nivel actual e inicializa los enemigos llamando a la función `initEnemies()` y pasando al estado `CLevelState.INIT_LEVEL`.

Nota: Al pasar de estados según el tiempo transcurrido, estamos asumiendo que el framerate está puesto fijo (a *60 FPS*). Dicho de otra manera, estamos usando frames para medir el tiempo. Esto no es lo mejor. Deberíamos usar el tiempo real por si la máquina es muy rápida y queremos dibujar más fluido, pero es aceptable usar tiempo basado en frames mientras se aprende a programar juegos, porque simplifica los cálculos.

En `CGame` hay una constante `FPS` para tener los frames por segundo al que corre el juego. Este valor lo necesitamos para indicar cuántos cuadros corresponde un segundo al poner los textos en el juego.

Otro cambio que necesitamos hacer en `CLevelState` es que los enemigos no disparen cuando está comenzando el nivel. Para eso, se hace una función `canShoot()` en el manager de enemigos (`CEnemyManager`), y se le pasa como parámetro `True` si pueden disparar y `False` si no pueden. Este valor se establece en la función `setState()` según el estado del juego. Al inicio no pueden disparar, y cuando se pasa al estado `CLevelState.PLAYING`, pueden disparar.

La función `canShoot()` del manager de enemigos recorre todos los enemigos e invoca a la función `canShoot()` del enemigo para indicarle si puede disparar o no. Al crear el enemigo se lo crea invocando a la sentencia `n.setCanShoot(False)` así no dispara. Luego, al pasar al estado `CLevelState.PLAYING`, se habilita el disparo.

También para mostrar u ocultar el texto según el estado del nivel, invocamos a la función `setVisible()` con `True` o `False` según el caso. Recuerda que `CTextSprite` hereda de `CSprite`, que tiene este comportamiento de visible/invisible.

Nota: Este y otros cambios hechos en el código del libro, ya no se van a listar todos. Debes revisar las variables y funciones relacionadas, y ser capaz de incorporarlas a tu propio proyecto. Si algo no sale bien, siempre tienes la opción de partir desde el proyecto del libro, o

consultar las diferencias entre tu proyecto y el código del libro con un profesor. De esta forma, ya iremos acelerando en el desarrollo, para hacer juegos más complejos. Recuerda que la práctica es lo que hace bueno a un programador.

Animación de la Llama (Usar un Sprite dentro de Otro)

En esta sección le agregaremos una llama a la nave del jugador. Para esto, crearemos un sprite animado con la llama, y lo colocaremos en la parte inferior de la nave. Tendremos un sprite que es usado dentro de otro sprite.

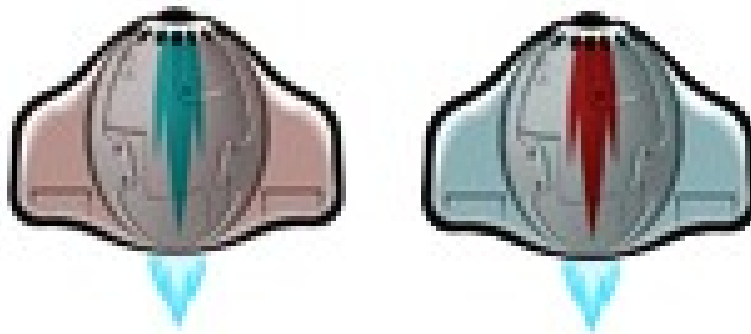


Figura 18-2: Fuego de la nave del jugador.

Podríamos hacer que el fuego sea parte del sprite animado, pero como la nave gira, es mejor hacerlo de esta manera, y además aprenderemos cómo se puede componer un sprites de otros sprites.

Mira el ejemplo que se encuentra en la carpeta `capitulo_18\007_llama_de_los_jugadores` y ejecútalo. Verás que las naves de los jugadores tienen una llama abajo.

Comenzamos creando un sprite animado para la llama, que implementamos en la clase `Flame`. Como siempre, ponemos las imágenes de la llama en la carpeta de imágenes en la carpeta de assets, y cargamos los frames al inicio. La animación de la llama es cíclica, por lo cual la clase no tiene mucho código.

```
# -*- coding: utf-8 -*-

# -----
# Clase CFlame.
# Fuego de la nave del jugador.
#
# Autor: Fernando Sansberro - Batovi Games.
# Proyecto: Hacete tu Videojuego.
# Licencia: Creative Commons. BY-NC-SA.
```

```

# -----

import pygame
from api.CAnimatedSprite import *

class CFlame(CAnimatedSprite):

    def __init__(self):

        CAnimatedSprite.__init__(self)

        self.mFrames = []
        i = 0
        while i <= 2:
            num = "0" + str(i)
            tmpImg = pygame.image.load("assets/images/fire" + num +
".png")
            tmpImg = tmpImg.convert_alpha()
            self.mFrames.append(tmpImg)
            i = i + 1

        self.initAnimation(self.mFrames, 0, 2, True)

    def update(self):
        CAnimatedSprite.update(self)

    def render(self, aScreen):
        CAnimatedSprite.render(self, aScreen)

    def destroy(self):
        CAnimatedSprite.destroy(self)

        i = len(self.mFrames)
        while i > 0:
            self.mFrames[i-1] = None
            self.mFrames.pop(i-1)
            i = i - 1

```

En la clase `CPlayer`, es donde colocaremos la llama del jugador. En el constructor creamos la instancia de la llama y luego en las funciones `update()` y `render()` invocamos a las funciones `update()` y `render()` de la llama respectivamente. En cada llamada a la función `update()` de la nave del jugador, se invoca a la función `setFlamePosition()`, que coloca al sprite de la llama centrado en la parte inferior de la nave. De esta forma, cada vez que se mueve la nave, la llama lo sigue. Esta es la forma de implementar un sprite compuesto, o sea, un sprite que contiene otros sprites.

Vemos en el código de la clase `CPlayer` los cambios que tienen que ver con la llama.

```

...

from game.CFlame import *

# La clase CPlayer hereda de CAnimatedSprite.
class CPlayer(CAnimatedSprite):

    ...

    def __init__(self, aType):

        ...

        # Crear la llama de la nave y posicionarla.
        self.mFlame = CFlame()
        self.setFlamePosition()

        # Estado inicial.
        self.setState(CPlayer.NORMAL)

# Mover el objeto.
def update(self):

    # Invocar update() de CAnimatedSprite.
    CAnimatedSprite.update(self)

    # Luego de mover la nave, actualizar la llama.
    self.mFlame.update()
    self.setFlamePosition()

    # Obtener el enemigo con el cual chocamos.
    enemy = CEnemyManager.inst().collides(self)

    ...

# Dibuja el objeto en la pantalla.
# Parámetros:
# aScreen: La superficie de la pantalla en donde dibujar.
def render(self, aScreen):

    # En el estado game over la nave no se dibuja.
    if self.getState() == CPlayer.GAME_OVER:
        return

    # Poner visible o invisible según el caso.
    if self.getState() == CPlayer.DYING:
        if self.getTimeState() % 8 == 0:

```

```

        self.setVisible(True)
    else:
        self.setVisible(False)

elif self.getState() == CPlayer.EXPLODING:
    pass

elif self.getState() == CPlayer.START:
    if self.getTimeState() % 16 == 0:
        self.setVisible(True)
    else:
        self.setVisible(False)

CAnimatedSprite.render(self, aScreen)

# Dibujar la llama.
self.mFlame.render(aScreen)

# Liberar lo que haya creado el objeto.
def destroy(self):

    ...

if self.mFlame != None:
    self.mFlame.destroy()
    self.mFlame = None

# Establece el estado actual e inicializa
# las variables correspondientes al estado.
def setState(self, aState):
    CAnimatedSprite.setState(self, aState)

# Por defecto poner el sprite visible.
self.setVisible(True)

if self.getState() == CPlayer.NORMAL:
    self.initAnimation(self.mFrames, 0, 0, False)
    self.gotoAndStop(0)

elif self.getState() == CPlayer.DYING:
    self.stopMove()
    self.initAnimation(self.mFrames, 0, 0, False)
    self.gotoAndStop(0)

    # Ejecutar sonido de morir.
    CAudioManager.inst().play(CAudioManager.mSoundHitPlayer)

elif self.getState() == CPlayer.EXPLODING:

```

```

        self.initAnimation(self.mFramesExplosion, 0, 2, False)

        # Ejecutar el sonido de la explosión.

CAudioManager.inst().play(CAudioManager.mSoundExplosionPlayer)

        self.mFlame.setVisible(False)

elif self.getState() == CPlayer.START:
    self.initAnimation(self.mFrames, 0, 0, False)
    self.gotoAndStop(0)

elif self.getState() == CPlayer.GAME_OVER:
    self.setVisible(False)

...

# Establece si el sprite es visible o no.
# Parámetro: True para que se dibuje, False para que no se dibuje.
def setVisible(self, aIsVisible):

    # Invocar destroy() de CAnimatedSprite.
    CAnimatedSprite.setVisible(self, aIsVisible)

    # Mostrar u ocultar la llama.
    self.mFlame.setVisible(aIsVisible)

# Posicionar la llama debajo de la nave.
def setFramePosition(self):
    self.mFlame.setX(self.getX() + self.getWidth() / 2 -
self.mFlame.getWidth() / 2)
    self.mFlame.setY(self.getY() + self.getHeight())

```

En la función `update()` del jugador, se coloca la llama en su posición, luego de que se haya movido la nave, o sea, luego de invocar a la función `update()` de la clase base que es la que la coloca en su nueva posición. Esto es para que la llama se coloque usando la posición del frame actual. Es un error común colocar la llama antes de mover la nave, con lo cual quedarían desfasadas.

En la función `setState()`, se coloca la llama visible o invisible, según corresponda el estado. Por defecto la nave siempre está visible, por eso se llama a `setVisible(True)` al inicio de `setState()`. Hay estados en los cuales la nave parpadea, y lo mismo hay que hacer con la llama. Esto lo hacemos en una propia función `setVisible()` en la clase, la cual pone la visibilidad de la nave, y de todos sus componentes (en este caso, la llama).

```

# Establece si el sprite es visible o no.

```

```

# Parámetro: True para que se dibuje, False para que no se dibuje.
def setVisible(self, aIsVisible):

    # Invocar destroy() de CAnimatedSprite.
    CAnimatedSprite.setVisible(self, aIsVisible)

    # Mostrar u ocultar la llama.
    self.mFlame.setVisible(aIsVisible)

```

Nota: Este concepto es muy importante al componer sprites. Se sobrescriben las funciones de la clase base, pero se agrega la parte para los componentes del sprite.

Luego, por último, se pone invisible la llama cuando explota la nave (en el estado `CPlayer.EXPLODING`).

Modo de Uno o Dos Jugadores

Para terminar con el nivel del juego, haremos que se pueda jugar con uno o con dos jugadores. Para eso, cambiaremos el mensaje que se muestra en el menú principal, por el mensaje: "Pulsa: [Space] = Un jugador, [Q] = Dos jugadores...".

Cuando se pulsa [Space] o [Q] para seleccionar uno o dos jugadores respectivamente, se le pasa como parámetro al constructor de la clase `CLevelState` el número de jugadores (1 o 2).

Mira el ejemplo que se encuentra en la carpeta `capitulo_18\008_uno_o_dos_jugadores`. En la clase `CMenuState`, se invoca al constructor de `CLevelState` con la cantidad de jugadores como parámetro:

```

def update(self):
    CGameState.update(self)

    if CKeyboard.inst().fire():
        from game.states.CLevelState import CLevelState
        nextState = CLevelState(1)
        CGame.inst().setState(nextState)
        return

    if CKeyboard.inst().fire2():
        from game.states.CLevelState import CLevelState
        nextState = CLevelState(2)
        CGame.inst().setState(nextState)
        return

    self.mTextTitle.update()

```

```
self.mTextPressFire.update()
```

Luego, en la clase `CLevelState`, tenemos una variable `mNumPlayers` donde guardamos la cantidad de jugadores que hay en el juego (1 o 2). Según esta variable, se crea o no el segundo jugador, y luego se actualiza o no, se dibuja o no, o se muestra o no los indicadores del segundo jugador. También se usa esta variable para tener en cuenta uno o dos jugadores a la hora de morir y chequear la condición de perder. Veamos el código de `CLevelState` con los cambios:

```
...
```

```
class CLevelState(CGameState):
```

```
...
```

```
# Cantidad de jugadores.
```

```
mNumPlayers = 1
```

```
def __init__(self, aNumPlayers):
```

```
    CGameState.__init__(self)
```

```
    print("LevelState constructor")
```

```
# Establecer la cantidad de jugadores.
```

```
self.mNumPlayers = aNumPlayers
```

```
self.mImgSpace = None
```

```
self.mPlayer1 = None
```

```
self.mPlayer2 = None
```

```
self.mTextScore1 = None
```

```
self.mTextScore2 = None
```

```
if self.mNumPlayers == 2:
```

```
    self.mTextLives1 = None
```

```
    self.mTextLives2 = None
```

```
# Comienzo del juego.
```

```
self.mLevel = 1
```

```
self.mText = None
```

```
# Función donde se inicializan los elementos necesarios del nivel.
```

```
def init(self):
```

```
...
```

```
# Crear el jugador 1.
```

```
self.mPlayer1 = CPlayer(CPlayer.TYPE_PLAYER_1)
```

```
self.mPlayer1.setXY(CGame.SCREEN_WIDTH / 4 -
```



```

self.mPlayer1.getWidth() / 2, CGame.SCREEN_HEIGHT -
self.mPlayer1.getHeight() - 20)
    self.mPlayer1.setBounds(0, 0, CGame.SCREEN_WIDTH -
self.mPlayer1.getWidth(), CGame.SCREEN_HEIGHT)
    self.mPlayer1.setBoundAction(CGameObject.STOP)

    if self.mNumPlayers == 2:
        # Crear el jugador 2.
        self.mPlayer2 = CPlayer(CPlayer.TYPE_PLAYER_2)
        self.mPlayer2.setXY(CGame.SCREEN_WIDTH / 4 * 3 -
self.mPlayer2.getWidth() / 2, CGame.SCREEN_HEIGHT -
self.mPlayer2.getHeight() - 20)
        self.mPlayer2.setBounds(0, 0, CGame.SCREEN_WIDTH -
self.mPlayer2.getWidth(), CGame.SCREEN_HEIGHT)
        self.mPlayer2.setBoundAction(CGameObject.STOP)

    ...

    self.mTextScore1 = CTextSprite("SCORE: " +
str(CGameData.inst().getScore1()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
    self.mTextScore1.setXY(5, 5)
    self.mTextLives1 = CTextSprite("VIDAS: " +
str(CGameData.inst().getLives1()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
    self.mTextLives1.setXY(5, CGame.SCREEN_HEIGHT - 20 - 5)

    if self.mNumPlayers == 2:
        self.mTextScore2 = CTextSprite("SCORE: " +
str(CGameData.inst().getScore2()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
        self.mTextScore2.setXY(530, 5)
        self.mTextLives2 = CTextSprite("VIDAS: " +
str(CGameData.inst().getLives2()), 20, "assets/fonts/days.otf",
(0xFF, 0xFF, 0xFF))
        self.mTextLives2.setXY(540, CGame.SCREEN_HEIGHT - 20 - 5)

    ...

# Actualizar los objetos del nivel.
def update(self):
    CGameState.update(self)

    # Actualizar los enemigos.
    CEnemyManager.inst().update()

    # Mover las balas.
    CBulletManager.inst().update()

```

```

# Lógica de los jugadores.
self.mPlayer1.update()
if self.mNumPlayers == 2:
    self.mPlayer2.update()

print(self.getState())

if self.getState() == CLevelState.INIT_LEVEL:
    if self.getTimeState() >
CLevelState.TIME_SHOWING_LEVEL_TEXT:
        self.setState(CLevelState.PLAYING)
        return

    elif self.getState() == CLevelState.PLAYING:
        if (self.mNumPlayers == 1 and self.mPlayer1.isGameOver())
or (self.mNumPlayers == 2 and self.mPlayer1.isGameOver() and
self.mPlayer2.isGameOver()):
            print("AMBOS JUGADORES MUEREN - LOSE CONDITION")
            self.setState(CLevelState.GAME_OVER)
            return

        if CEnemyManager.inst().getLength() == 0:
            print("TODOS LOS ENEMIGOS MUEREN - WIN CONDITION")
            self.setState(CLevelState.TRANSITION)
            return

...

self.mTextScore1.update()
self.mTextLives1.update()
if self.mNumPlayers == 2:
    self.mTextScore2.update()
    self.mTextLives2.update()

if self.mText != None:
    self.mText.update()

# Dibujar el frame del nivel.
def render(self):
    CGameState.render(self)

# Obtener la referencia a la pantalla.
screen = CGame.inst().getScreen()

# Dibujar los enemigos.
CEnemyManager.inst().render(screen)

```

```

# Dibujar las balas.
CBulletManager.inst().render(screen)

# Dibujar los jugadores.
self.mPlayer1.render(screen)
if self.mNumPlayers == 2:
    self.mPlayer2.render(screen)

# Dibujar el texto del score y las vidas.
self.mTextScore1.setText("SCORE: " +
str(CGameData.inst().getScore1()))
self.mTextLives1.setText("VIDAS: " +
str(CGameData.inst().getLives1()))
if self.mNumPlayers == 2:
    self.mTextScore2.setText("SCORE: " +
str(CGameData.inst().getScore2()))
    self.mTextLives2.setText("VIDAS: " +
str(CGameData.inst().getLives2()))

self.mTextScore1.render(screen)
self.mTextLives1.render(screen)
if self.mNumPlayers == 2:
    self.mTextScore2.render(screen)
    self.mTextLives2.render(screen)

if self.mText != None:
    self.mText.render(screen)

# Destruir los objetos creados en el nivel.
def destroy(self):
    CGameState.destroy(self)

# Destruir la nave de los jugadores.
self.mPlayer1.destroy()
self.mPlayer1 = None
if self.mNumPlayers == 2:
    self.mPlayer2.destroy()
    self.mPlayer2 = None

self.mImgSpace = None

# Destruir las balas.
CBulletManager.inst().destroy()

# Destruir los enemigos.
CEnemyManager.inst().destroy()

self.mTextScore1.destroy()

```

```

self.mTextScore1 = None
self.mTextLives1.destroy()
self.mTextLives1 = None
if self.mNumPlayers == 2:
    self.mTextScore2.destroy()
    self.mTextScore2 = None
    self.mTextLives2.destroy()
    self.mTextLives2 = None

if self.mText != None:
    self.mText.destroy()
    self.mText = None

CGameData.inst().destroy()

...

```

Como vemos en el código, hay una sentencia `if` para cada vez que existe código relacionado al segundo jugador. Esto es porque el segundo jugador podría no existir, y si este es el caso, no se debe invocar a sus funciones `update()`, `render()` y `destroy()`. Del mismo modo, si hay un solo jugador, entonces no hay que actualizar ni dibujar los sprites de texto correspondientes a los marcadores del segundo jugador.

Cuando se está en el estado `CLevelState.PLAYING`, hay que preguntar si todos los jugadores han muerto para saber si termina el juego. La condición para terminar el juego se ve reflejada en el siguiente `if`:

```

elif self.getState() == CLevelState.PLAYING:
    if (self.mNumPlayers == 1 and self.mPlayer1.isGameOver()) or
    (self.mNumPlayers == 2 and self.mPlayer1.isGameOver() and
    self.mPlayer2.isGameOver()):
        print("AMBOS JUGADORES MUEREN - LOSE CONDITION")
        self.setState(CLevelState.GAME_OVER)
        return

```

Esto es, si la cantidad de jugadores es uno y el primer jugador está muerto, se termina (la sentencia `if` vale `True`). O, si la cantidad de jugadores es dos y tanto el jugador uno como el dos han muerto. Analiza bien los paréntesis y lee la frase en español para entender cómo funciona esta pregunta.

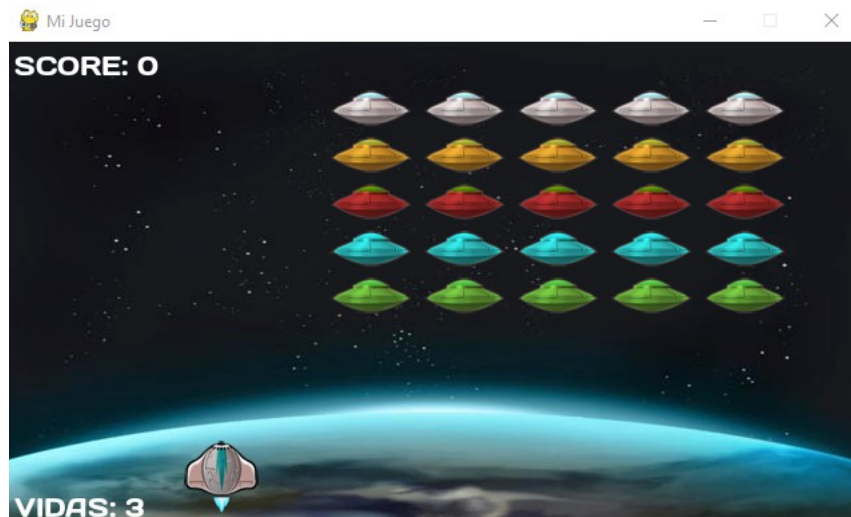


Figura 18-3: El juego con un solo jugador.

Con esto terminamos nuestro primer juego. ¡Felicitaciones! Ha sido un largo camino hasta este punto, y mucho de lo que hemos hecho lo reutilizaremos en nuestros próximos juegos.

Nota: Aún quedan detalles y muchos temas por aprender. Algunos temas irán en los futuros libros, y otros en forma de artículos que serán publicados en el sitio del libro (www.fsansberro.com). Visita frecuentemente este sitio para ver las novedades.